



CI13410-a High-Speed USB Device Controller Core

Doc.Revision: 1.0A

Approval: Luis Miguel Sampaio Date: 2006-05-11

Company Confidential
Chipidea® - Microelectrónica, S.A.
ISO 9001:2000 - Certified Company

Av. Dr. Mário Soares, Lote 21 - TAGUSPARK
2740-119 Porto Salvo **Portugal**
Tel.: (+351) 210336300
Fax: (+351) 210336396
<http://www.chipidea.com>

Revision History

Doc. Revision	Date	Prepared	Description
0.1	2006-01-04	Joana Braga	From Template TP02_DEP_GP
0.2	2006-01-10	Joana Braga	Document update
0.3	2006-01-11	Joana Braga	New text format
0.4	2006-01-19	Joana Braga	- Updated Chapters 8.2 and 11.5.3 - Deleted Chapters 11.5.3.1, 11.5.4 and 12.3
0.5	2006-02-03	Joana Braga	Updated Chapter 3
1.0A	2006-05-11	Joana Braga	- Converted document to OpenOffice - Updated figures in "API Header Files and API Source Files" Chapter - added the bit description for the TTCTRL register - USBCMD register bit description updated - Updated ULPI Block Diagram - Replaced CtesterExcuteList, Cexecutetestlist, CTesterExecuteList_MasterList and CtestExecuteListfile by CTesterExecuteList - Replaced "Precise USB2.0 Device API" by "USB2.0 Device API" - Updated "Synthesis script" Chapter - Updated "Running the Verilog Test Bench for Synopsys VCS" Chapter

		<i>Title:</i> CI13410-a High-Speed USB Device Controller Core			<i>Revision:</i> 1.0A	
<i>Template:</i> TP02_DEP_GP	<i>Department:</i> IPCS	<i>Prepared by:</i> Joana Braga	<i>Approved by:</i> Luis Miguel Sampaio	<i>Security Level:</i> confidential	<i>Date:</i> 2006-05-11	<i>Page:</i> 3-191

ChipIdea® Microelectrónica, S.A. reserves the right to make changes to the information contained herein without notice. No liability shall be incurred as a result of its use or application.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 4-191

Table of Contents

1 INTRODUCTION.....	14
1.1 DOCUMENT OVERVIEW.....	14
1.1.1 Style Conventions.....	14
1.2 USB	15
1.3 USB 2.0.....	15
1.4 USB ON-THE-GO.....	16
1.5 USB-HS BLOCK DIAGRAM	16
1.6 USB-HS CORE FEATURES	17
2 FUNCTIONAL DESCRIPTION.....	18
2.1 SOFTWARE MODEL	18
2.1.1 Device Data Structure.....	18
2.2 HARDWARE MODEL.....	18
2.2.1 USB-HS Block Diagram	18
2.2.2 Microprocessor Interface.....	20
2.2.3 DMA Engine	20
2.2.4 Dual Port RAM Controller.....	21
2.2.5 Protocol Engine	22
2.2.6 Port Controller.....	23
2.2.7 System Bus Interface.....	24
2.2.7.1 BVCI Overview.....	24
2.2.7.2 AMBA-AHB Overview.....	24
3 REGISTER INTERFACE.....	25
3.1 CONFIGURATION, CONTROL AND STATUS REGISTER SET.....	25
3.2 SUMMARY OF REGISTER LAYOUTS.....	28
3.3 IDENTIFICATION REGISTERS.....	31
3.3.1 ID.....	31
3.3.2 HWGENERAL.....	31
3.3.3 HWHOST.....	32
3.3.4 HWDEVICE.....	32
3.3.5 HWTXBUF.....	33
3.3.6 HWRXBUF.....	33
3.4 DEVICE/HOST CAPABILITY REGISTERS.....	33
3.4.1 CAPLENGTH – EHCI Compliant.....	33
3.4.2 HCIVERSION – EHCI Compliant.....	34
3.4.3 HCSPARAMS – EHCI Compliant with extensions.....	34
3.4.4 HCCPARAMS – EHCI Compliant.....	35
3.4.5 DCIVERSION (Non-EHCI).....	36
3.4.6 DCCPARAMS (Non-EHCI).....	37
3.5 DEVICE/HOST TIMER REGISTERS (Non-EHCI).....	37
3.5.1 GPTIMER0LD (Non-EHCI).....	37
3.5.2 GPTIMER0CTRL (Non-EHCI).....	38
3.5.3 GPTIMER1LD (Non-EHCI).....	38
3.5.4 GPTIMER1CTRL (Non-EHCI).....	39
3.6 DEVICE/HOST OPERATIONAL REGISTERS.....	39
3.6.1 USBCMD.....	39
3.6.2 USBSTS.....	42
3.6.3 USBINTR.....	45

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 5-191

3.6.4	FRINDEX.....	47
3.6.5	CTRLDSSEGMENT.....	48
3.6.6	PERIODICLISTBASE; DEVICEADDR.....	48
3.6.6.1	Host Controller (PERIODICLISTBASE).....	48
3.6.6.2	Device Controller (USB DEVICEADDR).....	49
3.6.7	ASYNCLISTADDR; ENDPOINTLISTADDR.....	49
3.6.7.1	Host Controller (ASYNCLISTADDR).....	50
3.6.7.2	Device Controller (ENDPOINTLISTADDR).....	50
3.6.8	BURSTSIZE.....	50
3.6.9	ULPI VIEWPORT.....	51
3.6.10	ENDPTNAK.....	52
3.6.11	ENDPTNAKEN.....	53
3.6.12	CONFIGFLAG.....	53
3.6.13	PORTSCx.....	54
3.6.13.1	Device Controller.....	54
3.6.14	USBMODE.....	60
3.6.15	ENDPTSETUPSTAT.....	62
3.6.16	ENDPTPRIME.....	62
3.6.17	ENDPTFLUSH.....	63
3.6.18	ENDPTSTAT.....	64
3.6.19	ENDPTCOMPLETE.....	65
3.6.20	ENDPTCTRL0.....	65
3.6.21	ENDPTCTRL1 – ENDPTCTRL15.....	67
4	DEVICE DATA STRUCTURES.....	71
4.1	ENDPOINT QUEUE HEAD (dQH).....	71
4.1.1	Endpoint Capabilities/Characteristics.....	72
4.1.2	Transfer Overlay.....	73
4.1.3	Current dTD Pointer.....	73
4.1.4	Set-up Buffer.....	73
4.2	ENDPOINT TRANSFER DESCRIPTOR (dTD).....	74
5	DEVICE OPERATIONAL MODEL.....	77
5.1	DEVICE CONTROLLER INITIALIZATION.....	77
5.2	PORT STATE AND CONTROL.....	78
5.2.1	Bus Reset.....	79
5.2.2	Suspend/Resume.....	79
5.2.2.1	Suspend.....	79
5.2.2.1.1	Suspend Description.....	79
5.2.2.1.2	Suspend Operational Model	80
5.2.2.2	Resume.....	80
5.3	MANAGING ENDPOINTS.....	80
5.3.1	Endpoint Initialization.....	81
5.3.2	Stalling.....	81
5.3.3	Data Toggle	82
5.3.3.1	Data Toggle Reset.....	82
5.3.3.2	Data Toggle Inhibit.....	82
5.4	OPERATIONAL MODEL FOR PACKET TRANSFERS.....	82
5.4.1.1	Priming Transmit Endpoints.....	83
5.4.1.2	Priming Receive Endpoints.....	83
5.4.2	Interrupt/Bulk Endpoint Operational Model.....	83
5.4.2.1	Interrupt/Bulk Endpoint Bus Response Matrix.....	84

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 6-191

5.4.3 Control Endpoint Operation Model.....	85
5.4.3.1 Setup Phase.....	85
5.4.3.1.1 Setup Packet Handling (Pre-2.3 hardware).....	85
5.4.3.1.2 Setup Packet Handling (2.3 hardware and later).....	85
5.4.3.2 Data Phase.....	86
5.4.3.3 Status Phase.....	86
5.4.3.4 Control Endpoint Bus Response Matrix.....	86
5.4.4 Isochronous Endpoint Operational Model.....	87
5.4.4.1 Isochronous Pipe Synchronization.....	88
5.4.4.2 Isochronous Endpoint Bus Response Matrix.....	88
5.5 MANAGING QUEUE HEADS.....	89
5.5.1 Queue Head Initialization.....	89
5.5.2 Operational Model For Setup Transfers.....	90
5.6 MANAGING TRANSFERS WITH TRANSFER DESCRIPTORS.....	90
5.6.1 Software Link Pointers.....	90
5.6.2 Building a Transfer Descriptor.....	90
5.6.3 Executing A Transfer Descriptor.....	91
5.6.4 Transfer Completion.....	91
5.6.5 Flushing/De-priming an Endpoint.....	92
5.6.6 Device Error Matrix.....	92
5.7 SERVICING INTERRUPTS.....	93
5.7.1 High-Frequency Interrupts.....	93
5.7.2 Low-Frequency Interrupts.....	93
5.7.3 Error Interrupts.....	93
6 SIGNALS	95
6.1 SIGNAL DESCRIPTIONS.....	95
6.1.1 System Clock/Resets.....	95
6.1.2 BVCI Bus Target Interface.....	95
6.1.3 BVCI Bus Initiator Interface.....	96
6.1.4 AMBA-AHB Bus Slave Interface.....	96
6.1.5 AMBA-AHB Bus Master Interface.....	96
6.1.6 Interrupt.....	97
6.1.7 USB-HS Transceiver Interfaces.....	97
6.1.7.1 Transceiver select signals.....	97
6.1.7.2 UTMI signals.....	97
6.1.7.3 ULPI signals.....	98
6.1.7.4 Full Speed Serial signals.....	99
6.1.7.5 USB port power control signals	99
6.1.7.6 USB power suspend control signals.....	100
6.1.8 High Speed Receive Buffer RAM.....	100
6.1.8.1 High Speed Receive Buffer Wiring.....	101
6.1.8.2 High Speed Receive Buffer Timing.....	102
6.1.9 High Speed Transmit Buffer RAM.....	102
6.1.9.1 High Speed Transmit Buffer Wiring.....	103
6.1.9.2 High Speed Transmit Buffer Timing.....	103
7 SYSTEM LEVEL ISSUES AND CORE CONFIGURATION.....	104
7.1 CONFIGURATION CONSTANTS.....	104
7.2 BUS CONFIGURATION.....	106
7.3 CORE AND SYSTEM CLOCKING.....	107
7.3.1 Transceiver clocked design.....	108

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 7-191

7.3.2 System clocked design.....	109
7.3.3 Mixed Clocked Design.....	110
7.3.4 Triple Clocked Design.....	110
7.3.5 Negative edge clocks.....	111
7.4 RESET STRATEGY.....	112
7.5 BANDWIDTH AND LATENCY ISSUES	112
7.5.1 Sizing the device controller endpoint latency buffers.....	113
7.6 INTERRUPT FREQUENCY.....	114
7.7 POWER.....	114
7.7.1 USB power states.....	115
7.7.2 Device Power States.....	115
7.7.3 Host Power States.....	117
7.7.4 Suspend Control Logic.....	117
7.7.4.1 UTMI Suspend Control Logic.....	119
7.7.4.2 ULPI Suspend Control Logic.....	121
7.7.4.3 Serial (Legacy) Suspend Control Logic.....	123
7.7.5 Transceiver Selection.....	123
8 VERIFICATION SIMULATION ENVIRONMENT	125
8.1 INTRODUCTION	125
8.1.1 Verification of the USB-HS Core.....	125
8.1.2 Integration Verification Environment.....	125
8.1.3 Integration Verification Environment Features.....	125
8.1.4 Test Bench Architecture.....	125
8.1.4.1 The Integration Verification Environment Software.....	125
8.1.4.2 The HDL Responder Model.....	126
8.1.4.3 The PHY Behavioral Model.....	126
8.1.4.4 Test Control Command Channel	126
8.1.5 The Simulation tests Shipped With The USB-HS Core.....	126
8.1.5.1 Running The Delivered USB-HS Simulation tests.....	127
8.2 GETTING STARTED.....	134
8.2.1 System Requirements For Simulation.....	134
8.2.2 Location of the binaries.....	134
8.2.3 Running the VHDL/Verilog Test Bench for Modelsim.....	134
8.2.4 Running the Verilog Test Bench for NC Verilog.....	135
8.2.5 Running the Verilog Test Bench for Synopsys VCS.....	135
8.2.6 Changing USB-HS Tests Executed	135
8.3 CUSTOMER SIMULATION ENVIRONMENT.....	136
8.3.1 HDL USB Bus Modeling Using 5 bits.....	136
8.3.1.1 Overview	136
8.3.1.2 Description	137
8.3.1.3 Bus States	137
8.3.1.4 Resolution Function	139
8.3.1.4.1 PULL_STATE = ZZZZ.....	139
8.3.1.4.2 PULL_STATE = PULL00 (HIGH SPEED).....	140
8.3.1.4.3 PULL_STATE = PULL01 (LOW SPEED).....	140
8.3.1.4.4 PULL_STATE = PULL10 (FULL SPEED).....	141
8.3.1.4.5 Test Bench	141
8.3.1.4.6 PHY Model Integration	141
8.3.2 Processor System Requirements for the Integration Test Software.....	142
8.3.3 Integration steps.....	142
8.4 DEVICE AND HOST INTEGRATION SOFTWARE TESTS.....	143

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 8-191

8.4.1	<i>Brief Summary of the Tests for both Host and Device</i>	143
8.4.1.1	Register Interfaces:.....	143
8.4.1.2	Interrupt Tests:.....	143
8.4.1.3	Data Transfers:.....	143
8.4.1.3.1	All USB data types are transferred including:.....	143
8.4.1.3.2	All transfer attributes are varied including:.....	143
8.4.1.4	Bus Signaling:.....	143
8.4.1.5	OTG signaling:.....	143
8.4.1.6	Example of a grep for “Testing” in the file SoftwareMessagesIO.txt.....	144
8.5	PORTING THE SOFTWARE TESTING API LAYER FOR AN EMBEDDED PROCESSOR.....	144
8.5.1	<i>Portability features of the Integration Software</i>	144
8.5.2	<i>The Common Include Directory</i>	145
8.5.3	<i>API Header Files and API Source Files</i>	145
8.5.4	<i>Examples Of Ported Sections of Software</i>	146
8.5.4.1	Class CMessageIO.....	146
8.5.4.2	Class CVirtualComponentInterface writing	146
8.5.4.3	Function Delete Memory.....	147
8.5.4.4	Function SystemAbort.....	147
8.5.4.5	Function AllocMemory.....	147
8.5.4.6	Function SystemWait	147
8.5.4.7	Function InitRandomNumber and RandomNumber.....	147
8.5.4.8	Function SystemCreateThread.....	147
8.5.4.9	Function main defined.....	148
8.6	DETAILED DESCRIPTION OF THE INTEGRATION VERIFICATION ENVIRONMENT.....	148
8.6.1	<i>Layered Approach of the Test Software and the Co-Simulation BFM</i>	149
8.6.2	<i>Common Abstractions Used In The Software Test Code And Basic Framework Information</i>	150
8.6.3	<i>Test Software Dependencies</i>	150
8.6.3.1	Key Class Naming Conventions.....	150
8.6.4	<i>Register Testing</i>	151
8.6.5	<i>Interrupt Tests</i>	152
8.6.6	<i>Bus Signaling Tests for Host Mode</i>	152
8.6.7	<i>Host Data Type Transfers Testing</i>	153
8.6.8	<i>OTG Signaling</i>	153
8.6.9	<i>Running Device Only Or Host Only Functions with an OTG core</i>	155
8.6.10	<i>Altering the regression tests</i>	155
8.6.11	<i>The recommend methodology for utilizing a compute grid.</i>	156
8.6.12	<i>Software Class information for the Host and Device Tests</i>	156
8.6.12.1	Top Level Managers	156
8.6.12.2	Host Controller Manager and Sub Components.....	156
8.6.12.3	Host Frame List Element Pointer Sub Components.....	157
8.6.12.4	Host Queue Head Interface Sub Components.....	157
8.6.12.5	Host QueueElement Transfer Descriptor Interface Sub Components.....	157
8.6.12.6	Host Interrupt, Bulk and Control Data Sub Components.....	158
8.6.12.7	Host Isochronous High Speed Interface Sub Components.....	158
8.6.12.8	Host SplitTransactionIsochronousTransferDescriptorInterface Sub Components.....	158
8.6.12.9	CUSBIsochronousData Sub Components.....	158
8.6.12.10	Device Manager And Sub Components.....	159
8.6.12.11	Device Endpoint Queue Head Interface Sub Components.....	159
8.6.12.12	Device Queue Element Transfer Descriptor Interface Sub Components.....	159
8.6.12.13	Device Interrupt, Bulk, Control and Isochronous Data Sub Components.....	160
8.7	INTEGRATION TEST CLASSES	160
8.7.1	<i>Key Testing Classes</i>	160

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 9-191

8.7.2 Device Testing Classes.....	161
8.7.2.1 Device CUSBMultiFunctionComboPeripheralTester Dependencies	161
8.7.2.2 CUSBMultiFunctionComboPeripheralTester Sub Tests.....	161
8.7.2.2.1 Bulk Testing	161
8.7.2.2.2 ISO Testing.....	161
8.7.2.2.3 Interrupt Testing	161
8.7.2.2.4 Control Testing	162
8.7.2.2.5 USB Address Change Testing.....	162
8.7.2.2.6 HandShake Sequence Testing.....	162
8.7.3 Host Testing Classes.....	162
8.7.3.1 EHCI Test 2_2 Information.....	162
8.7.3.2 EHCI Test 3_11 Sub Components.....	162
8.7.3.3 EHCI Test 3_13 Sub Components.....	163
8.8 INTEGRATION TESTS EXECUTION.....	163
8.8.1 List of Events from the Host and Device Integration Test Simulation.....	163
8.8.1.1 Responder HDL at Time Zero.....	163
8.8.1.2 OTG TESTING:.....	163
8.8.1.3 HOST TESTING:.....	164
8.8.1.4 DEVICE TESTING:.....	164
8.9 SOFTWARE FLOW USING HOST BULK TRANSACTION AS AN EXAMPLE.....	166
8.9.1 Step 1 - Create endpoint information for an IN and the OUT pipe using CUSBEndPointInformation.Create()	166
8.9.2 Step 2 - Use the CUSBResponderInterface::SetupEchoEndPoint() and pass the CUSBEndPointInformation	166
8.9.3 Step 3 - Create CTestData, one for the IN data and one for the OUT data.....	166
8.9.4 Step 4 - Create two CUSBBulkData one for the IN and one for the OUT. In the create function pass the address of the CTestData buffers.....	167
8.9.5 Step 5 - Process the OUT data on the CAsynchronousListManager.....	167
8.9.6 Step 6 - Process the IN data on the CAsynchronousListManager.....	167
8.9.7 Step 7 - Check the data echoed back correctly.....	167
8.10 SOFTWARE FLOW USING DEVICE BULK IN TRANSACTION AS AN EXAMPLE.....	167
8.10.1 Step 1 - Create the data to transfer as an IN.....	167
8.10.2 Step 2 - Put a pattern into the data so that the responder will get the correct pattern.....	168
8.10.3 Step 3 - Get a pointer to the endpoint configured for bulk.....	168
8.10.4 Step 4 - Create the CBULKDevice data using the endpoint information and test data.....	168
8.10.5 Step 5 - Queue the data for transfer.....	168
8.10.6 Step 6 - Request the transfer from the host responder.....	168
8.10.7 Step 7 - Wait for the transaction to complete.....	168
8.11 CONTROLLING THE LENGTH OR THE NUMBER OF TESTS AT COMPILE TIME AND RUNTIME.....	168
8.11.1 Run time selection: Specifying an individual Test in the CTesterExecuteList file1.1.1	169
8.11.2 Test Code Handling of the test strings in CTesterExecuteList file.....	169
8.11.3 Adding a Test to the CTesterExecuteList file.....	170
8.11.3.1 Compile time selection: Specifying Tests (Preferred Method).....	170
8.11.3.1.1 CTesterExecuteList.h Example.....	170
8.11.3.1.2 CTesterExecuteList.cfg Example.....	171
8.11.3.2 Configuration Test Strings in the CTesterExecuteList.....	171
8.11.3.2.1 Changing the above test to only run full speed ISO.....	171
8.11.3.2.2 Change the above test to only run full speed ISO with a max packet of 8.....	172
8.11.3.2.3 Change the above test to run the ISO and the Bulk full speed with both set to max packet of size 8	172
8.11.3.2.4 Adding a Match Limit of 10 to the number of Bulk tests that run during the simulation.....	172
8.11.3.3 Using grep, the following Host Execute Test Strings can be found.....	172

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 10-191

8.11.3.4 Using grep, the following Shared Host And Device Execute Test Strings can be found in the /APILevelTestCode Directory.....	173
8.11.3.5 Using grep the following Device Execute Test Strings can be found in /APILevelTestCode Directory.....	173
8.11.3.6 OTG Execute Test Strings.....	173
8.11.3.7 Running All Tests And Controlling The Number Of Tests Executed (This method of controlling the tests is NOT preferred , see section on specifying tests in the configuration file.).....	174
8.11.4 Increasing and Decreasing the Messaging Levels.....	174
8.11.5 Co-Simulation BFM Logging Results.....	174
8.12 INTERRUPT PROPAGATION INTO THE SOFTWARE OBJECTS.....	174
8.12.1 Example of a host interrupt calling:.....	174
8.13 SOFTWARE DETERMINATION OF HARDWARE PARAMETERS.....	175
8.13.1 How does the test software know which connection speeds are supported during simulation?	175
8.13.1.1 How does the software determine the operational mode of the core.	175
8.13.1.2 Example Setting the Mode Bit to Host for a Host Capable Core In the SoftwareMessage.txt File.....	175
8.13.2 References to Lab Code In The Integration Tests.....	175
8.14 ADDING TESTS AND FUNCTIONALITY TO THE EXISTING TEST SOFTWARE.....	176
8.14.1 Adding a New Register Definition to the OTG Register Interface.....	176
8.14.1.1.1 Step 1 - Assign an enumerated value to the register in CUSBOTGRegisterInterface.h.....	176
8.14.1.1.2 Step 2 - Create a description of the register in the register description structure in CUSBOTGRegisterInterface.cpp.....	176
8.14.1.1.3 Step 3 - Add a function to set specific values.....	176
8.14.2 Creating A New Test for the USB Host Functionality	177
8.14.2.1.1 Step 1 - Define a new class to contain the test code and derive it from CTester. On create pass in all the objects needed to perform the testing.....	177
8.14.2.1.2 Step 2 - Define the Create() function.....	177
8.14.2.1.3 Step 3 - Define the Start() function.....	177
8.14.2.1.4 Step 4 - Insert the class as a member of CUSBHostControllerTester in the CUSBHostControllerTester.h file.....	178
8.14.2.1.5 Step 5 - Call the create function in the CUSBHostControllerTester::Create function and pass in the needed objects for the new tester.....	178
8.14.2.1.6 Step 6 - Insert the new tester into the CUSBHostControllerTester: code using the host tester class.....	178
8.15 REDUCING TEST INITIALIZATION OVERHEAD IN NON-BFM TEST ENVIRONMENTS.....	178
8.15.1 Hardware additions for simulation acceleration.....	179
8.15.2 Steps to force the Peripheral controller into the connected state.....	179
9 SYNTHESIS.....	180
9.1 USB-HS HDL FILE HIERARCHY.....	180
9.2 CONFIGURATION.....	182
9.3 SYNTHESIS SCRIPT.....	182
10 REFERENCES.....	184
11 GLOSSARY OF TERMS AND ABBREVIATIONS.....	185

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 11-191

Illustration Index

Figure 1 Example USB 2.0 System Configuration.....	15
Figure 2 Example USB 2.0 On-The-Go Configurations.....	16
Figure 3 USB-HS Block Diagram	17
Figure 4 End Point Queue Head Organization.....	18
Figure 5 USB-HS Block Diagram	19
Figure 6 Microprocessor Interface Block Diagram.....	20
Figure 7 DMA Engine Block Diagram.....	21
Figure 8 Protocol Engine Block Diagram.....	22
Figure 9 Port Controller Block Diagram.....	23
Figure 10 ID - Identification Register.....	31
Figure 11 HWGENERAL – General Hardware Parameters.....	32
Figure 12 HWHOST – Host Hardware Parameters.....	32
Figure 13 HWDEVICE – Device Hardware Parameters.....	32
Figure 14 HWTXBUF – TX Buffer Hardware Parameters.....	33
Figure 15 HWRXBUF – RX Buffer Hardware Parameters.....	33
Figure 16 CAPLENGTH – Capability Register Length.....	34
Figure 17 HCIVERSION – Host Interface version number.....	34
Figure 18 HCSPARAMS – Host Control Structural Parameters.....	34
Figure 19 HCCPARAMS – Host Control Capability Parameters.....	36
Figure 20 DCIVERSION – Device Interface Version Number.....	37
Figure 21 DCCPARAMS - Device Control Capability Parameters.....	37
Figure 22 GPTIMER0LD – General Purpose Timer #0 Load Register.....	37
Figure 23 GPTIMER0CTRL – General Purpose Timer #0 Control Register.....	38
Figure 24 USBCMD – USB Command Register.....	39
Figure 25 USBSTS – USB Status.....	42
Figure 26 USBINTR – USB Interrupt Enable.....	45
Figure 27 FRINDEX – USB Frame Index.....	47
Figure 28 PERIODICLISTBASE - Host Controller Frame List Base Address.....	48
Figure 29 DEVICEADDR - Device Controller USB Device Address.....	49
Figure 30 ASYNCLISTADDR - Host Controller Next Asynch. Address.....	50
Figure 31 ENDPOINTLISTADDR - Device Controller Endpoint List Address.....	50
Figure 32 BURSTSIZE - Host Controller Embedded TT Async. Buffer Status.....	51
Figure 33 PORTSCx - Port Status Control[1:8].....	54
Figure 34 USBMODE -USB Device Mode.....	60
Figure 35 ENDPTSETUPSTAT - Endpoint Setup Status.....	62
Figure 36 ENDPTPRIME - Endpoint Initialization.....	63
Figure 37 ENDPTFLUSH - Endpoint De-Initialize.....	63
Figure 38 ENDPTSTAT - Endpoint Status.....	64

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 12-191

Figure 39 ENDPTCOMPLETE - Endpoint Complete	65
Figure 40 ENDPTCTRL0 – Endpoint Control 0	66
Figure 41 ENDPTCTRL1 – ENDPTCTRL15 – Endpoint Control 1 to 15	67
Figure 42 End Point Queue Head Organization	71
Figure 43 Endpoint Queue Head (dQH)	72
Figure 44 Endpoint Transfer Descriptor (dTD)	74
Figure 45 Device State Diagram	78
Figure 46 End Point Queue Head Diagram	89
Figure 47 Software Link Pointers	90
Figure 48 High Speed Receive Buffer Wiring	101
Figure 49 High Speed Receive Buffer Timing	102
Figure 50 High Speed Transmit Buffer Wiring	103
Figure 51 Transceiver clocked design	108
Figure 52 System clocked design	109
Figure 53 Mixed Clocked Design	110
Figure 54 Triple Clocked Design	111
Figure 55 Reset Strategy	112
Figure 56 Latency Buffer Architecture	113
Figure 57 Device Power State Diagram	116
Figure 58 Host/OTG Power State Diagram	117
Figure 59 Suspend Wake-Up Handshake	119
Figure 60 Software commanded Suspend Wake-Up Handshake	119
Figure 61 Suspend Control Connections	120
Figure 62 Device Asynchronous Suspend Control Example	121
Figure 63 ULPI Power control interface behavior when entering low power mode	122
Figure 64 ULPI power control Interface behavior when exiting low power mode	122
Figure 65 Test Bench Architecture Block Diagram	125
Figure 66 USB PHY Simulation Test Bench Block Diagram	136
Figure 67 Resolution Function Block Diagram	137
Figure 68 USB Bus State 5 Bit Encoding	139
Figure 69 Resolution Table For PULL_STATE ZZZZ	140
Figure 70 Resolution Table For PULL_STATE PULL00	140
Figure 71 Resolution Table For PULL_STATE PULL01	141
Figure 72 Resolution Table For PULL_STATE PULL10	141
Figure 73 Device HDL File Hierarchy	181

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 13-191

Index of Tables

Table 1 Interface Register Sets.....	25
Table 2 Device/Host Capability Registers.....	25
Table 3 Endpoint Capabilities/Characteristics.....	72
Table 4 Next dTD Pointer.....	73
Table 5 Multiple Mode Control (HCCPARAMS).....	73
Table 6 Next dTD Pointer.....	74
Table 7 dTD Token.....	74
Table 8 dTD Buffer Page Pointer List.....	75
Table 9 Device Controller State Information Bits.....	78
Table 10 Device Controller Endpoint Initialization.....	81
Table 11 Device Controller Stall Response Matrix.....	81
Table 12 Variable Length Transfer Protocol Example (ZLT = 0).....	83
Table 13 Variable Length Transfer Protocol Example (ZLT = 1).....	83
Table 14 Interrupt/Bulk Endpoint Bus Response Matrix.....	84
Table 15 Control Endpoint Bus Response Matrix.....	86
Table 16 Isochronous Endpoint Bus Response Matrix.....	88
Table 17 Device Error Matrix.....	92
Table 18 High Frequency Interrupt Events.....	93
Table 19 Low Frequency Interrupt Events.....	93
Table 20 Error Interrupt Events.....	93
Table 21 Configuration Controls.....	104
Table 22 Negative edge triggered Flip Flops.....	111
Table 23 Latency tolerance as a function of buffer and bus burst size.....	114
Table 24 Transceiver Selection Control Matrix.....	123
Table 25 Transceiver Serial Selection Control Matrix.....	123
Table 26 Transceiver Selection Control Reset Values.....	124
Table 27 Transceiver Serial Selection Control Reset Values.....	124

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 14-191

1 Introduction

The USB-HS core is a standards based “Serial Interface Engine” for implementing a USB interface in compliance with the USB 2.0 specification [3]. The USB-HS is a family of products constructed around a common code base. Depending on the options selected the USB-HS core is able to act as peripheral controller, or host controller or a dual role OTG controller that is able to negotiate the host or peripheral role on the bus in compliance with the On-The-Go (OTG) supplement to the USB specification [4].

The core interfaces to two different standard USB 2.0 transceivers types. Transceiver interfaces are provided for direct connections transceivers that conform to the UTMI (USB Transceiver Macrocell Interface) specification [1] and the ULPI (UTMI+ Low-pincount Interface) specification [9].

One of two standards based 32-bit synchronous system bus interfaces may be selected for the USB-HS core. The BVCI interface complies with the Basic Virtual Component Interface (BVCI) standard from VSI alliance [2], and the AMBA™ interface complies with ARM Limited Advanced Microcontroller Bus Architecture (AMBA™) Advanced High-performance Bus (AHB) for smooth integration into a System-on-Chip (SoC).

The register and data structure interfaces to the core are based on the Enhanced Host Controller Interface (EHCI) specification from Intel™ Corporation [5]. By leveraging industry standards throughout the USB-HS controller family, the compatibility and quality of the final product are insured.

The USB-HS core is architected to make efficient use of the system resources in an SOC design. The 32-bit system bus interface contains a chaining DMA (Direct Memory Access) engine that reduces the interrupt load on the application processor, and reduces the total system bus bandwidth that must be dedicated to servicing the USB interface requirements. By transferring the data to system memory at wire rates, the buffer memory requirement within the core is minimized. The USB-HS also makes strategic use of the processor for tasks that do not require timing critical responses to reduce the amount of special purpose logic.

The USB-HS product family includes a device controller core, a single port host controller core, a multi-port host controller core and a USB On-The-Go capable dual role host / device controller core. The HDL code required to implement the device controller core is a subset of the HDL code for the OTG core. The OTG controller is able to operate as a device controller, a host controller or a dual role controller under software control. The HDL code is structured in such a way that if only the device controller is required that the extra logic to implement the OTG and host controller function is removed during synthesis to reduce the size of the resulting design. Like wise the overhead associated with the device controller function is removed if the product is synthesized as a USB host controller. Separate product installers are delivered for each of the four products in the USB-HS product family. Only the HDL code required to synthesize the products that are actually licensed are delivered in the product installers.

1.1 Document Overview

This document contains user documentation for the USB-HS core. The USB-HS supports both device hardware and software interfaces.

It is assumed that the reader has an understanding of USB technology. For additional information, please consult the USB 2.0 specification [3], the Enhanced Host Controller Interface specification for universal serial bus [5], the Virtual Component Interface for information on the BVCI bus [2], the AMBA™ Specification for information on AMBA™-AHB [8].

1.1.1 Style Conventions

Normal text is displayed using this font.

Any code segments are displayed in this mono-space font.

This product contains both AMBA-AHB and BVCI system interfaces and will make references to files and scripts with the notation [bvci|amba]. This means BVCI or AMBA as demonstrated in this example:

usb_hs_core_dev_[bvci|amba].v refers to:

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 15-191

vusb_hs_core_dev_bvci.v for customers using the BVCI version OR
vusb_hs_core_dev_amba.v for customers using the AMBA-AHB version.

Notes, tips and cautions use the following style.

NOTE Notes, tips and cautions use this style.

1.2 USB

The Universal Serial Bus (USB) is a cable bus that supports data exchange between a host computer and a wide range of simultaneously accessible peripherals. The attached peripherals share USB bandwidth through a host-scheduled, token-based protocol. The bus allows peripherals to be attached, configured, used, and detached while the host and other peripherals are in operation.

USB software provides a uniform view of the IP system for all application software, hiding implementation details making application software more portable. It manages the dynamic attach and detach of peripherals.

There is only one host in any USB system. The USB interface to the host computer system is referred to as the Host Controller. The Host Controller may be implemented in a combination of hardware, firmware, or software.

There may be multiple USB devices in any system such as hubs, joysticks, speakers, printers, etc. USB devices present a standard USB interface in terms of comprehension, response, and standard capability.

The host initiates transactions to specific peripherals, while the device responds to control transactions. The device sends and receives data to and from the host using a standard USB data format. The early versions of the USB specification (USB 1.0 and 1.1) operated at 12Mb/s or 1.5 MB/s.

1.3 USB 2.0

The USB 2.0 specification supersedes the earlier versions of the USB specification. USB 2.0 offers the user a larger bandwidth increasing data throughput by a factor of 40. All the peripherals used with the previous versions of USB work perfectly with USB 2.0 while also offering a larger choice of higher performance peripherals, such as video-conferencing cameras, next-generation scanners and printers, a fast storage device, etc.

In addition to the 1.5Mbit/sec and 12Mbit/sec data rates of USB 1.1, the evolution from USB 1.1 to USB 2.0 adds an additional data rate of 480Mbit/sec. Existing USB 1.1 connectors and full-speed cables support the higher speeds of the USB 2.0 without any changes. USB 2.0 specifies a microframe, is 1/8th of a 1msec frame allowing small buffers even at the high data rate.

USB2.0 system configurations look identical to the end user to configurations based on earlier revisions of the USB specification. However, the end user will need to distinguish between USB 2.0 hubs and USB 1.1 hubs to optimize the placement of USB 2.0 high speed devices. A USB 2.0 hub accepts high-speed transactions at a faster rate but also matches the data rate of the peripherals, while the USB 1.1 hub will only support the lower 12Mb/sec and 1.5Mb/sec data rates.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 16-191

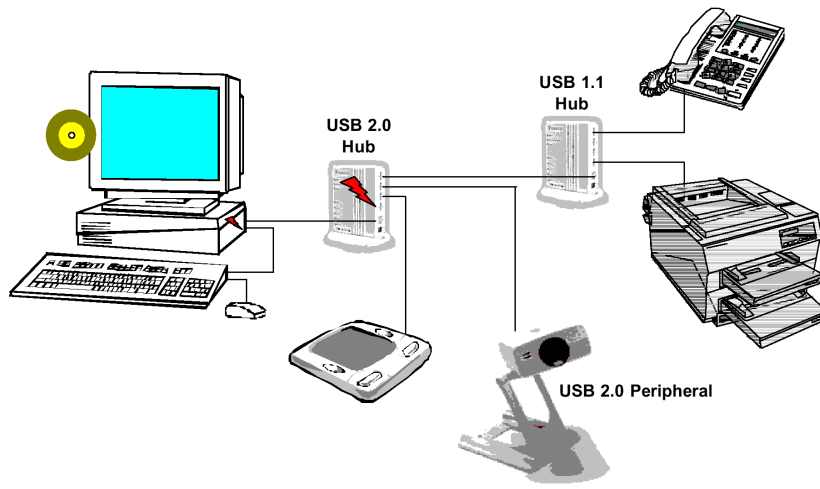


Figure 1 Example USB 2.0 System Configuration

The roles of the components of the USB 2.0 system have minor changes from the roles in a USB 1.1 system. However, current peripheral products work with the USB 2.0 system without any changes. Additional performance is not required for many of the human interface devices such as mice, keyboards, and game pads. Both USB 1.1 and USB 2.0 devices will operate in a USB 2.0 system while opening up the possibilities of exciting new higher bandwidth peripherals.

For additional information, refer to USB 2.0 specification [1].

1.4 USB On-The-Go

USB is a popular standard for connecting peripherals and portable consumer electronic devices such as digital cameras and hand-held computers to host PCs. The On-The-Go (OTG) Supplement to the USB Specification extends USB to peer-to-peer application. Using USB OTG technology consumer electronics, peripherals and portable devices can connect to each other (for example, a digital camera can connect directly to a printer, or a keyboard can connect to a Personal Digital Assistant) to exchange data.

With USB On-The-Go you can develop a fully USB compliant peripheral device that can also assume the role of a USB host. The OTG state machines determine the role of the device based on connector signals, and then initializes the device in the appropriate mode of operation (host or peripheral) based on how it is connected. After connecting the devices can negotiate using the OTG protocols to assume the role of host or peripheral based on the task to be accomplished.

For additional information, refer to the *On-The-Go Supplement to the USB 2.0 Specification* [4].

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 17-191

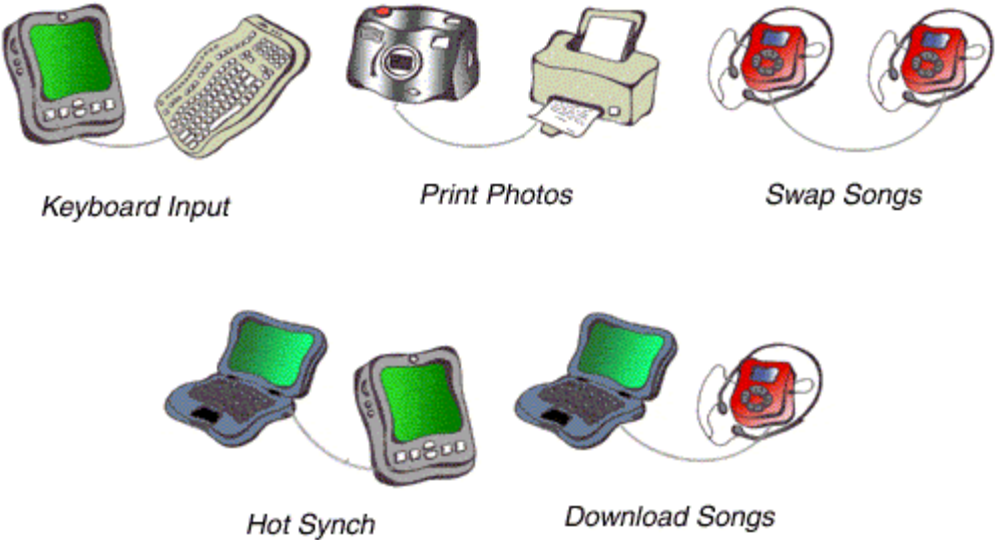


Figure 2 Example USB 2.0 On-The-Go Configurations

1.5 USB-HS Block Diagram

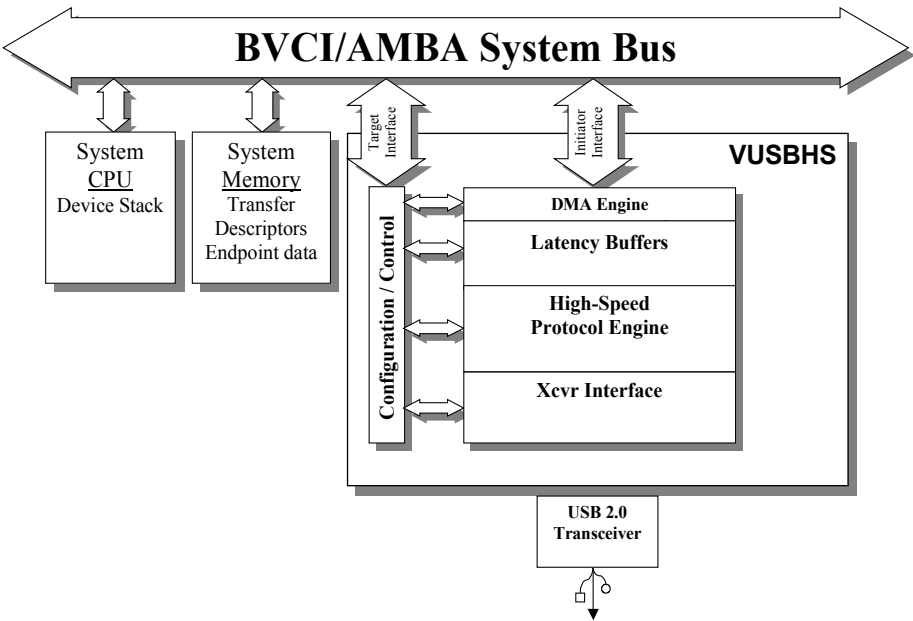


Figure 3 USB-HS Block Diagram

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 18-191

1.6 USB-HS Core Features

- **USB 2.0 High Speed Device**
- **BVCI compliant system bus interface OR**
- **AMBA-AHB compliant system bus interface.**
- **Direct support for USB Transceiver Macrocell Interface (UTMI) and UTMI+ Low Pin Interface (ULPI).**
- **Integrated USB Full Speed transceiver.** The integrated Full speed DPLL and NRZI CODECs allow a single piece of silicon to perform as a Full speed device with an internal transceiver, or as a high speed device with an external transceiver.
- **Configurable dual port RAM buffers isolate memory latency on the system bus from the timing requirements of the USB.**
- **Fully static and synchronous, synthesizable RTL design available in VHDL or Verilog formats.** The synchronous static design simplifies synthesis and test insertion within the tool flow, and provides a framework for low power design.
- **Integrated simulation and verification test bench.** A verification environment is provided to simulate host and device operations in a standalone or processor integrated environment.
- **FPGA prototype available.** A Field Programmable Gate Array (FGPA) implementation of the USB-HS core integrated with a 32-bit RISC microprocessor is available to reduce the prototype design time for technology evaluation and software development.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 20-191

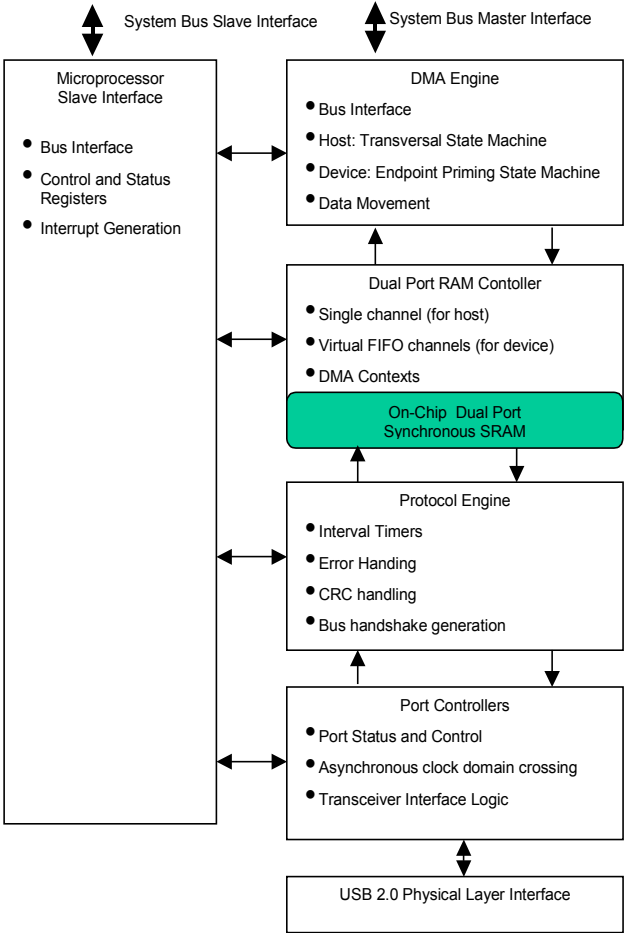


Figure 5 USB-HS Block Diagram

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 21-191

2.2.2 Microprocessor Interface

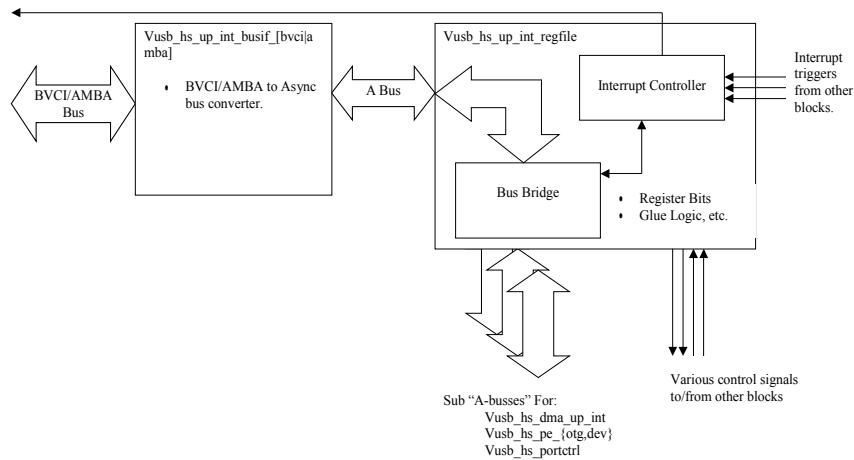


Figure 6 Microprocessor Interface Block Diagram

The USB-HS Microprocessor interface block (**vusb_hs_up_int_[bvci|amba]**) provides a target (slave) interface on the system processor bus. A simple synchronous bus interface signaling is used for the microprocessor bus allowing the core to be easily connected to a number of microprocessors.

The Microprocessor interface block contains all the control and status registers that allow a processor to interface to the USB-HS core. These registers allow a microprocessor to control the configuration of the core, ascertain the capabilities of the core and control the core in operation for both host and device modes.

Two groups of registers exist in the interface. The USB host controller registers are compatible with the USB host controller registers defined in the Intel™ EHCI specification. An additional set of registers and bit usages for the existing registers allows control of a target device or function controller. In applications that do not make use of bit one in host or device mode, the unused registers may be configured out of the design to reduce the gate count, size and power of the block.

2.2.3 DMA Engine

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 22-191

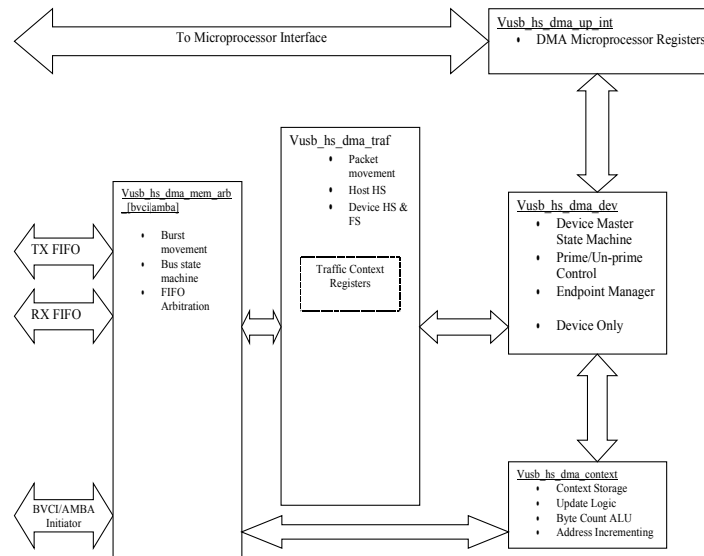


Figure 7 DMA Engine Block Diagram

The **DMA Engine Block** (**vusb_hs_dma _ dev _[bvc|amba]**) presents a bus initiator (master) interface to the system memory bus. It is responsible for moving all of the data to be transferred over the USB between the USB-HS core and buffers in system memory. Like the microprocessor interface block the DMA block uses a simple synchronous bus signaling protocol that eases connection to a number of different standard buses.

The DMA controller must access both control information and packet data from system memory. The control information is contained in link list based queue structures. The DMA controller has state machines that are able to parse all of the data structures defined in this controller specification. In device mode, the data structures designed to be similar to those in the EHCI specification are used to allow device responses to be queued for each of the active pipes in the device.

The DMA Engine Block has several sub-blocks:

- **vusb_hs_dma_up_int**: Contains DMA specific microprocessor interface registers.
- **vusb_hs_dma_traf** : Move packet data through the memory arbiter in bursts of configurable size.
- **vusb_hs_dma_mem_arb_[bvc|amba]**: Data path arbitration & BVCI|AMBA bus master state machine.
- **vusb_hs_dma_context** : Context storage for data structures.
- **vusb_hs_dma_dev_sm** : Master DMA controller state machine .

2.2.4 Dual Port RAM Controller

The **Dual Port RAM Controller** (**dpsram.v(hd1)**, **vusb_hs_rx_buf.v(hd1)**, **vusb_hs_tx_buf.v(hd1)**) is used for context information and to build configurable FIFOs between the Protocol Engine block and the DMA controller. These FIFOs decouple the system processor memory bus request from the

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 23-191

extremely tight timing required by the USB itself. The use of the FIFO buffers differs between host and device mode operation. In host mode, a single data channel is maintained in each direction through the dual port memory. In device mode, multiple FIFO channels are maintained for each of the active endpoints in the system.

The size of the dual port RAM is configurable and is dependent on several factors. A simple set of constants will allow the size of the dual port RAM to be set prior to synthesis.

The sizing of the dual port RAM must take into account the intended usage of the core: host, device or both, the number of device endpoints supported, and the worst case latency to acquire the bus and fetch a block of data.

2.2.5 Protocol Engine

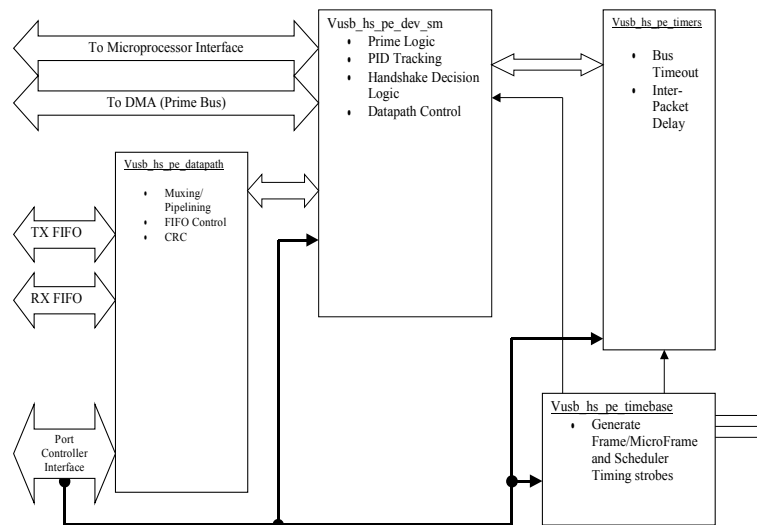


Figure 8 Protocol Engine Block Diagram

The **Protocol Engine** (`vusb_hs_pe_dev`) parses all the USB tokens and generates the response packets. It is responsible for all error checking, check field generation, formatting all the handshake, ping and data response packets on the bus, and for any signals that must be generated based on a USB based timeframe. In host mode, the Protocol engine also generates all of the token packets required by the USB protocol. The Protocol engine contains several sub-functions:

- The token state machines track all of the tokens on the bus and filter the traffic based on the address and endpoint information in the token. In host mode, these state machines also generate the tokens required for data transfer and bus control.
- The **CRC5** and **CRC16** CRC generator/checker circuits check and generate the CRC check fields for the token and data packets.
- The data and handshake state machines generate any responses required on the USB and move the packet data through the dual port memory FIFOs to the DMA controller block.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 24-191

- The Interval timers provide timing strobes that identify important bus timing events: the bus timeout interval, the microframe interval, the start of frame interval, and the bus reset, resume, and suspend intervals.
- Reports all transfer status to the DMA engine.

2.2.6 Port Controller

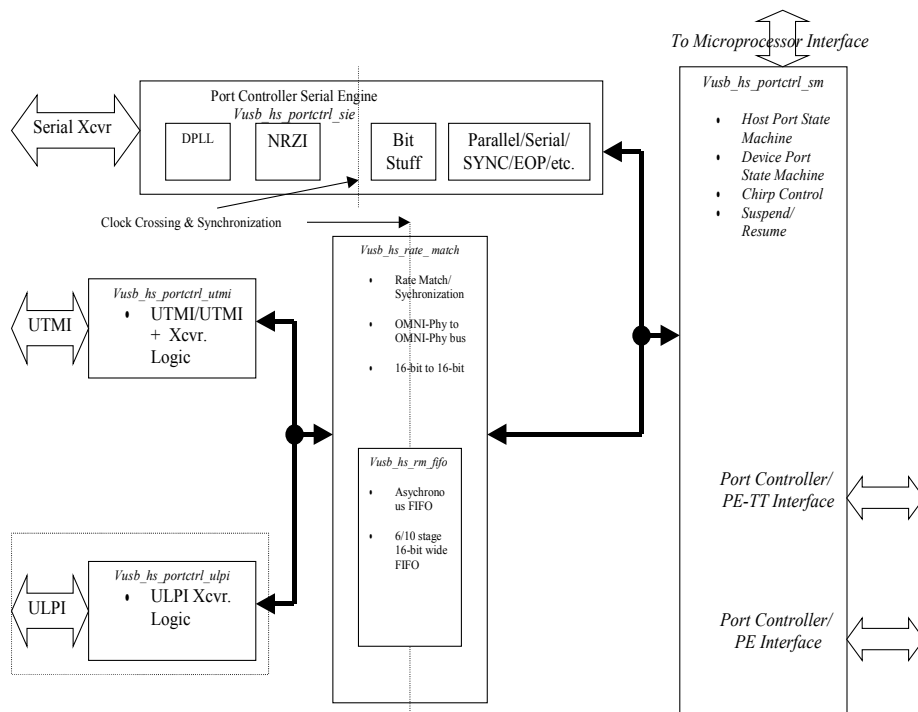


Figure 9 Port Controller Block Diagram

The **Port Controller** block (**vusb_hs_portctrl**) interfaces to UTMI, UTMI+ or ULPI compatible transceiver macrocell core or component. In addition, Full Speed Low Speed SIE logic is provided that allows support for direct connection to USB Full Speed I/O cells. The ULPI block shown above is an interface block to support ULPI (UTMI+ Low Pin count Interface) external transceivers.

The primary function of the **Port Controller** block is to isolate the rest of the USB-HS core from the transceiver, and to move all of the transceiver signaling into the primary clock domain of the USB-HS core. This allows the core to run synchronously with the system processor and it's associated resources.

The port controller block has five sub-blocks:

- **vusb_hs_portctrl_sm** : Connection state machines and port status and control
- **vusb_hs_portctrl_utmi** : UTMI transceiver specific logic
- **vusb_hs_portctrl_ulpi** : ULPI transceiver specific logic
- **vusb_hs_portctrl_rm & vusb_hs_portctrl_rm_fifo** : Asynchronous clock crossing logic
- **vusb_hs_portctrl_sie** : Full and Low Speed DPLL, NRZI and SERDES logic.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 25-191

2.2.7 System Bus Interface

The USB-HS core interfaces to the SOC memory and processing resources using one of two standard bus interfaces. Either a BVCI or a AMBA interface can be selected. Both of these bus interface standards provide the features required for high performance/high clock frequency systems including:

- Synchronous Single clock edge operation
- Non-tristate on chip implementation
- Initiator (master) and Target (slave) interfaces
- Burst transfers

See the Bus Configuration section of 7System Level Issues and Core Configuration for more information on the choice of system bus interface type.

2.2.7.1 BVCI Overview

The USB-HS core can interface to the system processor and memory system using industry standard Basic Virtual Component Interface (BVCI) interconnect signaling [2]. The BVCI interface is the result of the work of the On-Chip Bus Development Working Group of the Virtual Socket Interface Alliance (VSIA). The objective of the VCI standards is to provide a general interface, such that Intellectual Property (IP), in the shape of Virtual Components (VCs) of any origin, can be connected to create Systems-on-Chips (SoC). BVCI is an open standard specifically design as an on-chip bus and is inherently technology independent. There are three complexity levels for the VCI: Peripheral (PVC), Basic VCI (BVCI), and Advanced VCI (AVCI). These interfaces are supersets, and subsets of one another and are designed to be compatible with each other. BVCI defines an interface that is suitable for most applications. It has a powerful, but not overly complex protocol.

The BVCI interface is efficient and easy to design. It provides a synchronous signaling interface for reads, writes, and burst cycles. Transactions between an initiator (master) interface and a responder (slave) interface are defined. BVCI is an interface rather than a bus defining request-response protocol where the requests may be separate (split) from the responses. Straightforward signaling allows interfaces to be designed that can have a variable number of wait states, and a variable pipeline depth with one or more outstanding transfers between an initiator and a target in process. Because of these characteristics BVCI is well suited for RISC based processors and DMA style peripherals. The split request response architecture and variable depth pipelining of request can be used to construct efficient interfaces to medium latency targets such as SDRAMs and pipelined SRAMs.

The BVCI specification can be obtained from the Virtual Socket Interface Alliance [2].

2.2.7.2 AMBA-AHB Overview

The other option for interfacing the USB HS core with the system is via the AMBA bus defined by ARM Limited. AMBA is an open standard, on-chip bus specification to provide a common method for the interconnection and management of functional blocks that comprise a System-on-Chip (SoC). Like BVCI, AMBA's goal is to provide a framework for seamless interoperability between ARM processors based systems and third party peripheral Intellectual Property. The industry acceptance of the ARM processor architecture as a basis for embedded system design had lead this AMBA bus architecture to become a de facto standard in the industry.

The AMBA bus defines a number of flavors of the bus to enable designers to select the bus structure that best meets their system needs. The AMBA-APB is a single master bus targeting peripheral subsystems, the AMBA-AHB defines a High Performance multimaster bus capability, AMBA-lite defines a simplified subset of the AHB specification for lower system implementation costs, and the AMBA multi-layer AHB defines approaches for hierarchically intermixing the above busses for advanced topologies. The USB HS cores are fully compatible with the AHB and AHB-lite system implementations, and present a slave interfaces and a master interface on the AHB system bus.

The AMBA-AHB specification can be obtained from ARM's website at http://www.arm.com/products/solutions/AMBA_Spec.html [8].

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 26-191

3 Register Interface

Slave accesses from the controlling processor enables access to the configuration, control, and status registers. One function of the system address map is the registers base address, which must begin on a **DWord** (32-bit) boundary. Register offset definitions are listed in the table below.

Configuration, control and status registers are divided into three categories, identification, capability and operational registers.

- Identification registers are used to declare the slave interface presence along with the complete set of the hardware configuration parameters.
- Static, read only capability registers define the software limits, restrictions, and capabilities of the host/device controller.
- Operational registers are comprised of dynamic control or status registers that may be read only, read/write, or read/write to clear. The following sections define the use of these registers.

EHCI registers are listed alongside device registers to show the complementary nature of host and device control.

NOTE Host mode EHCI compatibility begins at offset 0x100. If it is necessary to begin the EHCI register set at offset 0x000, the identification registers are disabled from the address map by connecting the upper most address bit of the slave interface to a logic level '1' and adjusting the offsets below accordingly.

Table 1 Interface Register Sets

Offset	Register Set	Explanation
000h to 0fch	Identification Registers	Identification registers are used to declare the slave interface presence and include a table of the hardware configuration parameters.
100h to 124h	Capability Registers	Capability registers specify the limits, restrictions, and capabilities of a host/device controller implementation. These values are used as parameters to the host/device controller driver.
140h to 1FCh	Operational Registers	Operational registers are used by the system software to control and monitor the operational state of the host/device controller.

3.1 Configuration, Control and Status register set

Table 2 Device/Host Capability Registers

Offset	Size (bytes)	Mnemonic	Register Name	DEV	OTG	SPH	MPH
000h	4	ID	Identification Register	√	√	√	√
004h	4	HWGENERAL	General Hardware Parameters	√	√	√	√
008h	4	HWHOST	Host Hardware Parameters		√	√	√
00Ch	4	HWDEVICE	Device Hardware Parameters	√	√		

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 27-191

Offset	Size (bytes)	Mnemonic	Register Name	DEV	OTG	SPH	MPH
010h	4	HWTXBUF	TX Buffer Hardware Parameters	√	√	√	√
014h	4	HWRXBUF	RX Buffer Hardware Parameters	√	√	√	√
018h	4	HWTXXBUF	TT-TX Buffer Hardware Parameters				√
01Ch	4	HWTTRXBUF	TT-RX Buffer Hardware Parameters				√
020h-07Ch	96	Reserved	N/A				
080h	4	GPTIMER0LD	General Purpose Timer #0 Load Register	√	√	√	√
084h	4	GPTIMER0CTRL	General Purpose Timer #0 Control Register	√	√	√	√
088h	4	GPTIMER1LD	General Purpose Timer #1 Load Register	√	√	√	√
08Ch	4	GPTIMER1CTRL	General Purpose Timer #1 Control Register	√	√	√	√
090h-0FCh	112	Reserved	N/A				
100h	1	CAPLENGTH	Capability Register Length	√	√	√	√
101h	1	Reserved	N/A				
102h	2	HCIVERSION	Host Interface Version Number		√	√	√
104h	4	HCSPARAMS	Host Ctrl. Structural Parameters		√	√	√
108h	4	HCCPARAMS	Host Ctrl. Capability Parameters		√	√	√
10Ch – 11Fh	20	Reserved	N/A				
120h	2	DCIVERSION	Dev. Interface Version Number	√	√		
122h	2	Reserved	N/A	√			

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 28-191

Offset	Size (bytes)	Mnemonic	Register Name	DEV	OTG	SPH	MPH
124h	4	DCCPARAMS	Device Ctrl. Capability Parameters	√	√		
128h – 13Ch	24	Reserved	N/A				
140h	4	USBCMD	USB Command	√	√	√	√
144h	4	USBSTS	USB Status	√	√	√	√
148h	4	USBINTR	USB Interrupt Enable	√	√	√	√
14Ch	4	FRINDEX	USB Frame Index	√	√	√	√
150h	4	Reserved	4G Segment Selector				
154h	4	PERIODICLISTBASE	Frame List Base Address		√	√	√
		Device Addr	USB Device Address	√	√		
158h	4	ASYNCLISTADDR	Next Asynchronous List Address		√	√	√
		Endpointlist Addr	Address at Endpoint list in memory	√	√		
15Ch	4	TTCTRL	TT status and control.		√	√	√
160h	4	BURSTSIZE	Programmable Burst Size	√	√	√	√
164h	4	TXFILLTUNING	Host Transmit Pre-Buffer Packet Tuning		√	√	√
168h	4	TXTTFILLTUNING	Host TT Transmit Pre-Buffer Packet Tuning				√
16Ch	4	N/A	Reserved				
170h	4	ULPI Viewport	ULPI Viewport	√	√	√	√
174h	4	N/A	Reserved				
178h	4	ENDPTNAK	Endpoint NAK	√	√		
17Ch	4	ENDPTNAKEN	Endpoint NAK Enable	√	√		
180h	4	CONFIGFLAG	Configured Flag Register		√	√	√

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 29-191

Offset	Size (bytes)	Mnemonic	Register Name	DEV	OTG	SPH	MPH
184h	4	PORTSC1	Port Status/Control 1	√	√	√	√
188h	4	PORTSC2	Port Status/Control 2				√
...	4	PORTSCx	Port Status/Control x				√
1A0h	4	PORTSC8	Port Status/Control 8				√
1A4h	4	OTGSC	On-The-Go (OTG) Status and Control		√		
1A8h	4	USBMODE	USB Device Mode	√	√	√	√
1ACh	4	ENPDTSETUPSTAT	Endpoint Setup Status	√	√		
1B0h	4	ENDPTPRIME	Endpoint Initialization	√	√		
1B4h	4	ENDPTFLUSH	Endpoint De-Initialize	√	√		
1B8h	4	ENDPTSTATUS	Endpoint Status	√	√		
1BCh	4	ENDPTCOMPLETE	Endpoint Complete	√	√		
1C0h	4	ENDPTCTRL0	Endpoint Control 0	√	√		
1C4h	4	ENDPTCTRL1	Endpoint Control 1	√	√		
...	4	ENDPTCTRLx	Endpoint Control x	√	√		
1FCh	4	ENDPTCTRL15	Endpoint Control 15	√	√		

NOTE *Italic Text* indicates a deviation from EHCI for Device

3.2 Summary of register layouts

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
reserved								REVISION								1	1	NID								ID								000h ID					
reserved																				S	M	PHYM				PHYW			B	C	L	W	K	C	RT	004h HWGENERAL			
TTPER								TTASY								reserved												NPORT				HC	008h HWHOST						
reserved																								DEVEP				DC				00Ch HWDEVICE							
TXLCR	reserved								TXCHANADD								TXADD								TCBRUST								010h HWTXBUF						

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 30-191

reserved																RXADD																RXBURST																014h HWRXBUF																																																													
reserved																reserved																reserved																020h reserved																																																													
reserved																reserved																... reserved																																																																													
reserved																reserved																0FCh reserved																																																																													
																																CAPLENGTH																100h CAPLENGTH																																																													
																																reserved																101h reserved																																																													
																																HCVERSION																102h HCVERSION																																																													
reserved				N_TT				N_PTT				reserved				R				N_CC				N_PCC				reserved				PPC				N_PORTS				104h HCSPARAMS																																																																					
reserved																EECP[7:0]																IST[7:4]				R				ASP				PFL				ADC				108h HCCPARAMS																																																									
reserved																reserved																reserved																10Ch reserved																																																													
reserved																reserved																... reserved																																																																													
reserved																reserved																11Fh reserved																																																																													
																																DCVERSION																120h DCVERSION																																																													
																																reserved																122h reserved																																																													
reserved																reserved																HC				DC				R				R				DEN[4:0]				124h DCCPARAMS																																																									
reserved																reserved																reserved																128h reserved																																																													
reserved																reserved																... reserved																																																																													
reserved																reserved																13Ch reserved																																																																													
reserved								ITC[7:0]								FS2				ATDTW				SUTW				R				ASPE				R				ASP1				ASP0				LR				IAA				ASE				PSE				FS1				FS0				RST				RS				140h USBCMD																													
reserved						TI0		TI1		reserved				UPI				UAI				R				NAKI				AS				PS				RCL				HCH				R				ULPII				R				SLI				SRI				URI				AAI				SEI				FRI				PCI				UEI				UI				144h USBSTS															
reserved						TIE0		TIE1		reserved				UPIE				UAIE				R				NAKE				reserved								ULPIE				R				SLE				SRE				URE				AAE				SEE				FRE				PCE				UEE				UE				148h USBINTR																											
reserved														FRINDEX[13:0]														14Ch FRINDEX																																																																																	
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																																																														
reserved																																150h reserved																																																																													
PERBASE[31:12]																reserved																154h PERIODICLISTBASE																																																																													
USBADR[31:25]								USBADRA								reserved																Device Addr																																																																													
ASYBASE[31:5]																reserved																158h ASYNCLISTADDR																																																																													
EPBASE[31:11]																reserved																Endpoint list Addr																																																																													

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 31-191

31	TTHA		reserved															TTAC	TTAS	15Ch TTCTRL (this register is not used in device mode) (bits TTAS and TTAC only used in multiport h													
reserved										TXPBURST					RXPBURST					160h BURSTSIZE													
reserved										TXFIFOTHRES		R	TXSCHEALTH			TXSCHOH				164h TXFILLTUNNING (not used in device)													
reserved										TXTTSCHEALTH					R	TXTTSCHOH				168h TXTTFILLTUNNING (only in multiport h													
reserved																				16Ch reserved													
ULPI ViewPort																				170h ULPI Viewport													
reserved																				174h reserved													
EPTN[15:0]										EPRN[15:0]										178h ENDPTNAK													
EPTNE[15:0]										EPRNE[15:0]										17Ch ENDPTNAKEN													
set to zero																			1	180h CONFIGFLAG													
PTS	STS	PTW	PSPD	R	PFSC	PHCD	WKOC	WKDS	WKCEN	PTC[3:0]	PIC	PO	PP	LS	HSP	PR	SUSP	FPR	OCC	OCA	PEC	PE	CSC	CCS	184h PORTSC1								
PTS	STS	PTW	PSPD	R	PFSC	PHCD	WKOC	WKDS	WKCEN	PTC[3:0]	PIC	PO	PP	LS	HSP	PR	SUSP	FPR	OCC	OCA	PEC	PE	CSC	CCS	188h PORTSC2								
PTS	STS	PTW	PSPD	R	PFSC	PHCD	WKOC	WKDS	WKCEN	PTC[3:0]	PIC	PO	PP	LS	HSP	PR	SUSP	FPR	OCC	OCA	PEC	PE	CSC	CCS	... PORTSCx								
PTS	STS	PTW	PSPD	R	PFSC	PHCD	WKOC	WKDS	WKCEN	PTC[3:0]	PIC	PO	PP	LS	HSP	PR	SUSP	FPR	OCC	OCA	PEC	PE	CSC	CCS	1A0h PORTSC8								
R	DPIE	1mSE	BSEIE	ASVIE	ASVIE	IDIE	R	DPLS	1mSS	BSEIS	BSVIS	ASVIS	AVVIS	IDIS	R	DPS	1mST	BSE	BSV	ASV	AVV	ID	HABA	HADP	IDPU	VBPS	SDIS	SLOW	ES	HAAR	VC	VD	1A4h OTGSC (only used in OTG)
reserved																			CM				1A8h USBMODE										
reserved										ENDPTSETUPSTAT[15:0]													1ACh ENDPTSETUPSTAT										
PETB[15:0]										PERB[15:0]													1B0h ENDPTPRIME										
FETB[15:0]										FERB[15:0]													1B4h ENDPTFLUSH										
ETBR[15:0]										ERBR[15:0]													1B8h ENDPTSTAT										
ETCE[15:0]										ERCE[15:0]													1BCh ENDPTCOMPLETE										
reserved										TXE	reserved					TXT	R	TXS	reserved					RXE	reserved					RXT	R	RXS	1C0h ENDPTCTRL0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		

		Title: CII3410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 32-191

reserved	TXE	TXR	TXI	R	TXT	TXD	TXS	reserved	RXE	RXR	RXI	R	RXT	RXD	RXS	1C4h ENDPTCTRL1
reserved	TXE	TXR	TXI	R	TXT	TXD	TXS	reserved	RXE	RXR	RXI	R	RXT	RXD	RXS	... ENDPTCTRLx
reserved	TXE	TXR	TXI	R	TXT	TXD	TXS	reserved	RXE	RXR	RXI	R	RXT	RXD	RXS	1FCh ENDPTCTRL15

3.3 Identification Registers

Identification registers are used to declare the slave interface presence and include a table of the hardware configuration parameters.

3.3.1 ID

Address: Base + 000h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

The Identification register (ID) provides a simple way to determine if the USB-HS USB 2.0 core is provided in the system. The ID register identifies the USB-HS USB 2.0 core and its revision.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								REVISION								1	1	NID					0	0	ID						

Figure 10 ID - Identification Register

The register fields are described in the following table.

Field	Description
ID[5:0]	Configuration number. This number is set to 0x05 and indicates that the peripheral is the USB-HS USB 2.0 core.
NID[5:0]	Ones complement version of ID[5:0].
REVISION[7:0]	Revision number of the core
Reserved	These bits are reserved and should be set to zero

3.3.2 HWGENERAL

Address: Base + 004h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

General hardware parameters as defined in [System Level Issues and Core Configuration](#).

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0																															
reserved																SM	PHYM		PHYW		BWT		CLKC		RT						

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 33-191

Figure 11 HWGENERAL – General Hardware Parameters

The register fields are described in the following table.

Field	Description
reserved	Reserved. These bits are reserved and should be set to zero.
SM	VUSB_HS_PHY_SERIAL
PHYM	VUSB_HS_PHY_TYPE
PHYW	VUSB_HS_PHY16_8
BWT	Reserved for internal testing.
CLKC	VUSB_HS_CLOCK_CONFIGURATION
RT	VUSB_HS_RESET_TYPE

3.3.3 HWHOST

Address: Base + 008h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

Host hardware parameters as defined in [System Level Issues and Core Configuration](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TTPER										TTASY										reserved										NPORT	HC

Figure 12 HWHOST – Host Hardware Parameters

The register fields are described in the following table.

Field	Description
TTPER	VUSB_HS_TT_PERIODIC_CONTEXTS
TTASY	VUSB_HS_TT_ASYNC_CONTEXTS
reserved	Reserved. These bits are reserved and should be set to zero.
NPORT	VUSB_HS_NUM_PORT-1
HC	VUSB_HS_HOST

3.3.4 HWDEVICE

Address: Base + 00Ch

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

Device hardware parameters as defined in [System Level Issues and Core Configuration](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																									DEVEP					DC	

Figure 13 HWDEVICE – Device Hardware Parameters

The register fields are described in the following table.

Field	Description
reserved	Reserved. These bits are reserved and should be set to zero.
DEVEP	VUSB_HS_DEV_EP

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 34-191

Field	Description
DC	device capable; [VUSB_HS_DEV != 0]

3.3.5 HWTXBUF

Address: Base + 010h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

TX buffer hardware parameters as defined in [System Level Issues and Core Configuration](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TXLCR		reserved							TXCHANADD								TXADD						TCBURST								

Figure 14 HWTXBUF – TX Buffer Hardware Parameters

The register fields are described in the following table.

Field	Description
reserved	Reserved. These bits are reserved and should be set to zero.
TXCHANADD	VUSB_HS_TX_CHAN_ADD
TXADD	VUSB_HS_TX_ADD
TCBURST	VUSB_HS_TX_BURST

3.3.6 HWRXBUF

Address: Base + 014h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

RX buffer hardware parameters as defined in [System Level Issues and Core Configuration](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																RXADD						RXBURST									

Figure 15 HWRXBUF – RX Buffer Hardware Parameters

The register fields are described in the following table.

Field	Description
reserved	Reserved. These bits are reserved and should be set to zero.
RXADD	VUSB_HS_RX_ADD
RXBURST	VUSB_HS_RX_BURST

3.4 Device/Host Capability Registers

Device/Host Capability registers specify the software limits, restrictions, and capabilities of the host/device controller implementation.

3.4.1 CAPLENGTH – EHCI Compliant

Address: Base + 100h

Default Value: 40h

Attribute: Read Only

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 35-191

Size: 8 bits

This register is used to indicate which offset to add to the register base address at the beginning of the Operational Register.

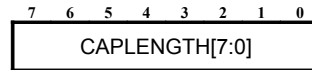


Figure 16 CAPLENGTH – Capability Register Length

3.4.2 HCVERSION – EHCI Compliant

Address: Base + 102h

Default Value: 0100h

Attribute: Read Only

Size: 16 bits

This is a two-byte register containing a BCD encoding of the EHCI revision number supported by this host controller. The most significant byte of this register represents a major revision and the least significant byte is the minor revision.

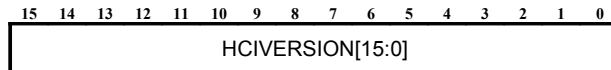


Figure 17 HCVERSION – Host Interface version number

3.4.3 HCSPARAMS – EHCI Compliant with extensions

Address: Base + 104h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 32 bits

Port steering logic capabilities are described in this register.

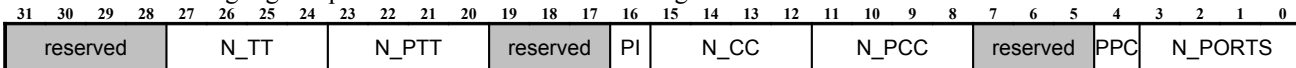


Figure 18 HCSPARAMS – Host Control Structural Parameters

The register fields are described in the following table.

HCSPARAMS – EHCI Compliant with extensions	
Field	Description
reserved	Reserved. These bits are reserved and should be set to zero.
PI	Port Indicators (P INDICATOR). This bit indicates whether the ports support port indicator control. When set to one, the port status and control registers include a read/writeable field for controlling the state of the port indicator. This field will always be "1".
N_TT[3:0]	Number of Transaction Translators (N_TT). This field indicates the number of embedded transaction translators associated with the USB2.0 host controller. For Multi-Port Host this field will always equal "0001". For all other implementation, N_TT = "0000". This is a non-EHCI field to support embedded TT.
N_PTT[3:0]	Number of Ports per Transaction Translator (N_PTT). This field indicates the number of ports assigned to each transaction translator within the USB2.0 host controller.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 36-191

HCSPARAMS – EHCI Compliant with extensions	
Field	Description
	For Multi-Port Host this field will always equal N_PORTS. For all other implementations, N_PTT = "0000". This in a non-EHCI field to support embedded TT.
N_CC[3:0]	Number of Companion Controller (N_CC). This field indicates the number of companion controllers associated with this USB2.0 host controller. A zero in this field indicates there are no internal Companion Controllers. Port-ownership hand-off is not supported. A value larger than zero in this field indicates there are companion USB1.1 host controller(s). Port-ownership hand-offs are supported. High, Full- and Low-speed devices are supported on the host controller root ports. In this implementation this field will always be "0".
N_PCC[3:0]	Number of Ports per Companion Controller. This field indicates the number of ports supported per internal Companion Controller. It is used to indicate the port routing configuration to the system software. For example, if N_PORTS has a value of 6 and N_CC has a value of 2 then N_PCC could have a value of 3. The convention is that the first N_PCC ports are assumed to be routed to companion controller 1, the next N_PCC ports to companion controller 2, etc. In the previous example, the N_PCC could have been 4, where the first 4 are routed to companion controller 1 and the last two are routed to companion controller 2. The number in this field must be consistent with N_PORTS and N_CC. In this implementation this field will always be "0".
PPC	Port Power Control. This field indicates whether the host controller implementation includes port power control. A one indicates the ports have port power switches. A zero indicates the ports do not have port power switches. The value of this field affects the functionality of the Port Power field in each port status and control register. This field will always be "0" for a device only implementation.
N_PORTS[3:0]	Number of downstream ports. This field specifies the number of physical downstream ports implemented on this host controller. The value of this field determines how many port registers are addressable in the Operational Register. Valid values are in the range of 1h to Fh. A zero in this field is undefined. The number of ports for a host implementation is parameterizable from 1 to 8. This field will always be 1 for device only implementation.

3.4.4 HCCPARAMS – EHCI Compliant

Address: Base + 108h

Default Value: 0006h

Attribute: Read Only

Size: 32 bits

This register identifies multiple mode control (time-base bit functionality) addressing capability.

This register identifies multiple mode control (time base or functionality), addressing capability.																															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																EECP[7:0]				IST[7:4]				R	ASPPFL	ADC					

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 37-191

Figure 19 HCCPARAMS – Host Control Capability Parameters

The register fields are described in the following table.

Field	HCCPARAMS – EHCI Compliant Description
reserved	reserved. These bits are reserved and should be set to zero.
EECP[7:0]	EHCI Extended Capabilities Pointer. Default = 0. This optional field indicates the existence of a capabilities list. A value of 00h indicates no extended capabilities are implemented. A non-zero value in this register indicates the offset in PCI configuration space of the first EHCI extended capability. The pointer value must be 40h or greater if implemented to maintain the consistency of the PCI header defined for this class of device. For this implementation this field is always "0".
IST[7:4]	Isochronous Scheduling Threshold. Default = implementation dependent. This field indicates, relative to the current position of the executing host controller, where software can reliably update the isochronous schedule. When bit [7] is zero, the value of the least significant 3 bits indicates the number of micro-frames a host controller can hold a set of isochronous data structures (one or more) before flushing the state. When bit [7] is a one, then host software assumes the host controller may cache an isochronous data structure for an entire frame. This field will always be "0".
R	Reserved. These bits are reserved and should be set to zero.
ASP	Asynchronous Schedule Park Capability. Default = 1. If this bit is set to a one, then the host controller supports the park feature for high-speed queue heads in the Asynchronous Schedule. The feature can be disabled or enabled and set to a specific level by using the <i>Asynchronous Schedule Park Mode Enable</i> and <i>Asynchronous Schedule Park Mode Count</i> fields in the USBCMD register. This field will always be "1"
PFL	Programmable Frame List Flag. If this bit is set to zero, then the system software must use a frame list length of 1024 elements with this host controller. The USBCMD register Frame List Size field is a read-only register and must be set to zero. If set to a one, then the system software can specify and use a smaller frame list and configure the host controller via the USBCMD register Frame List Size field. The frame list must always be aligned on a 4K-page boundary. This requirement ensures that the frame list is always physically contiguous. This field will always be "1".
ADC	64-bit Addressing Capability. This field will always be "0". No 64-bit addressing capability is supported.

3.4.5 DCIVERSION (Non-EHCI)

Address: Base + 120h

Default Value: Implementation Dependent

Attribute: Read Only

Size: 16 bits

The device controller interface conforms to the two-byte BCD encoding of the interface version number contained in this register.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 38-191

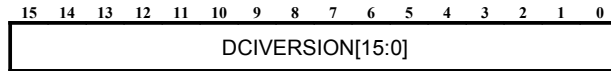


Figure 20 DCIVERSION – Device Interface Version Number

3.4.6 DCCPARAMS (Non-EHCI)

Address: Base + 124h

Default Value: Implementation Dependant

Attribute: Read Only

Size: 32 bits

These fields describe the overall host/device capability of the controller.

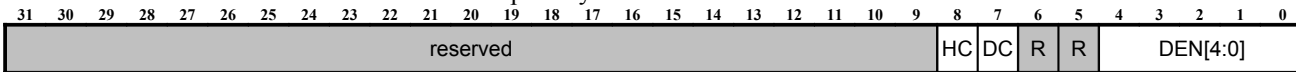


Figure 21 DCCPARAMS - Device Control Capability Parameters

The register fields are described in the following table.

Field	DCCPARAMS (Non-EHCI) Description
reserved	Reserved. These bits are reserved and should be set to zero.
HC	Host Capable. When this bit is 1, this controller is capable of operating as an EHCI compatible USB 2.0 host controller.
DC	Device Capable. When this bit is 1, this controller is capable of operating as a USB 2.0 device.
R	Reserved. These bits are reserved and should be set to zero.
DEN[4:0]	Device Endpoint Number. This field indicates the number of endpoints built into the device controller. If this controller is not device capable, then this field will be zero. Valid values are 0 – 16.

3.5 Device/Host Timer Registers (Non-EHCI)

The host/device controller drivers can measure time related activities using these timer registers. These registers are not part of the standard EHCI controller.

3.5.1 GPTIMER0LD (Non-EHCI)

Address: Base + 80h

Default Value: 00000000h

Attribute: Read/Write

Size: 32 bits

This register contains the timer duration or load value. See the GPTIMER0CTRL (Non-EHCI) for a description of the timer functions.

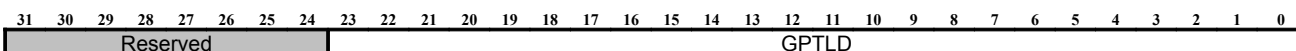


Figure 22 GPTIMER0LD – General Purpose Timer #0 Load Register

The register fields are described in the following table.

Field	GPTIMER0LD (Non-EHCI) Description
-------	--------------------------------------

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 39-191

Reserved **Reserved.** These bits are reserved and should be set to zero.

GPTLD **General Purpose Timer Load Value. (Read/Write)** This field is the value to be loaded into the GPTCNT countdown timer on a reset action. This value in this register represents the the time in microseconds minus 1 for the timer duration.

Example: for a one millisecond timer, load 1000-1=999 or 0x0003E7.

Note: Max value is 0xFFFFFFFF or 16.777215 seconds.

3.5.2 GPTIMER0CTRL (Non-EHCI)

Address: Base + 84h
Default Value: 00000000h
Attribute: Read Only, Write Only, Read/Write
Size: 32 bits

This register contains the control for the timer and a data field can be queried to determine the running count value. This timer has granularity on 1 us and can be programmed to a little over 16 seconds. There are two modes supported by this timer, the first is a one shot and the second is a looped count which is described in the register table below. When the timer counter value transitions to zero, an interrupt can be generated through the use of the timer interrupts in the USBSTS and USBINTR registers.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
GPTRUN	GPTRST	Reserved						GPTMODE	GPTCNT																						

Figure 23 GPTIMER0CTRL – General Purpose Timer #0 Control Register

The register fields are described in the following table.

GPTIMER0CTRL (Non-EHCI)	
Field	Description
GPTRUN	General Purpose Timer Run. Read/Write. (0) – Timer stop; (1) – Timer Run. This bit enables the general-purpose timer to run. Setting or clearing this bit will not have an effect on the GPTCNT except
GPTRST	General Purpose Timer Reset. Write-only. (0) – No action; (1) – Load Counter Value. Writing a one to this bit will reload the GPTCNT with the value in GPTLD.
reserved	Reserved. These bits are reserved and should be set to zero.
GPTMODE	General Purpose Timer Mode. Read/Write. (0) – One Shot ; (1) – Repeat. This bit selects between a single timer countdown and a looped count down. In one-shot mode, the timer will count down to zero, generate and interrupt, and stop until the counter is reset by software. In repeat mode, the timer will count down to zero, generate and interrupt, and automatically reload the counter to begin again.
GPTCNT	General Purpose Timer Counter. Read-only. This field is the value of the running timer.

3.5.3 GPTIMER1LD (Non-EHCI)

Address: Base + 88h
Default Value: 00000000h
Attribute: Read/Write
Size: 32 bits

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 40-191

Same as GPTIMER0LD.

3.5.4 GPTIMER1CTRL (Non-EHCI)

Address: Base + 8Ch
Default Value: 00000000h
Attribute: Read Only, Write Only, Read/Write
Size: 32 bits
Same as GPTIMER0CTRL.

3.6 Device/Host Operational Registers

Operational registers are comprised of dynamic control or status registers that may be read only, read/write, or read/write to clear. The following sections define the use of these registers.

3.6.1 USBCMD

Address: Base + 140h
Default Value: 00080B00h (host mode)
00080000h (device mode)
Attribute: Read Only, Read/Write, Write Only (field dependent)
Size: 32 bits

The serial bus host/device controller executes the command indicated in this register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ITC[7:0]								FS2	ATDTW	SUTW	R	ASPE	R	ASP1	ASP0	LR	IAA	ASE	PSE	FS1	FS0	RST	RS

Figure 24 USBCMD – USB Command Register

The register fields are described in the following table.

Field	USBCMD Description																		
R, reserved	Reserved. These bits are reserved and should be set to zero.																		
ITC[7:0]	Interrupt Threshold Control —Read/Write. Default 08h. The system software uses this field to set the maximum rate at which the host/device controller will issue interrupts. ITC contains the maximum interrupt interval measured in micro-frames. Valid values are shown below. <table> <tr> <th>Value</th><th>Maximum Interrupt Interval</th></tr> <tr> <td>00h</td><td>Immediate (no threshold)</td></tr> <tr> <td>01h</td><td>1 micro-frame</td></tr> <tr> <td>02h</td><td>2 micro-frames</td></tr> <tr> <td>04h</td><td>4 micro-frames</td></tr> <tr> <td>08h</td><td>8 micro-frames</td></tr> <tr> <td>10h</td><td>16 micro-frames</td></tr> <tr> <td>20h</td><td>32 micro-frames</td></tr> <tr> <td>40h</td><td>64 micro-frames</td></tr> </table>	Value	Maximum Interrupt Interval	00h	Immediate (no threshold)	01h	1 micro-frame	02h	2 micro-frames	04h	4 micro-frames	08h	8 micro-frames	10h	16 micro-frames	20h	32 micro-frames	40h	64 micro-frames
Value	Maximum Interrupt Interval																		
00h	Immediate (no threshold)																		
01h	1 micro-frame																		
02h	2 micro-frames																		
04h	4 micro-frames																		
08h	8 micro-frames																		
10h	16 micro-frames																		
20h	32 micro-frames																		
40h	64 micro-frames																		
SUTW	Setup TripWire – Read/Write. [device mode only] This bit is used as a semaphore to ensure that the setup data payload of 8 bytes is extracted from a QH by the DCD without being corrupted. If the setup lockout mode is off (See USBMODE) then there exists a hazard when new setup data arrives while the DCD is copying the setup data payload from the QH for a previous setup packet. This bit is set and cleared by software and will be cleared by hardware when a hazard exists. For more information on the use of this bit, see the Device Operational Model section of the USB-HS DEV reference manual.																		

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 41-191

Field	USBCMD Description
ATDTW	Add dTD TripWire – Read/Write. [device mode only] This bit is used as a semaphore to ensure the to proper addition of a new dTD to an active (primed) endpoint's linked list. This bit is set and cleared by software. This bit shall also be cleared by hardware when is state machine is hazard region for which adding a dTD to a primed endpoint may go unrecognized. For more information on the use of this bit, see the Device Operational Model section of the USB-HS DEV reference manual.
ASPE	Asynchronous Schedule Park Mode Enable (OPTIONAL) — Read/Write. If the <i>Asynchronous Park Capability</i> bit in the HCCPARAMS register is a one, then this bit defaults to a 1h and is R/W. Otherwise the bit must be a zero and is RO. Software uses this bit to enable or disable Park mode. When this bit is one, Park mode is enabled. When this bit is a zero, Park mode is disabled. This field is set to "1" in this implementation.
ASP[1:0]	Asynchronous Schedule Park Mode Count (OPTIONAL) — Read/Write. If the <i>Asynchronous Park Capability</i> bit in the HCCPARAMS register is a one, then this field defaults to 3h and is R/W. Otherwise it defaults to zero and is RO. It contains a count of the number of successive transactions the host controller is allowed to execute from a high-speed queue head on the Asynchronous schedule before continuing traversal of the Asynchronous schedule. See Section 4.10.3.2 for full operational details. Valid values are 1h to 3h. Software must not write a zero to this bit when <i>Park Mode Enable</i> is a one as this will result in undefined behavior. This field is set to 3h in this implementation.
LR	Light Host/Device Controller Reset (OPTIONAL) — Read Only. Not Implemented. This field will always be "0".
IAA	Interrupt on Async Advance Doorbell — Read/Write. This bit is used as a doorbell by software to tell the host controller to issue an interrupt the next time it advances asynchronous schedule. Software must write a 1 to this bit to ring the doorbell. When the host controller has evicted all appropriate cached schedule states, it sets the Interrupt on Async Advance status bit in the USBSTS register. If the Interrupt on Sync Advance Enable bit in the USBINTR register is one, then the host controller will assert an interrupt at the next interrupt threshold. The host controller sets this bit to zero after it has set the Interrupt on Sync Advance status bit in the USBSTS register to one. Software should not write a one to this bit when the asynchronous schedule is inactive. Doing so will yield undefined results. This bit is only used in host mode. Writing a one to this bit when device mode is selected will have undefined results.
ASE	Asynchronous Schedule Enable — Read/Write. Default 0b. This bit controls whether the host controller skips processing the Asynchronous Schedule. Values meaning 0 Do not process the Asynchronous Schedule. 1 Use the ASYNCLISTADDR register to access the Asynchronous Schedule. Only the host controller uses this bit.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 42-191

Field	USBCMD Description																
PSE	Periodic Schedule Enable— Read/Write. Default Ob. This bit controls whether the host controller skips processing the Periodic Schedule. Values meaning <table> <tr> <td>0</td><td>Do not process the Periodic Schedule</td></tr> <tr> <td>1</td><td>Use the PERIODICLISTBASE register to access the Periodic Schedule.</td></tr> </table> Only the host controller uses this bit.	0	Do not process the Periodic Schedule	1	Use the PERIODICLISTBASE register to access the Periodic Schedule.												
0	Do not process the Periodic Schedule																
1	Use the PERIODICLISTBASE register to access the Periodic Schedule.																
FS[2:0]	Frame List Size — (Read/Write or Read Only). Default 000b. This field is Read/Write only if Programmable Frame List Flag in the HCCPARAMS registers is set to one. This field specifies the size of the frame list that controls which bits in the Frame Index Register should be used for the Frame List Current index. Note that this field is made up from USBCMD bits 15, 3 and 2. Values meaning <table> <tr> <td>000</td><td>1024 elements (4096 bytes) Default value</td></tr> <tr> <td>001</td><td>512 elements (2048 bytes)</td></tr> <tr> <td>010</td><td>256 elements (1024 bytes)</td></tr> <tr> <td>011</td><td>128 elements (512 bytes)</td></tr> <tr> <td>100</td><td>64 elements (256 bytes)</td></tr> <tr> <td>101</td><td>32 elements (128 bytes)</td></tr> <tr> <td>110</td><td>16 elements (64 bytes)</td></tr> <tr> <td>111</td><td>8 elements (32 bytes)</td></tr> </table> Only the host controller uses this field.	000	1024 elements (4096 bytes) Default value	001	512 elements (2048 bytes)	010	256 elements (1024 bytes)	011	128 elements (512 bytes)	100	64 elements (256 bytes)	101	32 elements (128 bytes)	110	16 elements (64 bytes)	111	8 elements (32 bytes)
000	1024 elements (4096 bytes) Default value																
001	512 elements (2048 bytes)																
010	256 elements (1024 bytes)																
011	128 elements (512 bytes)																
100	64 elements (256 bytes)																
101	32 elements (128 bytes)																
110	16 elements (64 bytes)																
111	8 elements (32 bytes)																
RST	Controller Reset (RESET) — Read/Write. Software uses this bit to reset the controller. This bit is set to zero by the Host/Device Controller when the reset process is complete. Software cannot terminate the reset process early by writing a zero to this register. <u>Host Controller:</u> When software writes a one to this bit, the Host Controller resets its internal pipelines, timers, counters, state machines etc. to their initial value. Any transaction currently in progress on USB is immediately terminated. A USB reset is not driven on downstream ports. Software should not set this bit to a one when the HCHalted bit in the USBSTS register is a zero. Attempting to reset an actively running host controller will result in undefined behavior. <u>Device Controller:</u> When software writes a one to this bit, the Device Controller resets its internal pipelines, timers, counters, state machines etc. to their initial value. Writing a one to this bit when the device is in the attached state is not recommended, since the effect on an attached host is undefined. In order to ensure that the device is not in an attached state before initiating a device controller reset, all primed endpoints should be flushed and the USBCMD Run/Stop bit should be set to 0.																

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 43-191

Field	USBCMD Description
RS	Run/Stop (RS) – Read/Write. Default 0b. 1=Run. 0=Stop. Host Controller: When set to a 1, the Host Controller proceeds with the execution of the schedule. The Host Controller continues execution as long as this bit is set to a one. When this bit is set to 0, the Host Controller completes the current transaction on the USB and then halts. The HC Halted bit in the status register indicates when the Host Controller has finished the transaction and has entered the stopped state. Software should not write a one to this field unless the host controller is in the Halted state (i.e. HCHalted in the USBSTS register is a one). Device Controller: Writing a one to this bit will cause the device controller to enable a pull-up on D+ and initiate an attach event. This control bit is not directly connected to the pull-up enable, as the pull-up will become disabled upon transitioning into high-speed mode. Software should use this bit to prevent an attach event before the device controller has been properly initialized. Writing a 0 to this will cause a detach event.

3.6.2 USBSTS

Address: Base + 144h

Default Value: 00001000h (host mode)
00000000h (device mode)

Attribute: Read Only, Read/Write, Read/Write-Clear (field dependant)

Size: 32 bits

This register indicates various states of the Host/Device Controller and any pending interrupts. This register does not indicate status resulting from a transaction on the serial bus. Software clears certain bits in this register by writing a 1 to them.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
reserved						T11	T10	reserved				UP 	UA 		NAKI	AS	PS	RCL	HCH		R 	ULPII		R 	SL 	SR 	URI	AA 	SE 	FR 	PC 	UE 	U

Figure 25 USBSTS – USB Status

The register fields are described in the following table.

Field	USBSTS Description
reserved	Reserved. These bits are reserved and should be set to zero.
T11	General Purpose Timer Interrupt 1 (GPTINT1) — R/WC. This bit is set when the counter in the GPTIMER1CTRL (Non-EHCI) register transitions to zero. Writing a one to this bit will clear it.
T10	General Purpose Timer Interrupt 0 (GPTINT0) — R/WC. This bit is set when the counter in the GPTIMER0CTRL (Non-EHCI) register transitions to zero. Writing a one to this bit will clear it.
reserved	Reserved. These bits are reserved and should be set to zero.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 44-191

Field	USBSTS Description
UPI	USB Host Periodic Interrupt (USBHSTPERINT) — R/WC. This bit is set by the Host Controller when the cause of an interrupt is a completion of a USB transaction where the Transfer Descriptor (TD) has an interrupt on complete (IOC) bit set and the TD was from the periodic schedule. This bit is also set by the Host Controller when a short packet is detected AND the packet is on the periodic schedule. A short packet is when the actual number of bytes received was less than the expected number of bytes. This bit is not used by the device controller and will always be zero.
UAI	USB Host Asynchronous Interrupt (USBHSTASYNCINT) — R/WC. This bit is set by the Host Controller when the cause of an interrupt is a completion of a USB transaction where the Transfer Descriptor (TD) has an interrupt on complete (IOC) bit set AND the TD was from the asynchronous schedule. This bit is also set by the Host when a short packet is detected AND the packet is on the asynchronous schedule. A short packet is when the actual number of bytes received was less than the expected number of bytes. This bit is not used by the device controller and will always be zero.
reserved	Reserved. These bits are reserved and should be set to zero.
NAKI	NAK Interrupt Bit — Read Only. This bit is read-only. It is set by hardware when for a particular endpoint both the TX/RX Endpoint NAK bit and the corresponding TX/RX Endpoint NAK Enable bit are set. This bit is automatically cleared by hardware when the all the enabled TX/RX Endpoint NAK bits are cleared.
AS	Asynchronous Schedule Status — Read Only. 0=Default. This bit reports the current real status of the Asynchronous Schedule. When set to zero the asynchronous schedule status is disabled and if set to one the status is enabled. The Host Controller is not required to immediately disable or enable the Asynchronous Schedule when software transitions the Asynchronous Schedule Enable bit in the USBCMD register. When this bit and the Asynchronous Schedule Enable bit are the same value, the Asynchronous Schedule is either enabled (1) or disabled (0). Only used by the host controller.
PS	Periodic Schedule Status — Read Only. 0=Default. This bit reports the current real status of the Periodic Schedule. When set to zero the periodic schedule is disabled, and if set to one the status is enabled. The Host Controller is not required to immediately disable or enable the Periodic Schedule when software transitions the Periodic Schedule Enable bit in the USBCMD register. When this bit and the Periodic Schedule Enable bit are the same value, the Periodic Schedule is either enabled (1) or disabled (0). Only used by the host controller.
RCL	Reclamation — Read Only. 0=Default. This is a read-only status bit used to detect an empty asynchronous schedule. Only used by the host controller.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 45-191

Field	USBSTS Description
HCH	HCHalted — Read Only. 1=Default. This bit is a zero whenever the Run/Stop bit is a one. The Host Controller sets this bit to one after it has stopped executing because of the Run/Stop bit being set to 0, either by software or by the Host Controller hardware (e.g. internal error). Only used by the host controller.
R	Reserved. These bits are reserved and should be set to zero.
ULPII	ULPI Interrupt – R/WC. 0=Default. When the ULPI Viewport is present in the design, an event completion will set this interrupt. Used by both host & device controller. Only present in designs where configuration constant VUSB_HS_PHY_ULPI = 1.
R	Reserved. These bits are reserved and should be set to zero.
SLI	DCSuspend – R/WC. 0=Default. When a device controller enters a suspend state from an active state, this bit will be set to a one. The device controller clears the bit upon exiting from a suspend state. Only used by the device controller.
SRI	SOF Received – R/WC. 0=Default. When the device controller detects a Start Of (micro) Frame, this bit will be set to a one. When a SOF is extremely late, the device controller will automatically set this bit to indicate that an SOF was expected. Therefore, this bit will be set roughly every 1ms in device FS mode and every 125ms in HS mode and will be synchronized to the actual SOF that is received. Since the device controller is initialized to FS before connect, this bit will be set at an interval of 1ms during the prelude to connect and chirp. In host mode, this bit will be set every 125us and can be used by host controller driver as a time base. Software writes a 1 to this bit to clear it. This is a non-EHCI status bit.
URI	USB Reset Received – R/WC. 0=Default. When the device controller detects a USB Reset and enters the default state, this bit will be set to a one. Software can write a 1 to this bit to clear the USB Reset Received status bit. Only used by the device controller.
AAI	Interrupt on Async Advance — R/WC. 0=Default. System software can force the host controller to issue an interrupt the next time the host controller advances the asynchronous schedule by writing a one to the Interrupt on Async Advance Doorbell bit in the USBCMD register. This status bit indicates the assertion of that interrupt source. Only used by the host controller.
SEI	System Error— R/WC. This bit is not used in this implementation and will always be set to "0".

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 46-191

Field	USBSTS Description
FRI	Frame List Rollover — R/WC. The Host Controller sets this bit to a one when the Frame List Index rolls over from its maximum value to zero. The exact value at which the rollover occurs depends on the frame list size. For example. If the frame list size (as programmed in the Frame List Size field of the USBCMD register) is 1024, the Frame Index Register rolls over every time FRINDEX [1:3] toggles. Similarly, if the size is 512, the Host Controller sets this bit to a one every time FHINDEX [12] toggles. Only used by the host controller.
PCI	Port Change Detect — R/WC. The Host Controller sets this bit to a one when on any port a Connect Status occurs, a Port Enable/Disable Change occurs, or the Force Port Resume bit is set as the result of a J-K transition on the suspended port. The Device Controller sets this bit to a one when the port controller enters the full or high-speed operational state. When the port controller exits the full or high-speed operation states due to Reset or Suspend events, the notification mechanisms are the USB Reset Received bit and the DCSuspend bits respectively. This bit is not EHCI compatible.
UEI	USB Error Interrupt (USBERRINT) — R/WC. When completion of a USB transaction results in an error condition, this bit is set by the Host/Device Controller. This bit is set along with the USBINT bit, if the TD on which the error interrupt occurred also had its interrupt on complete (IOC) bit set See Section (Reference Host Operation Model: Transfer/Transaction Based Interrupt – i.e. 4.15.1 in EHCI 10) for a complete list of host error interrupt conditions. See section Device Error Matrix in the USB-HS DEV reference manual. The device controller detects resume signaling only.
UI	USB Interrupt (USBINT) — R/WC. This bit is set by the Host/Device Controller when the cause of an interrupt is a completion of a USB transaction where the Transfer Descriptor (TD) has an interrupt on complete (IOC) bit set. This bit is also set by the Host/Device Controller when a short packet is detected. A short packet is when the actual number of bytes received was less than the expected number of bytes.

3.6.3 USBINTR

Address: Base + 148h
Default Value: 00000000h
Attribute: Read/Write
Size: 32 bits

The interrupts to software are enabled with this register. An interrupt is generated when a bit is set and the corresponding interrupt is active. The USB Status register (USBSTS) still shows interrupt sources even if they are disabled by the USBINTR register, allowing polling of interrupt events by the software.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TIE1	TIE0	Reserved				UP	IA	UA	IE	R	NAKE	Reserved				ULPIE	R	SLE	SRE	URE	AA	E	SEE	FRE	PCE	UEE	UE

Figure 26 USBINTR – USB Interrupt Enable

The register fields are described in the following table.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 47-191

Field	USBINTR Interrupt Source	Description
reserved	Reserved.	These bits are reserved and should be set to zero.
TIE1	General Purpose Timer Interrupt Enable 1	When this bit is a one, and the <i>GPTINT1</i> bit in the USBSTS register is a one, the controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>GPTINT1</i> bit.
TIE0	General Purpose Timer Interrupt Enable 0	When this bit is a one, and the <i>GPTINT0</i> bit in the USBSTS register is a one, the controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>GPTINT0</i> bit.
reserved	Reserved.	These bits are reserved and should be set to zero.
UPIE	USB Host Periodic Interrupt Enable	When this bit is a one, and the <i>USBHSTPERINT</i> bit in the USBSTS register is a one, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by software clearing the <i>USBHSTPERINT</i> bit.
UAIE	USB Host Async. Interrupt Enable	When this bit is a one, and the <i>USBHSTASYNCINT</i> bit in the USBSTS register is a one, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by software clearing the <i>USBHSTASYNCINT</i> bit.
reserved	Reserved.	These bits are reserved and should be set to zero.
NAKE	NAK Interrupt Enable	This bit is set by software if it wants to enable the hardware interrupt for the NAK Interrupt bit. If both this bit and the corresponding NAK Interrupt bit are set, a hardware interrupt is generated.
reserved	Reserved.	These bits are reserved and should be set to zero.
ULPIE	ULPI Enable	When this bit is a one, and the ULPI Interrupt bit in the USBSTS register transitions, the controller will issue an interrupt. The interrupt is acknowledged by software writing a one to the ULPI Interrupt bit. Used by both host & device controller. Only present in designs where configuration constant <i>VUSB_HS_PHY_ULPI</i> = 1.
reserved	Reserved.	These bits are reserved and should be set to zero.
SLE	Sleep Enable	When this bit is a one, and the <i>DCSuspend</i> bit in the USBSTS register transitions, the device controller will issue an interrupt. The interrupt is acknowledged by software writing a one to the <i>DCSuspend</i> bit. Only used by the device controller.
SRE	SOF Received Enable	When this bit is a one, and the <i>SOF Received</i> bit in the USBSTS register is a one, the device controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>SOF Received</i> bit.
URE	USB Reset Enable	When this bit is a one, and the <i>USB Reset Received</i> bit in the USBSTS register is a one, the device controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>USB Reset Received</i> bit. Only used by the device controller.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 48-191

Field	USBINTR Interrupt Source	Description
AAE	Interrupt on Async Advance Enable	When this bit is a one, and the <i>Interrupt on Async Advance</i> bit in the USBSTS register is a one, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by software clearing the <i>Interrupt on Async Advance</i> bit. Only used by the host controller.
SEE	System Error Enable	When this bit is a one, and the <i>System Error</i> bit in the USBSTS register is a one, the host/device controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>System Error</i> bit.
FRE	Frame List Rollover Enable	When this bit is a one, and the <i>Frame List Rollover</i> bit in the USBSTS register is a one, the host controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>Frame List Rollover</i> bit. Only used by the host controller.
PCE	Port Change Detect Enable	When this bit is a one, and the <i>Port Change Detect</i> bit in the USBSTS register is a one, the host/device controller will issue an interrupt. The interrupt is acknowledged by software clearing the <i>Port Change Detect</i> bit.
UEE	USB Error Interrupt Enable	When this bit is a one, and the <i>USBERRINT</i> bit in the USBSTS register is a one, the host controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by software clearing the <i>USBERRINT</i> bit in the USBSTS register.
UE	USB Interrupt Enable	When this bit is a one, and the <i>USBINT</i> bit in the USBSTS register is a one, the host/device controller will issue an interrupt at the next interrupt threshold. The interrupt is acknowledged by software clearing the <i>USBINT</i> bit.

3.6.4 FRINDEX

Address: Base + 14Ch

Default Value: Undefined (free running counter)

Attribute: Read/Write in host mode, Read in device mode

Size: 32 bits

This register is used by the host controller to index the periodic frame list. The register updates every 125 microseconds (once each micro-frame). Bits [N: 3] are used to select a particular entry in the Periodic Frame List during periodic schedule execution. The number of bits used for the index depends on the size of the frame list as set by system software in the Frame List Size field in the USBCMD register.

This register must be written as a DWord. Byte writes produce undefined results. This register cannot be written unless the Host Controller is in the 'Halted' state as indicated by the HCHalted bit. A write to this register while the Run/Stop bit is set to a one produces undefined results. Writes to this register also affect the SOF value.

In device mode this register is read only and, the device controller updates the FRINDEX [13:3] register from the frame number indicated by the SOF marker. Whenever a SOF is received by the USB bus, FRINDEX [13:3] will be checked against the SOF marker. If FRINDEX [13:3] is different from the SOF marker, FRINDEX [13:3] will be set to the SOF value and FRINDEX [2:0] will be set to zero (i.e. SOF for 1 ms frame). If FRINDEX [13:3] is equal to the SOF value, FRINDEX [2:0] will be incremented (i.e. SOF for 125 us micro-frame.)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																		FRINDEX[13:0]													

Figure 27 FRINDEX – USB Frame Index

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 49-191

The register fields are described in the following table.

Field	FRINDEX Description																											
reserved	Reserved. These bits are reserved and should be set to zero.																											
FRINDEX	Frame Index. The value, in this register, increments at the end of each time frame (e.g. micro-frame). Bits [N: 3] are used for the Frame List current index. This means that each location of the frame list is accessed 8 times (frames or micro-frames) before moving to the next index. The following illustrates values of N based on the value of the Frame List Size field in the USBCMD register, when used in host mode. <table><tr><th>USBCMD</th><th>[Frame List Size] Number</th><th>Elements N</th></tr><tr><td>000b</td><td>(1024)</td><td>12</td></tr><tr><td>001b</td><td>(512)</td><td>11</td></tr><tr><td>010b</td><td>(256)</td><td>10</td></tr><tr><td>011b</td><td>(128)</td><td>9</td></tr><tr><td>100b</td><td>(64)</td><td>8</td></tr><tr><td>101b</td><td>(32)</td><td>7</td></tr><tr><td>110b</td><td>(16)</td><td>6</td></tr><tr><td>111b</td><td>(8)</td><td>5</td></tr></table> In device mode the value is the current frame number of the last frame transmitted. It is not used as an index. In either mode bits 2:0 indicate the current microframe.	USBCMD	[Frame List Size] Number	Elements N	000b	(1024)	12	001b	(512)	11	010b	(256)	10	011b	(128)	9	100b	(64)	8	101b	(32)	7	110b	(16)	6	111b	(8)	5
USBCMD	[Frame List Size] Number	Elements N																										
000b	(1024)	12																										
001b	(512)	11																										
010b	(256)	10																										
011b	(128)	9																										
100b	(64)	8																										
101b	(32)	7																										
110b	(16)	6																										
111b	(8)	5																										

3.6.5 CTRLDSSEGMENT

Address: Base + 150h

Default Value: 00000000h

Attribute: Read Only

Size: 32 bits

This register is not used in this implementation.

3.6.6 PERIODICLISTBASE; DEVICEADDR

Address: Base + 154h

Default Value: 00000000h

Attribute: Read/Write (Writes must be DWord Writes)

Size: 32 bits

This register is shared between the host controller and the device controller operation.

3.6.6.1 Host Controller (PERIODICLISTBASE)

This 32-bit register contains the beginning address of the Periodic Frame List in the system memory. HCD loads this register prior to starting the schedule execution by the Host Controller. The memory structure referenced by this physical memory pointer is assumed to be 4-Kbyte aligned. The contents of this register are combined with the Frame Index Register (FRINDEX) to enable the Host Controller to step through the Periodic Frame List in sequence.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PERBASE[31:12]																reserved															

Figure 28 PERIODICLISTBASE - Host Controller Frame List Base Address

The register fields are described in the following table.

		Title: CII3410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 50-191

Field	PERIODICLISTBASE Description
BASEADR	Base Address (Low). These bits correspond to memory address signals [31:12], respectively. Only used by the host controller.
reserved	Reserved. Must be written as zeros. During runtime, the values of these bits are undefined.

3.6.6.2 Device Controller (USB DEVICEADDR)

The upper seven bits of this register represent the device address. After any controller reset or a USB reset, the device address is set to the default address (0). The default address will match all incoming addresses. Software shall reprogram the address after receiving a SET_ADDRESS descriptor.

The USBADRA is used to accelerate the SET_ADDRESS sequence by allowing the DCD to preset the USBADR register before the status phase of the SET_ADDRESS descriptor.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
USBADR[31:25]							USBADRA	reserved																							

Figure 29 DEVICEADDR - Device Controller USB Device Address

The register fields are described in the following table.

Field	DEVICEADDR Description
USBADR	Device Address. These bits correspond to the USB device address
USBADRA	Device Address Advance. Default=0. When this bit is '0', any writes to USBADR are instantaneous. When this bit is written to a '1' at the same time or before USBADR is written, the write to the USBADR field is staged and held in a hidden register. After an IN occurs on endpoint 0 and is ACKed, USBADR will be loaded from the holding register. Hardware will automatically clear this bit on the following conditions: 1) IN is ACKed to endpoint 0. (USBADR is updated from staging register). 2) OUT/SETUP occur to endpoint 0. (USBADR is not updated). 3) Device Reset occurs (USBADR is reset to 0). Note: After the status phase of the SET_ADDRESS descriptor, the DCD has 2 ms to program the USBADR field. This mechanism will ensure this specification is met when the DCD can not write of the device address within 2ms from the SET_ADDRESS status phase. If the DCD writes the USBADR with USBADRA=1 after the SET_ADDRESS data phase (before the prime of the status phase), the USBADR will be programmed instantly at the correct time and meet the 2ms USB requirement.
reserved	Reserved. Must be written as zeros. During runtime, the values of these bits are undefined.

3.6.7 ASYNCLISTADDR; ENDPOINTLISTADDR

Address: Base + 158h

Default Value: 00000000h

Attribute: Read/Write (Writes must be DWord Writes)

Size: 32 bits

This register is shared between the host controller and the device controller operation.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 51-191

3.6.7.1 Host Controller (ASYNCLISTADDR)

This 32-bit register contains the address of the next asynchronous queue head to be executed by the host. Bits [4:0] of this register cannot be modified by the system software and will always return a zero when read.

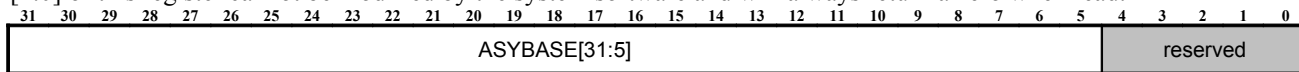


Figure 30 ASYNCLISTADDR - Host Controller Next Asynch. Address

The register fields are described in the following table.

ASYNCLISTADDR	
Field	Description
ASYBASE[31:5]	Link Pointer Low (LPL). These bits correspond to memory address signals [31:5], respectively. This field may only reference a Queue Head (OH). Only used by the host controller.
reserved	Reserved. These bits are reserved and their value has no effect on operation.

3.6.7.2 Device Controller (ENDPOINTLISTADDR)

In device mode, this register contains the address of the top of the endpoint list in system memory. Bits [10:0] of this register cannot be modified by the system software and will always return a zero when read.

The memory structure referenced by this physical memory pointer is assumed 64-byte.

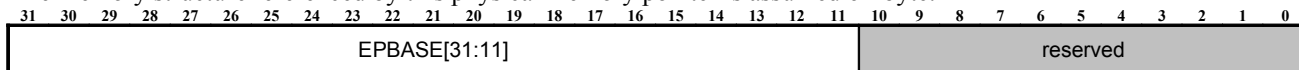


Figure 31 ENDPOINTLISTADDR - Device Controller Endpoint List Address

The register fields are described in the following table.

ENDPOINTLISTADDR	
Field	Description
EPBASE[31:11]	Endpoint List Pointer (Low). These bits correspond to memory address signals [31:11], respectively. This field will reference a list of up to 32 Queue Heads (QH). (i.e. one queue head per endpoint & direction.)
reserved	Reserved. These bits are reserved and their value has no effect on operation.

3.6.8 BURSTSIZE

Address: Base + 160h

Default Value: Implementation Dependent

Attribute: Read/Write (Writes must be DWord Writes)

Size: 32 bits

This register is used to control dynamically change the burst size used during data movement on the initiator (master) interface.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 52-191

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																TXPBURST								RXPBURST							

Figure 32 BURSTSIZE - Host Controller Embedded TT Async. Buffer Status

The register fields are described in the following table.

Field	BURSTSIZE Description
reserved	Reserved. These bits are reserved and their value has no effect on operation.
TXPBURST	Programmable TX Burst Length. (Read/Write) Default is the constant VUSB_HS_TX_BURST. This register represents the maximum length of a the burst in 32-bit words while moving data from system memory to the USB bus.
RXPBURST	Programmable RX Burst Length. (Read/Write) Default is the constant VUSB_HS_RX_BURST. This register represents the maximum length of a the burst in 32-bit words while moving data from the USB bus to system memory.

3.6.9 ULPI VIEWPORT

Address: Base + 170h

Default Value: 00000000h

Attribute: Read/Write

Size: 32 bits

The register provides indirect access to the ULPI PHY register set. Although the core performs access to the ULPI PHY register set, there may be extraordinary circumstances where software may need direct access.

CAUTION: WRITES TO THE ULPI THROUGH THE VIEWPORT CAN SUBSTANTIALLY HARM STANDARD USB OPERATIONS. CURRENTLY NO USAGE MODEL HAS BEEN DEFINED WHERE SOFTWARE SHOULD NEED TO EXECUTE WRITES DIRECTLY TO THE ULPI – SEE EXCEPTION REGARDING OPTIONAL FEATURES BELOW.

NOTE: EXECUTING READ OPERATIONS THROUGH THE ULPI VIEWPORT SHOULD HAVE NO HARMFUL SIDE EFFECTS TO STANDARD USB OPERATIONS.

NOTE: THE ULPI VIEWPORT IS ONLY SYNTHESIZED IN THE DESIGN IF THE CONSTANT VUSB_HS_PHY_ULPI IS SET TO 1. IF THE ULPI INTERFACE IS NOT ENABLED, THIS REGISTER WILL ALWAYS READ ZEROS.

There are two operations that can be performed with the ULPI Viewport, wakeup and read /write operations. The wakeup operation is used to put the ULPI interface into normal operation mode and re-enable the clock if necessary. A wakeup operation is required before accessing the registers when the ULPI interface is operating in low power mode, serial mode, or carkit mode. The ULPI state can be determined by reading the sync. state bit (ULPISS). If this bit is a one, then ULPI interface is running in normal operation mode and can accept read/write operations. If the ULPISS indicates a '0' then then read/write operations will not be able execute. Undefined behavior will result if ULPISS = 0 and a read or write operation is performed. To execute a wakeup operation, write all 32-bits of the ULPI Viewport where ULPIPORT is constructed appropriately and the ULPIWU bit is a '1' and ULPIRUN bit is a '0'. Poll the ULPI Viewport until ULPIWU is zero for the operation to complete.

To execute a read or write operation, write all 32-bits of the ULPI Viewport where ULPIDATWR, ULPIADDR, ULPIPORT, ULPIRW are constructed appropriately and the ULPIRUN bit is a '1'. Poll the ULPI Viewport until ULPIRUN is zero for the operation to complete. Once ULPIRUN is zero, the ULPIDATRD will be valid if the operation was a read.

The polling method above could also be replaced and interrupt driven using the ULPI interrupt defined in the USBSTS and USBINTR registers. When a wakeup or read/write operation complete, the ULPI interrupt will be set.

There are several optional features that may need to be enabled or disabled by system software as part of system configuration. These bits are contained in the Interface and OTG Control registers of the ULPI PHY register set. These registers also contain bits which are controlled by the link dynamically and therefore should only modify by system software using the Set/Clear access method. **Direct writes to these registers could have harmful side effects to the standard USB operations.** The optional bits are as follows: Bits 3 through 7 in the Interface Control register and Bits 6

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 53-191

and 7 in the OTG Control register. Please refer to the ULPI Specification Revision 1.1 for further information on the use of the optional features.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ULPIWU	ULPIRUN	ULPIRW	R	ULPISS	ULPIPORT	ULPIADDR				ULPIDATRD				ULPIDATWR																	

Field	ULPI VIEWPORT Description
ULPIWU	ULPI Wakeup – Read/Write. Writing the ‘1’ to this bit will begin the wakeup operation. The bit will automatically transition to 0 after the wakeup is complete. Once this bit is set, the driver can not set it back to ‘0’. Note: The driver must never execute a wakeup and a read/write operation at the same time.
ULPIRUN	ULPI Read/Write Run – Read/Write. Writing the ‘1’ to this bit will begin the read/write operation. The bit will automatically transition to 0 after the read/write is complete. Once this bit is set, the driver can not set it back to ‘0’. Note: The driver must never execute a wakeup and a read/write operation at the same time.
ULPIRW	ULPI Read/Write Control – Read/Write. (0) – Read; (1) – Write. This bit selects between running a read or write operation.
reserved	Reserved. This bit is reserved and its value has no effect on operation.
ULPISS	ULPI Sync State – Read Only. (1) – Normal Sync. State. (0) In another state (i.e. carkit, serial, low power) This bit represents the state of the ULPI interface. Before reading this bit, the ULPIPORT field should be set accordingly if used with the multi-port host. Otherwise, this field should always remain 0.
ULPIPORT	ULPI Port Number – Read/Write. For the wakeup or read/write operation to be executed, this value selects the port number the ULPI PHY is attached to in the multi-port host. The range is 0 to 7. This field should always be written as a 0 for the non-multi port products.
ULPIADDR	ULPI Data Address – Read/Write. When a read or write operation is commanded, the address of the operation is written to this field.
ULPIDATRD	ULPI Data Read – Read Only. After a read operation completes, the result is placed in this field.
ULPIDATWR	ULPI Data Write – Read/Write. When a write operation is commanded, the data to be sent is written to this field.

3.6.10 ENDPTNAK

Address: Base + 178h
Default Value: 00000000h
Attribute: Read/Write Clear
Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EPTN[15:0]																EPRN[15:0]															

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 54-191

Field	ENDPTNAK Description
EPTN [15:0]	TX Endpoint NAK – R/WC. Each TX endpoint has 1 bit in this field. The bit is set when the device sends a NAK handshake on a received IN token for the corresponding endpoint. Bit 15 – Endpoint #15 Bit 1 – Endpoint #1 Bit 0 – Endpoint #0
EPRN [15:0]	RX Endpoint NAK – R/WC. Each RX endpoint has 1 bit in this field. The bit is set when the device sends a NAK handshake on a received OUT or PING token for the corresponding endpoint. Bit 15 – Endpoint #15 Bit 1 – Endpoint #1 Bit 0 – Endpoint #0

3.6.11 ENDPTNAKEN

Address: Base + 17Ch
Default Value: 00000000h
Attribute: Read/Write Clear
Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EPTNE[15:0]																EPRNE[15:0]															

Field	ENDPTNAKEN Description
EPTNE [15:0]	TX Endpoint NAK Enable – R/W. Each bit is an enable bit for the corresponding TX Endpoint NAK bit. If this bit is set and the corresponding TX Endpoint NAK bit is set, the NAK Interrupt bit is set. Bit 15 – Endpoint #15 Bit 1 – Endpoint #1 Bit 0 – Endpoint #0
EPRNE [15:0]	RX Endpoint NAK Enable – R/W. Each bit is an enable bit for the corresponding RX Endpoint NAK bit. If this bit is set and the corresponding RX Endpoint NAK bit is set, the NAK Interrupt bit is set. Bit 15 – Endpoint #15 Bit 1 – Endpoint #1 Bit 0 – Endpoint #0

3.6.12 CONFIGFLAG

Address: Base + 180h
Default Value: 00000001h
Attribute: Read Only
Size: 32 bits

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 55-191

This register is not used in this implementation. A read from this register returns a constant of a 00000001h to indicate that all port routings default to this host controller.

3.6.13 PORTSCx

Address: Base + 184h + (4*(Port Number - 1))

where Port Number = 1, 2, 3, ... 8

Default Value: IIII0000000000000000XX0000000000b (host mode)

IIII00000000000000001XX0000000100b (device mode)

I = Implementation dependent

X = Unknown

Attribute: R0, Read/Write, R/WC (field dependent)

Size: 32 bits

3.6.13.1 Device Controller

A device controller must implement only port register one and it does not support power control. Port control in device mode is only used for status port reset, suspend, and current connect status. It is also used to initiate test mode or force signaling and allows software to put the PHY into low power suspend mode and disable the PHY clock.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PTS	STS	PTW	PSPD	R	PFSC	PHCD	WKOC	WKDS	WKCN	PTC[3:0]				PIC	PO	PP	LS	HS P	PR	SU SP	FP R	OC C	OC A	PE C	PE	CS C	CC S				

Figure 33 PORTSCx - Port Status Control[1:8]

The register fields are described in the following table.

Field	PORTSCx Description
PTS	Parallel Transceiver Select – Read/Write. This register bit pair is used in conjunction with the configuration constant VUSB_HS_PHY_TYPE to control which parallel transceiver interface is selected. If VUSB_HS_PHY_TYPE is set for 0, 1, 2, or 3 then this bit is read only. If VUSB_HS_PHY_TYPE is 4, 5, 6, or 7 then this bit is read/write. This field is reset to: “00” if VUSB_HS_PHY_TYPE = 0, 4 – UTMI/UTMI+ “01” if VUSB_HS_PHY_TYPE = 1, 5 – Reserved “10” if VUSB_HS_PHY_TYPE = 2, 6 – ULPI “11” if VUSB_HS_PHY_TYPE = 3, 7 – Serial/1.1 PHY (FS Only)
STS	This bit is not defined in the EHCI specification. Serial Transceiver Select – Read/Write. This register bit is used in conjunction with the configuration constant VUSB_HS_PHY_SERIAL to control whether the parallel or serial transceiver interface is selected for FS and LS operation. The Serial Interface Engine can be used in combination with the UTMI+ or ULPI physical interface to provide FS/LS signaling instead of the parallel interface. If VUSB_HS_PHY_SERIAL is set for 0 or 1 then this bit is read only. If VUSB_HS_PHY_SERIAL is 3 or 4 then this bit is read/write. This bit has no effect unless Parallel Transceiver Select is set to UTMI+ or ULPI. The Serial/1.1 physical interface will use the Serial Interface Engine for FS/LS signaling regardless of this bit value. Note: This bit is reserved for future operation and is a placeholder adding dynamic use of the serial engine in accord with UMTI+ and ULPI characterization logic. This bit is not defined in the EHCI specification.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 56-191

Field	PORTSCx Description
PTW	Parallel Transceiver Width – Read/Write. This register bit is used in conjunction with the configuration constant VUSB_HS_PHY16_8 to control whether the data bus width of the UTMI transceiver interface. If VUSB_HS_PHY16_8 is set for 0 or 1 then this bit is read only. If VUSB_HS_PHY16_8 is 2 or 3 then this bit is read/write. This bit is reset to 1 if VUSB_HS_PHY16_8 selects a default UTMI interface width of 16-bits else it is reset to 0. Writing this bit to 0 selects the 8-bit [60MHz] UTMI interface. Writing this bit to 1 selects the 16-bit [30MHz] UTMI interface. This bit has no effect if the Serial interface is selected. This bit is not defined in the EHCI specification.
PSPD	Port Speed – Read Only. This register field indicates the speed at which the port is operating. For HS mode operation in the host controller and HS/FS operation in the device controller the port routing steers data to the Protocol engine. For FS and LS mode operation in the host controller, the port routing steers data to the Protocol Engine w/ Embedded Transaction Translator. 00 – Full Speed 01 – Low Speed 10 – High Speed This bit is not defined in the EHCI specification.
PFSC	Port Force Full Speed Connect – Read/Write. Default = 0b. Writing this bit to a 1b will force the port to only connect at Full Speed. It disables the chirp sequence that allows the port to identify itself as High Speed. This is useful for testing FS configurations with a HS host, hub or device. This bit is not defined in the EHCI specification. This bit is for debugging purposes.
PHCD	PHY Low Power Suspend - Clock Disable (PLPSCD) – Read/Write. Default = 0b. Writing this bit to a 1b will disable the PHY clock. Writing a 0b enables it. Reading this bit will indicate the status of the PHY clock. NOTE: The PHY clock cannot be disabled if it is being used as the system clock. In device mode, The PHY can be put into Low Power Suspend – Clock Disable when the device is not running (USBCMD Run/Stop=0b) or the host has signaled suspend (PORTSC SUSPEND=1b). Low power suspend will be cleared automatically when the host has signaled resume if using a circuit similar to that in 7. Before forcing a resume from the device, the device controller driver must clear this bit. In host mode, the PHY can be put into Low Power Suspend – Clock Disable when the downstream device has been put into suspend mode or when no downstream device is connected. Low power suspend is completely under the control of software. See 7 System Level Issues and Core Configuration for more discussion on clock disable and power down issues. This bit is not defined in the EHCI specification.
WKOC	Wake on Over-current Enable (WKOC_E) — Read/Write.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 57-191

Field	PORTSCx Description																				
	Default = 0b. Writing this bit to a one enables the port to be sensitive to over-current conditions as wake-up events. This field is zero if <i>Port Power(PP)</i> is zero. This bit is output from the controller as signal pwrctl_wake_ovrcurr_en (OTG/host core only) for use by an external power control circuit.																				
WKDC	Wake on Disconnect Enable (WKDSCNNT_E) — Read/Write. Default=0b. Writing this bit to a one enables the port to be sensitive to device disconnects as wake-up events. This field is zero if <i>Port Power(PP)</i> is zero or in device mode. This bit is output from the controller as signal pwrctl_wake_dscnnt_en (OTG/host core only) for use by an external power control circuit.																				
WKN	Wake on Connect Enable (WKCNNT_E) — Read/Write. Default=0b. Writing this bit to a one enables the port to be sensitive to device connects as wake-up events. This field is zero if <i>Port Power(PP)</i> is zero or in device mode. This bit is output from the controller as signal pwrctl_wake_dscnnt_en (OTG/host core only) for use by an external power control circuit.																				
PTC[3:0]	Port Test Control — Read/Write. Default = 0000b. Any other value than zero indicates that the port is operating in test mode. <table> <tr> <th>Value</th><th>Specific Test</th></tr> <tr> <td>0000b</td><td>TEST_MODE_DISABLE</td></tr> <tr> <td>0001b</td><td>J_STATE</td></tr> <tr> <td>0010b</td><td>K_STATE</td></tr> <tr> <td>0011b</td><td>SE0 (host) / NAK (device)</td></tr> <tr> <td>0100b</td><td>Packet</td></tr> <tr> <td>0101b</td><td>FORCE_ENABLE_HS</td></tr> <tr> <td>0110b</td><td>FORCE_ENABLE_FS</td></tr> <tr> <td>0111b</td><td>FORCE_ENABLE_LS</td></tr> <tr> <td>1000b to 1111b</td><td>Reserved</td></tr> </table> Refer to Chapter 7 of the USB Specification Revision 2.0 10 for details on each test mode. The FORCE_ENABLE_FS and FORCE_ENABLE_LS are extensions to the test mode support specified in the EHCI specification. Writing the PTC field to any of the FORCE_ENABLE_{HS/FS/LS} values will force the port into the connected and enabled state at the selected speed. Writing the PTC field back to TEST_MODE_DISABLE will allow the port state machines to progress normally from that point. <i>Note: Low speed operations are not supported as a peripheral device.</i>	Value	Specific Test	0000b	TEST_MODE_DISABLE	0001b	J_STATE	0010b	K_STATE	0011b	SE0 (host) / NAK (device)	0100b	Packet	0101b	FORCE_ENABLE_HS	0110b	FORCE_ENABLE_FS	0111b	FORCE_ENABLE_LS	1000b to 1111b	Reserved
Value	Specific Test																				
0000b	TEST_MODE_DISABLE																				
0001b	J_STATE																				
0010b	K_STATE																				
0011b	SE0 (host) / NAK (device)																				
0100b	Packet																				
0101b	FORCE_ENABLE_HS																				
0110b	FORCE_ENABLE_FS																				
0111b	FORCE_ENABLE_LS																				
1000b to 1111b	Reserved																				
PIC[1:0]	Port Indicator Control — Read/Write. Default = 0b. Writing to this field has no effect if the P_INDICATOR bit in the HCSPARAMS register is a zero. If P_INDICATOR bit is a one, then the bit is: <table> <tr> <th>Bit Value</th><th>Meaning</th></tr> <tr> <td>00b</td><td>Port indicators are off</td></tr> <tr> <td>01b</td><td>Amber</td></tr> <tr> <td>10b</td><td>Green</td></tr> <tr> <td>11b</td><td>Undefined</td></tr> </table>	Bit Value	Meaning	00b	Port indicators are off	01b	Amber	10b	Green	11b	Undefined										
Bit Value	Meaning																				
00b	Port indicators are off																				
01b	Amber																				
10b	Green																				
11b	Undefined																				

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 58-191

Field	PORTSCx Description										
	Refer to the USB Specification Revision 2.0 10 for a description on how these bits are to be used. This field is output from the controller as signals port_ind_ctl_1 & port_ind_ctl_0 for use by an external led driving circuit.										
PO	Port Owner—Read Only. Port owner handoff is not implemented in this design, therefore this bit will always read back as 0. The EHCI definition is include here for reference: Default = 0. This bit unconditionally goes to a 0 when the configured bit in the CONFIGFLAG register makes a 0 to 1 transition. This bit unconditionally goes to 1 whenever the Configured bit is zero System software uses this field to release ownership of the port to a selected host controller (in the event that the attached device is not a high-speed device). Software writes a one to this bit when the attached device is not a high-speed device. A one in this bit means that an internal companion controller owns and controls the port.										
PP	Port Power (PP)—Read/Write or Read Only. The function of this bit depends on the value of the Port Power Switching (PPC) field in the HCSPARAMS register. The behavior is as follows: <table> <tr> <td>PPC</td><td>PP Operation</td></tr> <tr> <td>0b</td><td>0b Read Only— A device controller with no OTG capability does not have port power control switches.</td></tr> <tr> <td>1b</td><td>1b/0b – RW. Host/OTG controller requires port power control switches. This bit represents the current setting of the switch (0=off, 1=on). When power is not available on a port (i.e. PP equals a 0), the port is non-functional and will not report attaches, detaches, etc.</td></tr> </table> When an over-current condition is detected on a powered port and PPC is a one, the PP bit in each affected port may be transitioned by the host controller driver from a one to a zero (removing power from the port). This feature is implemented in the host/OTG controller (PPC = 1). In a device only implementation port power control is not necessary, thus PPC and PP = 0.	PPC	PP Operation	0b	0b Read Only— A device controller with no OTG capability does not have port power control switches.	1b	1b/0b – RW. Host/OTG controller requires port power control switches. This bit represents the current setting of the switch (0=off, 1=on). When power is not available on a port (i.e. PP equals a 0), the port is non-functional and will not report attaches, detaches, etc.				
PPC	PP Operation										
0b	0b Read Only— A device controller with no OTG capability does not have port power control switches.										
1b	1b/0b – RW. Host/OTG controller requires port power control switches. This bit represents the current setting of the switch (0=off, 1=on). When power is not available on a port (i.e. PP equals a 0), the port is non-functional and will not report attaches, detaches, etc.										
LS[1:0]	Line Status—Read Only. These bits reflect the current logical levels of the D+ (bit 11) and D- (bit 10) signal lines. The encoding of the bits are: <table> <tr> <td>Bits [11:10]</td><td>Meaning</td></tr> <tr> <td>00b</td><td>SE0</td></tr> <tr> <td>10b</td><td>J-state</td></tr> <tr> <td>01b</td><td>K-state</td></tr> <tr> <td>11b</td><td>Undefined</td></tr> </table> In host mode, the use of linestate by the host controller driver is not necessary (unlike EHCI), because the port controller state machine and the port routing manage the connection of LS and FS. In device mode, the use of linestate by the device controller driver is not necessary.	Bits [11:10]	Meaning	00b	SE0	10b	J-state	01b	K-state	11b	Undefined
Bits [11:10]	Meaning										
00b	SE0										
10b	J-state										
01b	K-state										
11b	Undefined										
HSP	High-Speed Port — Read Only. Default = 0b. When the bit is one, the host/device connected to the port is in high-speed mode and if set to										

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 59-191

Field	PORTSCx Description								
	zero, the host/device connected to the port is not in a high-speed mode. Note: HSP is redundant with PSPD(27:26) but will remain in the design for compatibility. This bit is not defined in the EHCI specification.								
PR	Port Reset This field is zero if <i>Port Power(PP)</i> is zero. In Host Mode: Read/Write. 1=Port is in Reset. 0=Port is not in Reset. Default 0. When software writes a one to this bit the bus-reset sequence as defined in the USB Specification Revision 2.0 is started. <i>This bit will automatically change to zero after the reset sequence is complete. This behavior is different from EHCI where the host controller driver is required to set this bit to a zero after the reset duration is timed in the driver.</i> In Device Mode: This bit is a read only status bit. Device reset from the USB bus is also indicated in the USBSTS register.								
SUSP	Suspend In Host Mode: Read/Write. 1=Port in suspend state. 0=Port not in suspend state. Default=0. Port Enabled Bit and Suspend bit of this register define the port states as follows: <table> <tr> <td>Bits [Port Enabled, Suspend]</td><td>Port State</td></tr> <tr> <td>0x</td><td>Disable</td></tr> <tr> <td>10</td><td>Enable</td></tr> <tr> <td>11</td><td>Suspend</td></tr> </table> When in suspend state, downstream propagation of data is blocked on this port, except for port reset. The blocking occurs at the end of the current transaction if a transaction was in progress when this bit was written to 1. In the suspend state, the port is sensitive to resume detection. Note that the bit status does not change until the port is suspended and that there may be a delay in suspending a port if there is a transaction currently in progress on the USB. The host controller will unconditionally set this bit to zero when software sets the <i>Force Port Resume</i> bit to zero. The host controller ignores a write of zero to this bit. If host software sets this bit to a one when the port is not enabled (i.e. <i>Port enabled</i> bit is a zero) the results are undefined. This field is zero if <i>Port Power(PP)</i> is zero in host mode. In Device Mode: Read Only. 1=Port in suspend state. 0=Port not in suspend state. Default=0. In device mode this bit is a read only status bit.	Bits [Port Enabled, Suspend]	Port State	0x	Disable	10	Enable	11	Suspend
Bits [Port Enabled, Suspend]	Port State								
0x	Disable								
10	Enable								
11	Suspend								
FPR	Force Port Resume —Read/Write. 1= Resume detected/driven on port. 0=No resume (K-state) detected/driven on port. Default = 0. In Host Mode: Software sets this bit to one to drive resume signaling. The Host Controller sets this bit to one if a J-to-K transition is detected while the port is in the Suspend state. When this bit transitions to a one because a J-to-K transition is detected, the <i>Port Change Detect</i> bit in the USBSTS register is also set to one. <i>This bit will automatically change to zero after the resume sequence is complete. This behavior is different from EHCI where the host controller driver is required to set this bit to a zero after the resume duration is timed in the driver.</i>								

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 60-191

PORTSCx	
Field	Description
	Note that when the Host controller owns the port, the resume sequence follows the defined sequence documented in the USB Specification Revision 2.0. The resume signaling (Full-speed 'K') is driven on the port as long as this bit remains a one. This bit will remain a one until the port has switched to the high-speed idle. Writing a zero has no affect because the port controller will time the resume operation clear the bit the port control state switches to HS or FS idle. This field is zero if <i>Port Power(PP)</i> is zero in host mode. This bit is not-EHCI compatible. In Device mode: After the device has been in Suspend State for 5ms or more, software must set this bit to one to drive resume signaling before clearing. The Device Controller will set this bit to one if a J-to-K transition is detected while the port is in the Suspend state. The bit will be cleared when the device returns to normal operation. Also, when this bit transitions to a one because a J-to-K transition detected, the <i>Port Change Detect</i> bit in the USBSTS register is also set to one.
OCC	Over-current Change—R/WC. Default=0. 1=This bit gets set to one when there is a change to Over-current Active. Software clears this bit by writing a one to this bit position. For host/OTG implementations the user can provide over-current detection to the vbus_pwr_fault input for this condition. For device-only implementations this bit shall always be 0.
OCA	Over-current Active—Read Only. Default 0. 1=This port currently has an over-current condition. 0=This port does not have an over-current condition. This bit will automatically transition from one to zero when the over current condition is removed. For host/OTG implementations the user can provide over-current detection to the vbus_pwr_fault input for this condition. For device-only implementations this bit shall always be 0.
PEC	Port Enable/Disable Change—R/WC. 1=Port enabled/disabled status has changed. 0=No change. Default = 0. In Host Mode: For the root hub, this bit gets set to a one only when a port is disabled due to disconnect on the port or due to the appropriate conditions existing at the EOF2 point (See Chapter 11 of the USB Specification). Software clears this by writing a one to it. This field is zero if <i>Port Power(PP)</i> is zero. In Device mode: The device port is always enabled. (This bit will be zero)
PE	Port Enabled/Disabled—Read/Write. 1=Enable. 0=Disable. Default 0. In Host Mode: Ports can only be enabled by the host controller as a part of the reset and enable. Software cannot enable a port by writing a one to this field. Ports can be disabled by either a fault condition (disconnect event or other fault condition) or by the host software. Note that the bit status does not change until the port state actually changes. There may be a delay in disabling or enabling a port due to other host controller and bus events.

		Title: CII3410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 61-191

Field	PORTSCx Description
	When the port is disabled, (0b) downstream propagation of data is blocked except for reset. This field is zero if <i>Port Power(PP)</i> is zero in host mode. In Device Mode: The device port is always enabled. (This bit will be one)
CSC	Connect Status Change—R/WC. 1 =Change in Current Connect Status. 0=No change. Default 0. In Host Mode: Indicates a change has occurred in the port's Current Connect Status. The host/device controller sets this bit for all changes to the port device connect status, even if system software has not cleared an existing connect status change. For example, the insertion status changes twice before system software has cleared the changed condition, hub hardware will be 'setting' an already-set bit (i.e., the bit will remain set). Software clears this bit by writing a one to it. This field is zero if <i>Port Power(PP)</i> is zero in host mode. In Device Mode: This bit is undefined in device controller mode.
CCS	Current Connect Status—Read Only. In Host Mode: 1=Device is present on port. 0=No device is present. Default = 0. This value reflects the current state of the port, and may not correspond directly to the event that caused the <i>Connect Status Change</i> bit (Bit 1) to be set. This field is zero if <i>Port Power(PP)</i> is zero in host mode. In Device Mode: 1=Attached. 0=Not Attached. Default=0. A one indicates that the device successfully attached and is operating in either high speed or full speed as indicated by the High Speed Port bit in this register. A zero indicates that the device did not attach successfully or was forcibly disconnected by the software writing a zero to the Run bit in the USBCMD register. It does not state the device being disconnected or suspended.

3.6.14 USBMODE

Address: Base + 1A8h

Default Value: 00000000h (otg implementation – mode not selected)
00000003h (host mode)
00000002h (device mode))

Attribute: R/WO, Read Only

Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																								VB PS	SDI S	SL OM	ES	CM			

Figure 34 USBMODE -USB Device Mode

The register fields are described in the following table.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 62-191

Field	USBMODE Description								
reserved	Reserved. These bits are reserved and should be set to zero.								
VBPS	Vbus Power Select (0 – Output is 0; 1 – Output is one) This bit is connected to the vbus_pwr_select output and can be used for any generic control but is named to be used by logic that selects between an on-chip Vbus power source (charge pump) and an off-chip source in systems when both are available.								
SDIS	Stream Disable Mode. (0 – Inactive [default] ; 1 – Active) Device Mode: Setting to a ‘1’ disables double priming on both RX and TX for low bandwidth systems. This mode, when enabled, ensures that the RX and TX buffers are sufficient to contain an entire packet, so the usual double buffering scheme is disabled to prevent overruns/underruns in bandwidth limited systems. Note: In High Speed Mode, all packets received will be responded to with a NYET handshake when stream disable is active. Host Mode: Setting to a ‘1’ ensures that overruns/underruns of the latency FIFO are eliminated for low bandwidth systems where the RX and TX buffers are sufficient to contain the entire packet. Enabling stream disable also has the effect of ensuring the the TX latency is filled to capacity before the packet is lunched onto the USB. Note: Time duration to pre-fill the FIFO becomes significative when stream disable is active. See TXFILLTUNING and TXTTFILLTUNING [MPH Only] to characterize the adjustments needed for the scheduler when using this feature. Note: The use of this feature substantially limits of the overall USB performance that can be achieved.								
SLOM	Setup Lockout Mode. In device mode, this bit controls behavior of the setup lock mechanism. See Control Endpoint Operation Model. 0 – Setup Lockouts On (default); 1 – Setup Lockouts Off (DCD requires use of Setup Data Buffer Tripwire in USBCMD)								
ES	Endian Select – Read/Write. This bit can change the byte ordering of the transfer buffers to match the host microprocessor bus architecture. The bit fields in the microprocessor interface and the DMA data structures (including the setup buffer within the device QH) are unaffected by the value of this bit, because they are based upon 32-bit words. <table> <tr> <th>Bit</th><th>Meaning</th></tr> <tr> <td>0</td><td>Little Endian [Default] – first byte referenced in least significant byte of 32-bit word.</td></tr> <tr> <td>1</td><td>Big Endian - first byte referenced in most significant byte of 32-bit word.</td></tr> </table>	Bit	Meaning	0	Little Endian [Default] – first byte referenced in least significant byte of 32-bit word.	1	Big Endian - first byte referenced in most significant byte of 32-bit word.		
Bit	Meaning								
0	Little Endian [Default] – first byte referenced in least significant byte of 32-bit word.								
1	Big Endian - first byte referenced in most significant byte of 32-bit word.								
CM[1:0]	Controller Mode – R/WO. Controller mode is defaulted to the proper mode for host only and device only implementations. For those designs that contain both host & device capability, the controller will default to an idle state and will need to be initialized to the desired operating mode after reset. For combination host/device controllers, this register can only be written once after reset. If it is necessary to switch modes, software must reset the controller by writing to the <i>RESET</i> bit in the USBCMD register before reprogramming this register. <table> <tr> <th>Bit</th><th>Meaning</th></tr> <tr> <td>00</td><td>Idle [Default for combination host/device]</td></tr> <tr> <td>01</td><td>Reserved</td></tr> <tr> <td>10</td><td>Device Controller [Default for device only controller]</td></tr> </table>	Bit	Meaning	00	Idle [Default for combination host/device]	01	Reserved	10	Device Controller [Default for device only controller]
Bit	Meaning								
00	Idle [Default for combination host/device]								
01	Reserved								
10	Device Controller [Default for device only controller]								

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 63-191

Field	USBMODE Description
11	Host Controller [Default for host only controller]

3.6.15 ENDPTSETUPSTAT

Address: Base + 1ACh
Default Value: 00000000h
Attribute: R/WC
Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved																ENDPTSETUPSTAT[15:0]															

Figure 35 ENDPTSETUPSTAT - Endpoint Setup Status

The register fields are described in the following table.

Field	ENDPTSETUPSTAT Description
reserved	Reserved. These bits are reserved and should be set to zero.
ENDPTSETUPSTAT [15:0]	Setup Endpoint Status. For every setup transaction that is received, a corresponding bit in this register is set to one. Software must clear or acknowledge the setup transfer by writing a one to a respective bit after it has read the setup data from Queue head. The response to a setup packet as in the order of operations and total response time is crucial to limit bus time outs while the setup lock our mechanism is engaged. See Managing Endpoints in the Device Operational Model.
This register is only used in device mode.	

3.6.16 ENDPTPRIME

Address: Base + 1B0h
Default Value: 00000000h
Attribute: R/WS
Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PETB[15:0]																PERB[15:0]															

Figure 36 ENDPTPRIME - Endpoint Initialization

This register is only used in device mode. The register fields are described in the following table.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 64-191

Field	ENDPTPRIME Description
PETB [15:0]	Prime Endpoint Transmit Buffer – R/WS. For each endpoint a corresponding bit is used to request that a buffer prepared for a transmit operation in order to respond to a USB IN/INTERRUPT transaction. Software should write a one to the corresponding bit when posting a new transfer descriptor to an endpoint. Hardware will automatically use this bit to begin parsing for a new transfer descriptor from the queue head and prepare a transmit buffer. Hardware will clear this bit when the associated endpoint(s) is (are) successfully primed. Note: These bits will be momentarily set by hardware during hardware re-priming operations when a dTD is retired, and the dQH is updated. PETB[15] – Endpoint #15 PETB[1] – Endpoint #1 PETB[0] – Endpoint #0
PERB [15:0]	Prime Endpoint Receive Buffer – R/WS. For each endpoint, a corresponding bit is used to request a buffer prepare for a receive operation for when a USB host initiates a USB OUT transaction. Software should write a one to the corresponding bit whenever posting a new transfer descriptor to an endpoint. Hardware will automatically use this bit to begin parsing for a new transfer descriptor from the queue head and prepare a receive buffer. Hardware will clear this bit when the associated endpoint(s) is (are) successfully primed. Note: These bits will be momentarily set by hardware during hardware re-priming operations when a dTD is retired, and the dQH is updated. Bit 15 – Endpoint #15 Bit 1 – Endpoint #1 Bit 0 – Endpoint #0

3.6.17 ENDPTFLUSH

Address: Base + 1B4h
Default Value: 00000000h
Attribute: R/WS
Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FETB[15:0]																FERB[15:0]															

Figure 37 ENDPTFLUSH - Endpoint De-Initialize

This register is only used in device mode. The register fields are described in the following table.

Field	ENDPTFLUSH Description
FETB [15:0]	Flush Endpoint Transmit Buffer – R/WS. Writing a one to a bit(s) in this register will cause the associated endpoint(s) to clear any primed buffers. If a packet is in progress for one of the associated endpoints, then that transfer will continue until completion. Hardware will clear this register after the endpoint flush operation is successful. FETB[15] – Endpoint #15 FETB[1] – Endpoint #1 FETB[0] – Endpoint #0

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 65-191

Field	ENDPTFLUSH Description
FERB [15:0]	Flush Endpoint Receive Buffer – R/WS. Writing a one to a bit(s) will cause the associated endpoint(s) to clear any primed buffers. If a packet is in progress for one of the associated endpoints, then that transfer will continue until completion. Hardware will clear this register after the endpoint flush operation is successful. Bit 15 – Endpoint #15 Bit 1 – Endpoint #1 Bit 0 – Endpoint #0

3.6.18 ENDPTSTAT

Address: Base + 1B8h
Default Value: 00000000h
Attribute: Read Only
Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETBR[15:0]																ERBR[15:0]															

Figure 38 ENDPTSTAT - Endpoint Status

This register is only used in device mode. The register fields are described in the following table.

Field	ENDPTSTAT Description
ETBR [15:0]	Endpoint Transmit Buffer Ready -- Read Only. One bit for each endpoint indicates status of the respective endpoint buffer. This bit is set to a one by the hardware as a response to receiving a command from a corresponding bit in the ENDPTPRIME register. There will always be a delay between setting a bit in the ENDPTPRIME register and endpoint indicating ready. This delay time varies based upon the current USB traffic and the number of bits set in the ENDPTPRIME register. Buffer ready is cleared by USB reset, by the USB DMA system, or through the ENDPTFLUSH register. Note: These bits will be momentarily cleared by hardware during hardware endpoint re-priming operations when a dTD is retired, and the dQH is updated. ETBR[15] – Endpoint #15 ETBR[1] – Endpoint #1 ETBR[0] – Endpoint #0
ERBR [15:0]	Endpoint Receive Buffer Ready -- Read Only. One bit for each endpoint indicates status of the respective endpoint buffer. This bit is set to a one by the hardware as a response to receiving a command from a corresponding bit in the ENDPTPRIME register. There will always be a delay between setting a bit in the ENDPTPRIME register and endpoint indicating ready. This delay time varies based upon the current USB traffic and the number of bits set in the ENDPTPRIME register. Buffer ready is cleared by USB reset, by the USB DMA system, or through the ENDPTFLUSH register. Note: These bits will be momentarily cleared by hardware during hardware endpoint re-priming operations when a dTD is retired, and the dQH is updated. ERBR[15] – Endpoint #15 ERBR[1] – Endpoint #1 ERBR[0] – Endpoint #0

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 66-191

3.6.19 ENDPTCOMPLETE

Address: Base + 1BCh

Default Value: 00000000h

Attribute: R/WC

Size: 32 bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETCE[15:0]																ERCE[15:0]															

Figure 39 ENDPTCOMPLETE - Endpoint Compete

This register is only used in device mode. The register fields are described in the following table.

Field	ENDPTCOMPLETE Description
ETCE [15:0]	Endpoint Transmit Complete Event – R/WC. Each bit indicates a transmit event (IN/INTERRUPT) occurred and software should read the corresponding endpoint queue to determine the endpoint status. If the corresponding IOC bit is set in the Transfer Descriptor, then this bit will be set simultaneously with the <i>USBINT</i> . Writing a one will clear the corresponding bit in this register. ETCE[15] – Endpoint #15 ETCE[1] – Endpoint #1 ETCE[0] – Endpoint #0
ERCE [15:0]	Endpoint Receive Complete Event – RW/C. Each bit indicates a received event (OUT/SETUP) occurred and software should read the corresponding endpoint queue to determine the transfer status. If the corresponding IOC bit is set in the Transfer Descriptor, then this bit will be set simultaneously with the <i>USBINT</i> . Writing a one will clear the corresponding bit in this register. ERCE[15] – Endpoint #15 ERCE[1] – Endpoint #1 ERCE[0] – Endpoint #0

3.6.20 ENDPTCTRL0

Address: Base + 1C0h

Default Value: 0080008h

Attribute: Read Only, Read/Write, R/WC (field dependent)

Size: 32 bits

Every Device will implement Endpoint0 as a control endpoint.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved								TX E	reserved				TXT	R	TX S	reserved								RX E	reserved				RXT	R	RXS

Figure 40 ENDPTCTRL0 – Endpoint Control 0

The register fields are described in the following table.

Field	ENDPTCTRL0 Description
reserved	Reserved. These bits are reserved and should be set to zero.
TXE	TX Endpoint Enable 1 – Enabled Endpoint0 is always enabled.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 67-191

Field	ENDPTCTRL0 Description
TXT[1:0]	TX Endpoint Type – Read/Write 00 – Control Endpoint0 is fixed as a Control End Point.
R	Reserved. Bit reserved and should be set to zero.
TXS	TX Endpoint Stall – Read/Write 0 – End Point OK [Default] 1 – End Point Stalled Software can write a one to this bit to force the endpoint to return a STALL handshake to the Host. It will continue returning STALL until the bit is cleared by software or it will automatically be cleared upon receipt of a new SETUP request. After receiving a SETUP request, this bit will continue to be cleared by hardware until the associated ENDPTSETUPSTAT bit is cleared. NOTE: THERE IS A SLIGHT DELAY (50 CLOCKS MAX.) BETWEEN THE ENDPTSETUPSTAT BEING CLEARED AND HARDWARE CONTINUING TO CLEAR THIS BIT. IN MOST SYSTEMS IT IS UNLIKELY THE DCD SOFTWARE WILL OBSERVE THIS DELAY. HOWEVER, SHOULD THE DCD OBSERVE THAT THE STALL BIT IS NOT SET AFTER WRITING A ONE TO IT THEN FOLLOW THIS PROCEDURE: CONTINUALLY WRITE THIS STALL BIT UNTIL IT IS SET OR UNTIL A NEW SETUP HAS BEEN RECEIVED BY CHECKING THE ASSOCIATED ENDPTSETUPSTAT BIT.
RXE	RX Endpoint Enable 1 – Enabled Endpoint0 is always enabled.
RXT[1:0]	RX Endpoint Type – Read/Write 00 – Control Endpoint0 is fixed as a Control End Point.
RXS	RX Endpoint Stall – Read/Write 0 – End Point OK. [Default] 1 – End Point Stalled Software can write a one to this bit to force the endpoint to return a STALL handshake to the Host. It will continue returning STALL until the bit is cleared by software or it will automatically be cleared upon receipt of a new SETUP request. After receiving a SETUP request, this bit will continue to be cleared by hardware until the associated ENDPTSETUPSTAT bit is cleared. NOTE: THERE IS A SLIGHT DELAY (50 CLOCKS MAX.) BETWEEN THE ENDPTSETUPSTAT BEING CLEARED AND HARDWARE CONTINUING TO CLEAR THIS BIT. IN MOST SYSTEMS IT IS UNLIKELY THE DCD SOFTWARE WILL OBSERVE THIS DELAY. HOWEVER, SHOULD THE DCD OBSERVE THAT THE STALL BIT IS NOT SET AFTER WRITING A ONE TO IT THEN FOLLOW THIS PROCEDURE: CONTINUALLY WRITE THIS STALL BIT UNTIL IT IS SET OR UNTIL A NEW SETUP HAS BEEN RECEIVED BY CHECKING THE ASSOCIATED ENDPTSETUPSTAT BIT.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 68-191

3.6.21 ENDPTCTRL1 – ENDPTCTRL15

Address: Base + 1C0h+(4*(EndPoint Number))

Default Value: 00000000h

Attribute: Read/Write

Size: 32 bits

There is an ENDPTCTRLx register for each endpoint in a device.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved								TX E	TX R	TX I	R	TXT		TX D	TX S	reserved								RX E	RX R	RX I	R	RXT		RX D	RXS

Figure 41 ENDPTCTRL1 – ENDPTCTRL15 – Endpoint Control 1 to 15

CAUTION: IF ONE ENDPOINT DIRECTION IS ENABLED AND THE PAIRED ENDPOINT OF OPPOSITE DIRECTION IS DISABLED THEN THE UNUSED DIRECTION TYPE MUST BE CHANGED FROM THE DEFAULT CONTROL-TYPE TO ANY OTHER TYPE (IE. BULK-TYPE). LEAVING AN UNCONFIGURED ENDPOINT CONTROL WILL CAUSE UNDEFINED BEHAVIOR FOR THE DATA PID TRACKING ON THE ACTIVE ENDPOINT/DIRECTION.

The register fields are described in the following table.

ENDPTCTRL1 – ENDPTCTRL15	
Field	Description
reserved	Reserved. These bits are reserved and should be set to zero.
TXE	TX Endpoint Enable 0 – Disabled [Default] 1 – Enabled An Endpoint should be enabled only after it has been configured.
TXR	TX Data Toggle Reset (WS) Write 1 – Reset PID Sequence Whenever a configuration event is received for this Endpoint, software must write a one to this bit in order to synchronize the data PID's between the Host and device.
TXI	TX Data Toggle Inhibit 0 – PID Sequencing Enabled. [Default] 1 – PID Sequencing Disabled. This bit is only used for test and should always be written as zero. Writing a one to this bit will cause this endpoint to ignore the data toggle sequence and always transmit DATA0 for a data packet.
R	Reserved. Bit reserved and should be set to zero.
TXT[1:0]	TX Endpoint Type – Read/Write 00 – Control 01 – Isochronous 10 – Bulk 11 – Interrupt
TXD	TX Endpoint Data Source – Read/Write 0 – Dual Port Memory Buffer/DMA Engine [DEFAULT] Should always be written as 0.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 69-191

Field	ENDPTCTRL1 – ENDPTCTRL15 Description
TXS	TX Endpoint Stall – Read/Write 0 – End Point OK 1 – End Point Stalled This bit will be cleared automatically upon receipt of a SETUP request if this Endpoint is configured as a Control Endpoint and this bit will continue to be cleared by hardware until the associated ENDPTSETUPSTAT bit is cleared. Software can write a one to this bit to force the endpoint to return a STALL handshake to the Host. This control will continue to STALL until this bit is either cleared by software or automatically cleared as above for control endpoints. NOTE [CONTROL ENDPOINT TYPES ONLY]: THERE IS A SLIGHT DELAY (50 CLOCKS MAX.) BETWEEN THE ENDPTSETUPSTAT BEING CLEARED AND HARDWARE CONTINUING TO CLEAR THIS BIT. IN MOST SYSTEMS IT IS UNLIKELY THE DCD SOFTWARE WILL OBSERVE THIS DELAY. HOWEVER, SHOULD THE DCD OBSERVE THAT THE STALL BIT IS NOT SET AFTER WRITING A ONE TO IT THEN FOLLOW THIS PROCEDURE: CONTINUALLY WRITE THIS STALL BIT UNTIL IT IS SET OR UNTIL A NEW SETUP HAS BEEN RECEIVED BY CHECKING THE ASSOCIATED ENDPTSETUPSTAT BIT.
RXE	RX Endpoint Enable 0 – Disabled [Default] 1 – Enabled An Endpoint should be enabled only after it has been configured.
RXR	RX Data Toggle Reset (WS) Write 1 – Reset PID Sequence Whenever a configuration event is received for this Endpoint, software must write a one to this bit in order to synchronize the data PID's between the host and device.
RXI	RX Data Toggle Inhibit 0 – Disabled [Default] 1 – Enabled This bit is only used for test and should always be written as zero. Writing a one to this bit will cause this endpoint to ignore the data toggle sequence and always accept data packet regardless of their data PID.
RXT[1:0]	RX Endpoint Type – Read/Write 00 – Control 01 – Isochronous 10 – Bulk 11 – Interrupt
RXD	RX Endpoint Data Sink – Read/Write – TBD 0 – Dual Port Memory Buffer/DMA Engine [Default] Should always be written as zero.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 70-191

ENDPTCTRL1 – ENDPTCTRL15	
Field	Description
RXS	RX Endpoint Stall – Read/Write 0 – End Point OK. [Default] 1 – End Point Stalled This bit will be cleared automatically upon receipt of a SETUP request if this Endpoint is configured as a Control Endpoint and this bit will continue to be cleared by hardware until the associated ENDPTSETUPSTAT bit is cleared. Software can write a one to this bit to force the endpoint to return a STALL handshake to the Host. This control will continue to STALL until this bit is either cleared by software or automatically cleared as above for control endpoints. NOTE [CONTROL ENDPOINT TYPES ONLY]: THERE IS A SLIGHT DELAY (50 CLOCKS MAX.) BETWEEN THE ENDPTSETUPSTAT BEING CLEARED AND HARDWARE CONTINUING TO CLEAR THIS BIT. IN MOST SYSTEMS IT IS UNLIKELY THE DCD SOFTWARE WILL OBSERVE THIS DELAY. HOWEVER, SHOULD THE DCD OBSERVE THAT THE STALL BIT IS NOT SET AFTER WRITING A ONE TO IT THEN FOLLOW THIS PROCEDURE: CONTINUALLY WRITE THIS STALL BIT UNTIL IT IS SET OR UNTIL A NEW SETUP HAS BEEN RECEIVED BY CHECKING THE ASSOCIATED ENDPTSETUPSTAT BIT.

		<i>Title:</i> CI13410-a High-Speed USB Device Controller Core				<i>Revision:</i> 1.0A
<i>Template:</i> TP02_DEP_GP	<i>Department:</i> IPCS	<i>Prepared by:</i> Joana Braga	<i>Approved by:</i> Luis Miguel Sampaio	<i>Security Level:</i> confidential	<i>Date:</i> 2006-05-11	<i>Page:</i> 71-191

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 72-191

4 Device Data Structures

This section defines the interface data structures used to communicate control, status, and data between Device Controller Driver (DCD) Software and the Device Controller. The data structure definitions in this chapter support a 32-bit memory buffer address space. The interface consists of device Queue Heads and Transfer Descriptors.

NOTE Software must ensure that no interface data structure reachable by the Device Controller spans a 4K-page boundary.

The data structures defined in the chapter are (from the device controller’s perspective) a mix of read-only and read/ writeable fields. The device controller must preserve the read-only fields on all data structure writes.

The USB-HS core includes DCD Software called the USB 2.0 Device API. The Device API provides an easy to use Application Program Interface for developing device (peripheral) applications using the USB-HS core. The Device API incorporates and abstracts for the application developer all of the elements of the program interface.

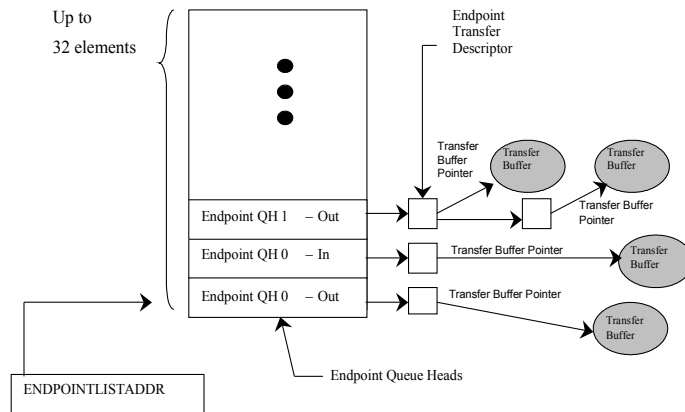


Figure 42 End Point Queue Head Organization

Device queue heads are arranged in an array in a continuous area of memory pointed to by the *ENDPOINTLISTADDR* pointer. The even –numbered device queue heads in the list support receive endpoints (OUT/SETUP) and the odd-numbered queue heads in the list are used for transmit endpoints (IN/INTERRUPT). The device controller will index into this array based upon the endpoint number received from the USB bus. All information necessary to respond to transactions for all primed transfers is contained in this list so the Device Controller can readily respond to incoming requests without having to traverse a linked list.

NOTE The Endpoint Queue Head List must be aligned to a 2k boundary.

4.1 Endpoint Queue Head (dQH)

The device Endpoint Queue Head (dQH) is where all transfers are managed. The dQH is a 48-byte data structure, but must be aligned on 64-byte boundaries. During priming of an endpoint, the dTD (device transfer descriptor) is copied into the overlay area of the dQH, which starts at the nextTD pointer DWord and continues through the end of the buffer pointers DWords. After a transfer is complete, the dTD status DWord is updated in the dTD pointed to by the currentTD pointer. While a packet is in progress, the overlay area of the dQH is used as a staging area for the dTD so that the Device Controller can access needed information with little minimal latency.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 73-191

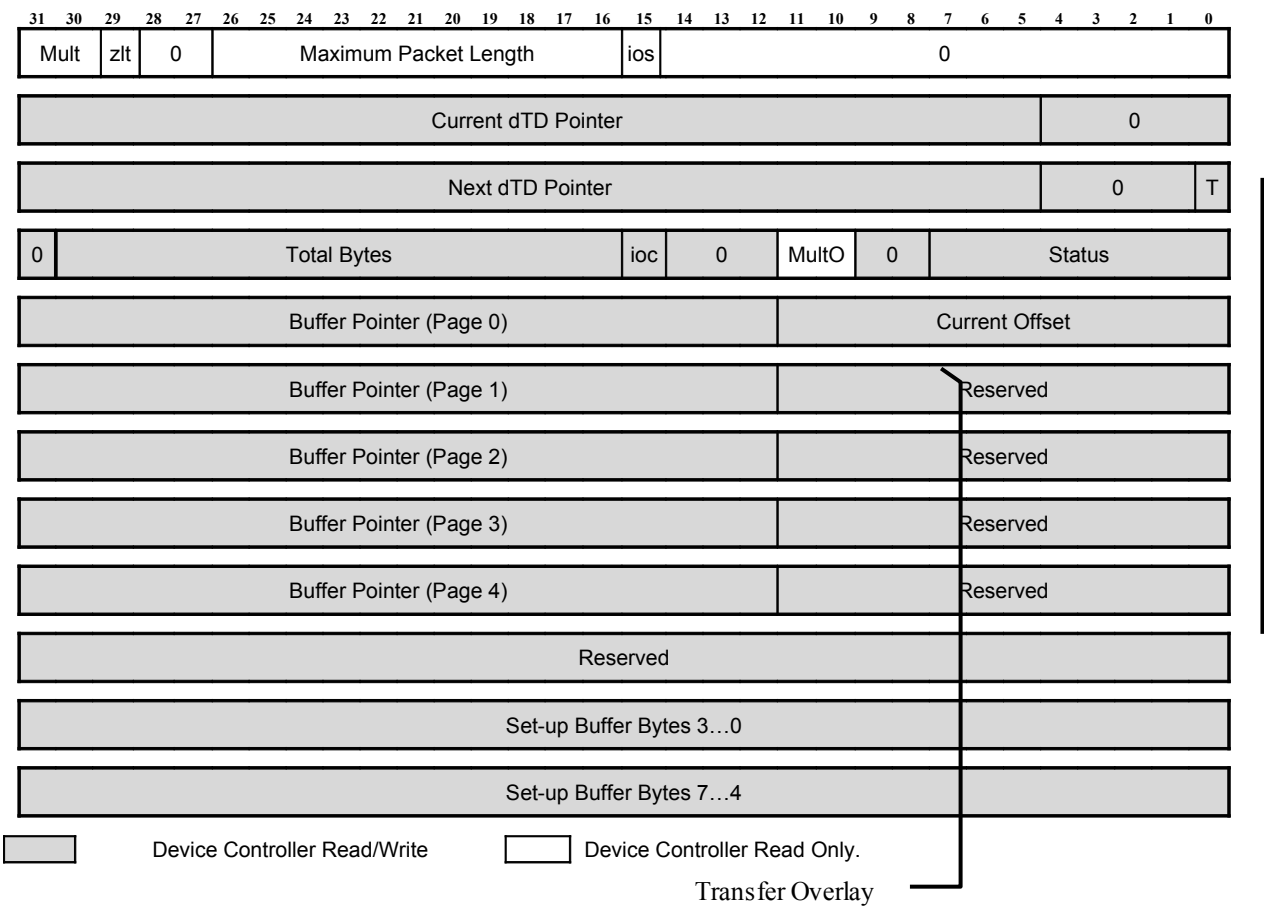


Figure 43 Endpoint Queue Head (dQH)

4.1.1 Endpoint Capabilities/Characteristics

This DWord specifies static information about the endpoint, in other words, this information does not change over the lifetime of the endpoint. Device Controller software should not attempt to modify this information while the corresponding endpoint is enabled.

Table 3 Endpoint Capabilities/Characteristics

Bit	Description
31:30	Mult. This field is used to indicate the number of packets executed per transaction description as given by the following: 00 – Execute N Transactions as demonstrated by the USB variable length packet protocol where N is computed using the Maximum Packet Length (dQH) and the Total Bytes field (dTD) 01 – Execute 1 Transaction. 10 – Execute 2 Transactions. 11 – Execute 3 Transactions. Note: Non-ISO endpoints must set Mult = "00". Note: ISO endpoints must set Mult = "01", "10", or "11" as needed.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 74-191

Bit	Description
29	Zero Length Termination Select. This bit is used to indicate when a zero length packet is used to terminate transfers where to total transfer length is a multiple . This bit is not relevant for Isochronous 0 – Enable zero length packet to terminate transfers equal to a multiple of the Maximum Packet Length. (default). 1 – Disable the zero length packet on transfers that are equal in length to a multiple Maximum Packet Length.
28:27	Reserved. These bit reserved for future use and should be set to zero.
26:16	Maximum Packet Length. This directly corresponds to the maximum packet size of the associated endpoint (wMaxPacketSize). The maximum value this field may contain is 0x400 (1024).
15	Interrupt On Setup (IOS). This bit is used on control type endpoints to indicate if USBINT is set in response to a setup being received.
14:0	Reserved. Bits reserved for future use and should be set to zero.

4.1.2 Transfer Overlay

The seven DWords in the overlay area represent a transaction working space for the device controller. The general operational model is that the device controller can detect whether the overlay area contains a description of an active transfer. If it does not contain an active transfer, then it will not read the associated endpoint.

After an endpoint is readied, the dTD will be copied into this queue head overlay area by the device controller. Until a transfer is expired, software must not write the queue head overlay area or the associated transfer descriptor. When the transfer is complete, the device controller will write the results back to the original transfer descriptor and advance the queue.

See dTD for a description of the overlay fields.

4.1.3 Current dTD Pointer

The current dTD pointer is used by the device controller to locate the transfer in progress. This word is for Device Controller (hardware) use only and should not be modified by DCD software.

Table 4 Next dTD Pointer

Bit	Description
31:5	Current dTD. This field is a pointer to the dTD that is represented in the transfer overlay area. This field will be modified by the Device Controller to next dTD pointer during endpoint priming or queue advance.
4:0	Reserved. Bit reserved for future use and should be set to zero.

4.1.4 Set-up Buffer

The set-up buffer is dedicated storage for the 8-byte data that follows a set-up PID.

NOTE Each endpoint has a TX and an RX dQH associated with it, and only the RX queue head is used for receiving setup data packets.

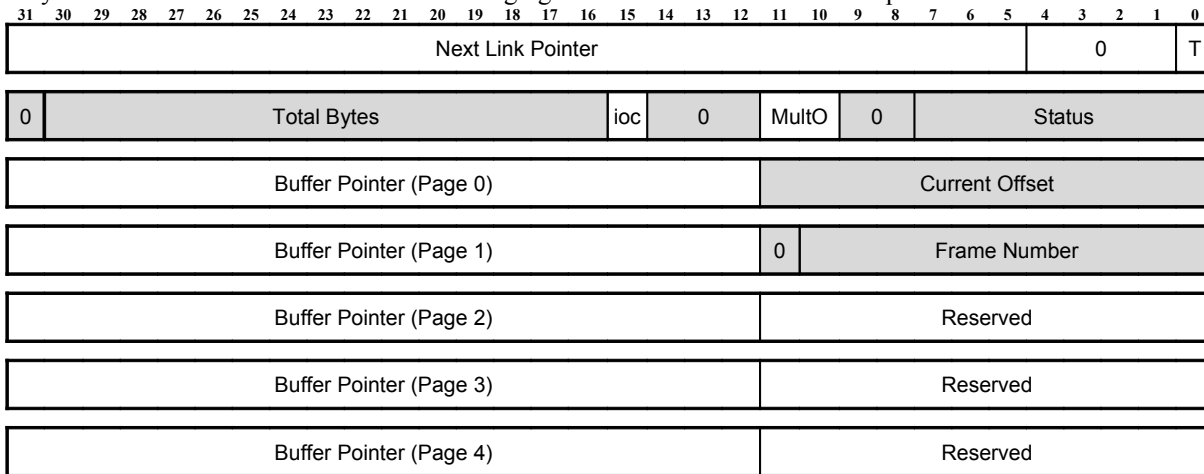
Table 5 Multiple Mode Control (HCCPARAMS)

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 75-191

DWord	Bits	Description
1	31:0	Setup Buffer 0. This buffer contains bytes 3 to 0 of an incoming setup buffer packet and is written by the device controller to be read by software.
2	31:0	Setup Buffer 1. This buffer contains bytes 7 to 4 of an incoming setup buffer packet and is written by the device controller to be read by software.

4.2 Endpoint Transfer Descriptor (dTD)

The dTD describes to the device controller the location and quantity of data to be sent/received for given transfer. The DCD should not attempt to modify any field in an active dTD except the Next Like Pointer, which should only be modified as described in section Managing Transfers with Transfer Descriptors.



 Device Controller Read/Write  Device Controller Read Only.

Figure 44 Endpoint Transfer Descriptor (dTD)

Table 6 Next dTD Pointer

Bit	Description
31:5	Next Transfer Element Pointer. This field contains the physical memory address of the next dTD to be processed. The field corresponds to memory address signals [31:5], respectively.
4:1	Reserved. Bits reserved for future use and should be set to zero.
0	Terminate (T). 1=pointer is invalid. 0=Pointer is valid (points to a valid Transfer Element Descriptor). This bit indicates to the Device Controller that there are no more valid entries in the queue.

Table 7 dTD Token

Bit	Description
31	Reserved. Bit reserved for future use and should be set to zero.
30:16	Total Bytes. This field specifies the total number of bytes to be moved with this transfer descriptor. This field is decremented by the number of bytes actually moved during the transaction and only on the successful completion of the transaction. The maximum value software may store in the field is 5*4K(5000H). This is the maximum number of bytes 5 page pointers can access. Although it is possible to create a transfer up to

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 76-191

Bit	Description												
	20K this assumes the 1st offset into the first page is 0. When the offset cannot be predetermined, crossing past the 5th page can be guaranteed by limiting the total bytes to 16K**. Therefore, the maximum recommended transfer is 16K(4000H). If the value of the field is zero when the host controller fetches this transfer descriptor (and the active bit is set), the device controller executes a zero-length transaction and retires the transfer descriptor. It is not a requirement for IN transfers that Total Bytes To Transfer be an even multiple of <i>Maximum Packet Length</i>. If software builds such a transfer descriptor for an IN transfer, the last transaction will always be less that <i>Maximum Packet Length</i>.												
15	Interrupt On Complete (IOC). This bit is used to indicate if USBINT is to be set in response to device controller being finished with this dTD.												
14:12	Reserved. Bits reserved for future use and should be set to zero.												
11:10	Multiplier Override (MultiO). This field can be used for transmit ISO's (i.e. ISO-IN) to override the multiplier in the QH. This field must be zero for all packet types that are not transmit-ISO. <u>Example:</u> if QH.multiplier = 3; Maximum packet size = 8; Total Bytes = 15; MultiO = 0 [default] Three packets are sent: {Data2(8); Data1(7); Data0(0)} if QH.multiplier = 3; Maximum packet size = 8; Total Bytes = 15; MultiO = 2 Two packets are sent: {Data1(8); Data0(7)} For maximal efficiency, software should compute MultiO = greatest integer of (Total Bytes / Max. Packet Size) except for the case when Total Bytes = 0; then MultiO should be 1. Note: Non-ISO and Non-TX endpoints must set MultiO = "00".												
9:8	Reserved. Bits reserved for future use and should be set to zero.												
7:0	Status. This field is used by the Device Controller to communicate individual command execution states back to the Device Controller software. This field contains the status of the last transaction performed on this qTD. The bit encodings are: <table> <tr> <td>Bit</td><td>Status Field Description</td></tr> <tr> <td>7</td><td>Active.</td></tr> <tr> <td>6</td><td>Halted.</td></tr> <tr> <td>5</td><td>Data Buffer Error.</td></tr> <tr> <td>3</td><td>Transaction Error.</td></tr> <tr> <td>4,2,0</td><td>Reserved.</td></tr> </table>	Bit	Status Field Description	7	Active.	6	Halted.	5	Data Buffer Error.	3	Transaction Error.	4,2,0	Reserved.
Bit	Status Field Description												
7	Active.												
6	Halted.												
5	Data Buffer Error.												
3	Transaction Error.												
4,2,0	Reserved.												

Table 8 dTD Buffer Page Pointer List

Bit	Description
31:12	Buffer Pointer. Selects the page offset in memory for the packet buffer. Non virtual memory systems will typically set the buffer pointers to a series of incrementing integers.
0;11:0	Current Offset. Offset into the 4kb buffer where the packet is to begin.

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 77-191

1;10:0 **Frame Number.** Written by the device controller to indicate the frame number in which a packet finishes. This is typically be used to correlate relative completion times of packets on an ISO endpoint.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 78-191

5 Device Operational Model

The function of the device operation is to transfer a request in the memory image to and from the Universal Serial Bus. Using a set of linked list **transfer descriptors**, pointed to by a **queue head**, the device controller will perform the data transfers. The following sections explain the use of the device controller from the device controller driver (DCD) point-of-view and further describe how specific USB bus events relate to status changes in the device controller programmer's interface.

The USB-HS is shipped with a DCD called the USB-HS Device API. The USB-HS Device API provides an easy to use application interface for developing USB device (peripheral) applications. The USB-HS Device API incorporates and abstracts for the application developer all of the information contained in the device operational model. For more information on the USB-HS Device API, refer to the "Software Design document for the USB 2.0 Device API".

5.1 Device Controller Initialization

After hardware reset, the device is disabled until the Run/Stop bit is set to a '1'. In the disabled state, the pull-up on the USB D+ is not active which prevents an attach event from occurring. At a minimum, it is necessary to have the queue heads setup for endpoint zero before the device attach occurs. Shortly after the device is enabled, a USB reset will occur followed by setup packet arriving at endpoint 0. A Queue head must be prepared so that the device controller can store the incoming setup packet.

In order to initialize a device, the software should perform the following steps:

1. Set Controller Mode in the USBMODE register to device mode.

NOTE Transitioning from host mode to device mode requires a device controller reset before modifying USBMODE.

2. Allocate and Initialize device queue heads in system memory.

— Minimum: Initialize device queue heads 0 Tx & 0 Rx.

NOTE All device queue heads associated with control endpoints must be initialized before the control endpoint is enabled. Non-Control device queue heads must be initialized before the endpoint is used and not necessarily before the endpoint is enabled.

— For information on device queue heads, refer to section Device Data Structures.

3. Configure [ENDPOINTLISTADDR](#) Pointer.

— For additional information on [ENDPOINTLISTADDR](#), refer to the register table.

4. Enable the microprocessor interrupt associated with the USB-HS core.

— Recommended: enable all device interrupts including: USBINT, USBERRINT, Port Change Detect, USB Reset Received, DCSuspend.

— For a list of available interrupts refer to the [USBINTR](#) and the [USBSTS](#) register tables.

5. Set Run/Stop bit to Run Mode.

— After the Run bit is set, a device reset will occur. The DCD must monitor the reset event and adjust the software state as described in the Bus Reset section of the following Port State and Control section below.

NOTE Endpoint 0 is designed as a control endpoint only and does not need to be configured using [ENDPTCTRL0](#) register.

It is also not necessary to initially prime Endpoint 0 because the first packet received will always be a setup packet. The contents of the first setup packet will require a response in accordance with USB device framework (Chapter 9) command set.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 79-191

5.2 Port State and Control

From a chip or system reset, the device controller enters the *powered* state. A transition from the *powered* state to the *attach* state occurs when the Run/Stop bit is set to a ‘1’. After receiving a reset on the bus, the port will enter the *defaultFS* or *defaultHS* state in accordance with the reset protocol described in Appendix C.2 of the USB Specification Rev. 2.0. The following state diagram depicts the state of a USB 2.0 device.

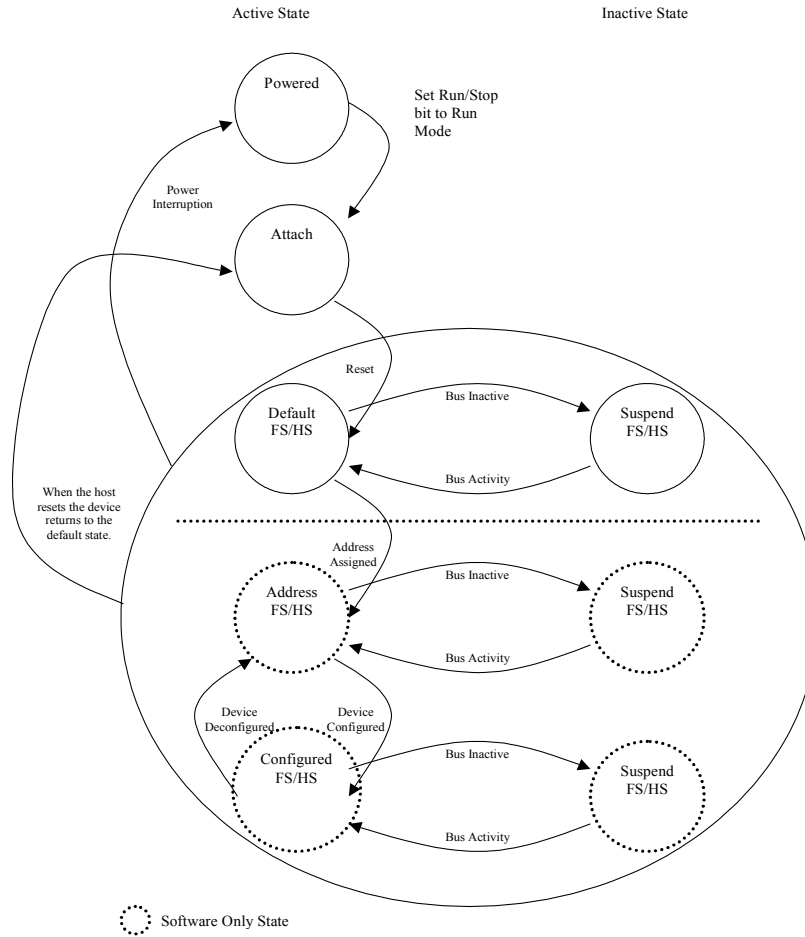


Figure 45 Device State Diagram

States *powered*, *attach*, *defaultFS/HS*, *suspendFS/HS* are implemented in the device controller and are communicated to the DCD using the following status bits:

Table 9 Device Controller State Information Bits

Bit	Register
DCSuspend	USBSTS
USB Reset Received	USBSTS
Port Change Detect	USBSTS
High-Speed Port	PORTSC

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 80-191

It is the responsibility of the DCD to maintain a state variable to differentiate between the *DefaultFS/HS* state and the *Address/Configured* states. Change of state from *Default* to *Address* and the *Configured* states is part of the enumeration process described in the device framework section of the USB 2.0 Specification.

As a result of entering the *Address* state, the device address register ([DEVICEADDR](#)) must be programmed by the DCD.

Entry into the *Configured* indicates that all endpoints to be used in the operation of the device have been properly initialized by programming the [ENDPTCTRLx](#) registers and initializing the associated queue heads.

5.2.1 Bus Reset

A bus reset is used by the host to initialize downstream devices. When a bus reset is detected, the device controller will renegotiate its attachment speed, reset the device address to 0, and notify the DCD by interrupt (assuming the USB Reset Interrupt Enable is set). After a reset is received, all endpoints (except endpoint 0) are disabled and any primed transactions will be cancelled by the device controller. The concept of priming will be clarified below, but the DCD must perform the following tasks when a reset is received:

Clear all setup token semaphores by reading the [ENDPTSETUPSTAT](#) register and writing the same value back to the [ENDPTSETUPSTAT](#) register.

Clear all the endpoint complete status bits by reading the [ENDPTCOMPLETE](#) register and writing the same value back to the [ENDPTCOMPLETE](#) register.

Cancel all primed status by waiting until all bits in the [ENDPTPRIME](#) are 0 and then writing 0xFFFFFFFF to [ENDPTFLUSH](#).

Read the reset bit in the [PORTSCx](#) register and make sure that it is still active. A USB reset will occur for a minimum of 3 ms and the DCD must reach this point in the reset cleanup before end of the reset occurs, otherwise a hardware reset of the device controller is recommended (rare.)

- A hardware reset can be performed by writing a one to the device controller reset bit in the USBCMD reset. Note: a hardware reset will cause the device to detach from the bus by clearing the Run/Stop bit. Thus, the DCD must completely re-initialize the device controller after a hardware reset.

Free all allocated dTDs because they will no longer be executed by the device controller. If this is the first time the DCD is processing a USB reset event, then it is likely that no dTDs have been allocated.

At this time, the DCD may release control back to the OS because no further changes to the device controller are permitted until a Port Change Detect is indicated.

After a Port Change Detect, the device has reached the default state and the DCD can read the [PORTSCx](#) to determine if the device is operating in FS or HS mode. At this time, the device controller has reached normal operating mode and DCD can begin enumeration according to the USB Chapter 9 - Device Framework.

NOTE The device DCD may use the FS/HS mode information to determine the bandwidth mode of the device.

In some applications, it may not be possible to enable one or more pipes while in FS mode. *Beyond the data rate issue, there is no difference in DCD operation between FS and HS modes.*

5.2.2 Suspend/Resume

5.2.2.1 Suspend

5.2.2.1.1 Suspend Description

In order to conserve power, USB devices automatically enter the suspended state when the device has observed no bus traffic for a specified period. When suspended, the USB device maintains any internal status, including its address and configuration. Attached devices must be prepared to suspend at any time they are powered, regardless of if they have been assigned a non-default address, are configured, or neither. Bus activity may cease due to the host entering a suspend mode of its own. In addition, a USB device shall also enter the suspended state when the hub port it is attached to is disabled.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 81-191

A USB device exits suspend mode when there is bus activity. A USB device may also request the host to exit suspend mode or selective suspend by using electrical signaling to indicate remote wakeup. The ability of a device to signal remote wakeup is optional. If the USB device is capable of remote wakeup signaling, the device must support the ability of the host to enable and disable this capability. When the device is reset, remote wakeup signaling must be disabled.

5.2.2.1.2 Suspend Operational Model

The device controller moves into the suspend state when suspend signaling is detected or activity is missing on the upstream port for more than a specific period. After the device controller enters the suspend state, the DCD is notified by an interrupt (assuming *DC Suspend Interrupt* is enabled). When the *DCSuspend* bit in the [PORTSCx](#) is set to a '1', the device controller is suspended.

DCD response when the device controller is suspended is application specific and may involve switching to low power operation.

Information on the bus power limits in suspend state can be found in USB 2.0 specification.

NOTE Review system level clocking issues defined in section (Ref: Signals-Clocking) for the clocking requirements of a suspended device controller.

5.2.2.2 Resume

If the device controller is suspended, its operation is resumed when any non-idle signaling is received on its upstream facing port. In addition, the device can signal the system to resume operation by forcing resume signaling to the upstream port. Resume signaling is sent upstream by writing a '1' to the Resume bit in the in the [PORTSCx](#) while the device is in suspend state. Sending resume signal to an upstream port should cause the host to issue resume signaling and bring the suspended bus segment (one more devices) back to the active condition.

NOTE Before resume signaling can be used, the host must enable it by using the Set Feature command defined in device framework (chapter 9) of the USB 2.0 Specification.

5.3 Managing Endpoints

The USB 2.0 specification defines an endpoint, also called a device endpoint or an address endpoint as a uniquely addressable portion of a USB device that can source or sink data in a communications channel between the host and the device. The endpoint address is specified by the combination of the endpoint number and the endpoint direction.

The channel between the host and an endpoint at a specific device represents a data pipe. Endpoint 0 for a device is always a *control* type data channel used for device discovery and enumeration. Other types of endpoints support by USB include *bulk*, *interrupt*, and *isochronous*. Each endpoint type has specific behavior related to packet response and error handling. More detail on endpoint operation can be found in the USB 2.0 specification.

The USB-HS device controller hardware supports up to the USB 2.0 maximum of 32 endpoint specified numbers. Each additional endpoint beyond the required endpoint position adds additional hardware logic. The maximum number of endpoint numbers available to the DCD is configured at hardware synthesis timer. After synthesis, the DCD can enable, disable and configure endpoint type up to the maximum selected during synthesis.

Each endpoint direction is essentially independent and can be configured with differing behavior in each direction. For example, the DCD can configure endpoint 1-IN to be a bulk endpoint and endpoint 1-OUT to be an isochronous endpoint. This helps to conserve the total number of endpoints required for device operation. The only exception is that control endpoints must use both directions on a single endpoint number to function as a control endpoint. Endpoint 0 is, for example, is always a control endpoint and uses the pair of directions.

Each endpoint direction requires a *queue head* allocated in memory. If the maximum of 16 endpoint numbers, one for each endpoint direction are being used by the device controller, then 32 *queue heads* are required. The operation of an endpoint and use of *queue heads* are described later in this document.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 82-191

5.3.1 Endpoint Initialization

After hardware reset, all endpoints except endpoint zero are uninitialized and disabled. The DCD must configure and enable each endpoint by writing to configuration bit in the [ENDPTCTRLx](#) register. Each 32-bit [ENDPTCTRLx](#) is split into an upper and lower half. The lower half of [ENDPTCTRLx](#) is used to configure the receive or OUT endpoint and the upper half is likewise used to configure the corresponding transmit or IN endpoint. Control endpoints must be configured the same in both the upper and lower half of the [ENDPTCTRLx](#) register otherwise the behavior is undefined. The following table shows how to construct a configuration word for endpoint initialization.

Table 10 Device Controller Endpoint Initialization

Field	Value
Data Toggle Reset	'1'
Data Toggle Inhibit	'0'
Endpoint Type	"00" – Control
	"01" – Isochronous
	"10" – Bulk
	"11" – Interrupt
Endpoint Stall	'0'

5.3.2 Stalling

There are two occasions where the device controller may need to return to the host a STALL. The first occasion is the **functional stall**, which is a condition set by the DCD as described in the USB 2.0 device framework (chapter 9). A functional stall is only used on non-control endpoints and can be enabled in the device controller by setting the endpoint stall bit in the [ENDPTCTRLx](#) register associated with the given endpoint and the given direction. In a functional stall condition, the device controller will continue to return STALL responses to all transactions occurring on the respective endpoint and direction until the endpoint stall bit is cleared by the DCD.

A **protocol stall**, unlike a function stall, is used on control endpoints is automatically cleared by the device controller at the start of a new control transaction (setup phase). When enabling a protocol stall, the DCD should enable the stall bits (both directions) as a pair. A single write to the [ENDPTCTRLx](#) register can ensure that both stall bits are set at the same instant.

NOTE Any write to the [ENDPTCTRLx](#) register during operational mode must preserve the endpoint type field (i.e. perform a read-modify-write).

Table 11 Device Controller Stall Response Matrix

USB Packet	Endpoint Stall Bit.	Effect on STALL bit.	USB Response
SETUP packet received by a non-control endpoint.	N/A	None.	STALL
IN/OUT/PING packet received by a non-control endpoint.	'1'	None.	STALL
IN/OUT/PING packet received by a non-control endpoint.	'0'	None.	ACK/ NAK/ NYET

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 83-191

SETUP packet received by a control endpoint.	N/A	Cleared	ACK
IN/OUT/PING packet received by a control endpoint	'1'	None	STALL
IN/OUT/PING packet received by a control endpoint.	'0'	None.	ACK/ NAK/ NYET

5.3.3 Data Toggle

Data toggle is a mechanism to maintain data coherency between host and device for any given data pipe. For more information on data toggle, refer to the USB 2.0 specification.

5.3.3.1 Data Toggle Reset

The DCD may reset the data toggle state bit and cause the data toggle sequence to reset in the device controller by writing a '1' to the data toggle reset bit in the [ENDPTCTRLx](#) register. This should only be necessary when configuring/initializing an endpoint or returning from a STALL condition.

5.3.3.2 Data Toggle Inhibit

This feature is for test purposes only and should never be used during normal device controller operation.

Setting the *data toggle Inhibit bit* active ('1') causes the device controller to ignore the data toggle pattern that is normally sent and accept all incoming data packets regardless of the data toggle state. In normal operation, the device controller checks the DATA0/DATA1 bit against the data toggle to determine if the packet is valid. If Data PID does not match the data toggle state bit maintained by the device controller for that endpoint, the Data toggle is considered not valid. If the data toggle is not valid, the device controller assumes the packet was already received and discards the packet (not reporting it to the DCD). To prevent the host controller from re-sending the same packet, the device controller will respond to the error packet by acknowledging it with either an ACK or NYET response.

5.4 Operational Model For Packet Transfers

All transactions on the USB bus are initiated by the host and in turn, the device must respond to any request from the host within the turnaround time stated in the USB 2.0 Specification. At USB 1.1 Full or Low Speed rates, this turnaround time was significant and the USB 1.1 device controllers were architected so that the device controller could access main memory or interrupt a host protocol processor in order to respond to the USB 1.1 transaction. The architecture of the USB 2.0 device controller must be different because same methods will not meet USB 2.0 High-speed turnaround time requirements by simply increasing clock rate.

A USB host will send requests to the device controller in an order that can not be precisely predicted as a single pipeline, so it is not possible to prepare a single packet for the device controller to execute. However, the order of packet requests is predictable when the endpoint number and direction is considered. For example, if endpoint 3 (transmit direction) is configured as a bulk pipe, then we can expect the host will send IN requests to that endpoint. This device controller is architected in such a way that it can prepare packets for each endpoint/direction in anticipation of the host request. The process of preparing the device controller to send or receive data in response to host initiated transaction on the bus is referred to as "**priming**" the endpoint. This term will be used throughout the following documentation to describe the device controller operation so the DCD can be architected properly use priming. Further, note that the term "**flushing**" is used to describe the action of clearing a packet that was queued for execution.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 84-191

5.4.1.1 Priming Transmit Endpoints

Priming a transmit endpoint will cause the device controller to fetch the device transfer descriptor (dTD) for the transaction pointed to by the device queue head (dQH). After the dTD is fetched, it will be stored in the dQH until the device controller completes the transfer described by the dTD. Storing the dTD in the dQH allows the device controller to fetch the operating context needed to handle a request from the host without the need to follow the linked list, starting at the dQH when the host request is received.

After the device has loaded the dTD, the leading data in the packet is stored in a FIFO in the device controller. This FIFO is split into virtual channels so that the leading data can be stored for any endpoint up to the maximum number of endpoints configured at device synthesis time.

After a priming request is complete, an endpoint state of primed is indicated in the ENDPTSTATUS register. For a primed transmit endpoint, the device controller can respond to an IN request from the host and meet the stringent bus turnaround time of High Speed USB.

Since only the leading data is stored in the device controller FIFO, it is necessary for the device controller to begin filling in behind leading data after the transaction starts. The FIFO must be sized to account for the maximum latency that can be incurred by the system memory bus. More information about FIFO sizing is presented in section Bandwidth and Latency Issues.

5.4.1.2 Priming Receive Endpoints

Priming receive endpoints is identical to priming of transmit endpoints from the point of view of the DCD. At the device controller the major difference in the operational model is that there is no data movement of the leading packet data simply because the data is to be received from the host.

Note as part of the architecture, the FIFO for the receive endpoints is not partitioned into multiple channels like the transmit FIFO. Thus, the size of the RX FIFO does not scale with the number of endpoints.

5.4.2 Interrupt/Bulk Endpoint Operational Model

The behaviors of the device controller for interrupt and bulk endpoints are identical. All valid IN and OUT transactions to bulk pipes will handshake with a NAK unless the endpoint had been primed. Once the endpoint has been primed, data delivery will commence.

A dTD will be retired by the device controller when the packets described in the transfer descriptor have been completed. Each dTD describes N packets to be transferred according to the USB Variable Length transfer protocol. The formula and table on the following page describe how the device controller computes the number and length of the packets to be sent/received by the USB vary according to the total number of bytes and maximum packet length.

With Zero Length Termination (ZLT) = 0

$$N = \text{INT}(\text{Number Of Bytes}/\text{Max. Packet Length}) + 1$$

With Zero Length Termination (ZLT) = 1

$$N = \text{MAXINT}(\text{Number Of Bytes}/\text{Max. Packet Length})$$

Table 12 Variable Length Transfer Protocol Example (ZLT = 0)

Bytes (dTD)	Max. Packet Length (dQH)	N	P1	P2	P3
511	256	2	256	255	
512	256	3	256	256	0
512	512	2	512	0	

Table 13 Variable Length Transfer Protocol Example (ZLT = 1)

Bytes (dTD)	Max. Packet Length (dQH)	N	P1	P2	P3
511	256	2	256	255	
512	256	2	256	256	

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 85-191

512

512

1

512

NOTE The MULT field in the dQH must be set to "00" for bulk, interrupt, and control endpoints.

TX-dTD is complete when:

- All packets described dTD were successfully transmitted. *** Total bytes in dTD will equal zero when this occurs.

RX-dTD is complete when:

- All packets described in dTD were successfully received. *** Total bytes in dTD will equal zero when this occurs.
- A short packet (number of bytes < maximum packet length) was received. *** This is a successful transfer completion; DCD must check Total Bytes in dTD to determine the number of bytes that are remaining. From the total bytes remaining in the dTD, the DCD can compute the actual bytes received.
- A long packet was received (number of bytes > maximum packet size) OR (total bytes received > total bytes specified). *** This is an error condition. The device controller will discard the remaining packet, and set the Buffer Error bit in the dTD. In addition, the endpoint will be flushed and the USBERR interrupt will become active.

On the successful completion of the packet(s) described by the dTD, the active bit in the dTD will be cleared and the next pointer will be followed when the Terminate bit is clear. When the Terminate bit is set, the device controller will flush the endpoint/direction and cease operations for that endpoint/direction.

On the unsuccessful completion of a packet (see long packet above), the dQH will be left pointing to the dTD that was in error. In order to recover from this error condition, the DCD must properly reinitialize the dQH by clearing the active bit and update the nextTD pointer before attempting to re-prime the endpoint.

NOTE All packet level errors such as a missing handshake or CRC error will be retried automatically by the device controller.

There is no required interaction with the DCD for handling such errors.

5.4.2.1 Interrupt/Bulk Endpoint Bus Response Matrix

Table 14 Interrupt/Bulk Endpoint Bus Response Matrix

	Stall	Not Primed	Primed	Underflow	Overflow
Setup	Ignore	Ignore	Ignore	N/A	N/A
In	STALL	NAK	Transmit	BS Error	N/A
Out	STALL	NAK	Receive + NYET/ACK	N/A	NAK
Ping	STALL	NAK	ACK	N/A	N/A
Invalid	Ignore	Ignore	Ignore	Ignore	Ignore

BS Error = Force Bit Stuff Error

NYET/ACK – NYET unless the Transfer Descriptor has packets remaining according to the USB variable length protocol then ACK.

SYSERR – System error should never occur when the latency FIFOs are correctly sized and the DCD is responsive.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 86-191

5.4.3 Control Endpoint Operation Model

5.4.3.1 Setup Phase

All requests to a control endpoint begin with a setup phase followed by an optional data phase and a required status phase. The device controller will always accept the setup phase unless the setup lockout is engaged.

The setup lockout will engage so that future setup packets are ignored. Lockout of setup packets ensures that while software is reading the setup packet stored in the queue head, that data is not written as it is being read potentially causing an invalid setup packet.

In hardware versions 2.3 and later, the setup lockout mechanism can be disabled and a new tripwire type semaphore will ensure that the setup packet payload is extracted from the queue head without being corrupted by an incoming setup packet. This is the preferred behavior because ignoring repeated setup packets due to long software interrupt latency would be a compliance issue.

5.4.3.1.1 Setup Packet Handling (Pre-2.3 hardware)

- After receiving an interrupt and inspecting USBMODE to determine that a setup packet was received on a particular pipe:
 - 1. Duplicate contents of dQH.SsetupBuffer into local software byte array.
 - 2. Write '1' to clear corresponding ENDPTSETUPSTAT bit and thereby disabling Setup Lockout. (i.e. the Setup Lockout activates as soon as a setup arrives. By writing to the ENDPTSETUPSTAT, the device controller will accept new setup packets.)
 - 3. Process setup packet using local software byte array copy and execute status/handshake phases.
 - Note: After receiving a new setup packet the status and/or handshake phases may still be pending from a previous control sequence. These should be flushed & deallocated before linking a new status and/or handshake dTD for the most recent setup packet.
 - 4. Before priming for status/handshake phases ensure that ENDPTSETUPSTAT is '0'.
 - The time from writing a '1' to ENDPTSETUPSTAT and reading back a '0' may vary according to the type of traffic on the bus up to nearly a 1ms, however it is absolutely necessary to ensure ENDPTSETUPSTAT has transitioned to '0' after step 1) and before priming for the status/handshake phases.

NOTE To limit the exposure of setup packets to the setup lockout mechanism (if used), the DCD should designate the priority of responding to setup packets above responding to other packet completions.

5.4.3.1.2 Setup Packet Handling (2.3 hardware and later)

- Disable Setup Lockout by writing '1' to Setup Lockout Mode (SLOM) in USBMODE. (once at initialization). Setup lockout is not necessary when using the tripwire as described below.

NOTE leaving the Setup Lockout Mode As '0' will result in pre-2.3 hardware behavior.

- After receiving an interrupt and inspecting ENDPTSETUPSTAT to determine that a setup packet was received on a particular pipe:
 - 1. Write '1' to clear corresponding bit ENDPTSETUPSTAT.
 - 2. Write '1' to Setup Tripwire (SUTW) in USBCMD register.
 - 3. Duplicate contents of dQH.SetupBuffer into local software byte array.
 - 4. Read Setup TripWire (SUTW) in USBCMD register. (if set - continue; if cleared - goto 2)
 - 5. Write '0' to clear Setup Tripwire (SUTW) in USBCMD register.
 - 6. Process setup packet using local software byte array copy and execute status/handshake phases.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 87-191

- 7. Before priming for status/handshake phases ensure that ENDPTSETUPSTAT is ‘0’.
- A poll loop should be used to wait until ENDPTSETUPSTAT transitions to ‘0’ after step 1) above and before priming for the status/handshake phases.
- In core versions 3.2 and later, the time from writing a ‘1’ to ENDPTSETUPSTAT and reading back a ‘0’ is very short (~1-2 us) so a poll loop in the DCD will not be harmful.
- In core versions 3.1 and earlier, the time from writing a ‘1’ to ENDPTSETUPSTAT and reading back a ‘0’ may vary according to the type of traffic on the bus up to nearly a 1ms, however the it is absolutely necessary to ensure ENDPTSETUPSTAT has transitioned to ‘0’ after step 1) and before priming for the status/handshake phases.

NOTE Note: After receiving a new setup packet the status and/or handshake phases may still be pending from a previous control sequence. These should be flushed & deallocated before linking a new status and/or handshake dTD for the most recent setup packet.

5.4.3.2 Data Phase

Following the setup phase, the DCD must create a device transfer descriptor for the data phase and prime the transfer.

After priming the packet, the DCD must verify a new setup packet has not been received by reading the ENDPTSETUPSTAT register immediately verifying that the prime had completed. A prime will complete when the associated bit in the ENDPTPRIME register is zero and the associated bit in the ENDPTSTATUS register is a one. If a prime fails, i.e. The ENDPTPRIME bit goes to zero and the ENDPTSTATUS bit is not set, then the prime has failed. This can only be due to improper setup of the dQH, dTD or a setup arriving during the prime operation. If a new setup packet is indicated after the ENDPTPRIME bit is cleared, then the transfer descriptor can be freed and the DCD must reinterpret the setup packet.

Should a setup arrive after the data stage is primed, the device controller will automatically clear the prime status (ENDPTSTATUS) to enforce data coherency with the setup packet.

NOTE The MULT field in the dQH must be set to “00” for bulk, interrupt, and control endpoints.

NOTE Error handling of data phase packets is the same as bulk packets described previously.

5.4.3.3 Status Phase

Similar to the data phase, the DCD must create a transfer descriptor (with byte length equal zero) and prime the endpoint for the status phase. The DCD must also perform the same checks of the ENDPTSETUPSTAT as described above in the data phase.

NOTE The MULT field in the dQH must be set to “00” for bulk, interrupt, and control endpoints.

NOTE Error handling of data phase packets is the same as bulk packets described previously.

5.4.3.4 Control Endpoint Bus Response Matrix

Shown in the following table is the device controller response to packets on a control endpoint according to the device controller state.

Table 15 Control Endpoint Bus Response Matrix

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 88-191

Token Type	Endpoint State					Setup Lockout
	Stall	Not Primed	Primed	Underflow	Overflow	
Setup	ACK	ACK	ACK	N/A	SYSERR	
In	STALL	NAK	Transmit	BS Error	N/A	N/A
Out	STALL	NAK	Receive + NYET/ACK	N/A	NAK	N/A
Ping	STALL	NAK	ACK	N/A	N/A	N/A
Invalid	Ignore	Ignore	Ignore	Ignore	Ignore	Ignore

BS Error = Force Bit Stuff Error

NYET/ACK – NYET unless the Transfer Descriptor has packets remaining according to the USB variable length protocol then ACK.

SYSERR – System error should never occur when the latency FIFOs are correctly sized and the DCD is responsive.

5.4.4 Isochronous Endpoint Operational Model

Isochronous endpoints are used for real-time scheduled delivery of data and their operational model is significantly different than the host throttled Bulk, Interrupt, and Control data pipes. Real time delivery by the device controller will be accomplished by the following:

- Exactly MULT Packets per (micro)Frame are transmitted/received. Note: MULT is a two-bit field in the device Queue Head. The variable length packet protocol is not used on isochronous endpoints.
- NAK responses are not used. Instead, zero length packets are sent in response to an IN request to an unprimed endpoints. For unprimed RX endpoints, the response to an OUT transaction is to ignore the packet within the device controller.
- Prime requests always schedule the transfer described in the dTD for the next (micro)frame. If the ISO-dTD is still active after that frame, then the ISO-dTD will be held ready until executed or canceled by the DCD.

An EHCI compatible host controller uses the periodic frame list to schedule data exchanges to Isochronous endpoints. The operational model for device mode does not use such a data structure. Instead, the same dTD used for Control/Bulk/Interrupt endpoints is also used for isochronous endpoints. The difference is in the handling of the dTD.

The first difference between bulk and ISO-endpoints is that priming an ISO-endpoint is a delayed operation such that an endpoint will become primed only after a SOF is received. After the DCD writes the prime bit, the prime bit will be cleared as usual to indicate to software that the device controller completed a priming the dTD for transfer. Internal to the design, the device controller hardware masks that prime start until the next frame boundary. This behavior is hidden from the DCD but occurs so that the device controller can match the dTD to a specific (micro)frame.

Another difference with isochronous endpoints is that the transaction must wholly complete in a (micro)frame. Once an ISO transaction is started in a (micro)frame it will retire the corresponding dTD when MULT transactions occur or the device controller finds a fulfillment condition.

The transaction error bit set in the status field indicates a fulfillment error condition. When a fulfillment error occurs, the frame after the transfer failed to complete wholly, the device controller will force retire the ISO-dTD and move to the next ISO-dTD.

It is important to note that fulfillment errors are only caused due to partially completed packets. If no activity occurs to a primed ISO-dTD, the transaction will stay primed indefinitely. This means it is up to software discard transmit ISO-dTDs that pile up from a failure of the host to move the data.

Finally, the last difference with ISO packets is in the data level error handling. When a CRC error occurs on a received packet, the packet is not retried similar to bulk and control endpoints. Instead, the CRC is noted by setting the *Transaction Error* bit and the data is stored as usual for the application software to sort out.

- TX Packet Retired
 - MULT counter reaches zero.
 - Fulfillment Error [*Transaction Error* bit is set]
 - # Packets Occurred > 0 AND # Packets Occurred < MULT

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 89-191

NOTE For TX-ISO, MULT Counter can be loaded with a lesser value in the dTD Multiplier Override field in hardware versions 2.3 and later. If the Multiplier Override is zero, the MULT Counter is initialized to the Multiplier in the QH.

- RX Packet Retired:
 - MULT counter reaches zero.
 - Non-MDATA Data PID is received**
 - ** Exit criteria only valid in hardware version 2.3 or later. Previous to hardware version 2.3, any PID sequence that did not match the MULT field exactly would be flagged as a transaction error due to PID mismatch or fulfillment error.
 - Overflow Error:
 - Packet received is > maximum packet length. [*Buffer Error* bit is set]
 - Packet received exceeds total bytes allocated in dTD. [*Buffer Error* bit is set]
 - Fulfillment Error [*Transaction Error* bit is set]
 - # Packets Occurred > 0 AND # Packets Occurred < MULT
 - CRC Error [*Transaction Error* bit is set]

NOTE For ISO, when a dTD is retired, the next dTD is primed for the next frame. For continuous (micro)frame to (micro)frame operation the DCD should ensure that the dTD linked-list is out ahead of the device controller by at least two (micro)frames.

5.4.4.1 Isochronous Pipe Synchronization

When it is necessary to synchronize an isochronous data pipe to the host, the (micro)frame number (FRINDEX register) can be used as a marker. To cause a packet transfer to occur at a specific (micro)frame number [N], the DCD should interrupt on SOF during frame N-1. When the FRINDEX=N-1, the DCD must write the prime bit. The device controller will prime the isochronous endpoint in (micro)frame N-1 so that the device controller will execute delivery during (micro)frame N.

CAUTION Priming an endpoint towards the end of (micro)frame N-1 will not guarantee delivery in (micro)frame N. The delivery may actually occur in (micro)frame N+1 if device controller does not have enough time to complete the prime before the SOF for packet N is received.

5.4.4.2 Isochronous Endpoint Bus Response Matrix

Table 16 Isochronous Endpoint Bus Response Matrix

	Stall	Not Primed	Primed	Underflow	Overflow
Setup	STALL	STALL	STALL	N/A	N/A
In	NULL Packet	NULL Packet	Transmit	BS Error	N/A
Out	Ignore	Ignore	Receive	N/A	Drop Packet
Ping	Ignore	Ignore	Ignore	Ignore	Ignore
Invalid	Ignore	Ignore	Ignore	Ignore	Ignore

BS Error = Force Bit Stuff Error

NULL Packet = Zero Length Packet

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 90-191

5.5 Managing Queue Heads

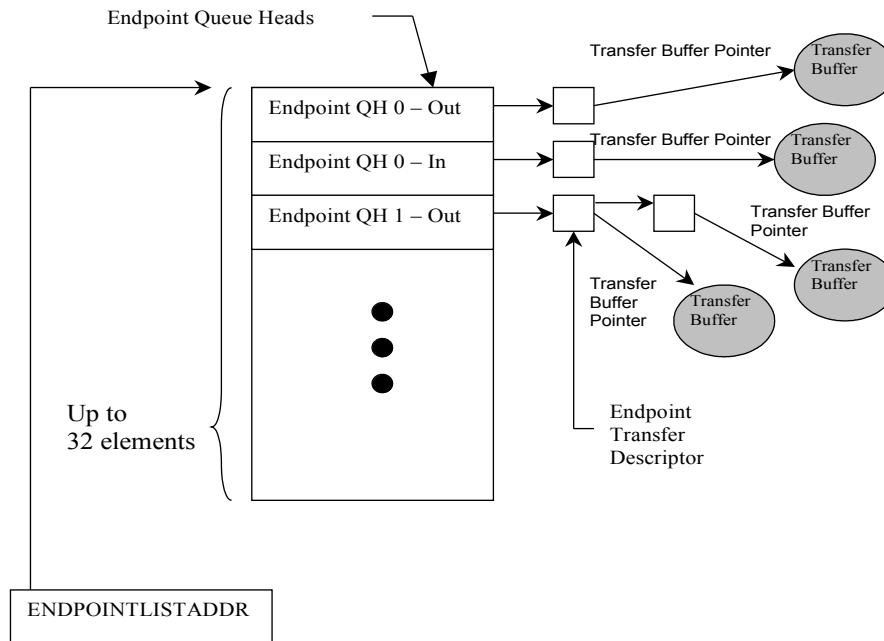


Figure 46 End Point Queue Head Diagram

The device queue head (dQH) points to the linked list of transfer tasks, each depicted by the device Transfer Descriptor (dTD). An area of memory pointed to by [ENDPOINTLISTADDR](#) contains a group of all dQH's in a sequential list as shown in Figure 46. The even elements in the list of dQH's are used for receive endpoints (OUT/SETUP) and the odd elements are used for transmit endpoints (IN/INTERRUPT). Device transfer descriptors are linked head to tail starting at the queue head and ending at a terminate bit. Once the dTD has been retired, it will no longer be part of the linked list from the queue head. Therefore, software is required to track all transfer descriptors since pointers will no longer exist within the queue head once the dTD is retired (see section Software Link Pointers).

In addition to the current and next pointers and the dTD overlay examined in section Operational Model For Packet Transfers, the dQH also contains the following parameters for the associated endpoint: Multiplier, Maximum Packet Length, Interrupt On Setup. The complete initialization of the dQH including these fields is demonstrated in the next section.

5.5.1 Queue Head Initialization

One pair of device queue heads must be initialized for each active endpoint. To initialize a device queue head:

- Write the wMaxPacketSize field as required by the USB Chapter 9 or application specific protocol.
- Write the multiplier field to 0 for control, bulk, and interrupt endpoints. For ISO endpoints, set the multiplier to 1, 2, or 3 as required bandwidth and in conjunction with the USB Chapter 9 protocol. *Note: In FS mode, the multiplier field can only be 1 for ISO endpoints.*
- Write the next dTD Terminate bit field to "1".
- Write the Active bit in the status field to "0".
- Write the Halt bit in the status field to "0".

NOTE The DCD must only modify dQH if the associated endpoint is not primed and there are no outstanding dTD's.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 91-191

5.5.2 Operational Model For Setup Transfers

As discussed in section Control Endpoint Operation Model, setup transfer requires special treatment by the DCD. A setup transfer does not use a dTD but instead stores the incoming data from a setup packet in an 8-byte buffer within the dQH.

Upon receiving notification of the setup packet, the DCD should handle the setup transfer as demonstrated here:

1. Copy setup buffer contents from dQH - RX to software buffer.
2. Acknowledge setup backup by writing a “1” to the corresponding bit in [ENDPTSETUPSTAT](#).

NOTE The acknowledge must occur before continuing to process the setup packet.

NOTE After the acknowledge has occurred, the DCD must not attempt to access the setup buffer in the dQH – RX. Only the local software copy should be examined.

1. Check for pending data or status dTD’s from previous control transfers and flush if any exist as discussed in section Flushing/De-priming an Endpoint.

NOTE It is possible for the device controller to receive setup packets before previous control transfers complete. Existing control packets in progress must be flushed and the new control packet completed.

2. Decode setup packet and prepare data phase [optional] and status phase transfer as required by the USB Chapter 9 or application specific protocol.

5.6 Managing Transfers with Transfer Descriptors

5.6.1 Software Link Pointers

It is necessary for the DCD software to maintain head and tail pointers to the for the linked list of dTDs for each respective queue head. This is necessary because the dQH only maintains pointers to the current working dTD and the next dTD to be executed. The operations described in next section for managing dTD will assume the DCD can use [reference the head and tail of the dTD linked list](#).

NOTE To conserve memory, the reserved fields at the end of the dQH can be used to store the Head & Tail pointers but it still remains the responsibility of the DCD to maintain the pointers.

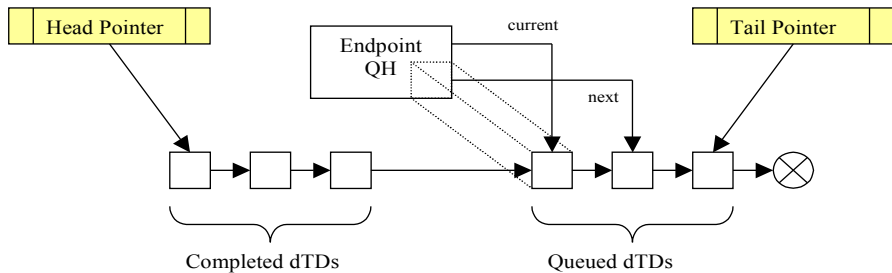


Figure 47 Software Link Pointers

5.6.2 Building a Transfer Descriptor

Before a transfer can be executed from the linked list, a dTD must be built to describe the transfer. Use the following procedure for building dTDs.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 92-191

Allocate 8-DWord dTD block of memory aligned to 8-DWord boundaries. Example: bit address 4:0 would be equal to “00000”

Write the following fields:

1. Initialize first 7 DWords to 0.
2. Set the terminate bit to “1”.
3. Fill in total bytes with transfer size.
4. Set the interrupt on complete if desired.
5. Initialize the status field with the active bit set to “1” and all remaining status bits set to “0”.
6. Fill in buffer pointer page 0 and the current offset to point to the start of the data buffer.
7. Initialize buffer pointer page 1 through page 4 to be one greater than each of the previous buffer pointer.

5.6.3 Executing A Transfer Descriptor

To safely add a dTD, the DCD must follow this procedure which will handle the event where the device controller reaches the end of the dTD list at the same time a new dTD is being added to the end of the list.

Determine whether the link list is empty:

- Check DCD driver to see if pipe is empty (internal representation of linked-list should indicate if any packets are outstanding)
- Case 1: Link list is empty
 - 1. Write dQH next pointer AND dQH terminate bit to 0 as a single DWord operation.
 - 2. Clear active & halt bit in dQH (in case set from a previous error).
 - 3. Prime endpoint by writing ‘1’ to correct bit position in [ENDPTPRIME](#).
- Case 2: Link list is not empty
 - 1. Add dTD to end of linked list.
 - 2. Read correct prime bit in [ENDPTPRIME](#) – if ‘1’ DONE.
 - 3. Set ATDTW bit in [USBCMD](#) register to ‘1’.
 - 4. Read correct status bit in [ENDPTSTAT](#). (store in tmp. variable for later)
 - 5. Read ATDTW bit in [USBCMD](#) register.
 - If ‘0’ goto 3.
 - If ‘1’ continue to 6.
 - 6. Write ATDTW bit in [USBCMD](#) register to ‘0’.
 - 7. If status bit read in (3) is ‘1’ DONE.
 - 8. If status bit read in (3) is ‘0’ then Goto Case 1: Step 1.

5.6.4 Transfer Completion

After a dTD has been initialized and the associated endpoint primed the device controller will execute the transfer upon the host-initiated request. The DCD will be notified with a USB interrupt if the Interrupt On Complete bit was set or alternately, the DCD can poll the endpoint complete register to find when the dTD had been executed. After a dTD has been executed, DCD can check the status bits to determine success or failure.

CAUTION Multiple dTD can be completed in a single endpoint complete notification. After clearing the notification, DCD must search the dTD linked list and retire all dTDs that have finished (Active bit cleared).

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 93-191

By reading the status fields of the completed dTDs, the DCD can determine if the transfers completed successfully. Success is determined with the following combination of status bits:

- Active = 0
- Halted = 0
- Transaction Error = 0
- Data Buffer Error = 0

Should any combination other than the one shown above exist, the DCD must take proper action. Transfer failure mechanisms are indicated in the [Device Error Matrix](#).

In addition to checking the status bit the DCD must read the Transfer Bytes field to determine the actual bytes transferred. When a transfer is complete, the Total Bytes transferred is by decremented by the actual bytes transferred. For Transmit packets, a packet is only complete after the actual bytes reaches zero, but for receive packets, the host may send fewer bytes in the transfer according the USB variable length packet protocol.

5.6.5 Flushing/De-priming an Endpoint

It is necessary for the DCD to flush to de-prime one more endpoints on a USB device reset or during a broken control transfer. There may also be application specific requirements to stop transfers in progress. The following procedure can be used by the DCD to stop a transfer in progress:

1. Write a '1' to the corresponding bit(s) in [ENDPTFLUSH](#).
2. Wait until all bits in [ENDPTFLUSH](#) are '0'.
 - Software note: this operation may take a large amount of time depending on the USB bus activity. It is not desirable to have this wait loop within an interrupt service routine.
3. Read [ENDPTSTAT](#) to ensure that for all endpoints commanded to be flushed, that the corresponding bits are now '0'. If the corresponding bits are '1' after step #2 has finished, then the flush failed as described in the following:
 - Explanation: In very rare cases, a packet is in progress to the particular endpoint when commanded flush using [ENDPTFLUSH](#). A safeguard is in place to refuse the flush to ensure that the packet in progress completes successfully. The DCD may need to repeatedly flush any endpoints that fail to flush by repeating steps 1-3 until each endpoint is successfully flushed.

5.6.6 Device Error Matrix

The following table summarizes packet errors that are not automatically handled by the Device Controller.

Table 17 Device Error Matrix

Error	Direction	Packet Type	Data Buffer Error Bit	Transaction Error Bit
Overflow **	RX	Any	1	0
ISO Packet Error	RX	ISO	0	1
ISO Fulfillment Error	Both	ISO	0	1

Notice that the device controller handles all errors on Bulk/Control/Interrupt Endpoints except for a data buffer overflow. However, for ISO endpoints, errors packets are not retried and errors are tagged as indicated.

Error Descriptions:

Overflow	Number of bytes received exceeded max. packet size or total buffer length. ** This error will also set the Halt bit in the dQH and if there are dTDs remaining in the linked list for the endpoint, then those will not be executed.
ISO Packet Error	CRC Error on received ISO packet. Contents not guaranteed to be

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 94-191

ISO Fulfillment Error correct.
Hst failed to complete the number of packets defined in the dQH mult field within the given (micro)frame. For scheduled data delivery the DCD may need to readjust the data queue because a fulfillment error will cause Device Controller to cease data transfers on the pipe for one (micro)frame. During the “dead” (micro)frame, the Device Controller reports error on the pipe and primes for the following frame.

5.7 Servicing Interrupts

The interrupt service routine must consider that there are high-frequency, low-frequency operations, and error operations and order accordingly.

5.7.1 High-Frequency Interrupts

High frequency interrupts in particular should be handed in the order below. The most important of these is listed first because the DCD must acknowledge a setup buffer in the timeliest manner possible.

Table 18 High Frequency Interrupt Events

Execution Order	Interrupt	Action
1a	USB Interrupt ** - ENDPTSETUPSTATUS	Copy contents of setup buffer and acknowledge setup packet (as indicated in section Managing Queue Heads). Process setup packet according to USB 2.0 Chapter 9 or application specific protocol.
1b	USB Interrupt ** - ENDPTCOMPLETE	Handle completion of dTD as indicated in section Managing Queue Heads.
2	SOF Interrupt	Action as deemed necessary by application. This interrupt may not have a use in all applications.

** It is likely that multiple interrupts to stack up on any call to the Interrupt Service Routine AND during the Interrupt Service Routine.

5.7.2 Low-Frequency Interrupts

The low frequency events include the following interrupts. These interrupt can be handled in any order since they don’t occur often in comparison to the high-frequency interrupts.

Table 19 Low Frequency Interrupt Events

Interrupt	Action
Port Change	Change software state information.
Sleep Enable (Suspend)	Change software state information. Low power handling as necessary.
Reset Received	Change software state information. Abort pending transfers.

5.7.3 Error Interrupts

Error interrupts will be least frequent and should be placed last in the interrupt service routine.

Table 20 Error Interrupt Events

Interrupt	Action
USB Error Interrupt.	This error is redundant because it combines USB Interrupt and an error status in the dTD. The DCD will more aptly handle packet-level errors by checking dTD status field upon receipt of USB Interrupt (w/ ENDPTCOMPLETE) .

		<i>Title:</i> CI13410-a High-Speed USB Device Controller Core			<i>Revision:</i> 1.0A	
<i>Template:</i> TP02_DEP_GP	<i>Department:</i> IPCS	<i>Prepared by:</i> Joana Braga	<i>Approved by:</i> Luis Miguel Sampaio	<i>Security Level:</i> confidential	<i>Date:</i> 2006-05-11	<i>Page:</i> 95-191

System Error

Unrecoverable error. Immediate Reset of core; free transfers buffers in progress and restart the DCD.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 96-191

6 Signals

6.1 Signal Descriptions

The following interfaces are provided:

- BVIC or AMBA
 - BVIC Target t_*
 - BVIC Initiator i_*
 - AMBA Slave s_*
 - AMBA Master m_*
- Interrupt interrupt
- PHY Interface xcvr_*
- RX buffer dual port memory rx_buf*
- TX buffer dual port memory tx_buf*

6.1.1 System Clock/Resets

System Clock/Resets		
Signal	Direction	Description
clk	IN	system clock
aux_clk	IN	Auxiliary clock input only used in clock configuration #3. Connect to logic zero for clock configurations #0, 1, and 2. Must be 33 MHz. Min.
xcvr_clk	IN	Transceiver interface clock for the parallel transceiver interface. A single pin is provided that should be connected to the clock from the HS transceiver that is selected. <i>Note: In clock architecture 2 if no parallel transceiver interface is implemented this clock must still be driven with a 30MHz or greater clock.</i>
xcvr_ser_clk	IN	60MHz Serial Interface Engine clock used to run the FS/LS serial engine DPLL. May be tied to logic 0 if the serial engine is not implemented. <i>Note: No relationship to the xcvr_clk is assumed, but if the xcvr_clk is 60MHz the same clock can be used for the xcvr_clk and xcvr_ser_clk inputs...</i>
rst	IN	active high reset input
test_mode	IN	Connect to logic zero for normal operation and logic one for scan test.
When high, the internally generated reset is disabled and the <i>rst</i> input directly controls all register resets.		

6.1.2 BVIC Bus Target Interface

Please refer to the VSI Alliance - Virtual Component Interface Standard Basic VCI [BVCI] Section for more detail.

Bus Target Interface		
Signal	Direction	Description
t_cmdack	OUT	Target command acknowledge
t_cmdval	IN	Target command valid
t_address[8:0]	IN	Target address

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 97-191

t_be[3:0]	IN	Target byte enables
t_cmd[1:0]	IN	Target command
t_wdata[31:0]	IN	Target write data
t_eop	IN	Target end of packet
t_rspack	IN	Target response acknowledge
t_rspval	OUT	Target response valid
t_rdata[31:0]	OUT	Target response data
t_reop	OUT	Target response end of packet

6.1.3 BVC I Bus Initiator Interface

Please refer to the VSI Alliance - Virtual Component Interface Standard Basic VCI [BVC I] Section for more detail.

Bus Initiator Interface

Signal	Direction	Description
i_cmdack	IN	Initiator command acknowledge
i_cmdval	OUT	Initiator command valid
i_address[31:0]	OUT	Initiator address
i_be[3:0]	OUT	Initiator byte enables
i_cmd[1:0]	OUT	Initiator command
i_wdata[31:0]	OUT	Initiator write data
i_eop	OUT	Initiator end of packet
i_rspack	OUT	Initiator response acknowledge
i_rspval	IN	Initiator response valid
i_rdata[31:0]	IN	Initiator response data
i_reop	IN	Initiator response end of packet
i_typeinfo	OUT	Initiator type. Leave open, this is not a BVC I signal.
i_xtra_in	IN	Contact customer support for more information.
i_xtra_out	OUT	Initiator alternate use port – connect each bit to logic zero.
		Initiator alternate use port – leave unconnected.

6.1.4 AMBA-AHB Bus Slave Interface

Please refer to the AMBA Specification for more detail.

Bus Slave Interface

Signal	Direction	Description
s_haddr(8:0)	IN	Slave Address
s_htrans(1:0)	IN	Slave Transaction
s_hrdata(31:0)	OUT	Slave Read Data
s_hwrite	IN	Slave Write
s_hwdata(31:0)	IN	Slave Write Data
s_hsize(2:0)	IN	Slave Data Size
s_hsel	IN	Slave Select
s_hready_in	IN	Slave Ready Input
s_hready_out	OUT	Slave Ready Output
s_hresp(1:0)	OUT	Slave Response

6.1.5 AMBA-AHB Bus Master Interface

Please refer to the AMBA Specification for more detail.

Bus Master Interface

Signal	Direction	Description
m_haddr(31:0)	OUT	Master Address
m_htrans(1:0)	OUT	Master Transaction

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 98-191

Bus Master Interface

Signal	Direction	Description
m_hwrite	OUT	Master Write
m_hsize(2:0)	OUT	Master Data Size
m_hburst(2:0)	OUT	Master Burst Type
m_hprot(3:0)	OUT	Master Protection Control
m_typeinfo	OUT	Master type. Leave open, this is not a AMBA-AHB signal. Contact customer support for more information.
m_hwdata(31:0)	OUT	Master Write Data
m_hrdata(31:0)	IN	Master Read Data
m_hready	IN	Master Ready
m_hresp	IN	Master Response
m_hbusreq	OUT	Master Bus Request
m_hgrant	IN	Master Bus Grant
m_hlock	OUT	Master Bus Lock

6.1.6 Interrupt

Interrupt Signal	Direction	Description
Interrupt	OUT	Interrupt to the processor

6.1.7 USB-HS Transceiver Interfaces

The USB-HS device transceiver interface signals allow direct connection of the core to UTMI, UTMI+ transceivers, legacy Full speed USB I/O macrocells, and ULPI transceivers. Individual interface signal groups are provided for each of the transceiver interfaces.

The tables below document the transceiver signals, as well as the how to tie-off the signals for the transceiver interfaces that are not in use. If a transceiver interface is to be used it must be configured at synthesis time. The value in the Tie-off column of the table shows how to connect signals for transceiver interfaces that are disabled at synthesis. For designs that make use of the software programmable transceiver selection no additional logic is required between the core and the transceivers that are connected.

6.1.7.1 Transceiver select signals

The transceiver select output signals gives an indication of which transceiver interface is currently selected. These signals may be used to control pin sharing logic outside the USB controller core.

USB-HS Transceiver Interfaces selection indicators

Signal	Direction	Tie-off	Description
utmi_phy_enable	OUT	Open	Asserted when the UTMI interface is selected
ser_phy_enable	OUT	Open	Asserted when the FS/LS serial interface is selected
ulpi_phy_enable	OUT	Open	Asserted when the ULPI (UTMI+ Low Pin-count interface) is selected.

6.1.7.2 UTMI signals

USB-HS Transceiver Interfaces [UTMI compatible]

Signal	Direction	Tie-off	Description
utmi_reset	OUT	Open	Reset out to the transceiver
utmi_xcvsselect[0]	OUT	Open	Selects FS or HS transceiver operation

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 99-191

USB-HS Transceiver Interfaces [UTMI compatible]

Signal	Direction	Tie-off	Description
utmi_termselect	OUT	Open	Selects FS or HS termination
utmi_linestate[1:0]	IN	00	Reflects the linestate of DM/DP
utmi_opmode[1:0]	OUT	Open	Selects the transceiver operating mode
utmi_datain[15:0]	OUT	Open	16-bit parallel TX data to the transceiver when using an 8-bit PHY (constant VUSB_HS_PHY16_8=0) only utmi_datain[7:0] are used
utmi_txvalid	OUT	Open	Indicates utmi_tx_data[7:0] is valid
utmi_txvalidh	OUT	Open	Indicates utmi_tx_data[15:8] is valid not used with an 8-bit PHY (constant VUSB_HS_PHY16_8=0)
utmi_txready	IN	0	Indicates transceiver wait state
utmi_dataout[15:0]	IN	0	16-bit parallel RX data from the transceiver when using an 8-bit PHY (constant VUSB_HS_PHY16_8=0) only utmi_rx_data[7:0] are used
utmi_rxvalid	IN	0	Indicates utmi_dataout[7:0] is valid
utmi_rxvalidh	IN	0	Indicates utmi_dataout[15:8] is valid
utmi_rxactive	IN	0	Indicates the transceiver has detected a sync and is active
utmi_rxerr	IN	0	Indicates receive error
utmi_dataoe	OUT	Open	Output enable for rx/tx data and rx/tx readyh for when using bi-directional signaling. Use of utmi_dataoe is preferred over utmi_txvalid for tri-state controls.
utmi_ddir	OUT	Open	Non-standard UTMI signal required by some PHYs which can't control their own tristate drivers
utmi_databus16_8	OUT	Open	Selects between 8-bit/16-bit modes for transceivers that support both signaling formats.
sess_valid	IN	0	Non UTMI standard signal that indicates that Power is present on VBus. <i>Note: This signal is not a standard UTMI signal but must be provided to pass the USB-IF back drive tests. For bus powered devices this signal may be tied to logic 1.</i>

6.1.7.3 ULPI signals

The ULPI interface is defined in the UTMI+ Low Pin Interface (ULPI) Specification. The specification is made available to companies that sign the ULPI adopters agreement. For more information refer to www.ulpi.org.

Note: All transceiver signals are vectors with widths equal to the number of downstream facing ports.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 100-191

USB-HS Transceiver Interfaces [ULPI compatible]

Signal	Direction	Tie-off	Description
ulpi_dir	IN	0	Controls the direction of the data bus. The PHY pulls dir high whenever the interface cannot accept data from the Link.
ulpi_stp	OUT	Open	Asserted to end or interrupt transfers on the interface
ulpi_nxt	IN	0	Data flow control signal from the PHY.
ulpi_tx_data[7:0]	OUT	Open	Transmit data
ulpi_tx_data_nxt	OUT	Open	Transmit data (unregistered version). This signal can be used for creating a DDR ULPI i/f but should otherwise be left open.
ulpi_tx_data_oe	OUT	Open	Link side data tri-state enable.
ulpi_rx_data[7:0]	IN	0	Receive data
ulpi_carkit	OUT	Open	Indication that the ULPI carkit interface is enabled. <i>Note: This is not a ULPI interface pin, but is made available to other system logic to control carkit interface signal multiplexers.</i>

6.1.7.4 Full Speed Serial signals

The following signals provide a means of interfacing the controller to a legacy Full or Low speed transceiver. These signals do not support High Speed USB operations and may be tied-off in applications that use a high speed capable transceiver.

USB-HS Transceiver Interfaces (Full Speed Serial signals)

Signal	Direction	Tie-off	Description
ser_tx_enable_n	OUT	Open	Output Enable. Active LOW, controls the transceiver output enable to the USB signals D+ and D-.
ser_tx_dat	OUT	Open	USB data D+ output to differential driver.
ser_tx_se0	OUT	Open	USB Single Ended Zero (SE0) output
ser_rx_rcv	IN	0	Receive data. USB data differential receiver input.
ser_rx_dp	IN	0	Single ended USB D+ Input.
ser_rx_dm	IN	0	Single ended USB D- Input.
ser_speed	OUT	Open	Edge rate control signal. Logic "1" selects "full speed" edge rates. Logic "0" selects "low speed" edge rates.
ser_dppullup	OUT	Open	Control for pull-up control DP for FS-only applications

6.1.7.5 USB port power control signals

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 101-191

USB-HS Transceiver Interfaces (Power control)

Signal	Direction	Description
sess_valid	IN	VBUS session valid indicator for control of the device DP pullup. <i>Note: This must be provided to pass the USB-IF back drive tests. For bus powered devices this signal may be tied to logic 1.</i>
port_ind_ctl_[1:0]	OUT	Port indicator LED control outputs

6.1.7.6 USB power suspend control signals

The following signals support an external asynchronous power control unit. If no power control is necessary these signal may be left unconnected.

USB-HS Transceiver Interfaces (Power control)		
Name	Direction	Description
utmi_pwrctl_suspend	Out	Suspend output synchronized to the xcvr_clk. Will be set until a wake-up event occurs and xcvr_clk begins running.
ulpi_pwrctl_suspend	Out	Suspend output synchronized to the xcvr_clk and is set after the write on ULPI bus. Will be set until a wake-up event occurs and the xcvr_clk begins running.
ser_pwrctl_suspend	Out	Suspend output synchronized to the xcvr_ser_clk. Will be set until a wake-up event occurs and the xcvr_ser_clk begins running.
pwrctl_suspend_clr	Out	Output to notify of software commanded wake-up. Not synchronized and will remain set until all the PHY_pwrctl_suspend output signals are cleared. (only after all clocks begin running)
pwrctl_wakeup	In	Input that clears the suspend output. All suspend outputs are synchronized to the appropriate clock and this input will not propagate to the suspend outputs until the related clock begins running. Thus is must remain asserted until the related suspend output transitions to zero.

6.1.8 High Speed Receive Buffer RAM

High Speed Receive Buffer RAM [dual-port]		
Signal	Direction	Description

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 102-191

rx_buf_addr_a[VUSB_HS_RX_ADD-1:0]	OUT	Address bus A
rx_buf_data_wr_a[35:0]	OUT	Data write bus A - bits [35:32] used as a tag field for the data
rx_buf_wr_en_a	OUT	Data write enable A
rx_buf_clk_a	OUT	Clock A - connect to pe_clk [see pe_clk notes]
rx_buf_addr_b[VUSB_HS_RX_ADD-1:0]	OUT	Address bus B
rx_buf_rd_en_b	OUT	Data read enable B
rx_buf_data_rd_b[35:0]	IN	Data read bus B - bits [35:32] used as a tag field for the data
rx_buf_clk_b	OUT	Clock B – Connect to system clock

6.1.8.1 High Speed Receive Buffer Wiring

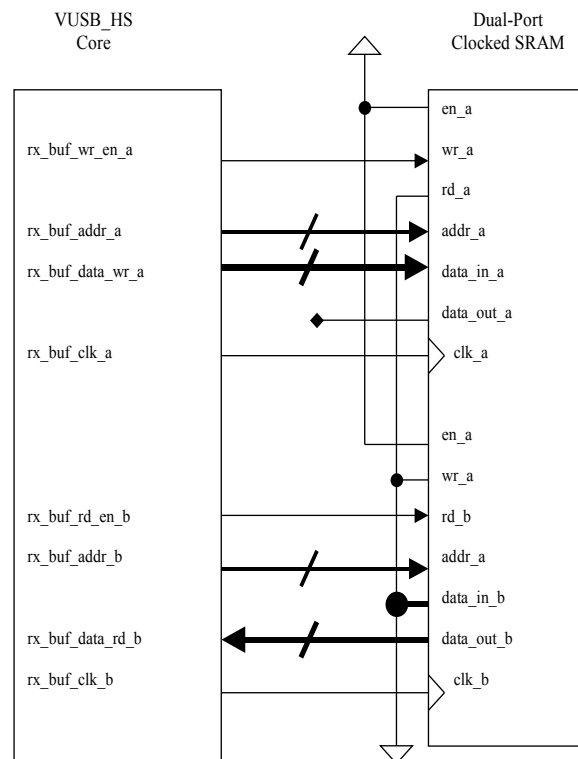


Figure 48 High Speed Receive Buffer Wiring

Optional: rx_buf_wr_en_a can also be used as a power control for port A.
Optional: rx_buf_rd_en_b can also be used as a power control for port B.

6.1.8.2 High Speed Receive Buffer Timing

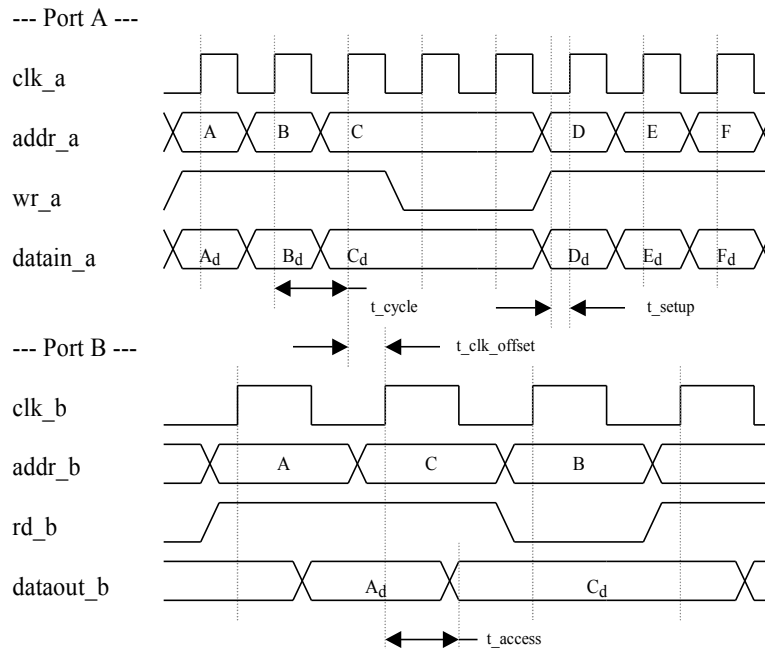


Figure 49 High Speed Receive Buffer Timing

SRAM Timing Parameters:

t_{setup}^{**} : SRAM Input Setup To Clock.

t_{access}^{**} : SRAM Clock To Output Delay.

****** The actual values for these parameters must be added to the synthesis script to ensure the correct timing interaction between the USB-HS design and the target silicon memory array.

t_{cycle}^{++} : SRAM min. clock period must be less than or equal to the device controller system clock period. i.e. $T_{cycle} (min.) < T_{sys}$.

$t_{clk_offset}^{++}$: SRAM min. clock offset where a read to write access at the same address will correctly provide the read data must be less than $\frac{1}{2}$ the system clock period. i.e. $T_{clk_offset} (min.) < \frac{1}{2} T_{sys}$;

++ These values are parameters of the SRAM that must meet the equations above relating the target silicon SRAM timing parameter to the USB-HS system clock.

NOTE Timing parameter t_{clk_offset} is only relevant to the mixed clocked design discussed in section 7.2. In mixed clocked design, the two clocks of the Dual-Port SRAM have no specified relationship and this parameter constrains the case where the rising edge of the write clock leads the rising edge of the read clock and the address of the read and the write are identical. The SRAM requirement is that when this situation occurs, the SRAM must be able to properly perform the read so long as the clock separation is greater than or equal to $\frac{1}{2}$ cycle of the read clock.

6.1.9 High Speed Transmit Buffer RAM

High Speed Transmit Buffer RAM [dual-port]		
Signal	Direction	Description
tx_buf_addr_a[VUSB_HS_TX_ADD-1:0]	OUT	Address bus A
tx_buf_data_wr_a[35:0]	OUT	Data write bus A - bits [35:32] used as a tag field for the data

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 104-191

tx_buf_wr_en_a	OUT	Data write enable A
tx_buf_clk_a	OUT	Clock A – Connect to system clock
tx_buf_addr_b[VUSB_HS_TX_ADD-1:0]	OUT	Address bus B
tx_buf_rd_en_b	OUT	Data read enable A
tx_buf_data_rd_b[35:0]	IN	Data read bus B - bits [35:32] used as a tag field for the data
tx_buf_clk_b	OUT	Clock B - Connect to system clock

6.1.9.1 High Speed Transmit Buffer Wiring

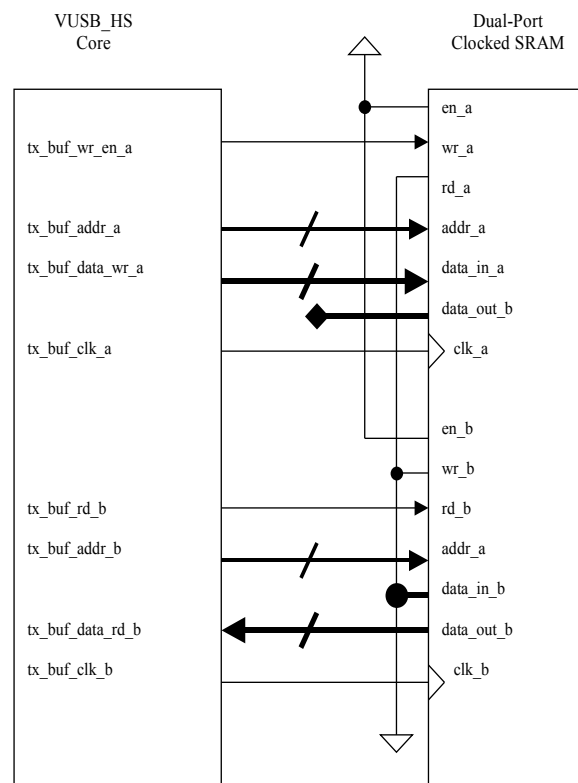


Figure 50 High Speed Transmit Buffer Wiring

Optional: tx_buf_wr_en_a can be combined as a clock enable power control for port A.

Optional: tx_buf_rd_en_b can also be used as a clock enable power control for port B.

6.1.9.2 High Speed Transmit Buffer Timing

See [High Speed Receive Buffer Timing](#)

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 105-191

7 System Level Issues and Core Configuration

The USB-HS core is designed to be part of a larger system containing a processor, a bus architecture, and a memory system. This section is intended to provide information about the USB-HS core that is required by a system designer to construct a complete system.

7.1 Configuration Constants

The USB-HS core has a small number of configuration options that a system engineer can use to adapt the core to the system environment. All the core configurations are controlled by constants in the `vusb_hs_cfg.v(hd1)` file. The following table summarizes the configuration controls. More detailed information is found in the sections below and in the comments contained in the `vusb_hs_cfg.v(hd1)` file.

Table 21 Configuration Controls

Constant	Description	Supported Values
VUSB_HS_RESET_TYPE	Controls the reset type.	0=Synchronous, 1=Asynchronous.
VUSB_HS_RESET_OPTIONAL	Adds resets to register files and other groups of registers where a reset is not functionally required. This is only used in the MPH design and has no effect on DEV, SPH, or OTG. The MPH product has this option because a large number of flops are affected and adding resets to those flops has considerable increase on the gate count. All registers have reset in DEV, SPH, and OTG.	0=No extra resets 1=Reset all flops
VUSB_HS_CLOCK_CONFIGURATION	Controls the clocking architecture.	[0,1,2,3] See following section on System Clocking.
VUSB_HS_NUM_PORT	Configures the number of ports for the multi-port host product. All other products must leave this value at 1.	[1-8] Number of downstream host ports.
VUSB_HS_PHY16_8	Controls the data interface to the UTMI transceiver. Should be set to match the transceiver connected to the core.	0 = 8-bit data bus [60MHz] 1 = 16-bit data bus [30MHz] 2 = Software Controlled reset to 8-bit data bus [60MHz] 3 = Software Controlled reset to 16-bit data bus [30MHz]

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 106-191

Constant	Description	Supported Values	
VUSB_HS_PHY_TYPE	Controls which USB transceiver interface is being used.	0 = UTMI Only 1 = Reserved 2 = ULPI Only 3 = Serial FS Only 4 = Software controlled reset to UTMI 5 = Reserved 6 = Software controller reset to ULPI 7 = Software controlled reset to Serial FS	
VUSB_HS_PHY_UTMI	Derived from VUSB_HS_PHY_TYPE. Controls presence of UTMI/UTMI+ characterization logic.	PHY_MODE	Value
		0	1
		1	0
		2	0
		3	0
		4	1
		5	0/1
		6	0/1
		7	0/1
		6	0/1
		7	0/1
VUSB_HS_PHY_ULPI	Derived from VUSB_HS_PHY_TYPE. Controls presence of ULPI characterization logic.	PHY_MODE	Value
		0	0
		1	0
		2	1
		3	0
		4	0/1
		5	0/1
		6	1
		7	0/1
VUSB_HS_PHY_SERIAL	Controls presence of SIE logic. (0) not present ; (>0) present. Must be coordinated with VUSB_HS_PHY_TYPE.	0 = Serial engine not present. Do not use if PHY_MODE = 1,3,5,7. 1 = Serial engine is present and used for all LS/FS operations. 2 = Serial engine is present & defaulted to not be used for FS/LS operations where possible. 3 = Serial engine is present & defaulted to be used for all FS/LS operations.	
VUSB_HS_DEV_EP	Controls the maximum number of endpoints supported by the core.	Integer values between 2 and 16. Set this to 1 for single and multi-port host-only products.	

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 107-191

Constant	Description	Supported Values
VUSB_HS_NUM_PORT	Controls the number of downstream ports in a host implementation.	Integer values between 1 and 8. Set this to 1 for non-multiport products.
VUSB_HS_TT_PERIODIC_CONTEXTS	USB 2.0 specification requires a Hub Transaction Translator to have 16 periodic contexts. However, for some host applications 4 may be adequate and a gate count savings can be realized.	4 or 16.
VUSB_HS_RX_DEPTH	Controls the size of the receive latency buffer. This value is measured in the number of 36 bit locations.	Powers of 2 from 8 to 4096.
VUSB_HS_RX_BURST	Controls the Bus burst size for Rx DMA data transfers. This value is measured in the number of 32 bit accesses.	Integer values from 4 to 128.
VUSB_HS_TX_CHAN	Controls the size of each of the transmit latency buffers. This value is measured in the number of 36 bit locations.	Powers of 2 from 16 to 256.
VUSB_HS_TX_BURST	Controls the Bus burst size for Tx DMA data transfers. This value is measured in the number of 32 bit accesses.	Integer values from 4 to 128.
VUSB_HS_HCIVERSION	Software readable host silicon version.	Unsigned 16-bit integer.
VUSB_HS_DCIVERSION	Software readable device silicon version.	Unsigned 16-bit integer.

7.2 Bus Configuration

USB-HS is delivered with two bus interfaces, BVCI and AMBA-AHB. The selection of the bus interface for running simulation or synthesis out-of-the-box is accomplished using the switches documented in the getting started section of the release notes. However, for customers that assemble the file list manually, refer to the 9 for the file hierarchy.

Several files in the hierarchy have both BVCI and AMBA-AHB versions starting from the top level core and going down the hierarchy tree to the leaf designs. Files that exclusively used with the BVCI bus have the suffix: `_bvci`. Likewise, files that are used for the AMBA-AHB design has the suffix: `_amba`.

The following example shows how the files differ:

BVCI File Set:

```

vusb_hs_core_   dev _bvci
...
vusb_hs_dma_    dev _bvci
...
vusb_hs_dma_mem_arb_bvci
...
vusb_hs_up_int_bvci

```

		<i>Title:</i> CI13410-a High-Speed USB Device Controller Core			<i>Revision:</i> 1.0A	
<i>Template:</i> TP02_DEP_GP	<i>Department:</i> IPCS	<i>Prepared by:</i> Joana Braga	<i>Approved by:</i> Luis Miguel Sampaio	<i>Security Level:</i> confidential	<i>Date:</i> 2006-05-11	<i>Page:</i> 108-191

vusb_hs_up_int_regfile
vusb_hs_up_int_busif_bvci

...

AMBA-AHB File Set:

vusb_hs_core_ dev _amba

...

vusb_hs_dma_ dev _amba

...

vusb_hs_dma__mem_arb_amba

...

vusb_hs_up_int_amba

vusb_hs_up_int_regfile

vusb_hs_up_int_busif_amba

...

7.3 Core and System Clocking

The USB-HS is designed to minimize the issues surrounding multiple clock domains. Four different clocking topologies are supported directly in the USB-HS core.

The optional serial (Full Speed) legacy transceiver interface has a separate 60MHz clock domain that is synchronized to the system clock with its own clock crossing logic. This 60MHz clock domain may share the same clock as the 60MHz used for the 8-bit UTMI interface. The serial interface 60Mhz clock domain will only be present when a serial interface is necessary and that is determined by the VUSB_HS_PHY_MODE constant (serial interface present when not equal to 0).

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 110-191

7.3.2 System clocked design

The next simplest clocking method is to use the customer's system or processor clock to clock all the circuits in the USB-HS core. The USB-HS core makes this possible by building a clock domain crossing and synchronization stage into the transceiver interface. There are several advantages to this clocking architecture:

- Single clock for synthesis and static timing analysis of the USB-HS core as part of the customer's SOC design.
- Synchronous Master and Slave interfaces onto the system bus eases integration with arbitration logic, synchronous memories, and bus signaling.
- Metastability analysis and multiple clock timing paths are grouped to a small well defined number of signals in ratematch clock crossing logic block.

This mode is selected by setting the **VUSB_HS_CLOCK_CONFIGURATION** constant equal to 1.

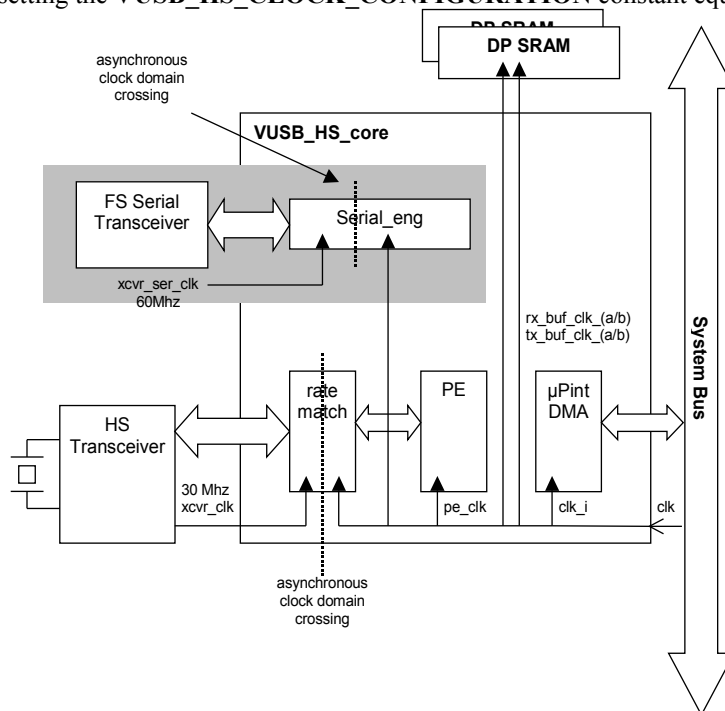


Figure 52 System clocked design

In order for the rate match block to function correctly, the system clock rate must exceed the transceiver 30MHz word rate. A minimum system clock frequency of 33MHz is needed to achieve this constraint.

Meeting the USB 2.0 turnaround times [IN-to-data(device only) & CRC-to-handshake] with the delay added by the clock crossing circuit at the transceiver boundary increases the constraint to the system clock frequency beyond the 33Mhz frequency mentioned above. Thus, the minimum system clock frequencies become:

UTMI 16-bit transceiver interface: 150Mhz (Not supported).

UTMI 8-bit transceiver interface: 53Mhz.

Other USB 2.0 parallel transceiver interfaces: Contact Support.

Because of the high system clock frequency required to meet the USB turnaround time specification with 16-bit High Speed PHY parts this clock architecture is not supported for 16-bit UTMI interface transceivers.

This clocking architecture requires the USB-HS core to run at clock frequencies that are dictated by the speed of the microprocessor system that the core is used with. The frequency target for the USB-HS core in 0.18micron CMOS is greater than 200MHz (see release notes for latest frequency targets).

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 112-191

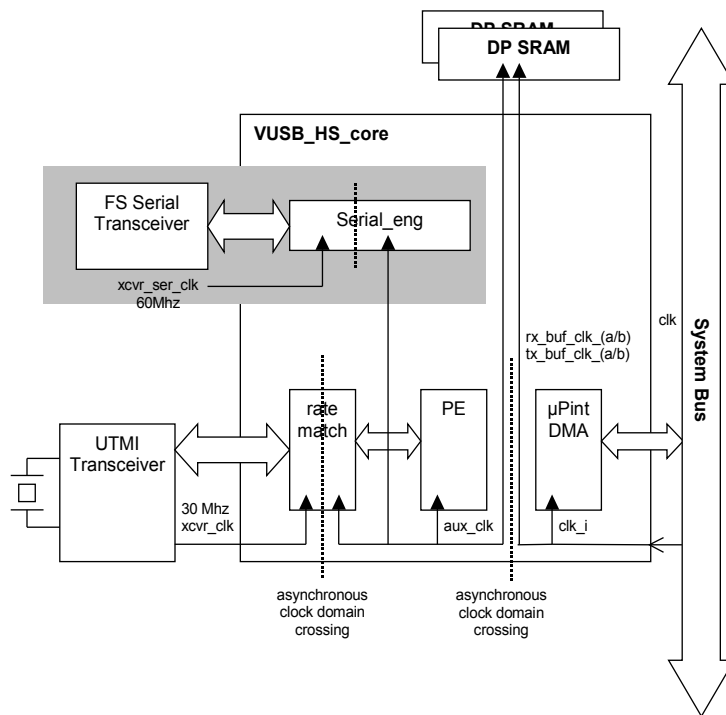


Figure 54 Triple Clocked Design

7.3.5 Negative edge clocks

The USB-HS design optionally uses the negative edge of clock to cut the amount of time lost when moving a signal across an asynchronous clock domain boundary using dual rank synchronizers. The negative edge triggered flip-flops are denoted by a `_neg_edge_s1` or `_ne_s1` suffix in the signal name.

The negative edge triggered flip-flops are optional mainly because in CLOCK CONFIGURATIONS #0 & #2, they are unused and will be removed by synthesis optimization. For CLOCK CONFIGURATION #1 & 3, the negative edge triggered flip-flops are included in the design by default. It is possible to switch the negative edge triggered flip-flops to a rising edge given a minimum system clock frequency of 2 times the transceiver clock. Chipidea Support can guide the user in making these changes if it is necessary to eliminate the negative edge-triggered flip-flops from CLOCK CONFIGURATION #1 & 3.

The following flip-flops operate on the negative edge of the clock:

Table 22 Negative edge triggered Flip Flops

Signal Name	Width	HDL Filename	Note
pe_rx_grey_ne_s1	2	vusb_hs_portctrl_rm.v(hdl)	1)
pe_linestate_ne_s1	1	vusb_hs_portctrl_rm.v(hdl)	1)
pe_txempty_ne_s1	1	vusb_hs_portctrl_rm.v(hdl)	1)
sync_wr2rd_ptr_gray_ne_s1	3	vusb_hs_portctrl_rm_fifo.v(hdl)	1)
sync_wr2rd_wr_en_ne_s1	1	vusb_hs_portctrl_rm_fifo.v(hdl)	1)
sync_wr2rd_full_ne_s1	1	vusb_hs_portctrl_rm_fifo.v(hdl)	1)
sync_rd2wr_ptr_gray_ne_s1	3	vusb_hs_portctrl_rm_fifo.v(hdl)	1)

Notes for table 63.

1) Used when `vusb_hs_clock_configuration = 1` or `3`.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 113-191

7.4 Reset Strategy

NOTE: ULPI INTERFACE LOCK-UP CAN RESULT IF SYNCHRONOUS RESETS ARE SELECTED (VUSB_HS_RESET_TYPE = 0) AND THE ULPI TRANSCEIVER CLOCK IS DISABLED.

The USB-HS core has two reset sources, an external reset signal, and a generated software reset. The two reset sources are combined into a single reset `rst_local`. Two reset nets are run through the core, the external reset and the combined `rst_local`. `rst_local` is used wherever possible to reset the core registers. The combined reset is also synchronized to the transceiver clock domain and supplied to the transceiver.

The register coding syntax in the USB-HS core supports either synchronous or asynchronous reset strategies. To select the reset strategy, use the `VUSB_HS_RESET_TYPE` constant.

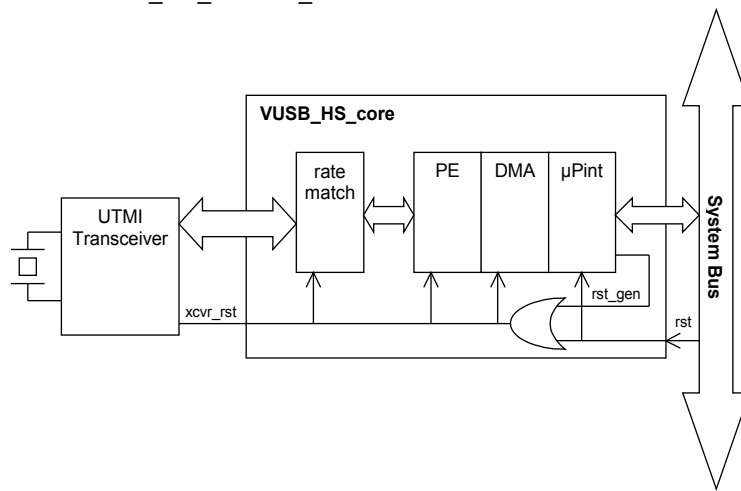


Figure 55 Reset Strategy

7.5 Bandwidth and Latency Issues

The USB-HS core is able to move data at a peak rate of 480Mb/s. This results in burst requirements of 15M 32-bit word per second transfer rate on the system memory bus. These transfers may be read or write and may be sustained for periods corresponding to the maximum packet size of 1kByte on the USB. At the start and end of each transfer the DMA controller interleaves access required to process the DMA descriptors in memory with the data transfers this results in short bursts of activity at 22M 32-bit word per second.

The time constants on the USB for data response to a token are measured in hundreds of nanoseconds. Because the time to reach system memory may exceed the required USB response times, small buffers are included in the USB-HS core architecture to overcome the latency. A single buffer is employed for all receive data, but each transmit endpoint is required to have a dedicated latency buffer. These buffers are preloaded or primed in anticipation of transmit data traffic from the USB-HS core to a remote USB host.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 114-191

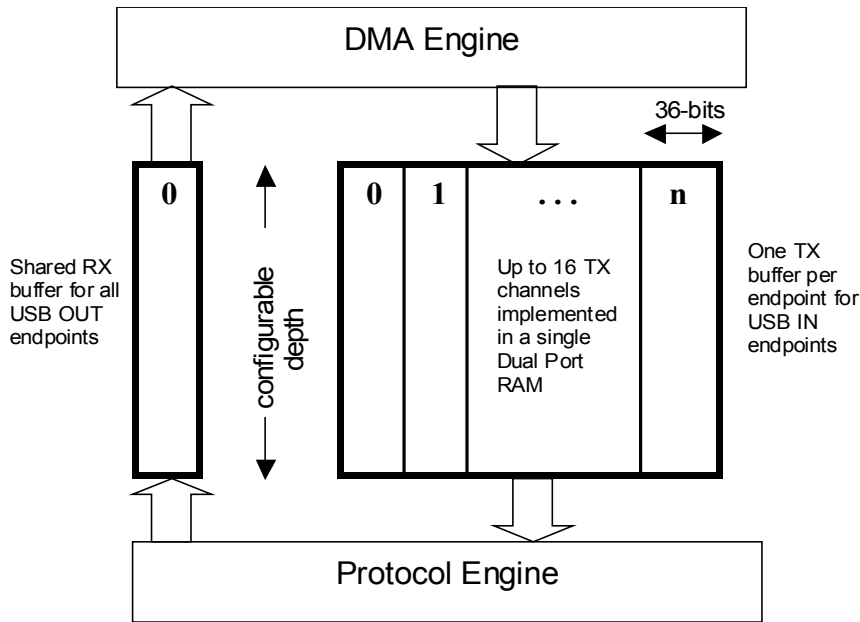


Figure 56 Latency Buffer Architecture

7.5.1 Sizing the device controller endpoint latency buffers

The endpoint latency buffers must be sized to allow a transfer to start on the USB with the data being sourced from or sunk to the buffer. The buffer must contain sufficient data space to allow the transfer to continue without any bubbles in the pipe for the period of time required to allow the DMA controller to start a data transfer from system memory. Having insufficiently sized buffers will cause an underrun or overrun error in the transfer. Signaling back to the host will cause the host to discard the packet. The underrun or overrun error is a soft error in that the host will retry the transfer under the USB protocols. The latency buffers should be sized to prevent these errors under normal conditions. It is, however, acceptable to size the buffers such that there are rare events that will cause these error conditions.

The buffers are built using 36-bit wide dual port RAM. The first 4 36-bit words are reserved for storing the DMA context information associated with an ongoing transfer. In addition the buffers are restricted to powers of 2 in size. The minimum size that can be implemented is 8 36-bit words. Common sizes in system application are 8, 16, 32 and 64 36-bit words.

The buffer space required (SIZE) by each of the transmit (IN) endpoint buffers of the device controller is a function of the transfer bandwidth on the USB (DR - buffer drain rate), the latency of the memory system to deliver the first byte of data (T(latency)) and the desired bus burst size (BBS) on the system bus. An additional 4 words are required in each buffer to hold the DMA context information for active transfers.

$SIZE = T(latency) * DR + BBS + 4$ where

$T(latency) = T(memory\ access) + T(DMA\ start)$

T(Memory Access) is a function of BVCI/AMBA bus interface, the memory system and other competing elements of the system architecture that may cause a delay in data return to the USB-HS core.

T(DMA start) is nominally four clock cycles at the system clock frequency.

In addition to the nominal DMA start delay there is a latency associated with the current state of the DMA controller when a token is received on the bus. In the worst case an IN token will arrive just after a processor request to prime one of the other endpoints. In this case the DMA controller will need to complete its current bus burst, store the priming DMA context in the buffer RAM, restore the DMA context for the new transfer from the buffer RAM, before it can initiate the data transfer required to support the token on the bus. In the worst case analysis these operations will

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 115-191

require an additional T(memory access) to initiate the priming operation, plus the time required to complete a bus burst, plus 16 additional cycles at the system clock frequency to save and restore DMA context and initiate the data transfer required to support the request from the USB.

The size of the Transmit and Receive latency buffers are controlled by manipulating the following constants in the vusb_hs_cfg file : **VUSB_HS_TX_CHAN**, **VUSB_HS_TX_BURST**, **VUSB_HS_RX_DEPTH**, **VUSB_HS_RX_BURST**.

The following table shows the total latency that may be accommodated by a latency buffer as a function of the buffer size (SIZE) and the bus burst size (BBS) for some common buffer and bus burst sizes. Note that the buffer sizes, and burst sizes are not limited to this subset of values.

Table 23 Latency tolerance as a function of buffer and bus burst size

Buffer size (in 36-bit words)	Bus Burst size BBS (in 32-bit words)		
	2-words	4-words	8-words
8-words	133 ns	0 ns	Not Possible
16-words	666 ns	533 ns	266 ns
32-words	1733 ns	1600 ns	1333 ns
64-words	3866 ns	3733 ns	3466 ns

7.6 Interrupt frequency

The data structures used for host controller and device controller application of the USB-HS core allow multiple packets of data to be queued in memory and scheduled for transmission over the USB. These structures contain a field that allows the hardware interface software to control which events cause an interrupt to the processor. Possible interrupt sources include:

- Connect and disconnect events.
- Bus reset or resume signaling
- Packet completion
- Completion of a multi-packet transfer
- Completion of all queued transfers
- Frame or microframe.

To allow the system processor to work efficiently, the USB-HS core allows a large amount of data to be queued for transfer, and the events that cause interrupts to be accumulated. This is so that the processor can deal with them as a group and cut the number of context switches required while still maintaining full transfer bandwidth on the USB.

7.7 Power

The USB-HS core is a fully synchronous static design. The power used by the design is dependant on the implementation technology used to fabricate the design, and by the application usage of the core. Applications that transfer more data or use a greater number of packets to be sent will consume a greater amount of power.

Because the design is synchronous and static, power may be conserved by reducing the transitions of the clock net. This may be done in several ways.

The first is to reduce the clock frequency to the core. The clock frequency may not be reduced below the minimum recommended operating frequency of the core without first disabling the USB operation. A second way to reduce transition on the clock net is through the use of clock gating methods like those employed by Synopsys power compiler or similar tools. Finally, the clock may be shut off to the core entirely to conserve power. Again this may only be done after the USB operations on the bus have been disabled. A device may suspend operations autonomously by disconnecting from the USB, or in response to the suspend signaling on the USB has moved it into the suspend state. A host can suspend operation autonomously, or it can command portions, or the entire USB to transition into the suspend state.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 116-191

7.7.1 USB power states

The USB provides a mechanism to place segments of the USB or the entire USB into a low power suspend state. USB bus powered devices are required to respond to a 3ms lack of activity on the USB bus by going into a suspend state. In the USB-HS core software is notified of the suspend condition via the transition in the PORTSC register, optionally an interrupt can be generated which is controlled by the port change Detect Enable bit in the VSBINTR control register. Software then has 7 ms to transition a bus powered device into the suspend state. In the suspend state, a USB device has a maximum USB bus power budget of 500uA. In general, to achieve that level of power conservation, most of the device circuits will need to be switched off, or clock at an extremely low frequency. This can be accomplished by suspending the clock, or additionally by implementing independent power wells in the fabrication of the design. When the clock is disabled to the USB-HS core all functions are disabled, but all operational state is retained. Asserting the SuspendM signal to the transceiver will disable the transceiver clock from the UTMI transceiver and will cause the LineState signals to become live (non-clocked) outputs. The LineState signals can be used to construct a circuit that is able to detect the resume signaling on the bus and restore the clocks to the rest of the circuit. Implementing power wells requires intimate knowledge of the characteristics of the silicon process used to isolate the active and inactive portions of the design in the low power states. Power wells are beyond the scope of this document.

The implementation of low power states in the USB-HS core is dependant on the use of the device role (host or peripheral), whether the device is bus powered, and the selected clock architecture of the core. Bus powered peripheral devices are required by the USB specification to support a low power suspend state. Self powered peripheral devices and hosts set their own power management strategies based on their system level requirements. The clocking architecture selected is important to consider as it determines what portions of the design will remain active when transitioned into the low power state. In clock configuration 0 the entire System is clocked from the transceiver clock. In clock architecture 1 the transceiver clock is only used to cross the asynchronous boundary in the transceiver interface. In clock architecture 2 the transceiver clock is used to clock all the port control and protocol Engine functions (for more on supported clock architectures see Core and System Clocking

Before the system clock is suspended or set to a frequency that is below the operational frequency of the USB-HS core, the core must be moved from the operational state to a low power state. The power strategies designed into the USB-HS core allow for the most challenging case, a self powered device that is clocked entirely by the transceiver clock.

7.7.2 Device Power States

A bus powered peripheral device must move through the power states as directed by the host. Optionally autonomously directed low power states may be implemented.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 117-191

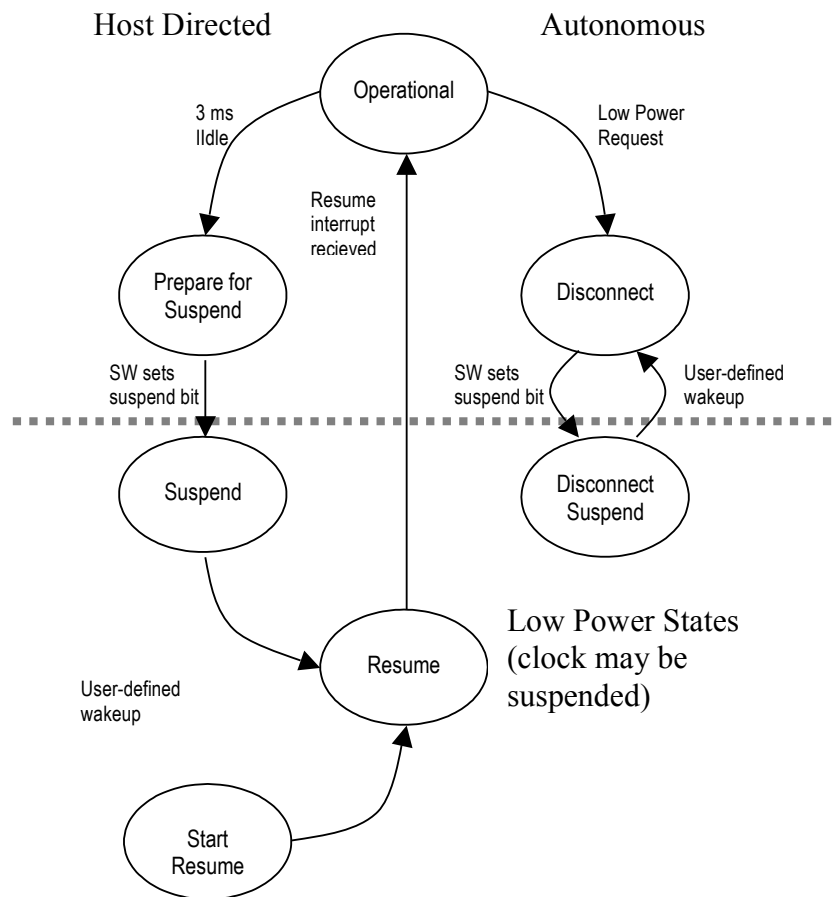


Figure 57 Device Power State Diagram

In the operational state both the transceiver clock and system clocks are running. Software can initiate a low power mode autonomously by disconnecting from the host to go into the disconnect state. Once in this state the software can set the suspend bit to turn off the transceiver clock putting the system in to the disconnect-suspend state. Since software cannot depend on the presents of a clock to clear the suspend bit, a wake-up event must be defined which would clear the suspend bit and allow the transceiver clock to resume.

The device can also go into suspend mode as a result of a suspend command from the host. Suspend is signaled on the bus by 3ms of idle time on the bus. This will generate a suspend interrupt to the software at which point the software must prepare to go into suspend then set the suspend bit. Once the suspend bit is set the transceiver clock may turn off and the device will be in the suspended state. The device has two ways of getting out of suspend. If remote wake-up is enabled a wake-up event could be defined which would clear the suspend bit. The software would then initiate the resume by setting the resume bit in the port controller then waiting for a port change interrupt indicating that the port is in an operational state. The other way for a device to get out of suspend is if the host puts resume signaling on the bus. This will clear the suspend bit and generate a port change interrupt when the resume is finished. In either case the system designer must insure an orderly restoration of the power and clocks to the suspended circuitry.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 118-191

7.7.3 Host Power States

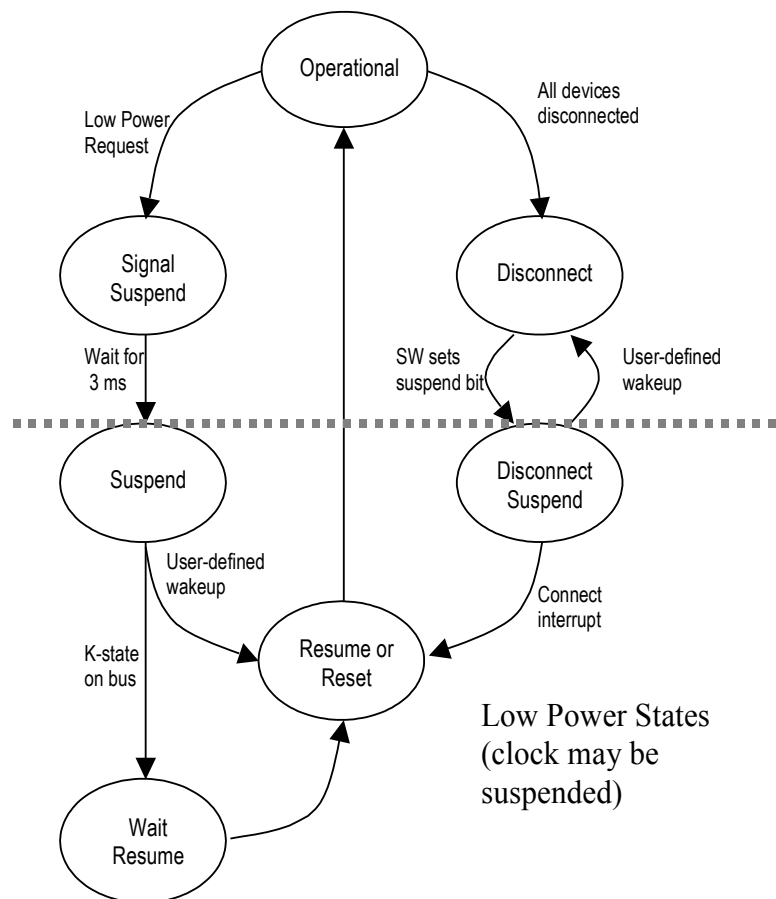


Figure 58 Host/OTG Power State Diagram

From an operational state when a host gets a low power request, it must set the suspend bit in the port controller. This will put an idle on the bus, block all traffic through the port and turn off the transceiver clock. There are two ways for a host controller to get out of the suspend state. If it has enabled remote wake-up, a K-state on the bus will turn the transceiver clock and generate an interrupt. The software will then have to wait 20 ms for the resume to complete and the port to go back to an active state. Alternatively an external event could clear the suspend bit and start the transceiver clock running again. The software can then initiate a resume by setting the resume bit in the port controller, or force a reconnect by setting the reset bit in the port controller.

If all devices have disconnected from the host, the host can go into a low power mode by the software setting the suspend bit. From the disconnect-suspend state a connect event would start the transceiver clock and interrupt the software. The software would then need to set the reset bit to start the connect process.

7.7.4 Suspend Control Logic

The suspend control logic is specialized logic block that is not included with the USB-HS core. The logic needs to be able to move between the low power states where the transceiver clock is suspended to the operational states in absence of a clock from the transceiver. To accomplish this, the suspend control block must be clocked on a clock that continues to run when the transceiver clock is suspended, or the block may be designed using asynchronous logic. The suspend control block needs to insure an orderly transition between the operational states and the low power states.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 119-191

The USB-HS core provides the following signals to control the suspend control block.

Name	Direction	Description
utmi_pwrctl_suspend	Out	Suspend output synchronized to the xcvr_clk. Will be set until a wake-up event occurs and xcvr_clk begins running.
ulpi_pwrctl_suspend	Out	Suspend output synchronized to the xcvr_clk and is set after the write on ULPI bus. Will be set until a wake-up event occurs and the xcvr_clk begins running.
ser_pwrctl_suspend	Out	Suspend output synchronized to the xcvr_ser_clk. Will be set until a wake-up event occurs and the xcvr_ser_clk begins running.
pwrctl_suspend_clr	Out	Output to notify of software commanded wake-up. Not synchronized and will remain set until all the PHY_pwrctl_suspend output signals are cleared. (only after all clocks begin running)
pwrctl_wakeup	In	Input that clears the suspend output. All suspend outputs are synchronized to the appropriate clock and this input will not propagate to the suspend outputs until the related clock begins running. Thus is must remain asserted until the related suspend output transitions to zero.

A two signal handshake is provided through the transceiver pwrctl_suspend and the pwrctl_wakeup signals that allows the suspend control signals to be passed to a lower frequency clock domain or an asynchronous power control block. The Transceiver pwrctl_suspend signal is asserted based on a write to the suspend bit in the PORTSC register, and will remain asserted until the pwrctl_wakeup signal is sampled on a rising edge of the transceiver clock. The deassertion of the transceiver pwrctl_suspend signal indicates that he pwrctl_suspend_signal has been seen, and can be deasserted by the power control block. The exception to this rule is a software commanded wake-up. In the case of a software commanded wake up the pwrctl_suspend signal will de-assert as soon as the pwrctl_suspend_clr is asserted and xcvr_clk is active. Pwrctl_suspend_clr will remain asserted until pwrctl_suspend de-asserts. To accommodate this behavior the power control block should not enter low power mode if pwrctl_suspend is not asserted, and when pwrctl_suspend de-assertion is used as a handshake to de-assert pwrctl_wakeup pwrctl_suspend should be treated as level sensitive not falling edge sensitive.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 120-191

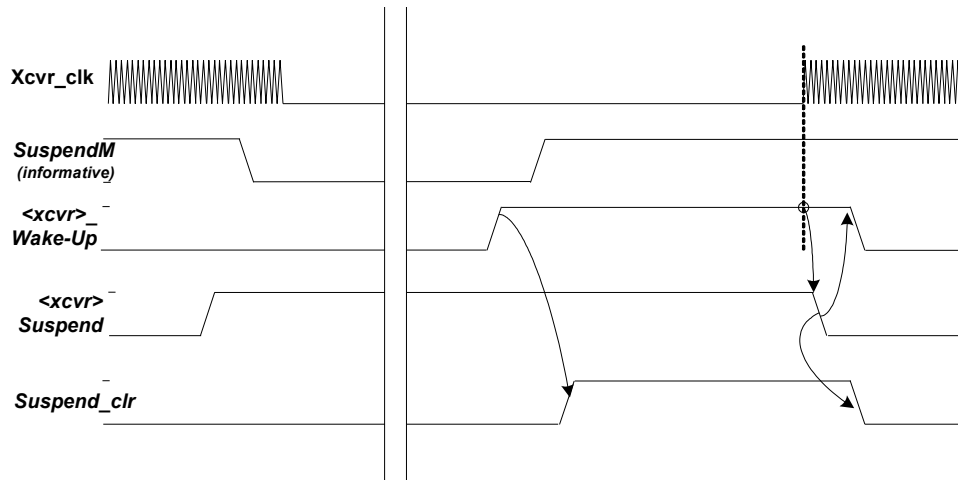


Figure 59 Suspend Wake-Up Handshake

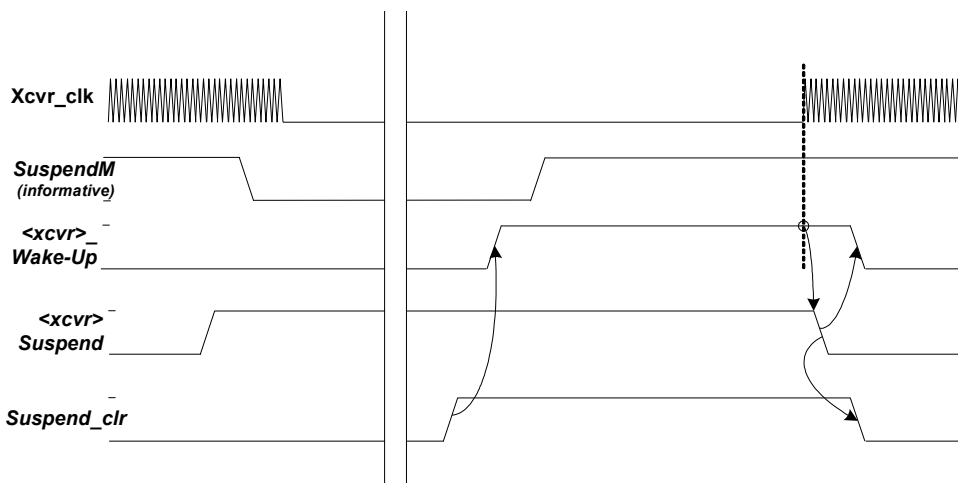


Figure 60 Software commanded Suspend Wake-Up Handshake

Note: If the xcvr_clk is running when pwrcfg_suspend_clr is asserted pwrcfg_suspend will de-assert at the next rising edge of clock.

7.7.4.1 UTMI Suspend Control Logic

The design of the UTMI suspend control block is simplified by the inclusion of clock conditioning logic in the UTMI transceiver. When the SuspendM signal is asserted to the UTMI transceiver, the transceiver shuts down the UTMI interface transceiver clock. Many clock suspend and clock resume elements of a traditional clock control block are specified as part of the UTMI transceiver. The UTMI transceiver handles:

- Orderly Shutdown of the PLL and Crystal Oscillator
- Sequenced startup and stabilization of the Crystal Oscillator
- Start up and Stabilization of the PLL

A compliant UTMI transceiver will deliver a stable frequency glitch free clock on either side of the suspend resume sequence.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 121-191

These signals can be used to control a simple asynchronous suspend control circuit that allows the low power states to be entered using the transceiver clock, but exited asynchronously based qualified on wake-up events. The following diagram shows how the suspend control block fits in the USB-HS based design.

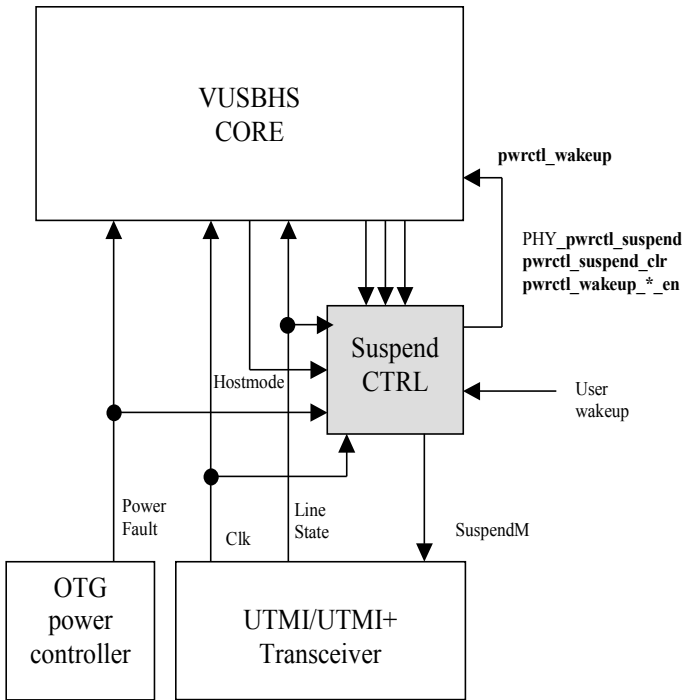


Figure 61 Suspend Control Connections

The circuit below is an example of a simple asynchronous Suspend control block. All of the clocked elements in the circuit below run on the transceiver clock. The circuit causes a wake up to occur whenever DP or DM change after the clock is halted. This handles the minimum wake up events required in a peripheral device application, USB Resume and USB Reset. Additional wake events can be added through the user wake signal, but the wake event used must not be dependant on the transceiver clock, as the transceiver clock will stop when SuspendM is asserted.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 122-191

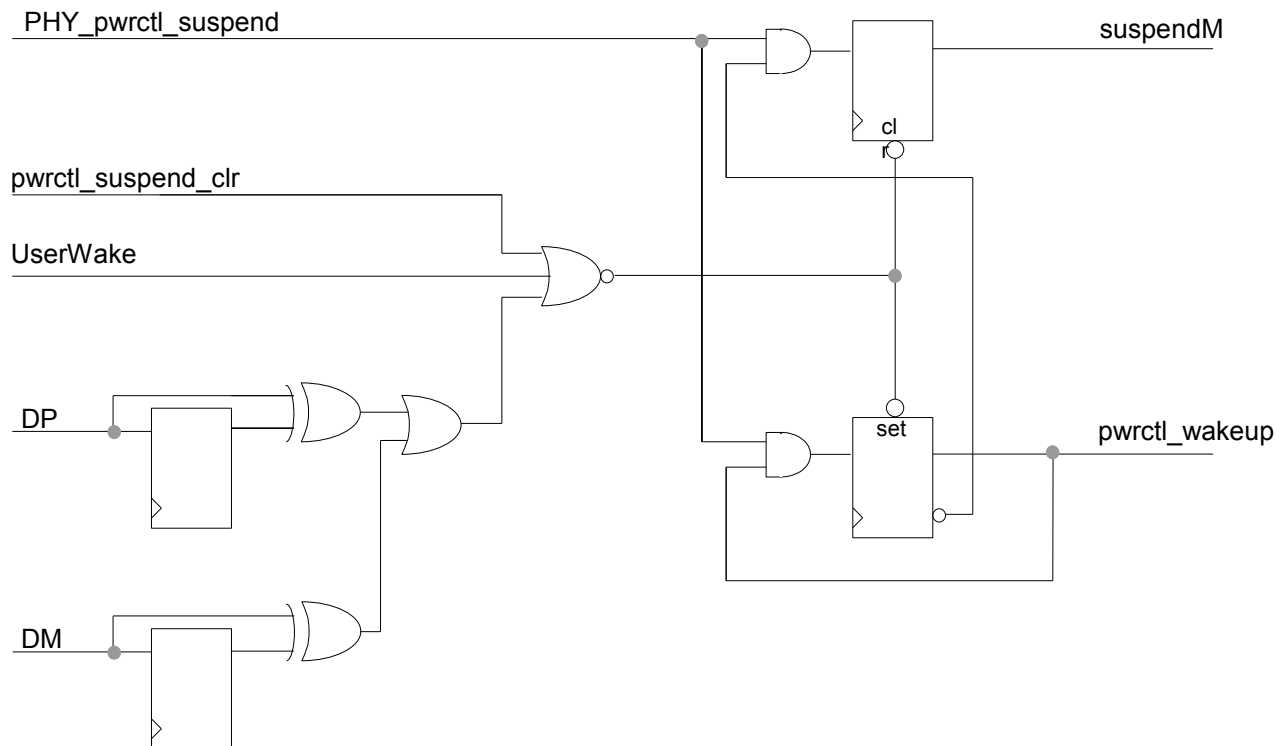


Figure 62 Device Asynchronous Suspend Control Example

This circuit makes use of asynchronous techniques that may require special attention when developing structured test circuits for the design. An alternate synchronous implementation can be developed that uses a separate low frequency clock that continues to run in the low power states.

The user must fully test the power control circuit and check all the wakeup conditions.

7.7.4.2 ULPI Suspend Control Logic

The UTMI circuits are adaptable to the ULPI with some notable exceptions:

NOTE: ULPI INTERFACE LOCK-UP CAN RESULT IF SYNCHRONOUS RESETS ARE SELECTED (VUSB_HS_RESET_TYPE = 0) AND THE ULPI TRANSCEIVER CLOCK IS DISABLED.

- The ulpi_pwrctl_suspend output will be asserted after the ULPI write to set low power mode.
- The suspendM is driven from the core through the ULPI bus and can be left out of the power control circuit.
- DP/DM can be tapped from the ULPI data bus (ulpi_rx_data(1:0)) which forwards the current line state asynchronously in low power mode..
- The ULPI low power mode INT signal (ulpi_rx_data(3)) should be added as an asynchronous wake-up signal for OTG designs.

- As with the UTMI interface, the user must fully test the power control circuit and check all the wakeup conditions.

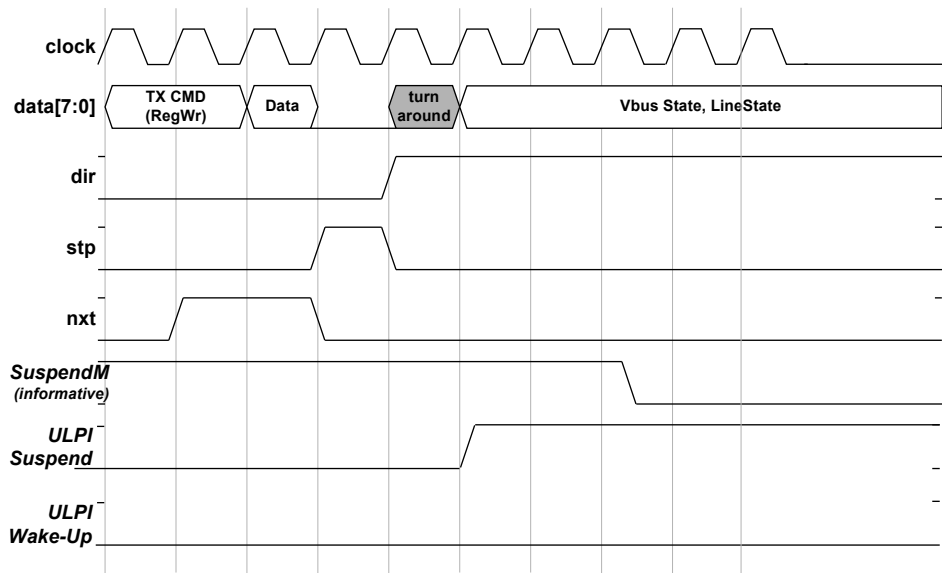


Figure 63 ULPI Power control interface behavior when entering low power mode

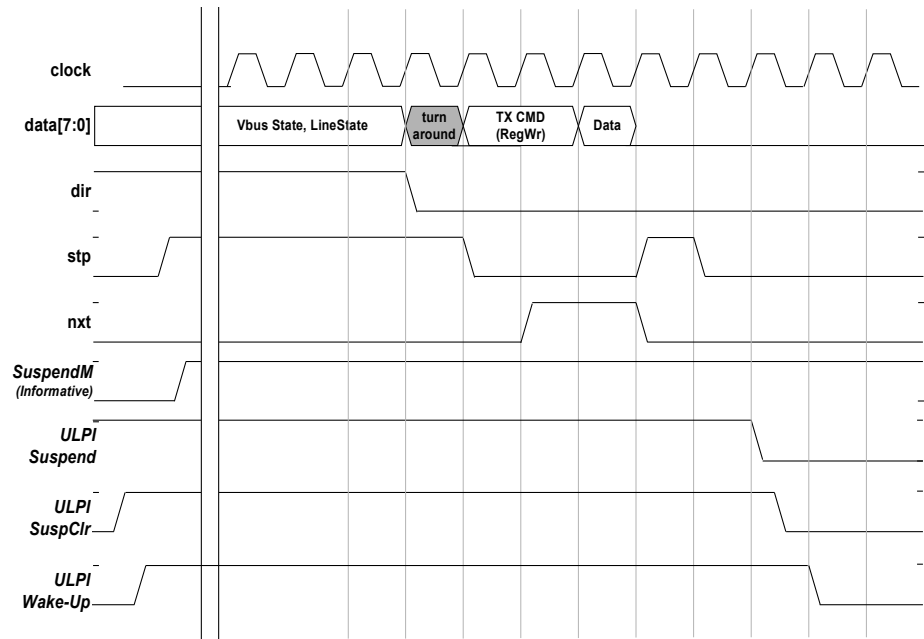


Figure 64 ULPI power control Interface behavior when exiting low power mode.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 124-191

7.7.4.3 Serial (Legacy) Suspend Control Logic

The UTMI circuits are adaptable to the Serial (Legacy) with some notable exceptions:

- The clock disable signal for the xcvr_ser_clk replaces the suspendM output.
- User must ensure the clock enable/disable transition is glitch free.
- DP/DM can be tapped from the digital serial interface pins of the legacy transceiver.
- As with the UTMI interface, the user must fully test the power control circuit and check all the wakeup conditions.

The USB-HS core is designed to interface with UTMI, ULPI, and Legacy Full speed transceiver chips, and macrocells. To function properly the core must be properly connected to the transceiver using the transceiver interface signals, and the proper transceiver must be selected.

7.7.5 Transceiver Selection

The USB-HS core is able to select the transceiver being used either statically, through configuration constants prior to synthesis, or dynamically under software control.

The static configuration is controlled through the VUSB_HS_PHY16_8 and VUSB_HS_PHY_TYPE, and VUSB_HS_PHY_SERIAL. These constants affect the logic synthesis along with the derived constants VUSB_HS_PHY_UTMI, and VUSB_HS_PHY_ULPI.

Configuring the transceiver selection statically will reduce the gate count of the port controller block in the USB-HS core, but it will eliminate the flexibility of using different transceiver parts with the same controller silicon. The transceiver selection constants are documented in Section Configuration Constants.

The dynamic configuration is controlled on a per port basis by bits in the Port Status and control register (PORTSCx). The three bits in the Port Status and control register that control the transceiver selection are called the Parallel Transceiver Select (PTS) and Serial Transceiver Select (STS). The first two bits allow you to select between UTMI, ULPI, and Full Speed Serial interfaces. The STS bit control when the Serial Interface Engine is used in place of the parallel transceiver when both exist. UTMI+, for example, has the option of running Full-speed traffic through the parallel interface or the serial interface. These transceiver selection control bits are documented in section PORTSCx.

The following table shows the interaction of the static and dynamic configuration controls on the transceiver selection. In the table RO means that the dynamic configuration control bit is Read Only, and can not be written to effect the operation of the device.

Table 24 Transceiver Selection Control Matrix

Static PHY_TYPE	PHY16_8	Dynamic PTS	PTW	Operation
0	0	RO-00	RO-0	8-bit UTMI
0	1	RO-00	RO-1	16-bit UTMI
1	X	RO-01	Ignore	Reserved
2	X	RO-10	Ignore	8-bit ULPI
3	X	RO-11	Ignore	Full Speed Serial
4,5,6, or 7	2 or 3	00	0	8-bit UTMI
4,5,6, or 7	2 or 3	00	1	16-bit UTMI
4,5,6, or 7	X	01	Ignore	Reserved
4,5,6, or 7	X	10	Ignore	8-bit ULPI
4,5,6, or 7	X	11	Ignore	Full Speed Serial

Table 25 Transceiver Serial Selection Control Matrix

Static PHY_TYPE	PHY_SERIAL	Dynamic STS	Operation
0,4	0	RO - 0	UTMI using parallel i/f for FS & LS

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 125-191

Static		Dynamic	
PHY_TYPE	PHY_SERIAL	STS	Operation
0,4	1	RO - 1	UTMI using serial i/f for FS & LS
0,4	2 or 3	0	UTMI using parallel i/f for FS & LS
0,4	2 or 3	1	UTMI using serial i/f for FS & LS
1,5	X	Ignore	Reserved
2,6	0	RO - 0	ULPI using parallel i/f for FS & LS
2,6	1	RO - 1	Not yet supported.
2,6	2 or 3	0	ULPI using parallel i/f for FS & LS
2,6	2 or 3	1	Not yet supported.
3,7	X	Ignore	FS/LS Serial

The **PHY_TYPE**, **PHY_SERIAL**, and the **PHY16_8** constant contribute to the default (reset) value of the PTS, STS and PTW bits in the port status and control register (PORTSCx). The following table shows the default operation after reset when the Transceiver selection is fully software selectable.

Table 26 Transceiver Selection Control Reset Values

Transceiver Selection Control Reset Values					
Static		Dynamic Reset Values			
PHY_MODE	PHY16_8	PTS	PTW	Operation	
4	2	00	0	8-bit UTMI	
4	3	00	1	16-bit UTMI	
5	X	01	X	Reserved	
6	X	10	X	8-bit ULPI	
7	X	11	X	Full Speed Serial	

Table 27 Transceiver Serial Selection Control Reset Values

Transceiver Selection Control Reset Values			
Static		Dynamic Reset Values	
PHY_SERIAL		STS	Operation
2		0	Using parallel interface for FS/LS operation where applicable
3		1	Using serial interface for FS/LS operation where applicable

The Values of the **PHY_TYPE**, **PHY_SERIAL** and **PHY16_8** constants are readable by software in the configuration constant read back value in the HWGENERAL Build configuration register (see: `vusb_hs_up_int.v(hd1)`).

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 126-191

8 Verification simulation environment

8.1 Introduction

8.1.1 Verification of the USB-HS Core

The verification of the USB-HS core makes use of an environment called the integration verification environment, consists of C++ software tests, an HDL host/device responder and the unit under test. The integration verification environment runs out of the box using a processor bus functional model, but allows the test to be ported and run on the SOC processor. The integration verification environment also supports testing with an integrated transceiver simulation model.

8.1.2 Integration Verification Environment

The integration verification environment has been designed to ease the validation of the integration of the USB-HS core in to the target system. The integration verification environment is delivered as part of the product deliverables.

8.1.3 Integration Verification Environment Features

The tests have been constructed to meet the following requirements:

- Port to the SOC processor
- Allow configuration of the tests that are executed without recompilation.
- Support incorporation of third party transceiver models
- Give an out of the box test system supporting multiple simulators.
- Identify problems typically found during a customer USB-HS core integration
- Test host and device and OTG functionality with all USB-HS data types.
- Identify problems in the integration of the USB-HS core with a new processor and memory system.
- Identify system level requirement violations at run-time, such as bus bandwidth.
- Generate useful information during a failure to help quickly identify the problem.
- Generate diagnostic information with instrumented software to allow our support staff to better assist a customer during integration.

8.1.4 Test Bench Architecture

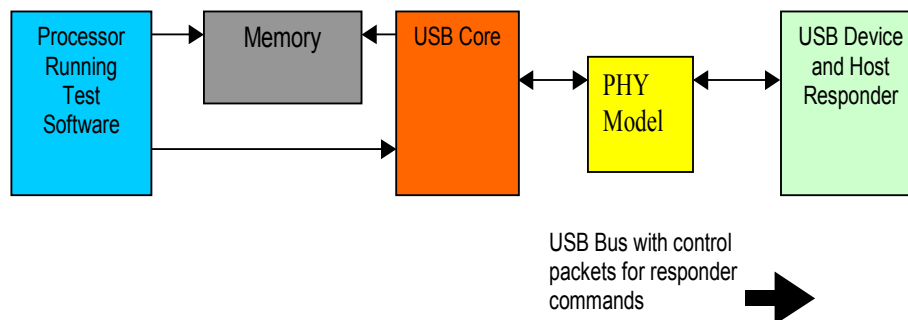


Figure 65 Test Bench Architecture Block Diagram

8.1.4.1 The Integration Verification Environment Software

The delivered software consists of two main components.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 127-191

1. The first is a co-simulation bus functional model that will connect into a simulator and behave like a processor. The model contains a SystemC behavioral processor and memory subsystem and is capable of running C and C++ software.
2. The second software component is a series of USB-HS tests designed to run in conjunction with the USB-HS HDL responder.

The test software is intended to be hosted on an embedded processor and is written in a portable manner which will allow hosting on the co-simulation bus functional model.

More information on the software layers is provided later in this chapter.

Many peripheral test bench designs use non-portable control ports (out of band control ports or side band signaling) to send messages to the responder from software, this makes the test software difficult to use in the customer's environment. The integration tests provided do not require special control port functionality implementation during processor integration.

8.1.4.2 The HDL Responder Model

The responder model is an HDL behavioral model of a USB2.0 device and host with support for OTG signaling. The test command interface is an in-band (it uses control pipe zero) connection between the software and the responder for both host and device testing.

The interface to the USB bus consists of a five bit representation that is used to model the characteristic states that can exist on the DP/DM wires. Additional details about the five bit modeling can be found later in this chapter.

The connection speeds supported include both High Speed and Full Speed operation as a Host Responder and High Speed, Full Speed and Low Speed as a Device Responder. The responder model acts as a USB host when the unit under test is a device, the responder initiates all USB bus transactions. The responder model acts as a USB device with four configurable loop back endpoints when the unit under test is the host. This models support all USB protocols associated with initialization, attachment, speed negotiation, OTG transitions and packet transmission for both host and device. The responder models also include both a USB2.0 bus protocol checker as well as test specific data and transaction checkers.

8.1.4.3 The PHY Behavioral Model

The verification environment includes a behavioral PHY (transceiver) model. This model is capable of behaving like one of three different types of transceivers. This model is a behavioral model and does not fully model all possible delays and latencies. The three transceiver types modeled include UTMI+, ULPI, and USB 1.1 serial. The model has 3 single bit enable signals that control the type of PHY to be modeled. The PHY behavioral model includes a super set of all I/O ports for all 3 transceiver types. Inputs that are not relevant to the currently selected PHY type are ignored and should be tied inactive. Outputs not relevant to the currently selected PHY type should be ignored. Customers are encouraged to simulate with models provided by their PHY vendors to verify compatibility.

8.1.4.4 Test Control Command Channel

The USB responder model is controlled using a test control command channel implemented as a command set on the USB endpoint 0 control pipe. This use of in-band signaling allows the test environment to be ported into a SOC environment without requiring any additional signals to support the stimulus generation or results checking. When the responder is behaving like a host, it will send setup packets to the device under tests and the software will send commands as the data phase of the control transfer. When the responder is behaving as a device, the responder will accept the commands sent in the control data phase.

8.1.5 The Simulation tests Shipped With The USB-HS Core

A complete library of tests is delivered with the USB-HS core. The library supports test of USB device (function), USB Host and USB OTG functions. The test software interrogates the hardware to determine which tests to run based on the USB HS core configuration. The level of testing is also configurable without recompiling the test code

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 128-191

through the test configuration control file. The test software reads this file to determine what test to run, and how much randomized testing to perform with each of the tests. The default test configuration control file delivered with the USB HS core is targeted for full integration interconnect verification in minimum simulation time.

8.1.5.1 Running The Delivered USB-HS Simulation tests

Running the tests out of the box requires setting up the simulator, copying the binaries onto the workstation and running the test script. Chipidea provides binaries for all the major simulators on all the major platforms in the standalone USB-HS release. See the Getting Started section later in this chapter for details of running the test scripts.

A compiled and linked version of the co-simulation BFM is provided with the release to reduce the number of steps required to get the simulation tests running.

The pre-compiled tests can be regenerated if desired. Chipidea provides all the source code for the integration tests, and the BVCI/AMBA bus functional model co-simulation environment that is used to create the compiled image.

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes
OTG	Register Interface default Read tests / "OTG, Register Interface"	Values are read in after reset and checked using the default values specified in the reference manual. Tests all reserved fields ensuring they always read back as zeros.	Note: When using the bus function models the configuration files will determine the level of BVCI/AMBA signaling variation. See Table for BVCI/AMBA below.
OTG	Register Interface Write Read tests / "OTG, Register Interface"	Writes a series of values to each field in the register interface and tests for the correct result.	
OTG	Register Interface Read Write Clear tests / "OTG, Register Interface"	Writes a series of values to each field in the register interface and tests for the correct result.	
Host	Register Interface default Read tests/ "Register Interface"	Values are read in after reset and checked using the default values specified in the reference manual. Tests all reserved fields ensuring they always read back as zeros.	
Host	Register Interface Write Read tests/ "Register Interface"	Writes a series of values to each field in the register interface and tests for the correct result.	
Host	Register Interface Read Write Clear tests/ "Register Interface"	Writes a series of values to each field in the register interface and tests for the correct result.	
Device	Register Interface default Read tests/ "Register Interface"	Values are read in after reset and checked using the default values specified in the reference manual. Tests all reserved fields ensuring they always read back as zeros.	
Device	Register Interface Write Read tests/ "Register Interface"	Writes a series of values to each field in the register interface and tests for the correct result.	
Device	Register Interface Read Write Clear tests/ "Register Interface"	Writes a series of values to each field in the register interface and tests for the correct result.	
Device	Interrupts Signal Connections tests/	Enables the test register for each interrupt type, checks for the	None

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 129-191

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes
Device	"Register Interface"	generation and clearing of each interrupt.	
Device	Interrupts Signal Connections tests/ "Interrupts Signal Connections"	Enables the test register for each interrupt type, checks for the generation and clearing of each interrupt. The test specifies a interrupt function for embedded software compatibility.	None
Host	Interrupts Signal Connections tests/ "Interrupts Signal Connections"	Enables the test register for each interrupt type, checks for the generation and clearing of each interrupt. The test specifies a interrupt function for embedded software compatibility..	None
Device	Test Packet Mode/ "Device, TestPacket"	Places a correctly formatted test packet into memory and enables the test packet mode using the test packet register. After the test is complete the test register is set back to the default mode and the test packet is removed from memory.	None
Host	Test Packet Mode/ "Host, HostTestModeTest"	Activates the host test packet mode using the test packet register. In the case of multi-port the test is issued on each of the ports. After the test is complete the test register is set back. In the case of multi-port all suspended ports are resumed.	None
Host	ULPI Register Interface testing/ "Testing: ULPI Register Mode"	Writes a series of values to each field in the register interface and tests for the correct result. In the case of multiport this test is completed for each port.	None
Device	ULPI Register Interface testing/ "Testing: ULPI Register Mode"	Writes a series of values to each field in the register interface and tests for the correct result.	None
OTG	Switch Modes Return Back To Start Mode MaxSpeed, NoSpeedToggle / "OTG, Switch Modes Return Back To Start Mode MaxSpeed, NoSpeedToggle*"	OTG transitions from Host to Device and Device to Host at the current connected speed. This test will not change to a new connect speed. 1-The core (AHost) will issue a CHANGE MODE request to the responder (BDevice). 2-The core (ADevice) will issue a CHANGE MODE request to the responder(BHost) 3-The core (AHost) will issue a reconnect command, which will swap the cable. 4-The core (BDevice) will issue a CHANGE MODE request to the responder(AHost) 5-The core (BHost) will issue a CHANGE MODE request to the responder(ADevice) 6-The core (BDevice) will issue a disconnect to the responder(AHost)	None
OTG	Switch Modes Return Back To Start Mode FullSpeed,	OTG transitions from Host to Device and Device to Host at Full Speed. During this test the connection will	None

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 130-191

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes
OTG	NoSpeedToggle / "OTG, Switch Modes Return Back To Start Mode FullSpeed, NoSpeedToggle "	remain full speed. 1-The core (AHost) will issue a CHANGE MODE request to the responder (BDevice). 2-The core (ADevice) will issue a CHANGE MODE request to the responder(BHost) 3-The core (AHost) will issue a reconnect command, which will swap the cable. 4-The core (BDevice) will issue a CHANGE MODE request to the responder(AHost) 5-The core (BHost) will issue a CHANGE MODE request to the responder(ADevice) 6-The core (BDevice) will issue a disconnect to the responder(AHost)	None
	Switch Modes Return Back To Start Mode FullSpeed, SpeedToggling / "OTG, Switch Modes Return Back To Start Mode FullSpeed, SpeedToggling"	OTG transitions from Host to Device and Device to Host changing speeds during the transitions, last connection is Full Speed. 1-The core (AHost) will issue a CHANGE MODE request to the responder (BDevice). 2-The core (ADevice) will issue a CHANGE MODE request to the responder(BHost) 3-The core (AHost) will issue a reconnect command, which will swap the cable. 4-The core (BDevice) will issue a CHANGE MODE request to the responder(AHost) 5-The core (BHost) will issue a CHANGE MODE request to the responder(ADevice) 6-The core (BDevice) will issue a disconnect to the responder(AHost)	
OTG	Switch Modes Return Back To Start Mode HighSpeed, SpeedToggling / "OTG, Switch Modes Return Back To Start Mode HighSpeed, SpeedToggling"	OTG transitions from Host to Device and Device to Host changing speeds during the transitions. Last connection is High Speed. 1-The core (AHost) will issue a CHANGE MODE request to the responder (BDevice). 2-The core (ADevice) will issue a CHANGE MODE request to the responder(BHost) 3-The core (AHost) will issue a reconnect command, which will swap the cable. 4-The core (BDevice) will issue a CHANGE MODE request to the responder(AHost) 5-The core (BHost) will issue a CHANGE MODE request to the responder(ADevice) 6-The core (BDevice) will issue a disconnect to the responder(AHost)	None

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 131-191

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes	
Host	Multi-Port Testing/ "Host,CmultiPort Tester"	Issues random BULK, ISO and INT transfers to existing and non-existing endpoints while issuing suspends, resumes, connects, disconnects on random ports. In the case of a single port core the possible combinations are limited.	Attribute Name MaxPacketSize Limit Iterations	Varying Method Fixed Fixed
Host	Multiple Echo Endpoint Traffic / "Host,Multiple Echo Endpoint Traffic,EndPointSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Loads random BULK transfers onto the Asynchronous list. For each iteration , the tests echoes data to 1 or more echo endpoints at one time.	Attribute Name MaxPacketSize TransferSize PortNumber USBAddress EndPointNumber HubAddress RootHubPortNum EndPointSpeed ForceFullSpeed Number Of Active Transfer Descriptors Variable Transfer Sizes In Parallel Data Direction Address Buffer Offsets	Varying Method Sequential Sequential Random (0 to MAXPORTS) Random Random Fixed Fixed Sequential Random 1 to Max Responder Endpoints Sequential Sequential Sequential
HOST	Bulk Echo / "Host,Bulk,Echo,EndPointSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates Bulk transfers and processes them using the Asynchronous list.	Attribute Name MaxPacketSize TransferSize PortNumber USBAddress EndPointNumber HubAddress RootHubPortNum EndPointSpeed ForceFullSpeed Number Of Active Transfer Descriptors Data Direction Address Buffer Offsets	Varying Method Sequential Sequential Random (0 to MAXPORTS) Random Random Fixed Fixed Sequential Random Random Sequential Sequential

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 132-191

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes	
Host	Interrupt Echo / "Host, Interrupt, Echo Interrupt EndPointSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates Interrupt transfers and processes them using the periodic list. This test will also exercise the LS_THRU_Hub feature of the responder.	Attribute Name MaxPacketSize TransferSize PortNumber USBAddress EndPointNumber HubAddress RootHubPortNum EndPointSpeed ForceFullSpeed Data Direction Address Buffer Offsets	Varying Method Sequential Sequential Random (0 to MAXPORTS) Random Random Fixed Fixed Sequential Random Sequential Sequential
Host	ISO Echo / "Testing:Host,ISO, Echo IsochronousData EndPointSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates ISO transfers and processes them using the periodic list.	Attribute Name VariablePacketMultiplier MicroFrame Index In Micro Frame Index Out MaxPacketSize TransferSize PortNumber USBAddress EndPointNumber HubAddress RootHubPortNum EndPointSpeed ForceFullSpeed Number Of Active Transfer Descriptors Data Direction	Varying Method Sequential Random Random Sequential Sequential Random (0 to MAXPORTS) Random Random Fixed Fixed Sequential Random Random Sequential
Host	Bus Signaling Suspend followed by a host resume / "Host,BusSignaling,ConnectionSpeed=%ld,PortNumber=%ld,%s, Suspend followed by a host resume*"	Connects at all the supported speeds and suspends the device and waits for the host to resume.	Attribute Name PortNumber EndPointSpeed OTGConnectionType	Varying Method Random (0 to MAXPORTS) Sequential Sequential
Host	Bus Signaling, Suspend followed by a port reset from the host / "Host,BusSignaling,ConnectionSpeed=%ld,PortNumber=%ld,%s Suspend followed by a wait for the device resume*"	Connects at all the supported speeds and suspends the device and then performs a reset from the host.	Attribute Name PortNumber EndPointSpeed OTGConnectionType	Varying Method Random (0 to MAXPORTS) Sequential Sequential
Host	Bus Signaling, Suspend followed by a port reset from the host. / "Host,BusSignaling,ConnectionSpeed=%ld,PortNumber=%ld,%s Suspend followed by a port reset from	Connects at all supported speeds followed by a suspend and then a port reset.	Attribute Name PortNumber EndPointSpeed OTGConnectionType	Varying Method Random (0 to MAXPORTS) Sequential Sequential

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 133-191

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes	
Device	ISO Packet Testing / "Device, ISOData ConnectSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates ISO transfers and places them onto endpoint and monitors the completion of the transfer. Test byte alignment of packets for both RX & TX. Find edges near word and burst boundaries. Tests Underflow.	Attribute Name	Varying Method
			VariablePacket Multiplier	Sequential
			MaxPacketSize	Sequential
			TransferSize	Sequential
			USBAddress	Random
			EndPointNumber	Random
			EndPointSpeed	Sequential
			Data Direction	Sequential
Device	Bulk Packet Testing / "Device, BulkData ConnectSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates BULK transfers and places them onto endpoint and monitors the completion of the transfer. Test byte alignment of packets for both RX & TX. Find edges near word and burst boundaries. Tests Underflow.	Attribute Name	Varying Method
			Buffer Address Offsets	Sequential
			Out Data UnderFlow Bytes	Sequential
			MaxPacketSize	Sequential
			TransferSize	Sequential
			USBAddress	Random
			EndPointNumber	Random
			EndPointSpeed	Sequential
			Data Direction	Sequential
Device	Control Packet Testing, / "Device, ControlData ConnectSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates Control transfers and places them onto endpoint and monitors the completion of the transfer. Test byte alignment of packets for both RX & TX. Find edges near word and burst boundaries. Tests Underflow	Attribute Name	Varying Method
			Buffer Address Offsets	Sequential
			MaxPacketSize	Sequential
			TransferSize	Sequential
			USBAddress	Random
			EndPointNumber	Random
			EndPointSpeed	Sequential
			Data Direction	Sequential
Device	Interrupt Packet Testing / "Device, InterruptData ConnectSpeed=%ld MaxPacketSize=%ld TransferSize=%ld *"	Randomly generates INTERRUPT transfers and places them onto endpoint and monitors the completion of the transfer. Test byte alignment of packets for both RX & TX. Find edges near word and burst boundaries. Tests Underflow	Attribute Name	Varying Method
			MaxPacketSize	Sequential
			TransferSize	Sequential
			USBAddress	Random
			EndPointNumber	Random
			EndPointSpeed	Sequential
			Data Direction	Sequential
Device	Handshake, Bulk / "Device, HandShake, Responder TestSequence=%ld Requesting Bulk Data"	Generates BULK traffic for testing handshakes. Executes each of the responder handshake tests for BULK.	Attribute Name	Varying Method
			Test Sequence	Sequential
			HandShake Code	Sequential
			EndPointSpeed	Sequential
			EndPointNumber	Random
Device	Handshake, Interrupt / "Device, HandShake, Responder TestSequence=%ld Requesting Interrupt Data"	Generates INTERRUPT traffic for testing handshakes. Executes each of the responder handshake tests for INTERRUPT.	Attribute Name	Varying Method
			Test Sequence	Sequential
			HandShake Code	Sequential
			EndPointSpeed	Sequential
			EndPointNumber	Random
Device	Handshake, Control Mixed with ISO transfer. / "Device, HandShake, Responder TestSequence=%ld Requesting ISO"	Generates ISO and Control traffic for testing of handshakes and proper interleaving of transfers.	Attribute Name	Varying Method
			Test Sequence	Sequential
			HandShake Code	Sequential
			EndPointSpeed	Sequential
			EndPointNumber	Random

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 134-191

Core Mode	Test Name / "Configuration String"	Test Description	Varied Testing Attributes	
Device	And Control Data" Handshake / "Device,HandShake, Responder TestSequence=1 Stalling Control Endpoint And Endpoint Does Not Send Payload For Setup"	Requesting Data And Endpoint Does Not Send Payload For Setup.	Attribute Name Test Sequence HandShake Code EndPointSpeed EndPointNumber BTO Retry PID_Err Retry PID Type Retry ACK Retry NAK Retry Stall Retry Nyet Retry CRC retry	Varying Method Sequential Sequential Sequential Random Sequential Sequential Sequential Sequential Sequential Sequential Sequential Sequential
Device	Bus Signaling / "Device,BusSignaling,Suspend with Host Resume, ConnectSpeed=%ld *"	Device is suspended followed by a wait for ForceResume bits enable.	Attribute Name Endpoint Speed	Varying Method Sequential
Device	Bus Signaling / "Device,BusSignaling,Suspend with Device Force Resume, ConnectSpeed=%ld *"	Devices is suspended with the device forcing Resume	Attribute Name Endpoint Speed	Varying Method Sequential

Co-simulation Environment Built-in Monitoring Areas.

Core Modes	Built-in co-simulation checks	Description
SPH, OTG, MPH, Device	Target Memory Bounds Checking	As traffic is generated from the core the BFM for the BVCI/AMBA target checks all memory access ranges are within range.
SPH, OTG, MPH, Device	BVCI/AMBA Target bus protocol	As traffic is generated from the core, the BVCI/AMBA BFM, monitors check all signaling is correct.
SPH, OTG, MPH, Device	BVCI/AMBA Initiator bus protocol	As traffic is generated from the co-simulation BVCI/AMBA BFM, monitors check all signaling is correct.

Co-simulation BVCI/AMBA Bus Model Configurable testing.

Core Modes	Co-Simulation BVCI/AMBA BFM Bus Protocol Testing	Varying Method
SPH, OTG, MPH, Device	Random Initiator Transaction Delay	Configurable to Random
SPH, OTG, MPH, Device	Random Initiator cmd_val delays	Configurable to Random
SPH, OTG, MPH, Device	Random Initiator rsp_ack delays	Configurable to Random
SPH, OTG, MPH, Device	Random Initiator byte enables	Configurable to Random
SPH, OTG, MPH, Device	Random Initiator splitting of data quads	Configurable to Random

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 136-191

8.2.4 Running the Verilog Test Bench for NC Verilog

To run the Verilog testbench use the following steps.

1. Verify that the ncverilog executable is in your PATH.
2. To run the Verilog test bench execute the following:
From a Unix Shell:

```
cd simulation/scripts
./runtest.pl --sim nc -b ←for BVCI
./runtest.pl --sim nc -a ←for AMBA-AHB
```

8.2.5 Running the Verilog Test Bench for Synopsys VCS

To run VCS simulator with our USB controller test environment you need the following:

```
VCS version: 7.1
Linux version: Red Hat Linux Enterprise 3.0
```

With this VCS should run with no major issues.

To run the Verilog testbench use the following steps.

1. Verify that the vcs/vcsi executable is in your PATH.
2. To run the Verilog testbench execute the following:
3. From a Unix Shell:

```
cd simulation/scripts
./runtest.pl --sim vcs -b ←for BVCI
./runtest.pl --sim vcs -a ←for AMBA-AHB
```

8.2.6 Changing USB-HS Tests Executed

The USBHS test software reads in a configuration file, (CTesterExecuteList.cfg) at run time when running on a non-embedded system like the integration verification environment. The configuration file contains a list of specific tests to run during the simulation. To change the tests executed, edit the configuration file by adding and removing test description strings. For the embedded system a header file, CTesterExecuteList.h, will need to be changed and the binary will need to be rebuilt. Later in this chapter there is more information on the configuration of the execution list.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 137-191

8.3 Customer Simulation Environment.

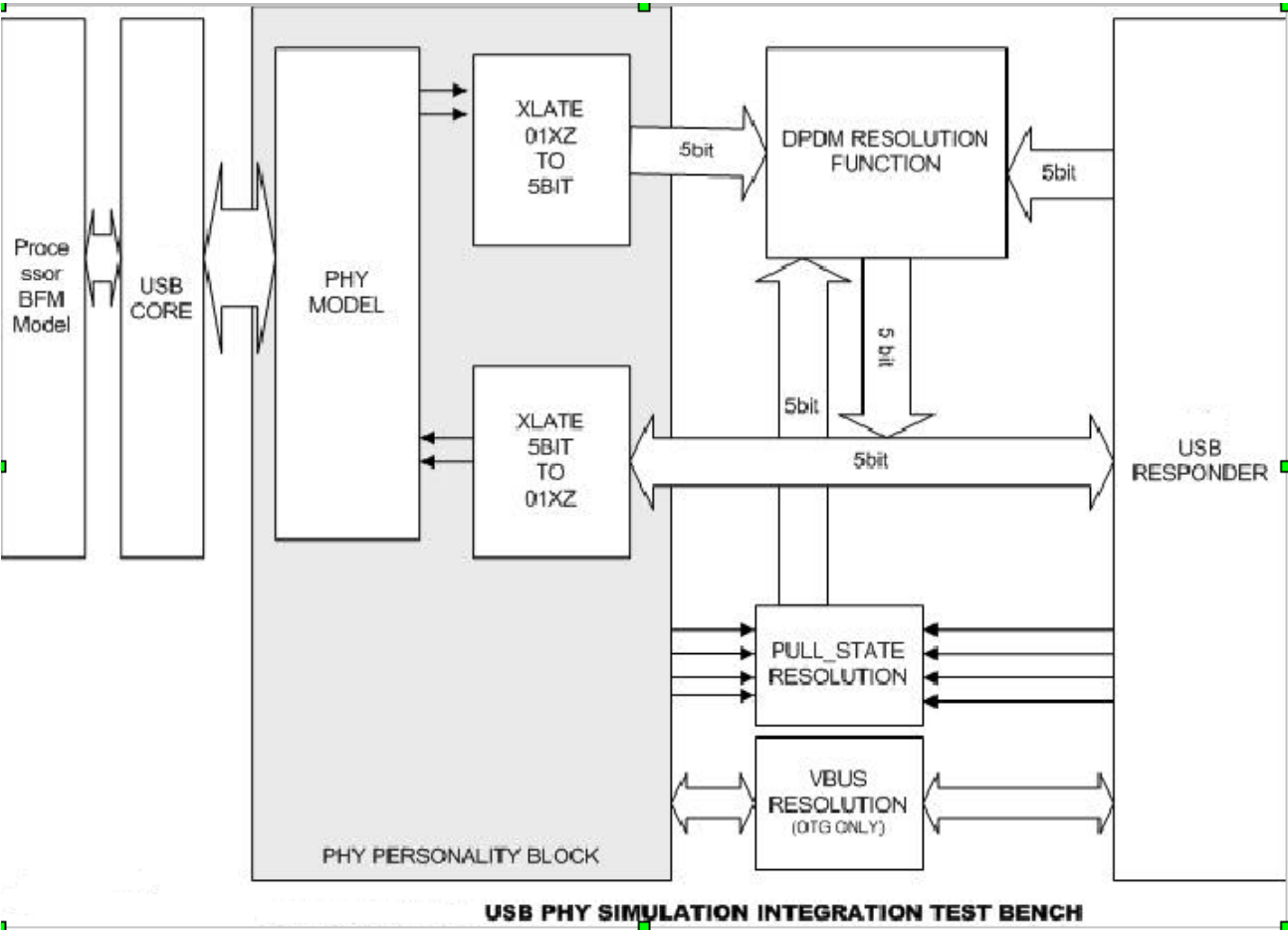


Figure 66 USB PHY Simulation Test Bench Block Diagram

8.3.1 HDL USB Bus Modeling Using 5 bits

8.3.1.1 Overview

For HDL simulation of the analog USB two wire bus traditional methods using HDL language supported multi value logic are inadequate. This section describes a modeling technique that uses 5 binary bits (17 values) for modeling the D+/D- USB data lines. This modeling technique was developed to enable integration of PHY models from multiple vendors in an easy to understand way. The modeling technique allows isolation of particular PHY (transceiver) model dependencies to a localized PHY personalization block. This modeling technique is HDL language independent. The major subblocks of the testbench become PHY independent and reusable.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 138-191

The USB-HS simulation environment is delivered to customers with an behavioral PHY model and a personalization block for that model. Customers wishing to integrate PHY models from other vendors can use this PHY model and personalization block as a template for their development. This model also provides a "known good" simulation environment for debug purposes.

8.3.1.2 Description

Conceptually you can visualize the USB bus as consisting of 3 sources of electrical information and a resolution function to calculate the resulting value. The 3 sources are:

- Upstream (Host)
- Downstream (Device)
- Pull Up/Down Resistors (From Both Host and Device)

The diagram below shows the conceptual model.

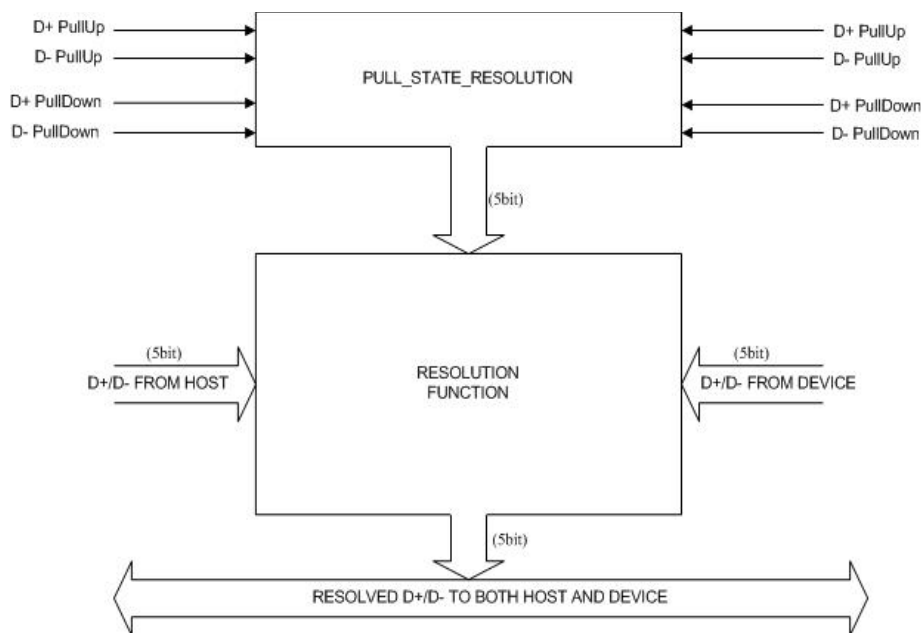


Figure 67 Resolution Function Block Diagram

8.3.1.3 Bus States

The following text describes the 17 values used for USB bus modeling. There are 3 general sets of values. Some values may belong to more than one set.

The first set are those values that are used to model the "Pull State". These values are the result of the combined pull-up and pull-down resistor contributions from both the host and device sides. These values are:

- DPDM_PULL00
- DPDM_PULL01

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 139-191

- DPDM_PULL11
- DPDM_ZZZZ

These are the only valid states that can be applied to the "Pull State" port of the resolution function. For these values the "D+" is the left most character and "D-" is the right most.

The second set of values are those that can be legally driven by the host and device. These values are:

- DPDM_SE0
- DPDM_RCV0
- DPDM_RCV1
- DPDM_RCVX
- DPDM_HS_0
- DPDM_HS_1
- DPDM_HS_X
- DPDM_HSZZ
- DPDM_ZZZZ

The last class is those values that can appear as the output of the resolution function. This class includes all members of the first two sets and the additional values:

- DPDM_CHIRP_J
- DPDM_CHIRP_K
- HDISC_0
- HDISC_1

The value DPDM_SE1 has been included for completeness it is not currently used.

BUS STATE NAME	D+	D-	Binary	Decimal	Notes
DPDM_SE_0	0	0	00000	0	Single Ended Zero
DPDM_RCV0	0	1	00001	1	Full/Low Differential 0 (LS-J, FS-K)
DPDM_RCV1	1	0	00010	2	Full/Low Differential 1 (LS-K, FS-J)
DPDM_HS_0	0	1	00011	3	High Speed 0 (HS-K)
DPDM_HS_1	1	0	00100	4	High Speed 1 (HS-J)
DPDM_PULL00	0	0	00101	5	Pull State = High Speed
DPDM_PULL01	0	1	00110	6	Pull State = Low Speed
DPDM_PULL10	1	0	00111	7	Pull State = Full Speed
DPDM_ZZZZ	Z	Z	01000	8	High Impedance

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 140-191

DPDM_RCVX	X	X	01001	9	FS/LS Contention
DPDM_HS_X	X	X	01010	10	HS Contention (also used as error state)
DPDM_CHIRP_J	1	0	01011	11	Never Driven Resolution Output
DPDM_CHIRP_K	0	1	01100	12	Never Driven Resolution Output
DPDM_HDISC_0	0	1	01101	13	Never Driven Resolution Output
DPDM_HDISC_1	1	0	01110	14	Never Driven Resolution Output
DPDM_PULLXX	X	X	01111	15	Uninitialized Pull State
DPDM_SE_1	1	1	10000	16	Single Ended One (not used)
DPDM_HSZZ	Z	Z	10001	17	High Speed Terminated

Figure 68 USB Bus State 5 Bit Encoding

8.3.1.4 Resolution Function

Below are 4 tables representing the resolution function. The resolution function is symmetric such that host and device inputs are interchangeable. The headers in the first row and column represent the inputs (either host or device) and the table entries represent the resolution function output. There are 4 tables one for each of the possible legal "PULL_STATE" inputs. "DPDM_HS_X" is used as an error output. A "***" in the last row or column represents any other value not listed. Other input combinations are considered error conditions that will resolve to "DPDM_HS_X".

The current implementation of the resolution function also includes a reset signal. This is used to suppress extraneous error messages at the start of simulation and is driven by the system reset signal. The output is driven to "DPDM_ZZZZ" when reset is active.

Most resolution is done in one simulation delta time except for "DPDM_HDISC_0" and "DPDM_HDISC_1". These two states are driven after 60 ns. These are shown in the table as "HS_0/HDISC_0" and "HS_1/HDISC_1". The first value is driven after one simulation delta; the second value is scheduled 60 ns later. This behavior is used to model the maximum delay of the reflected wave returning from an unterminated wire.

8.3.1.4.1 PULL_STATE = ZZZZ

	ZZZZ	HSZZ	**
ZZZZ	ZZZZ	SE_0	HS_X
HSZZ	SE_0	HS_X	HS_X
**	HS_X	HS_X	HS_X

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 141-191

--	--	--	--

Figure 69 Resolution Table For PULL_STATE ZZZZ

8.3.1.4.2 *PULL_STATE = PULL00 (HIGH SPEED)*

	ZZZZ	HSZZ	SE_0	RCV0	RCV1	RCVX	HS_0	HS_1	**
ZZZZ	PULL00	SE_0	SE_0	RCV0	RCV1	RCVX	HS_0/HDISC_0	HS_1/HDISC_1	HS_X
HSZZ	SE_0	SE_0	SE_0	RCV0	RCV1	HS_X	HS_0	HS_1	HS_X
SE_0	SE_0	SE_0	SE_0	RCVX	RCVX	RCVX	HS_0	HS_1	HS_X
RCV0	RCV0	RCV0	RCVX	RCV0	RCVX	RCVX	HS_0	HS_X	HS_X
RCV1	RCV1	RCV1	RCVX	RCVX	RCV1	RCVX	HS_X	HS_1	HS_X
RCVX	RCVX	HS_X	RCVX	RCVX	RCVX	RCVX	HS_X	HS_X	HS_X
HS_0	HS_0/HDISC_0	HS_0	HS_0	HS_0	HS_X	HS_X	HS_0	HS_X	HS_X
HS_1	HS_1/HDISC_1	HS_1	HS_1	HS_X	HS_1	HS_X	HS_X	HS_1	HS_X
**	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X

Figure 70 Resolution Table For PULL_STATE PULL00

8.3.1.4.3 *PULL_STATE = PULL01 (LOW SPEED)*

	ZZZZ	HSZZ	SE_0	RCV0	RCV1	RCVX	HS_0	HS_1	**
ZZZZ	PULL01	SE_0	SE_0	RCV0	RCV1	RCVX	HS_X	HS_X	HS_X
HSZZ	SE_0	SE_0	SE_0	RCV0	RCV1	RCVX	HS_X	HS_X	HS_X
SE_0	SE_0	SE_0	SE_0	RCVX	RCVX	RCVX	HS_X	HS_X	HS_X
RCV0	RCV0	RCV0	RCVX	RCV0	RCVX	RCVX	HS_X	HS_X	HS_X
RCV1	RCV1	RCV1	RCVX	RCVX	RCV1	RCVX	HS_X	HS_X	HS_X
RCVX	RCVX	RCVX	RCVX	RCVX	RCVX	RCVX	HS_X	HS_X	HS_X

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 142-191

HS_0	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X
HS_1	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X
**	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X

Figure 71 Resolution Table For PULL_STATE PULL01

8.3.1.4.4 *PULL_STATE = PULL10 (FULL SPEED)*

	ZZZZ	HSZZ	SE_0	RCV0	RCV1	RCVX	HS_0	HS_1	**
ZZZZ	PULL10	SE_0	SE_0	RCV0	RCV1	RCVX	CHIRP_K	CHIRP_J	HS_X
HSZZ	SE_0	SE_0	SE_0	RCV0	RCV1	RCVX	CHIRP_K	CHIRP_J	HS_X
SE_0	SE_0	SE_0	SE_0	RCVX	RCVX	RCVX	CHIRP_K	CHIRP_J	HS_X
RCV0	RCV0	RCV0	RCVX	RCV0	RCVX	RCVX	HS_0	HS_X	HS_X
RCV1	RCV1	RCV1	RCVX	RCVX	RCV1	RCVX	HS_X	HS_1	HS_X
RCVX	RCVX	RCVX	RCVX	RCVX	RCVX	RCVX	HS_X	HS_X	HS_X
HS_0	CHIRP_K	CHIRP_K	CHIRP_K	HS_0	HS_X	HS_X	HS_0	HS_X	HS_X
HS_1	CHIRP_J	CHIRP_J	CHIRP_J	HS_X	HS_1	HS_X	HS_X	HS_1	HS_X
**	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X	HS_X

Figure 72 Resolution Table For PULL_STATE PULL10

8.3.1.4.5 *Test Bench*

A simulation testbench is provided with the Chipidea USB core. The diagram at the beginning of this section shows a high level view of this test bench. The actual HDL should be consulted for actual block names and signal names.

The diagram shows the "PHY PERSONALITY BLOCK" that is used to isolate PHY specific modeling from the remainder of the testbench. Within that block are translation blocks. These blocks provide translation from the PHY specific modeling used for the D+ and D- wires to and from the 5 bit modeling documented above.

8.3.1.4.6 *PHY Model Integration*

PHY vendors have used several different modeling schemes in the past. Most of these schemes have been HDL language dependent. Integration of a PHY model within the USB-HS environment will typically involve creation of the 3 HDL files representing the "PHY PERSONALITY BLOCK", "XLATE 01XZ TO 5BIT" and "XLATE 5BIT TO 01XZ".

The major tasks that these 3 blocks provide are:

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 143-191

- Conversion of PHY specific DP/DM output modeling to 5 bit
- Conversion of 5 bit resolution function output to PHY specific DP/DM modeling input
- Loop breaking

It may be necessary in some instances to use some of the input signals to the PHY to determine the proper 5 bit value to drive into the resolution block (XLATE 01XZ TO 5BIT). The PHY input signals "xcvrselect" and "termselect" may need to be routed as inputs to the translation blocks for this purpose.

Loop breaking is required to prevent the output of the "DPDM RESOLUTION FUNCTION" from being driven back into the resolution function when the PHY is not driving the bus. Many PHY vendors will supply the tri-state enable signals as observe only signals to help with the loop-breaking problem. These enable signals may also be useful for determining high speed versus low/full speed signaling levels.

Consultation with the PHY vendor may be required to resolve ambiguities in the output modeling used for the particular PHY simulation model for the DP/DM PHY signals.

8.3.2 Processor System Requirements for the Integration Test Software

- Memory, all software tests have been designed to fit within the 2MB space.
- Interrupts, USB host and device tests require interrupts. The processor must support function calling via interrupt signal. All interrupts are assumed to be level type.
Text IO: The test software uses ASCII text messaging to give detailed test information. The information includes test descriptions, error messaging and test execution status. While a the Text IO port is not required some text IO ability is strongly recommended. For systems that do not have any text IO capability a TCL based console can be used to gain access to the messaging. An example TCL console (scripts/mti/console.tcl) is provided for the MTI simulation environment.
- The test code is written in portable C++, users will need a C++ compiler to recompile the test software for the targeted processor. All of the test code is compiler and processor independent.

8.3.3 Integration steps

The following is a list of key steps a user must perform to run the test software on an embedded processor. Software executable generation, Steps 5 and 6, are covered in detail in this chapter.

1. The embedded processor must have access either directly or through a bridge to the BVCI/AMBA target interface of the USB-HS core. The processor must have access to the shared memory where the data structures will be located.
2. The USB-HS core initiator interface must have access either directly or through a bridge to the shared memory where data structures reside and to the packet memory where the USB packet data will be stored.
3. An interrupt connection mechanism must be hooked up to allow the processor to react to events from the USB-HS Core.
4. The HDL USB Responder must be connected to the USB Core bus interface.
5. The software API layer must be ported to the target processor.
6. The test software and ported API layer must be built into an executable using the target processor compiler and linker.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 144-191

8.4 Device and Host Integration Software Tests

The test bench is capable of generating thousands of test sequences during a single simulation. The software tests implement a broad range of functional tests outside the set of typical integration tests. The delivered configuration will only run a small subset by default. All of the test software is included so the user may extend the number of tests executed.

8.4.1 Brief Summary of the Tests for both Host and Device

The test sequences include areas listed below. After running the tests in simulation a simple grep of `SoftwareMessageIO.txt` for the string, "RunTesting" will generate the full list of tests performed during the simulation.

8.4.1.1 Register Interfaces:

- Read tests for read only registers.
- Read and Write Tests for RW registers.
- Read Write Clear Tests for RWC registers.

8.4.1.2 Interrupt Tests:

- For each of the interrupt types an interrupt is forced.

8.4.1.3 Data Transfers:

8.4.1.3.1 All USB data types are transferred including:

- Bulk
- Interrupt
- Control
- ISO

8.4.1.3.2 All transfer attributes are varied including:

- Max packet size: 8, 64, 512, etc.
- Data buffer size: 1,2,3,4 etc.
- Transfer direction: IN-DATA OUT-DATA
- Transfer speed: Low, Full, High
- Buffer Offsets: 0,1,2,3

8.4.1.4 Bus Signaling:

- Suspend
- Resume
- Remote wakeup
- Etc.

8.4.1.5 OTG signaling:

- AHost to ADevice
- ADevice to AHost
- BHost to BDevice

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 145-191

- BDevice to BHost
- FS to HS switching
- HS to FS switching
- Etc.

NOTE The design team runs all of the test sequences, in all of the PHY configurations with several different clock configurations during regression testing of the core. Using a maximum length with and an execution file indicating all tests will result in several thousand test sequences for the host and device. Running the full set of tests after an integration at a customer site may be excessive, see section *Controlling the Length or the Number of Tests at Compile Time And Runtime* for trimming the number of tests.

8.4.1.6 Example of a grep for “Testing” in the file SoftwareMessagesIO.txt

```

19653>[123615727.180ns] RunTesting:Host,BusSignaling
19658>[123630424.038ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=0,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE
20220>[132531416.996ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=0,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE Suspend followed a port reset from the host.
20716>[145703741.128ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=0,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE Suspend followed a wait for the device resume
21116>[163462091.012ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=0,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE, Suspend followed by a host resume.
21421>[175303970.710ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=2,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE
21980>[182963457.952ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=2,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE Suspend followed a port reset from the host.
22458>[194822004.190ns] *RunTesting:Host,BusSignaling,ConnectionSpeed=2,PortNumber=0,OTGConnectionType =
B_DEV_DPULSE Suspend followed a wait for the device resume

```

8.5 Porting The Software Testing API Layer For an Embedded Processor

8.5.1 Portability features of the Integration Software

- All test code compiles using g++, MetaWare and MSVC++ without warnings.
- All test code uses the heap for volatile memory access between the core and the processor. All data structures and data buffers accessed by the USB core must be allocated on the heap.
- All volatile data take into consideration data-cache line fill accesses. Memory allocated for volatile access must not be accessed during data-cache line fills of non-volatile data.
- All core slave interfaces are accessed via an API Read and Write Functions. No direct memory de-referencing of the HDL core addresses is allowed in the test software.
- All functions return a true or a false for status. If a function fails, it must return a status indicating the exact location of the failure.
- All functions returning bad status must log a text searchable unique error. For example, an error reported from an action performed in the periodic list manager might be “0xPLM000192”. This string should be a unique instance in the /APITestSoftware directory for search reasons.
- All text messaging must be sent to a message API call. The use of **printf**, **fprintf**, etc. is not allowed in the test code.
- All endian dependent data structures must be managed using buffers and byte swap functions.
- Bit manipulation of volatile C structure data is not allowed. Volatile data structure manipulation needs to be compiler independent.
- All interrupt functions must be setup and controlled by the API calls found in the **common_include** directory.
- All volatile memory must be allocated using the **MemoryAlloc** API function call.
- All volatile memory must be deleted using the **MemoryDelete** API function call.
- All “while forever” (**while (1)**) functions must use the **SystemWait** API function call in the loop.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 146-191

8.5.2 The Common Include Directory

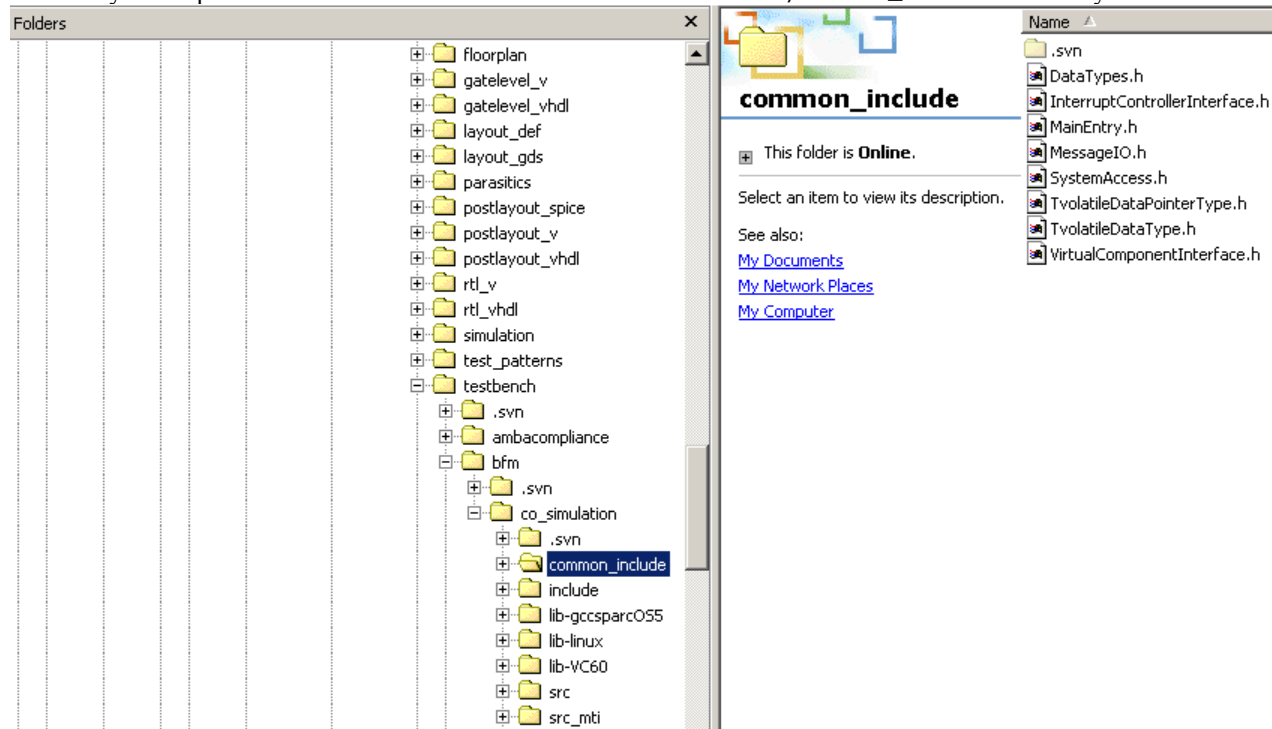
The API defined in the /common_include directory consists of a set of class and function definitions such as

- **CMessageIO** for text messaging,
- **CInterruptControllerInterface** for handing interrupts,
- **CVirtualComponentInterface** for reading and writing interfaces on the peripherals
- **SystemWait** for passing time,
- **AllocMemory** for heap allocations,
- **DeleteMemory** for heap deletions,
- **SystemAbort** for aborting the software,
- **RandomNumber** for issuing random numbers, etc.

Some common and simple function implementations of the API calls are listed below.

8.5.3 API Header Files and API Source Files

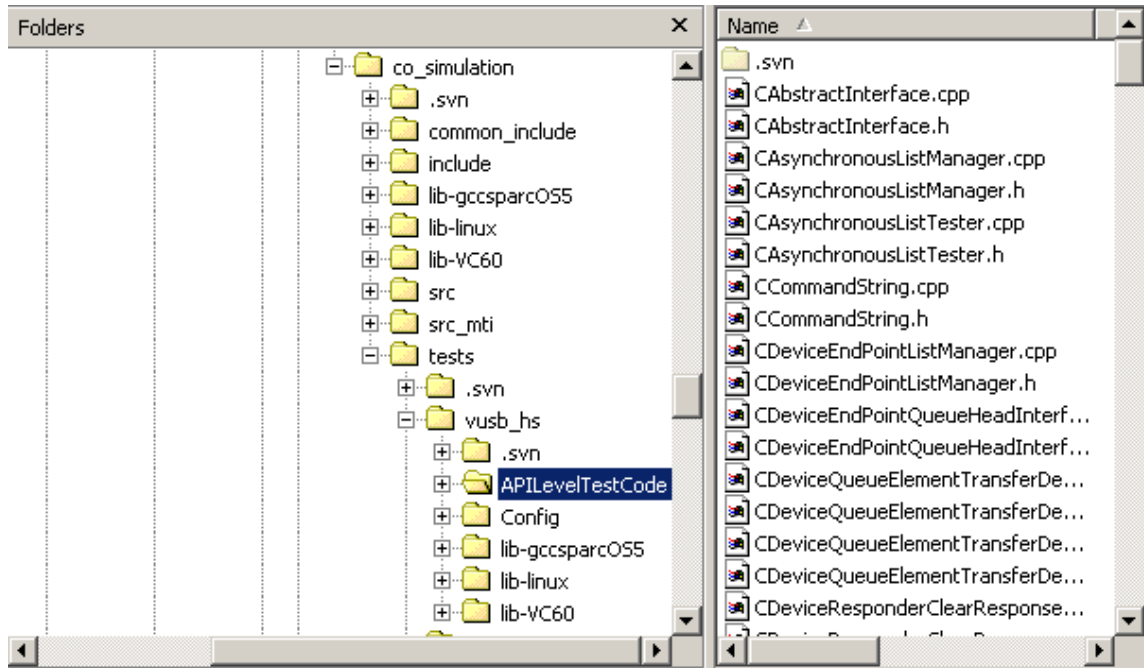
The header files found in the /common_include directory define the complete API layer. The ported files are essentially the implementation of the header file definitions found in the /common_include directory.



The files in /common_include define the common API that is used for all processor models. These files are independent of the actual processor model being used. Each processor must implement the API that is defined by these includes.

NOTE The code found in the /APITestSoftware directory should not require changes when porting to new processors.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 147-191



8.5.4 Examples Of Ported Sections of Software

The following examples of ported code to a generic embedded processor is intended to allow the user to gauge the level of effort required to perform a software port. The actual level of effort required to port the software would be processor and compiler specific.

8.5.4.1 Class CMessageIO

```
void CMessageIO::PrintStringStdio( char* string_buffer )
{
    printf("%s", string_buffer );
}

void CMessageIO::PrintStringStdioLF( char* string_buffer )
{
    printf("%s\n",string_buffer );
}
```

8.5.4.2 Class CVirtualComponentInterface writing

```
bool CVirtualComponentInterface::writeBuffer( const AddressType& dest_address, const
AddressType& src_address, const UnsignedLong& byte_count , const bool
asynchronous_operation)
{
    AddressType dest_offset_address = 0xFC5000 + dest_address;
    unsigned char* src_data = ( unsigned char*)src_address;
    volatile unsigned char* dest_data = (volatile unsigned char*)dest_offset_address;
    for( UnsignedLong i=0; i<byte_count; i++ )
    {
        dest_data[i] = src_data[i];
    }
}
```

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 148-191

```

    return(true);
}

```

NOTE This implementation is simply de-referencing the offset address added to the address passed in and assigning it the value passed in.

8.5.4.3 Function Delete Memory

```

bool DeleteMemory( AddressType allocated_data )
{
    if( (void*)allocated_data == NULL )
    {
        printf("Error: Attempt to delete a NULL pointer.");
        return( false );
    }
    else
    {
        delete( (void*)allocated_data );
        return( true );
    }
}

```

8.5.4.4 Function SystemAbort

```

void SystemAbort( char* message, UnsignedLong error_code )
{
    printf("Abort: message = %s, Code = %lu", message, error_code );
    abort(); //Keep for Lab
}

```

8.5.4.5 Function AllocMemory

```

AddressType AllocMemory( const UnsignedLong & size )
{
    AddressType new_address;
    new_address = (AddressType) new unsigned char[size];
    return( new_address );
}

```

8.5.4.6 Function SystemWait

```

bool SystemWait( const UnsignedLong& units, const TimeType time_type )
{
    wait(1*units); //System Call for1ms wait
    return(true);
}

```

8.5.4.7 Function InitRandomNumber and RandomNumber

```

void InitRandomNumber( unsigned int seed )
{
    srand( seed );
}

UnsignedLong RandomNumber()
{
    return( rand() );
}

```

8.5.4.8 Function SystemCreateThread

```

bool SystemCreateThread( SystemThread *thread_function )

```

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 149-191

```

{
    //Just execute it for single thread
    bool return_value = thread_function();
    return( return_value );
}

```

8.5.4.9 Function main defined

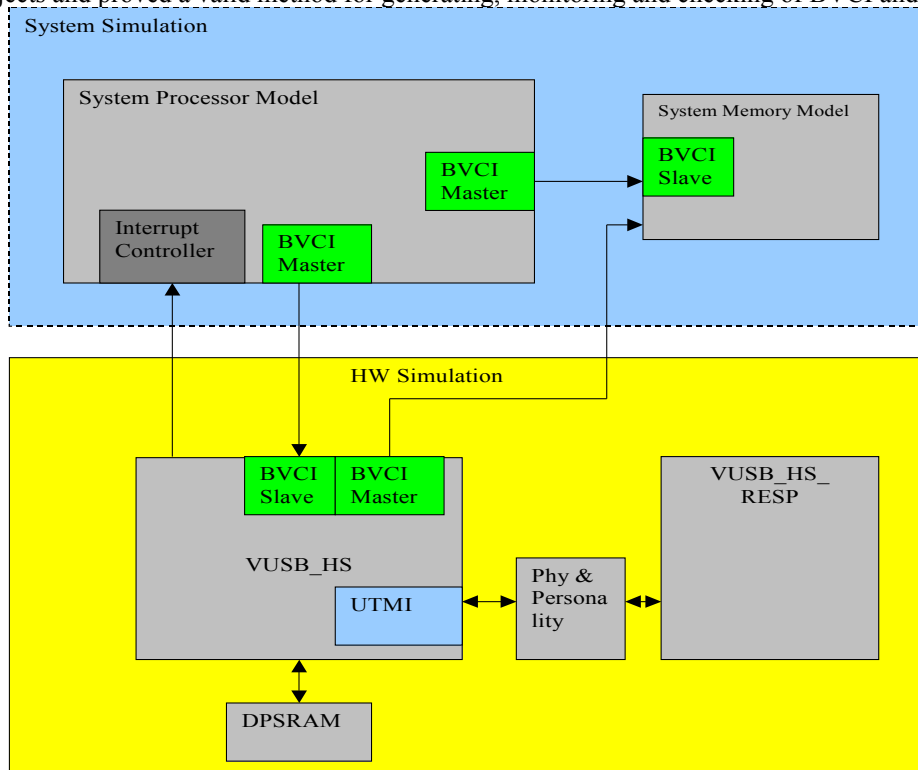
```

void main()
{
    if( HardwareCreate() == false )
    {
        SystemAbort( "Error: Bad status returned from main", 0 );
    }
    HardwareDestroy();
}

```

8.6 Detailed Description of the Integration Verification Environment.

Chipidea has built a co-verification/test bench library that includes the ability to connect any number of initiator and/or target BVCi or AMBA-AHB interfaces via the FLI or PLI interface among other things. This library has been used on past projects and proved a valid method for generating, monitoring and checking of BVCi and AMBA traffic.



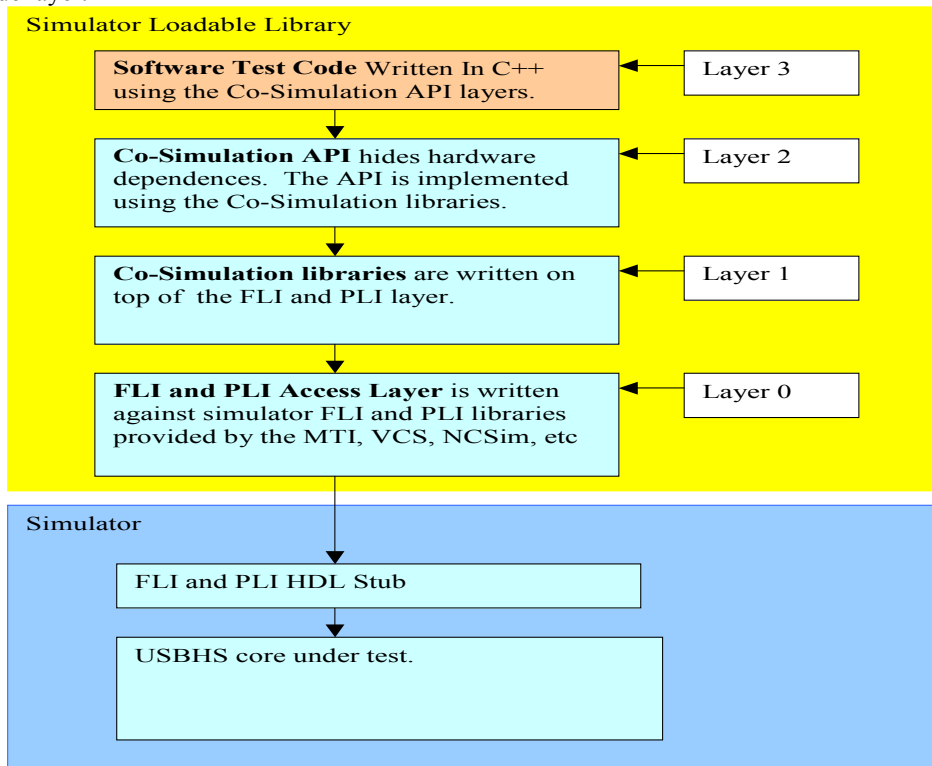
The figure above shows a generic processor model, written in C++, which functions as the system processor. The responder is an HDL model that emulates USB traffic through the UTMI or Serial interface. In the standalone USB-HS core simulation test bench, the System Processor Model and System Memory Model are combined into a single Bus Functional Model and emulated through Foreign Language Interface (FLI) for VHDL simulation and Programming Language Interface (PLI) for Verilog simulation. The integration verification environment is a transaction-based environment controlled by the test code running on the System Processor Model. The USB-HS responder model uses control endpoint 0 and various test specific commands to communicate with the processor. The host processor and the USB-HS responder set-up a communication pipe using the

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 150-191

USB protocol over endpoint 0. Information about test sequences and expected results are communicated to the responder model through this (over the USB bus) communication pipe.

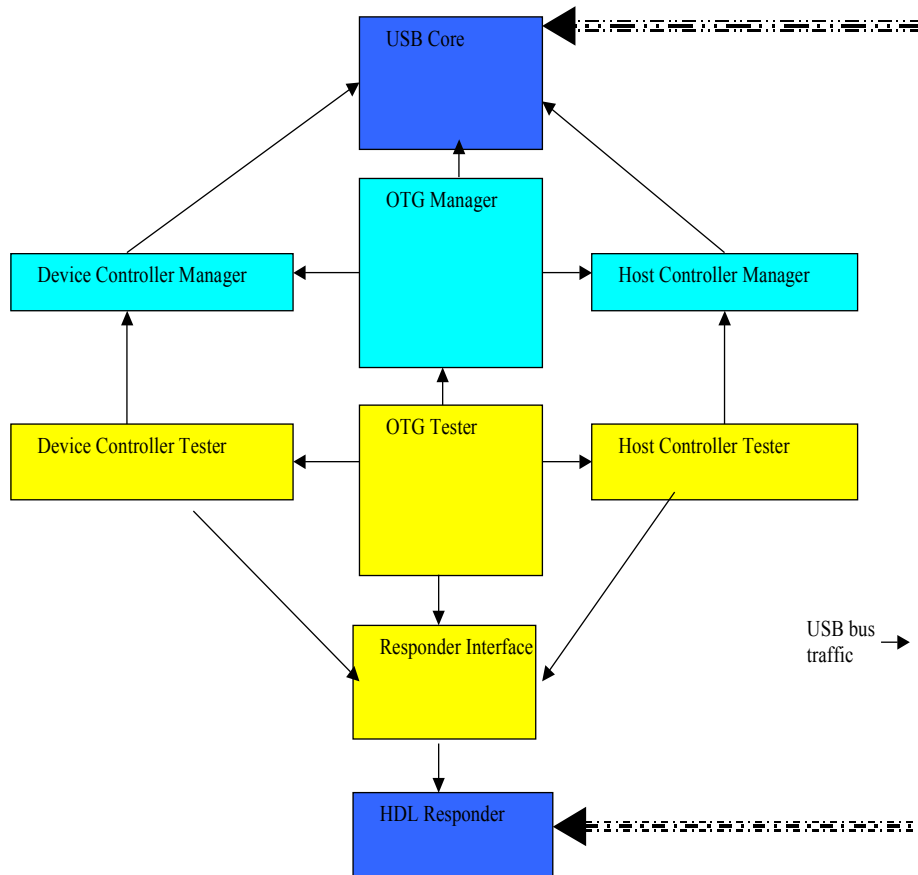
8.6.1 Layered Approach of the Test Software and the Co-Simulation BFM

The co-simulation BFM is composed of 3 major layers that are compiled into an object file and linked with the software test code layer.



- Layer 0 is the BFM physical bus transaction layer. This layer incorporates the Hardware pin level interface and the PLI/FLI communication layer, the system memory model including DMA accesses as well as an interrupt controller. The BFM is a System Simulation Model where the processor and memory subsystem are modeled using the CPU and memory of the simulation host. The BFM is shipped to the costumer as a dynamically linked binary (.dll) or a (.so) shared object.
- Layer 1 is the BFM API layer. This layer abstracts the low-level processor functional calls.
- Layer 2 is the API layer, which abstracts all functionality that can be found on typical processor system, for example, interrupts, reading and writing of memory etc.
- The actual tests are coded in C++. They are portable to other target processors. All of this test specific code can be found in the /APILevelTestCode directory.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 151-191



Top Level Control Structure Of The Test Software Modules

8.6.2 Common Abstractions Used In The Software Test Code And Basic Framework Information.

Test software breaks into a few abstractions including Interfaces, Managers and Testers.

- Classes with the string “Managers” in the class name are used to control types of functionality found in USB. For example, the object of type **CAsynchronousListManager** would be used to control the access to the USB asynchronous list data structures in memory. The class members would include functionality to append transfers, remove transfers, process transfers etc. A manager class may also just contain more managers.
- Classes with the string “Interface” in the name are used to access data or group functionality. For example, an object of class type **CHostQueueHeadInterface** would be used to access queue head data memory. The class members are used to help setup and help control and access of the data in memory.
- Classes with the string “Testers” in the name are used to exercise objects (typically **Manager** class objects) functionality. For example, **CUSBDeviceControllerTester** is used to test the functionality of the **CUSBDeviceControllerManager**. A Tester class may also just contain sub testers.

8.6.3 Test Software Dependencies

8.6.3.1 Key Class Naming Conventions

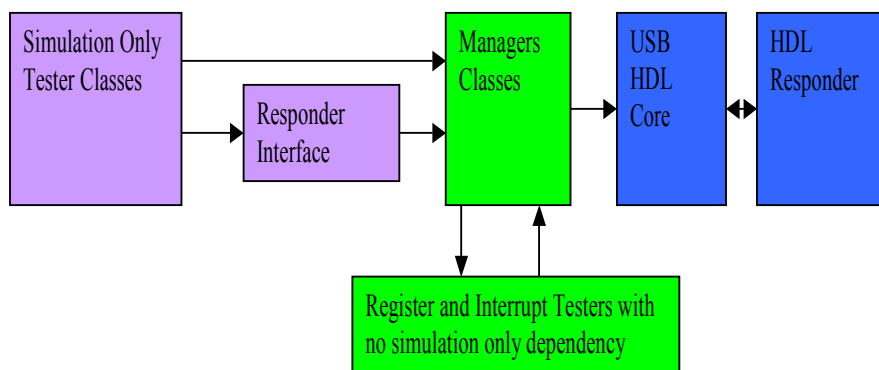
Careful consideration is given to class dependency hierarchy, reuse and multiple environments. Simulation only tester classes may depend on manager classes however no manager classes must depend on simulation tester classes. A

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 152-191

manager class will not make a reference to a class that can only be used in a simulation environment; an example is the tester class, which has dependencies such as the HDL responder. This allows for manager classes to be used outside the integration verification environment.

- The testers will check data using the Manager class Interfaces.
- The testers will request responder commands via the Responder class Interface.
- The Responder class will process commands using the Manager class Interfaces.
- The Manager class will control the data and register access to the USB core bus.

NOTE The register interface tester and the interrupt tester are an exception and do not depend on the simulation environment. These two testers make no references to simulation only objects. For example, these register and interrupt tests could be performed as a self-check when loading an actual device driver.



The structured dependency concept allows a very large percentage of the software to be reused in a variety of environments. For example Manager class software having no dependencies on the responder which allows replacement of the entire responder without making any changes to the manager class.

Software may also be ported to an actual hardware lab environment by replacing responder and tester classes with application level device driver software. This has been done internally, using hub and mass storage classes, to enumerate and test actual hardware.

8.6.4 Register Testing

Register testing is implemented in the base class **CRegisterArrayInterface**.

CRegisterArrayInterface will examine the assigned register definition table and exercise the register according to the assigned attributes found in the table during register testing.

The attributes include: access type, bit field size, location and default read data. Access type defines if the register is read, write, read write or read write clear. Bit field size and location define exactly where the bits are mapped in the memory space. The default read data value is used to test registers from a default state after reset.

The register field information for **CRegisterArrayInterface** also includes fields for turning on and off the register testing and the default read checking.

If the register test fails, the status output will include all the information found in the **CRegisterArrayInterface** field structure. This information is helpful with problem diagnosis.

The **CUSBRegisterInterfaceTester** uses the functions provided in the **CRegisterArrayInterface** to perform all the testing. The **CUSBRegisterInterfaceTester** contains all testing information required to execute a test properly in the framework. **CUSBRegisterInterfaceTester** is derived from **CTester** which defines all the pure virtual functions required of a test class.

The register interface testing is a test that is suitable for both simulation and lab testing, it contains no dependencies, which can only be demonstrated in simulation.

The register tester is called from the OTG manager, which is the top most manager in the system. The OTG manager has code to make sure the test is only run once for host mode and only run once for device mode during a simulation. The tests are not re-run each time the USB-HS core is switched between device and host mode.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 153-191

The register test is an excellent method for checking the target interface connections to the system.

8.6.5 Interrupt Tests

Interrupt testing is implemented in the class **CUSBInterruptTester** derived from the **CTester** class.

CUSBInterruptTester contains a set of tests that use the interrupt test register in the USB core to exercise all of the interrupts found in the core for a particular mode. The interrupt tests work in both host and device mode.

For each interrupt status bit field, the tester will cause the interrupt to trigger and provide a special interrupt test function to service it. In the interrupt test code will ensure the correct interrupt was enabled and only the correct interrupt was generated.

The interrupt tester uses the **CUSBRegisterInterface** to access the enable and status register during the testing. The **CUSBRegisterInterface** has a few helper functions for walking over the registers using an index value.

The interrupt tester is suitable for both simulation and lab testing, it contains no simulation only dependencies. The OTG manager has code to make sure the test is run only once for host mode and only once for device mode during a simulation. The tests are not re-run each time the USB-HS core is switched between device and host mode.

The Interrupt test is an excellent test to verify the interrupt signal is connected correctly and that the vector table for the interrupts are pointing to the correct interrupt function. This test is highly unlikely to complete with success if the interrupt path is incorrectly configured.

8.6.6 Bus Signaling Tests for Host Mode

The test **CEHCITest_2_2** test contains both speed connection testing and bus signaling testing for the USB host mode. The bus signaling would require a speed change and reconnect for each of the signaling tests, so these tests are merged into one and thus save simulation time.

Each time the speed has been changed a series of supported bus signaling tests are performed before the next speed change request is made. The speed changes are executed until all of the supported connection speeds have been exercised.

The bus signaling tests include four major tests.

- Resume from a suspended state by the host sending resume signaling.
- Remote wakeup by the device from the suspended state.
- Host asserts reset to the suspended device.
- The host forced reset on a non-suspended device.

The “suspend” and “resume” functions are controlled using members in the **CUSBPortManager**, which is called by the **USBHostControllerManager**. Suspend and resume are called from the **CUSBHostControllerManager** to maintain correct state information throughout the software framework.

The **CUSBPortManager** functions contain checks for the enable and disable of connect status. If the port connection is detected as enabled during a suspend request the call to the register interface will be made to suspend all devices on the port. If the port connection is detected as enabled during a resume, the function will issue a resume to the port.

In the **CUSBRegisterInterface**, the function for resume contains a read poll for the resume enable bit. The read poll function follows the setting of the resume enabling. When the resume enable bit goes low, the core has indicated it has completed the resume sequence.

In the **CUSBRegisterInterface**, the function for suspend contains a poll function after the suspend enable. When suspend is issued, the software will wait for this suspend bit to enable. To insure all the active (in progress) transaction packets have completed a wait for the enabling of the suspend bit is performed.

Note: The connection speeds are determined with the use of a register interface helper function in **CUSBRegisterInterface**. The helper function will return a list of supported speeds determined by the PHY configuration register values of the USB core. These register values are controlled by the PHY connection on USB core bus interface.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 154-191

8.6.7 Host Data Type Transfers Testing

The host data type transfers testing is done with several software testers, including **CEHCITest_3_11**, **CEHCITest_3_13** and **CEHCI_4_10**. All testers are derived from the **CTester** class. The **CAsynchronousListTester** and **CPeriodicListTester** are used to drive specific tests. All of the testers generate a series of valid tests to hand to the **CAsynchronousListTester** or **CPeriodicListTester** tester.

- **CEHCITest_3_11** - Performs the control, bulk and interrupt testing functions using all valid connection speeds, all valid transfer sizes, all transfer directions and all valid max packet sizes.
- **CEHCITest_3_13** - Performs the isochronous testing functions using all valid connection speeds, all valid transfer sizes, all transfer directions and all valid max packet sizes.
- **CEHCI_4_10** - Performs multi-endpoint testing in parallel using the **CAsynchronousListTester**. The **CAsynchronousListTester** has a parallel request member for bulk transfers, which will push as many echo transfers out in a single test as possible. The number of tests pushed out at one time is constrained by the responders max number of echo endpoints.

Top-level tester classes are mostly used to collect all the constraints for the test and formulate a valid set of hardware supported tests to be performed by the **CAsynchronousListTester** and/or **CPeriodicListTester**.

- The **CAsynchronousListTester** will construct the requested bulk tests and submits them to the **CAsynchronousListManager**. The **CAsynchronousListTester** uses both the blocking and non-blocking queuing functions on the **CAsynchronousListManager** interface to execute the tests.
- The **CPeriodicListTester** will construct the isochronous and interrupt requested tests and submits them to the **CPeriodicListManager**. The **CPeriodicListTester** currently uses blocking functions on the **CPeriodicListManager** to execute the tests. The blocking function provide a means for testing periodic transfers which have guaranteed scheduled delivery.

NOTE The valid connection speeds are detected with the register interface functions.

NOTE The transfer attributes are generated by the set of functions provided in the **CEndPointInformation** class. The Information extracted from **CEndPointInformation** allows the testers to create valid tests with the current hardware.

8.6.8 OTG Signaling

The OTG interface is a separate block of logic from the USB host and device core and can be tested separately from the host and device functionality. OTG testing requires many connections and reconnections.

Each of the basic transitions is used to generate the full set of possible test states found when connecting two OTG systems together. A list of system state transitions performed during the OTG testing follows.

NOTE The test 1-6 will be run at the max reconnection speed.

1. The core (AHost) will issue a **CHANGE MODE** request to the responder (BDevice)
 - The core will suspend the responder (BDevice)
 - The responder (BDevice) will see the suspend
 - The responder will change to BHost.
 - The core will change to ADevice with port power enabled.
 - The responder (BHost) will see the connect.
2. The core (ADevice) will issue a **CHANGE MODE** request to the responder (BHost)
 - The core will switch from (ADevice) to (AHost) with power disabled.
 - The responder (BHost) will see the disconnect.
 - The responder(BHost) will switch to (BDevice).

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 155-191

- The core (AHost) will see the data pulse.
 - The core (AHost) will enable the power.
 - The responder (BDevice) will connect when it sees the power from the core (AHost).
3. The core (AHost) will issue a reconnect command, which will swap the cable.
- The core (AHost) will see the responder (Bdevice) disconnect with a port disconnect on the core.
 - The core (AHost) will wait for the APin to change to BPin.
 - The responder (BDevice) will become (AHost) and connect the cable with the BPin into the core.
 - The core (AHost) will see the BPin and switch itself to a BDevice with the power disabled.
- The core (BDevice) will start the data pulsing.
 - The responder (AHost) will see the data pulsing and turn on port power.
 - The core (BDevice) will get the reset from the responder (AHost)
 - The core (BDevice) and the responder (AHost) will connect.
4. The core (BDevice) will issue a CHANGE MODE request to the responder (AHost)
- The responder (AHost) will suspend the core (BDevice)
- The core (BDevice) will see the suspend.
 - The core will switch from (BDevice) to (BHost) with power disabled.
 - The responder (AHost) will see the disconnect.
 - The responder (AHost) will switch to (ADevice) with power enabled..
 - The core (BHost) will see the connection.
 - The core (BHost) will connect with the responder(ADevice).
5. The core (BHost) will issue a CHANGE MODE request to the responder (ADevice)
- The core (BHost) will switch to the (BDevice).
- The responder (ADevice) will switch to(AHost).
 - The core (Bdevice) will enable data pulsing.
 - The responder (AHost) will see a data pulse from the core (BDevice).
 - The responder (AHost) will turn on port power.
 - The core (BDevice) will see the port set and connect to the responder (AHost).
6. The core (BDevice) will issue a disconnect to the responder (AHost)
- The core will see the BPin switch to APin.
- The core (BDevice) will switch to (AHost).
 - The responder (AHost) will switch to (BDevice) and enable data pulsing.
 - The core (AHost) will see the data pulsing from the responder (BDevice).
 - The core (AHost) will enable power on the port.
 - The core (AHost) will see the connection on the port.

The test function will exit with the core set to (AHost).

When the core does support both HS and FS, the test 7-12 will repeat tests 1-6 but the speed will be toggled from HS to FS and FS to HS.

The log file for a HS and FS core should do the following:

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 156-191

- 1-6 running with HS, Starting on HostA and ending on HostA. (CHANGE MODE, CHANGE MODE, reconnectB CHANGE MODE, CHANGE MODE, reconnectA).
- 7-12 running with FS, Starting on HostA and ending on HostA. (CHANGE MODE, CHANGE MODE, reconnectB CHANGE MODE, CHANGE MODE, reconnectA)
- 3-18 running with HS, Starting on HostA and ending on HostA. (CHANGE MODE-HS, CHANGE MODE-FS, reconnectB-HS CHANGE MODE-FS, CHANGE MODE-HS, reconnectA-FS)
- 19-24 running with FS, Starting on HostA and ending on HostA. (CHANGE MODE-FS, CHANGE MODE-HS, reconnectB-FS CHANGE MODE-HS, CHANGE MODE-FS, reconnectA-HS)

8.6.9 Running Device Only Or Host Only Functions with an OTG core.

If a customer is attempting to test an OTG USBHS core with either host or device functions disabled, some changes to the *CTesterExecuteList.cfg* should be made. If the customer chooses to test host only functions in the OTG core, the configuration string “Testing:OTG As A Host Only Core.” should exist in the *CTesterExecuteList.cfg* file. If the customer chooses to test device only functions in the OTG core, the configuration string “Testing:OTG As A Device Only Core.” should exist in the *CTesterExecuteList.cfg* file. The existence of these strings will cause the test bench to avoid calls to functions such as “mode change” and reconnect.

Important Note: If the string “Testing:OTG As A Device Only Core” is used the test bench responder must be configured to connect in correct mode at the start of the simulation. The delivered testbench for OTG will default to a host connection at the start of the sim. The responder will need to be configured to do the initial connection as a host responder vs. a device responder. These modes of connection are specified using generics to the responder HDL module. The *host_slash_not_device* should be set to 1 for the Device only test and 0 for Host only testing, see below.

Host Only Testing:

VHDL:

```
host_slash_not_device : integer := 0;
```

Verilog:

```
parameter host_slash_not_device = 1'b 0;
```

Device Only Testing:

VHDL:

```
host_slash_not_device : integer := 1;
```

Verilog:

```
parameter host_slash_not_device = 1'b 1;
```

8.6.10 Altering the regression tests.

Delivered are 4 levels of testing “scripts” in the form of configuration files.

- 1) *CTesterExecuteList.cfg* - This is basically a quick connection test.
- 2) *CTesterExecuteList_Bronze.cfg* - This will provide a low level of core coverage.
- 3) *CTesterExecuteList_Silver.cfg* - This will provide a medium level of core coverage.
- 4) *CTesterExecuteList_Gold.cfg* - This will provide the greatest coverage.

The test software is constructed to read and execute the tests existing in the configuration file. The configuration files have been created to correctly stimulate the test software to produce the optimal results in the shortest amount of simulation time. Reducing the configurations file tests into separate tests will reduce or completely disable the ability of the test bench to detect critical flaws in the core.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 157-191

If a customer attempted to enumerate all the tests require for a full regression a complete understanding of the USB packet protocol is a requirement. The legal combinations of attributes for each test is directly associated with the attributes of the core, the attributes of the USB specification and the attributes of the test bench setup. All of these attributes are generated and checked using the information provided by the core registers and the software itself.

8.6.11 The recommend methodology for utilizing a compute grid.

The selection of the gold testing configuration file will enable all the tests configurations in the USB test software. Many of the attributes in the test software are set using randomly constrained values. The re-running of the “gold testing” improves coverage typically. "In-house" regressions at Chipidea will typically run the “gold testing” many times before shipping. The random nature of the test bench will enable corner case testing which is critical in finding deep and difficult to find bugs.

A customer with a large compute grid who wishes to retest the core using the delivered test bench software should spawn a process on each machine using the gold test configuration file and allow the test software and its random capabilities to generate test cases.

8.6.12 Software Class information for the Host and Device Tests

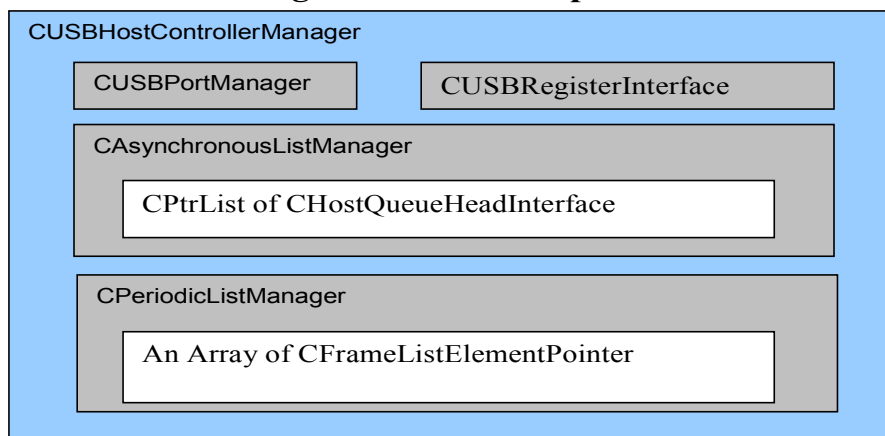
In this section a set of classes are presented. These classes are utilized to build the tests presented above.

8.6.12.1 Top Level Managers



- CUSBOTGRegisterInterface derived from CRegisterArray
- CUSBHostControllerManager derived from CUSBControllerManager
- CUSBDeviceControllerManager derived from CUSBControllerManager

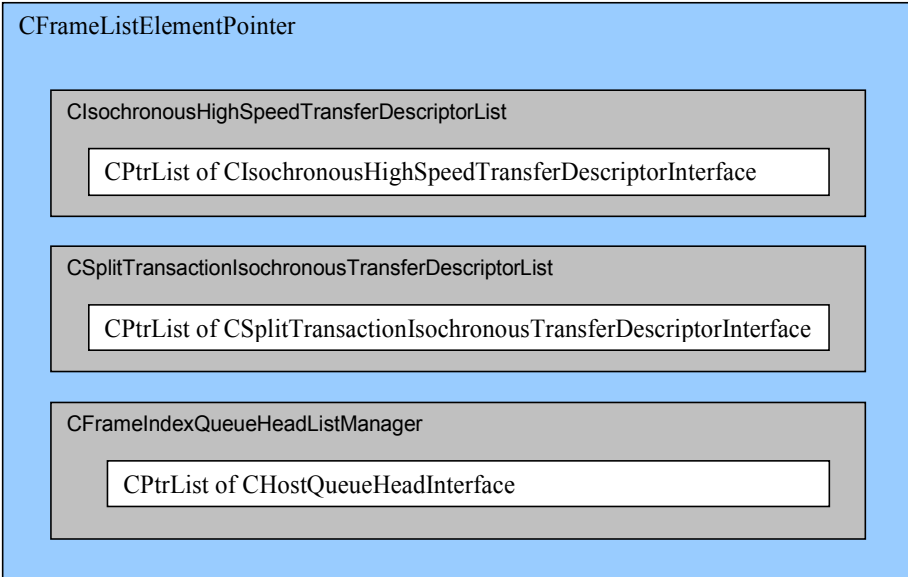
8.6.12.2 Host Controller Manager and Sub Components



		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 158-191

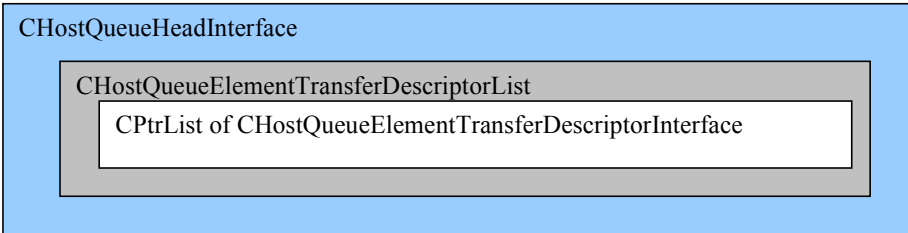
CUSBRegisterInterface derived from CRegisterArray
CAsynchronousListManager derived from CQueueListManager

8.6.12.3 Host Frame List Element Pointer Sub Components

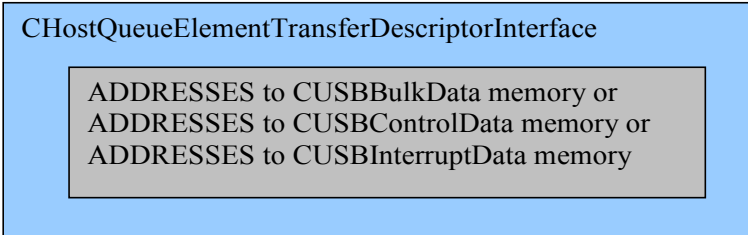


- CIsynchronousHighSpeedTransferDescriptorList derived from CTransferDescriptorList
- CHostQueueElementTransferDescriptorList derived from CTransferDescriptorList
- CSplitTransactionIsochronousTransferDescriptorList derived from CTransferDescriptorList
- CFrameIndexQueueHeadListManager derived from CQueueListManager
- CIsynchronousHighSpeedTransferDescriptorInterface derived from CTransferDescriptorInterface
- CSplitTransactionIsochronousTransferDescriptorInterface derived from CTransferDescriptorInterface

8.6.12.4 Host Queue Head Interface Sub Components

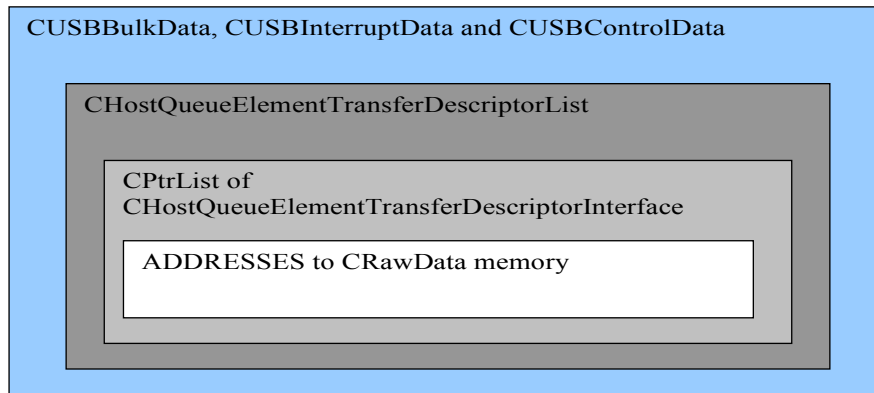


8.6.12.5 Host QueueElement Transfer Descriptor Interface Sub Components



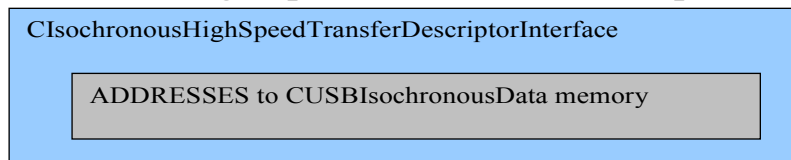
		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 159-191

8.6.12.6 Host Interrupt, Bulk and Control Data Sub Components

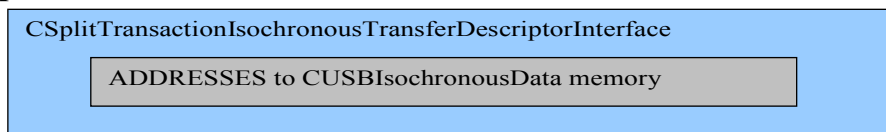


- **USBBulkData** derived from **CUSBAynchronousData**
- **CUSBInterruptData** derived from **CUSBPeriodicData**
- **CUSBControlData** derived from **CUSBAynchronousData**
- **CUSBAynchronousData** derived from **CUSBData**
- **CUSBPeriodicData** derived from **CUSBData**

8.6.12.7 Host Isochronous High Speed Interface Sub Components

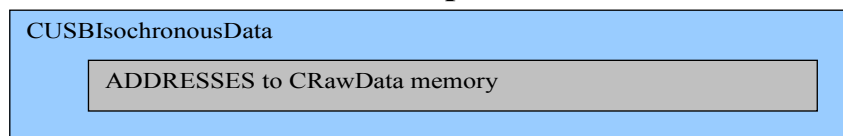


8.6.12.8 Host SplitTransactionIsochronousTransferDescriptorInterface Sub Components



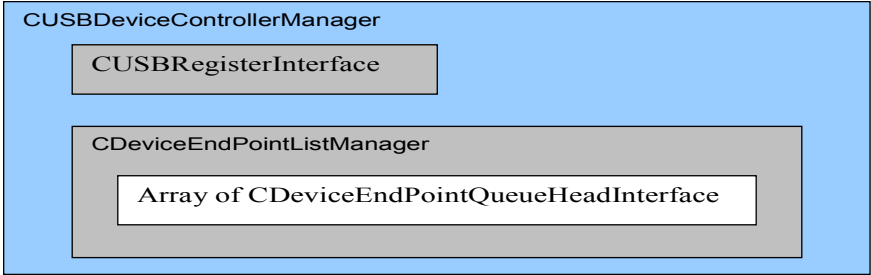
- **CUSBIsynchronousData** derived from **CUSBPeriodicData**

8.6.12.9 CUSBIsynchronousData Sub Components

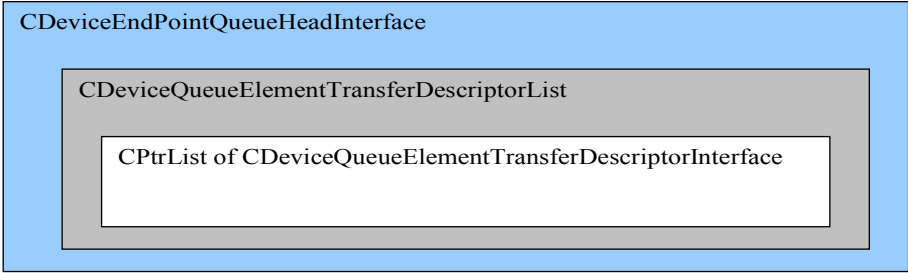


		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 160-191

8.6.12.10 Device Manager And Sub Components

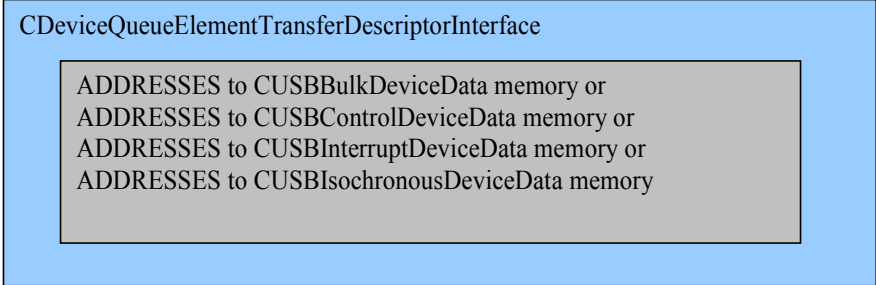


8.6.12.11 Device Endpoint Queue Head Interface Sub Components



- **CDeviceQueueElementTransferDescriptorList** derived from **CTransferDescriptorList**

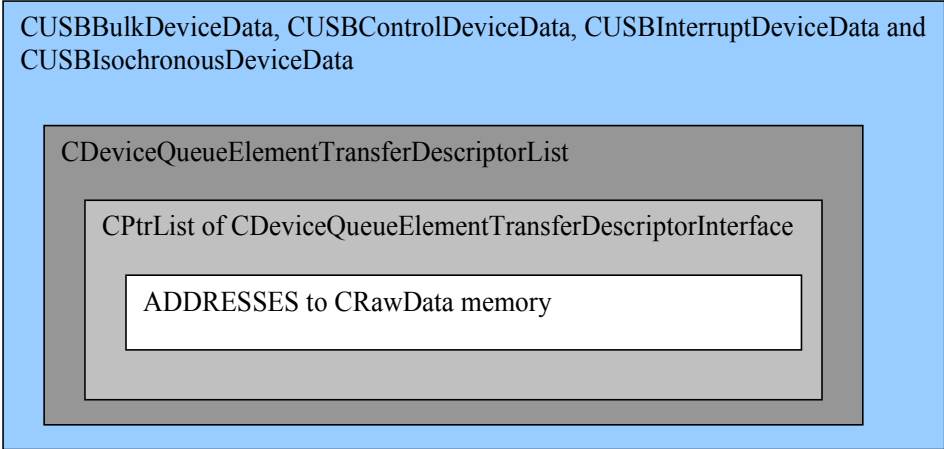
8.6.12.12 Device Queue Element Transfer Descriptor Interface Sub Components



- **CUSBBulkDeviceData** derived from **CUSBDeviceData**
- **USBControlDeviceData** derived from **CUSBDeviceData**
- **USBInterruptDeviceData** derived from **CUSBDeviceData**
- **USBIsochronousDeviceData** derived from **CUSBDeviceData**
- **CUSBDeviceData** derived from **CUSBData**

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 161-191

8.6.12.13 Device Interrupt, Bulk, Control and Isochronous Data Sub Components



- CDeviceQueueElementTransferDescriptorInterface derived from CTransferDescriptorInterface

8.7 Integration Test Classes

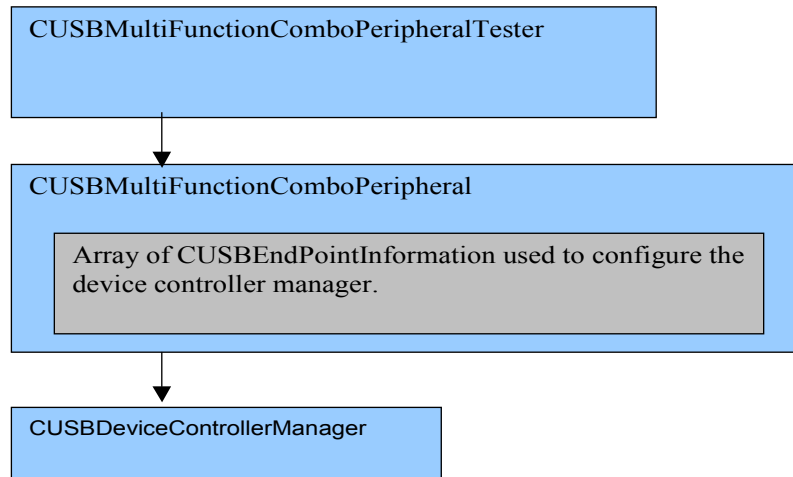
8.7.1 Key Testing Classes

- CUSBRegisterInterfaceTester derived from CTester
- CAsynchronousListTester derived from CTester
- CPeriodicListTester derived from CTester
- CUSBInterruptTester derived from CTester
- CUSBHostControllerTester derived from CTester
- CUSBDeviceControllerTester derived from CTester
- CUSBPeripheralTester derived from CTester
- CUSBOTGTester derived from CTester
- CEHCITest_2_2 derived from CTester
- CEHCITest_3_11 derived from CTester
- CEHCITest_3_13 derived from CTester
- CEHCITest_4_10 derived from CTester

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 162-191

8.7.2 Device Testing Classes

8.7.2.1 Device CUSBMultiFunctionComboPeripheralTester Dependencies



- The **CUSBMultiFunctionComboPeripheralTester** exercises the **CUSBMultiFunctionComboPeripheral**.
- The **CUSBMultiFunctionComboPeripheral** configures **CUSBDeviceControllerManager** and services transactions for the configuration.
- **CUSBMultiFunctionComboPeripheral** derived from **CUSBPeripheral**

8.7.2.2 CUSBMultiFunctionComboPeripheralTester Sub Tests

CUSBMultiFunctionComboPeripheralTester will drive tests for BULK, INT, CTRL and ISO.

8.7.2.2.1 Bulk Testing

Bulk Testing uses all key combinations of

- Address offset for data buffers.
- Max Packets Sizes
- Transfers Sizes
- Transfer Direction.
- Endpoint Configuration.
- UnderFlow values

8.7.2.2.2 ISO Testing

ISO Testing uses all key combinations of

- Address offset for data buffers.
- Max Packets Sizes
- Transfers Sizes
- Transfer Direction.
- Endpoint Configuration.
- Packet Multiplier.

8.7.2.2.3 Interrupt Testing

Interrupt Testing uses all key combinations of

- Address offset for data buffers.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 163-191

- Max Packets Sizes
- Transfers Sizes
- Transfer Direction.
- Endpoint Configuration.

8.7.2.2.4 Control Testing

Control Testing uses all key combinations of

- Address offset for data buffers.
- Max Packets Sizes
- Transfers Sizes
- Endpoint Configuration.

8.7.2.2.5 USB Address Change Testing

During All Packet Testing the Address is changed to a set of valid test address.

8.7.2.2.6 HandShake Sequence Testing

Testing with expected handshakes and error injection to validate error handling and error reporting.

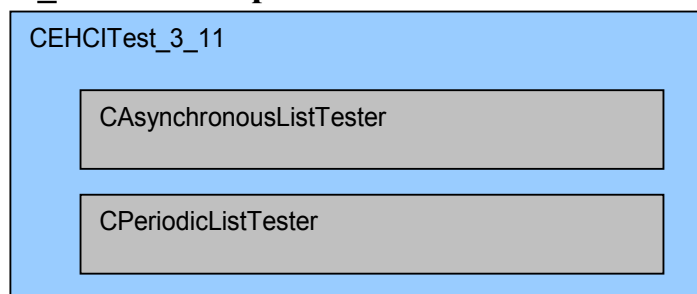
NOTE All the above tests have the built in DMA link test after priming. All the tests are driven via the USB interrupt signal. SOF interrupt is disabled.

8.7.3 Host Testing Classes

8.7.3.1 EHCI Test_2_2 Information

First, the test will start with a port suspend and resume and then proceed to reconnection tests. The test will walk over the list of speeds attempting all the supported speed connections. At the end of the test the tester will put the core back into the highest supported test speed.

8.7.3.2 EHCI Test 3_11 Sub Components



The 3.11 test contains a series of tests using BULK and Interrupt packets types. During the testing, the transfers are varied by

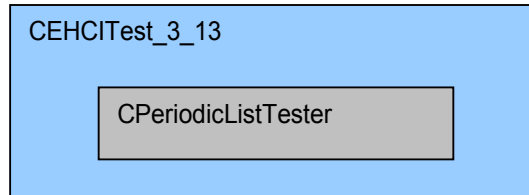
- Transfer size
- Max packet size
- USB address
- Endpoint number
- Endpoint speed.
- UnderFlow size

NOTE All tests account for PHY type.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 164-191

During the test the transfers are bounced off the responder echo endpoints and finally the incoming data is check against the previous out going transmitted data.

8.7.3.3 EHCI Test 3_13 Sub Components



8.8 Integration Tests Execution

The processor test software will begin by loading software to detect the core mode (host or device). If the mode is detected as host the responder will attach as a device and allow the software to control the responder to complete a set of host tests. If the core mode is detected as a device the responder will connect as a host and allow the device test software to complete a set of device tests. If the core supports both host and device modes the software will instruct the responder to switch modes after completing the host testing and run device testing to completion.

8.8.1 List of Events from the Host and Device Integration Test Simulation

8.8.1.1 Responder HDL at Time Zero

At time 0, the responder model performs sanity checks on the configuration constants in the `vsb_hs_cfg.v(hd1)` HDL core configuration file. For more information on these constants refer to the responder section. Any errors on these checks will cause a failure in the simulation.

NOTE This information can be extracted directly from the `SoftwareMessageIO.txt` file.

If the test configuration file is not setup to execute the tests listed below, the test will be skipped. The configuration file does not determine the order of the tests.

The following is a high level summary of the testing events, which skips over many of the details and checks performed

1. OTG registers are set to turn off all OTG interrupts.
2. The software will attempt to set the core into host mode.
3. The software will attempt to set the core into device mode.
4. If the core is only device capable the software will skip to the device mode setup step and testing procedures.
5. If the core is host and device capable the software will start attempting to run the OTG tests first.

8.8.1.2 OTG TESTING:

1. As an OTG capable core the tester will attempt to switch from AHost, BDevice, ADevice and BHost during the tests.
2. The OTG tester will run each test and return the connection to the state of AHost.

		<i>Title:</i> CI13410-a High-Speed USB Device Controller Core			<i>Revision:</i> 1.0A	
<i>Template:</i> TP02_DEP_GP	<i>Department:</i> IPCS	<i>Prepared by:</i> Joana Braga	<i>Approved by:</i> Luis Miguel Sampaio	<i>Security Level:</i> confidential	<i>Date:</i> 2006-05-11	<i>Page:</i> 165-191

8.8.1.3 HOST TESTING:

1. As a host, a check will be made to find the number of ports supported by the USB core. (In the current default build, this would be one port).
2. The register interface object will be extended to support the number of ports on the core.
3. The register interface class will analyze the register table and generate masks for proper register access . (Generating all the RWC fields for the register array...)
4. The Asynchronous List Manager is created.
5. The Period List Manager is created.
6. The Port Manager is created.
7. The register interface is tested. (Testing: Register Interface)
8. The periodic list size is set to a specific number of elements.
9. All of the interrupts for the host core are tested. (Testing Interrupt Index = 0, etc)
10. Host controller reset to cleanup any side effects from the register interface testing.
11. A wait for the responder starts.
12. An interrupt is detected and serviced by the interrupt handler; the control is passed to all of the sub-modules. (#Start of Interrupt Callback.)
13. The COMMON_PORTSC_CurrentConnectStatus is found set to one.
14. The port reset sequence is issued (-- Start Of Port Reset Sequence -)
15. A wait for the reset bit to go low is issued.
16. The connection speed is detected (Port Connection is detected as HS)
17. The port manager registers all the information about the new connection.
18. The responder interface adds a control endpoint to the Asynchronous manager for responder setup commands.
19. Next the HostController tester is loaded, which contains all the host specific testers to be executed.
20. The first tests consists of a series of connection tests at all the supported speeds, while inserting suspends and resumes. (Testing: Reconnections, ReconnectSpeed =0, etc)
21. Then the tester moves into testing with specific data types. (Testing: Echo InterruptData EndPointSpeed=1 MaxPacketSize=8 TransferSize=0 PortNumber=0 USBAddress=109 EndPointNumber=12, HubAddress=0, RootHubPortNum=0, etc)
22. Once all the data type tests have completed, the tester will request the next supported test speed and begin all the data type testing once again.
23. The HostControllerTester will finish all the tests.

8.8.1.4 DEVICE TESTING:

1. The HostControllerTester will issue a command to the responder for a reconnect as a device responder.
2. The device register interface will be loaded.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 166-191

3. The modeless reset will be asserted.
4. The software will attempt to change mode to a device core via the register interface. (Attempting Device Mode)
5. As a device core, the number of supported endpoints will be detected via the register interface.
6. The register interface object will be extended to support the detected number of endpoints.
7. The register interface class generates all the read/write access masks. (Generating all the RWC fields for the register array...)
8. The endpoint point registers are initialized.
9. The register interface is tested. (Testing: Register Interface)
10. The interrupts for the device is tested. (Testing: Interrupts Signal Connections.)
11. The controller is reset to a known state after the register test.
12. The mode bit will be set to device once again.
13. All of the interrupts for the device core are tested. (Testing Interrupt Index = 0, etc)
14. The interrupt callbacks are enabled.
15. The generic CUSBPeripheral class is created to perform the testing against. (Starting create of CUSBPeripheral....)
16. Control endpoint zero is created for device setup commands. (Creating data structures for endpoint zero...)
17. The CUSBPeripheral will change state to CompletedInit.
18. An interrupt is received. (#Start of Interrupt Callback.)
19. The OTG interface checks for any interrupt status enabled and find none.
20. The PortManager of the CUSBPeripheral detects a bus reset and does an endpoint cleanup. (Detected bus reset on DEV_USBSTS_USBReset...)
21. The CUSBPeripheral changes state to CompletedUSBReset.
22. The interrupt status is cleared.
23. Another interrupt is detected. (#Start of Interrupt Callback.)
24. The OTG status has no statuses enabled.
25. The PortManager finds the PortChangeDetect asserted. (Detected port change...)
26. The PortManager finds the new connection speed. (Port Connection is detected as HS)
27. The interrupt status is cleared for the PortChangeDetect.
28. Another Interrupt is received. (#Start of Interrupt Callback.)
29. The endpoint status is found as the reason for the interrupt. (Found DEV_ENDPOINTSETUPSTATUS_Status for EndpointNumber=0 Enabled.)

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 167-191

30. The testers are handed control in the interrupt and begin executing the tests. (Testing: BulkData ConnectSpeed=2 MaxPacketSize=8 AddressOffset=0 TransferSize=8 USBAddress=1 DeviceConfiguration=0 DataDirection=0)
31. The MultiFunctionPeripheralTester will switch states from issuing a test, to checking for completion of the test. (See CUSBMultiFunctionComboPeripheralTester.cpp for state machines in the form of a set if case(switch) statements).
32. Once all the data type tests have completed for a speed the tester will request a speed change and repeat the data type tests once again.
33. Once all the supported speeds and data types have been tested, a StopSim command will be issued to the responder.
34. The software will print a success message and exit.
35. The responder will halt the simulator.

8.9 Software flow using Host Bulk Transaction as an Example

8.9.1 Step 1 - Create endpoint information for an IN and the OUT pipe using CUSBEndPointInformation.Create()

```

if( request_end_point_information_out.Create( usb_address, usb_endpoint_number,
USBDataDirection_OUT, USBDataType_BULK, usb_max_packet_length, transfer_size,
endpoint_speed, hub_address, port_number, root_hub_port_number) == false )
{
PrintStringStdioErrorLF("0xEHCI484");
return( false );
}

if( request_end_point_information_in.Create( usb_address, usb_endpoint_number,
USBDataDirection_IN, USBDataType_BULK, usb_max_packet_length, transfer_size,
endpoint_speed, hub_address, port_number, root_hub_port_number) == false )
{
PrintStringStdioErrorLF("0xEHCI385");
return( false );
}

```

8.9.2 Step 2 - Use the CUSBResponderInterface::SetupEchoEndPoint() and pass the CUSBEndPointInformation

```

bool place_responder_behind_a_full_speed_downstream_hub = false;

if( m_usb_responder_interface->SetupEchoEndPoint( request_end_point_information_in,
request_end_point_information_out, port_number,
place_responder_behind_a_full_speed_downstream_hub ) == false)
{
PrintStringStdioErrorLF("0xAST0008");
return( false );
}

```

8.9.3 Step 3 - Create CTestData, one for the IN data and one for the OUT data.

```

CTestData test_data_out;

if( test_data_out.Create( transfer_size ) == false )
{
PrintStringStdioErrorLF("0xAST004");
return( false );
}

```

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 168-191

```

}
CTestData test_data_in;
if( test_data_in.Create( transfer_size ) == false )
{
PrintStringStdioErrorLF("0xAST005");
return( false );
}

```

8.9.4 Step 4 - Create two CUSBBulkData one for the IN and one for the OUT. In the create function pass the address of the CTestData buffers

```

CUSBBulkData out_data;

out_data.Create( USBDataDirection_OUT, transfer_size, &request_end_point_information_out,
test_data_out.GetDataAddress());

CUSBBulkData in_data;

in_data.Create( USBDataDirection_IN, transfer_size, &request_end_point_information_in,
test_data_in.GetDataAddress());

```

8.9.5 Step 5 - Process the OUT data on the CAsynchronousListManager.

```

UnsignedLong bytes_transferred_during_processing;

if( m_asynchronous_list_manager->ProcessAndWaitForCompletion( out_data ,
bytes_transferred_during_processing ) == false )
{
PrintStringStdioErrorLF("0xAST0978");
return( false);
}

```

8.9.6 Step 6 - Process the IN data on the CAsynchronousListManager

```

if( m_asynchronous_list_manager->ProcessAndWaitForCompletion( in_data,
bytes_transferred_during_processing ) == false )
{
PrintStringStdioErrorLF("0xAST073");
return( false );
}

```

8.9.7 Step 7 - Check the data echoed back correctly

```

if( test_data_out.DataContentsEqual( test_data_in ) == false )
{
PrintStringStdioErrorLF("0xAST92983");
return( false );
}

```

8.10 Software Flow using Device Bulk IN Transaction as an Example

8.10.1 Step 1 - Create the data to transfer as an IN

```

if( test_data_buffer.Create( transfer_size ) == false )
{
PrintStringStdioErrorLF("0xUPT048");
return( false);
}

```


		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 169-191

8.10.2 Step 2 - Put a pattern into the data so that the responder will get the correct pattern

```
test_data_buffer.SetContentsToIncrementPattern( m_default_start_index );
```

8.10.3 Step 3 - Get a pointer to the endpoint configured for bulk

```
CUSBEndPointInformation* endpoint_info;
    endpoint_info = m_multi_function_combo_peripheral->
GetEndpointBulkIN();
```

8.10.4 Step 4 - Create the CBULKDevice data using the endpoint information and test data

```
CUSBBulkDeviceData bulk_data;

if( bulk_data.Create( endpoint_information->
GetCopyEndpointDirection(), transfer_size, endpoint_information,
test_data_buffer.GetDataAddress()) == false )
{
    PrintStringStdioErrorLF("0xUPT0487");
    return( false );
}
```

8.10.5 Step 5 - Queue the data for transfer

```
if( m_peripheral_device_under_test->QueueDataTransfer( &device_data ) == false )
{
    PrintStringStdioErrorLF("0xUPT04837");
    return( false );
}
```

8.10.6 Step 6 - Request the transfer from the host responder

```
if( m_usb_responder_interface->RequestDataTransfer( bulk_data ) == false )
{
    PrintStringStdioErrorLF("0xUPT089887");
    return( false );
}
```

8.10.7 Step 7 - Wait for the transaction to complete.

```
if(m_peripheral_device_under_test ->Processwait( bulk_data ) == false )
{
    PrintStringStdioErrorLF("0xUMSPT08234873");
    return( false );
}
```

8.11 Controlling the Length or the Number of Tests at Compile Time And Runtime

The test length may be controlled in two ways. The first is specifying the exact tests to perform via the configuration file or header file. The second method is setting the test length iterations to limit the number of tests in each section that run. Please make note of the following:

1. The current parser uses a string match mechanism. To have a test execute from the CTesterExecuteList file, its name must be an exact match with that generated from the software test code. At run-time a simple comparison is made with the generated test string from the test software and the contents of the CTesterExecuteList file, if the two strings match (strcmp() == 0) the test will be executed. If the user

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 170-191

is not sure of the format of the test string a simple **grep** for **Testing:** in the test code `/APILevelTestCode` will reveal all of the **sprintf** statements used to generate the tests messages.

2. All test execute messages start with the string **Testing:** The software searches for the **Testing:** when attempting to determine the start of a command string.
3. A "*" can be placed at the end of a test execute string to specify the execution of all tests with a command string matching up to the *. The * is only valid at the end of a command string.
4. *:MatchLimit= can be used to limit the number of tests matching the string existing before the *. The command string *:MatchLimit=10 would only execute a maximum of 10 tests matching the string proceeding the * and succeeding the delimiter string **Testing:**.
5. A **CTestExecuteList** file containing a short set of tests will be included in the delivery. The file will be copied into the `/sim` directory at simulation run-time for the simulation software to find at run-time. Changing contents of this file will change the tests executed.
6. All attributes in command strings must exist in the test bench software. For example changing a transfer size to an arbitrary size like 18028 will cause the string to be ignored unless the transfer size exists in the list of valid transfer sizes in the software. If the user does not know which attributes are supported open the `CEndPointInformation.cpp` file, in the `/APILevelTestCode` directory, where all this information is defined.

When specifying the list of tests to be executed may be done at compile or run, time, use:

- **CTestExecuteList.cfg** at run time, this method may be used on systems that support file systems such as the co-simulation
- **CTestExecuteList.h** file at compile time, this method can be used in all simulation cases since the configuration for execution is compiled in.

8.11.1 Run time selection: Specifying an individual Test in the **CTestExecuteList** file 1.1.1

1. The **CTestExecuteList.cfg** lists the tests that will be done, you can choose to remove some of them.
2. Rerun the simulation

8.11.2 Test Code Handling of the test strings in **CTestExecuteList** file

1. The test software will always execute the tests specified in the same order. The **CTestExecuteList** does not control the order of the tests; it only controls the selection of the tests. Changing the order in the **CTestExecuteList** will have not effect on the testing.
2. The Test code has been constructed to execute tests with a reduced amount of overhead. Overhead reductions include reconnects, speed changes, responder reconfigurations and other expensive simulation operations.
3. The **CTestExecuteList** file is used at run time to determine if the test will be skipped or executed. When the test code is about to run a test the test code will generate a string describing the test and compare it to the list of strings in the **CTestExecuteList** file. If the list contains a matching string the test will be executed. If the test string does not match a string in the **CTestExecuteList** file the test will be skipped. The **CTestExecuteList** file contains a few special characters like * which is used ignore all characters

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 171-191

beyond the * and can only be used at the end of a string, MatchLimit to control the number of tests run matching the string and RandomIgnoreMatchValue which will randomly ignore tests matching the string if using a mod value.

4. The MatchLimit will stop the test software from executing tests matching the string once the limit number has been reached. The RandomIgnoreMatchValue will execute a test if the string matches and a random number mod the number specified is zero ($\text{rand}() \% \text{specified_number} == 0$). Examples of RandomIgnoreMatchValue exists in the config files delivered.

8.11.3 Adding a Test to the CTesterExecuteList file

1. **grep** for Testing: in the /APILevelTestCode directory - this is the best method for guaranteeing a correct string format.
2. Create the string in the exact format specified in the source code. It is suggested you cut and paste the string to guarantee you do not change the string contents.
3. Insert the attributes that are valid for the particular test. Use the information in CEndPointInformation.cpp if you are not sure of the correct attribute values. The string will contain formatting characters like %ld which will need to be updated with the desired values. For example MaxPacketsize=%ld would be updated with MaxPacketsize=8 in the case where a size of 8 was desired. String format commands may be verified by checking the **sprintf** command in any C compiler.
4. Paste the string into the CTesterExecuteList.cfg file if the simulation is running with a co-simulation system. Paste the string into a character array in the CTesterExecuteList.h file if the tests are going to be executed from an executable running on an embedded processor.
5. Rerun the simulation.
6. If the test is running on the co-simulation system check in the /Results/SoftwareMessageIO.txt file for the new command string. The command should have the string Run pre-pended to it, if the parser matched the string. If a **grep** for RunTesting: of the SoftwareMessageIO.txt file does not produce a string containing the new specified test and the simulation completed with a success message, check the format of the specified string for an incorrect attribute value or an incorrect string format.

8.11.3.1 Compile time selection: Specifying Tests (Preferred Method)

Specifying the minimum tests to provide the targeted functionality will result in the minimum simulation time.

Add the test strings to the CTesterExecuteList.h to have the test included at compile time.

USING_EMBEDDED_SYSTEM_WITH_NO_FILE_SYSTEM must be used included in the compile command.

CTesterExecuteList.h and CTesterExecuteList.cfg both contain a set of strings indicating the test to be executed at runtime. Note: if you have a valid CTesterExecuteList.h file, it can be used as a CTesterExecuteList.cfg file. This works because the parser for CTesterExecuteList.cfg will ignore all the header information used in CTesterExecuteList.h.

8.11.3.1.1 CTesterExecuteList.h Example

```
const int  g_tester_execute_list_string_count = 5;
const char* g_tester_execute_list[g_tester_execute_list_string_count];

g_tester_execute_list[0] = "\
Testing:OTG, Register Interface.\
Testing: Register Interface.\
Testing: Interrupts Signal Connections.\
Testing:OTG, Switch Modes Return Back To Start Mode MaxSpeed, NoSpeedToggle\
```

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 172-191

```

";
g_tester_execute_list[1] = "\
Testing:Host,BusSignaling*
";

g_tester_execute_list[2] = "\
Testing:Host,Multiple Echo Endpoint Traffic, EndPointSpeed=2 MaxPacketSize=8
TransferSize=32 *\
";

g_tester_execute_list[3] = "\
Testing:Host,Bulk,Echo, EndPointSpeed=1 MaxPacketSize=512 TransferSize=513 *\
Testing:Host,Bulk,Echo, EndPointSpeed=1 MaxPacketSize=1024 TransferSize=1023 *\
";

g_tester_execute_list[4] = "\
Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 MaxPacketSize=8 TransferSize=9 *\
Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 MaxPacketSize=32 TransferSize=31
*\
Testing:Host,ISO, Echo IsochronousData EndPointSpeed=2 MaxPacketSize=32 TransferSize=128
*\
";

```

NOTE The strings are broken into groups to keep from exceeding the max string length for some of the compilers.

8.11.3.1.2 CTesterExecuteList.cfg Example

```

Testing:OTG, Register Interface.
Testing: Register Interface.
Testing: Interrupts Signal Connections.
Testing:OTG, Switch Modes Return Back To Start Mode MaxSpeed, NoSpeedToggle

Testing:Host,BusSignaling*

Testing:Host,Multiple Echo Endpoint Traffic, EndPointSpeed=2 MaxPacketSize=8
TransferSize=32 *

Testing:Host,Bulk,Echo, EndPointSpeed=1 MaxPacketSize=512 TransferSize=513 *
Testing:Host,Bulk,Echo, EndPointSpeed=1 MaxPacketSize=1024 TransferSize=1023 *

Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 MaxPacketSize=8 TransferSize=9 *
Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 MaxPacketSize=32 TransferSize=31 *
Testing:Host,ISO, Echo IsochronousData EndPointSpeed=2 MaxPacketSize=32 TransferSize=128
*

```

NOTE CTesterExecuteList.h and CTesterExecuteList.cfg have all the same strings to perform the same tests. The CTesterExecuteList.h just has the proper syntax for creating an array of strings for a c compiler.

8.11.3.2 Configuration Test Strings in the CTesterExecuteList

Example of creating tests for host ISO using a specific string in the configuration file.

1. Open an empty text file called CTesterExecuteList.cfg
2. Put the string "Testing:Host,ISO*" in the file.
3. Place the file in the /Config directory.
4. Run test using the co-simulation environment.

8.11.3.2.1 Changing the above test to only run full speed ISO

```

"Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0*"

```

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 173-191

8.11.3.2.2 Change the above test to only run full speed ISO with a max packet of 8

"Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 MaxPacketSize=8 *"

8.11.3.2.3 Change the above test to run the ISO and the Bulk full speed with both set to max packet of size 8

"Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 \\
MaxPacketSize=8 *"

"Testing:Device, BulkData ConnectSpeed=2 MaxPacketSize=8 *"

8.11.3.2.4 Adding a Match Limit of 10 to the number of Bulk tests that run during the simulation

"Testing:Host,ISO, Echo IsochronousData EndPointSpeed=0 \\
MaxPacketSize=8 *"

"Testing:Device, BulkData ConnectSpeed=2 MaxPacketSize=8 *:MatchLimit=10"

NOTE The software generates all of the strings in the test bench as the software executes. The strings are the result of the software putting together the next test according to the constraints of the software, hardware and the list of attributes listed in the software.

For example in the test software a list of transfer sizes for FS BULK is generated by a function with statements for appending interesting ranges, centered around the typical corners. See below:

```
transfer_sizes.AppendValueRange( 7, 21 ); //add interesting cases first
transfer_sizes.AppendValueRange( 1, 6 );
transfer_sizes.AppendValueRange( 32 - edge_size, 32 + edge_size );
transfer_sizes.AppendValueRange( 64 - edge_size, 64 + edge_size );
transfer_sizes.AppendValueRange( 256 - edge_size, 256 + edge_size );
transfer_sizes.AppendValueRange( 512 - edge_size, 512 + edge_size );
transfer_sizes.AppendValueRange( 1024 - edge_size, 1024 );
```

8.11.3.3 Using grep, the following Host Execute Test Strings can be found.

```
Testing:Host,Multiple Echo Endpoint Traffic, \
EndPointSpeed=%ld MaxPacketSize=%ld \
TransferSize=%ld PortNumber=%ld USBAddress=%ld \
EndPointNumber=%ld, HubAddress=%ld, RootHubPortNum=%ld
```

```
Testing:Host,Bulk,Echo, EndPointSpeed=%ld MaxPacketSize=%ld \
TransferSize=%ld \
PortNumber=%ld USBAddress=%ld EndPointNumber=%ld, \
HubAddress=%ld, RootHubPortNum=%ld
```

```
Testing:Host,BusSignaling,ConnectionSpeed=%ld,\
PortNumber=%ld,%s, Suspend followed by a host resume.
```

```
Testing:Host,BusSignaling,ConnectionSpeed=%ld,\
PortNumber=%ld,%s Suspend followed by a wait for the device resume
```

```
Testing:Host,BusSignaling,ConnectionSpeed=%ld,\
PortNumber=%ld,%s Suspend followed by a port reset from the host.
```

```
Testing:Host,CmultiPortTester
```

```
Testing:Host,Interrupt, Echo Interrupt EndPointSpeed=%ld \
MaxPacketSize=%ld TransferSize=%ld \
PortNumber=%ld USBAddress=%ld EndPointNumber=%ld, \
HubAddress=%ld, RootHubPortNum=%ld
```

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 174-191

Testing:Host,ISO, Echo IsochronousData EndPointSpeed=%ld \\
MaxPacketSize=%ld TransferSize=%ld \\
PortNumber=%ld USBAddress=%ld EndPointNumber=%ld, \\
HubAddress=%ld, RootHubPortNum=%ld

8.11.3.4 Using grep, the following Shared Host And Device Execute Test Strings can be found in the /APILevelTestCode Directory.

Testing: Register Interface.

Testing: Interrupts Signal Connections.

8.11.3.5 Using grep the following Device Execute Test Strings can be found in /APILevelTestCode Directory

Testing:Device,TestPacket
Testing:Device, ISOData ConnectSpeed=%ld MaxPacketSize=%ld \\
AddressOffset=%ld TransferSize=%ld USBAddress=%ld \\
DeviceConfiguration=%ld DataDirection=%ld PacketMultiplier=%ld

Testing:Device, BulkData ConnectSpeed=%ld MaxPacketSize=%ld \\
AddressOffset=%ld TransferSize=%ld USBAddress=%ld \\
DeviceConfiguration=%ld DataDirection=%ld

Testing:Device, InterruptData ConnectSpeed=%ld \\
MaxPacketSize=%ld AddressOffset=%ld TransferSize=%ld \\
USBAddress=%ld DeviceConfiguration=%ld DataDirection=%ld

Testing:Device, ControlData ConnectSpeed=%ld \\
MaxPacketSize=%ld AddressOffset=%ld TransferSize=%ld \\
USBAddress=%ld DeviceConfiguration=%ld DataDirection=%ld

Testing:Device,HandShake, Responder TestSequence=%ld \\
Requesting Bulk Data ExpectedHandShakeCodes=%ld.

Testing:Device,HandShake, Responder TestSequence=%ld \\
Requesting Interrupt Data ExpectedHandShakeCodes=%ld.

Testing:Device,HandShake, Responder TestSequence=1 \\
Requesting Data And Endpoint Does Not Send Payload For Setup.

Testing:Device,HandShake, Responder TestSequence=1 Stalling \\
Control Endpoint And Endpoint Does Not Send Payload For Setup

8.11.3.6 OTG Execute Test Strings

Testing:OTG, Register Interface.

Testing:OTG, Switch Modes Return Back To Start Mode \\
MaxSpeed, NoSpeedToggle

Testing:OTG, Switch Modes Return Back To Start Mode \\
FullSpeed, NoSpeedToggle

Testing:OTG, Switch Modes Return Back To Start Mode \\
FullSpeed, SpeedToggling

Testing:OTG, Switch Modes Return Back To Start \\
Mode HighSpeed, SpeedToggling

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 175-191

8.11.3.7 Running All Tests And Controlling The Number Of Tests Executed (This method of controlling the tests is NOT preferred , see section on specifying tests in the configuration file.)

Many of the tests have the ability to trim the regression time by not running some of the unnecessary cases. The USB test software contains a constant called **TEST_LENGTH_ITERS** to control the maximum number of iterations a test will attempt to run. This number can be controlled in two ways.

1. On the compile line set **TEST_LENGTH_ITERS_COMPILE_TIME** to a value, this value will be used to set the value of **TEST_LENGTH_ITERS**.
2. The second method is to edit `CTest.h`, changing **TEST_LENGTH_ITERS** to a new default value and recompiling. A huge value like `0xFFFF` will cause all the test cases supported and available to run. A small value like 5 will only run the first 5 cases for each test. Note many of the tests have been ordered to execute some of the more interesting cases first and the less critical cases last.

8.11.4 Increasing and Decreasing the Messaging Levels

The USB test software contains a constant **g_debug_message_level** to control the level of messages printed to the message interface. This constant can be found in `CAbstractInterface.cpp`. The value of this number can be controlled in two ways.

1. Define **DEBUG_MESSAGE_LEVEL** to a value at compile time. Defining this value will cause the value of **g_debug_message_level** to be set in the code.
2. Edit `CAbstractInterface.cpp` and change **g_debug_message_level** to a new default value and recompile. A value of 99 will print the maximum number of messages and a value of zero will print minimal messages.

8.11.5 Co-Simulation BFM Logging Results

Tests were developed in a self-checking mode. Checking is performed both in the test code and in the responder. It is important to note that SW test code outputs it's messaging to the `SoftwareMessageIO.txt` file in the results directory, while the responder model outputs it's messaging to the transcript file of the simulation. Both transcript file and result directory must be checked at the end of the simulation for a complete report on the status of the simulation. Errors will always cause immediate termination of the simulation. Other interesting results files are `HostMemoryDevice.txt`, `device_1_Initiator.txt` and `device_1_Target.txt`. The description of these files is contained in the co-simulation document.

8.12 Interrupt Propagation into the Software Objects

Interrupts are started in the Interrupt Controller API function and make there way to the top level OTG manager via callback function pointer. The objects have been designed to trickle the interrupt down into the sub-objects, so that all object interrupt status checking can be completed with every interrupt.

8.12.1 Example of a host interrupt calling:

1. **CInterruptControllerInterface** calls the registered Interrupt callback **OTGInterruptCallbackIRQ**.
2. **OTGInterruptCallbackIRQ** calls **CUSBOTGManager::ProcessInterrupt** after it has checked the software framework has been created.
3. **CUSBOTGManager::ProcessInterrupt** calls **CUSBHostControllerManager::ProcessInterrupts** after it has completed processing the OTG status.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 176-191

4. **CUSBHostControllerManager::ProcessInterrupts** calls **CUSBRegisterInterface::ProcessInterrupt()**, **CAsynchronousListManager::ProcessInterrupt()**, **CPeriodicListManager::ProcessInterrupt()** and **CUSBPortManager::ProcessInterrupt()** after its has completed processing the controller status.

NOTE During the forced interrupt testing, the interrupts are blocked from the normal processing path. At the top level these interrupts are considered invalid interrupts and the control path is directed to the interrupt tester. This block is removed for all other testing.

8.13 Software determination of hardware parameters.

8.13.1 How does the test software know which connection speeds are supported during simulation?

The software will use the information provided by the USB core registers to control testing of the Low Full and High speed. If the register information for the PHY indicates no support for a connection speed, the test software will not attempt to connect and run tests at the unsupported speed

8.13.1.1 How does the software determine the operational mode of the core.

Test selection is a result of the HDL build, if the hardware does not exist the tests will not attempt to run the tests for the missing hardware.

The test bench software will start with an attempt to test the host USB functionality and then attempt to test the USB device functionality. When the test bench is attempting to test the host functionality the mode bit will be written and then checked. If the mode bit does not take the set to host mode value, the host testing will end with the assumption the host hardware does not exist. The test bench will print a message stating the fact it was unable to test the host functionality due to the HDL core capabilities.

8.13.1.2 Example Setting the Mode Bit to Host for a Host Capable Core In the SoftwareMessage.txt File

```

12> [ 16452.000ns] Attempting Host Mode
13> [ 16452.000ns] [Getting register COMMON_ModelessReset = 0...]
14> [ 16593.332ns] [Setting register COMMON_ModelessReset = 1...]
15> [ 16859.996ns]
16> [ 16859.996ns] > Waiting For: COMMON_ModelessReset to change to 0 BitMask=FFFFFFFh...
17> [ 16859.996ns] > Waiting:01
18> [ 50459.660ns] > Waiting Complete, polls required = 2, cycles per poll = 1000!
19> [ 50459.660ns]
20> [ 50459.660ns] Setting USBMODE bit to USB Host...
21> [ 50459.660ns] [Setting register COMMON_USBMode = 3...]
22> [ 50726.324ns]
23> [ 50726.324ns] > Waiting For: COMMON_USBMode to change to 3 BitMask=FFFFFFFh...
24> [ 50726.324ns] > Waiting:0
25> [ 50859.656ns] > Waiting Complete, polls required = 1, cycles per poll = 1000!
26> [ 50859.656ns]

```

8.13.2 References to Lab Code In The Integration Tests

A reference to `#ifdef RUN_IN_LAB` can be found in the source code `MainEntry.cpp` and should be ignored for all simulation software. The `RUN_IN_LAB` was defined during internal lab testing of the instrumented device driver manager integration software. These lab tests were used to verify the correctness of the models in a real-time environment using actual hardware in the lab.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 177-191

8.14 Adding Tests and Functionality to the existing Test Software

8.14.1 Adding a New Register Definition to the OTG Register Interface

8.14.1.1.1 Step 1 - Assign an enumerated value to the register in *CUSBOTGRegisterInterface.h*

```
enum USB200TGRegisters
{
//OTG registers
OTGMODE_NEW_REGISTER_NAME,    //New register name
OTGMODE_VBUSDischarge,
OTGMODE_VBUSCharge,
}
```

8.14.1.1.2 Step 2 - Create a description of the register in the register description structure in *CUSBOTGRegisterInterface.cpp*

```
RegisterInformationArray common_otg_echi_and_non_echi_registers =
{
{OTGMODE__NEW_REGISTER_NAME, false, false, DEBUG_STRING ("OTGMODE__NEW_REGISTER_NAME "),
DEFAULT_OTG_BASEADDRESS, 0x0000, 11, 11, REG_RW, 0x00, DEBUG_STRING ("TODO: added
common string here.")},

{OTGMODE_VBUSDischarge, false, false, DEBUG_STRING ("OTGMODE_VBUSDischarge"),
DEFAULT_OTG_BASEADDRESS, 0x0000, 0,0,REG_RW, 0x00, DEBUG_STRING ("Setting this bit causes
VBus to discharge through a resistor.")},

{OTGMODE_VBUSCharge, false, false, DEBUG_STRING ("OTGMODE_VBUSCharge"),
DEFAULT_OTG_BASEADDRESS, 0x0000, 1,1,REG_RW,0x00, DEBUG_STRING("Setting this bit causes
the VBus line to be charged. This is used for VBus pulsing during SRP.")},
}
```

8.14.1.1.3 Step 3 - Add a function to set specific values

```
bool CUSBOTGRegisterInterface::SetNewRegisterValueToResetValue()
{
if( SetRegisterValue(OTGMODE__NEW_REGISTER_NAME, status ) == false )
{
PrintStringStdioErrorLF("0xOTGRG0383828");
return( false );
}
return(true);
}
```

This is a simple example for extending the register interface for OTG. The CUSBRegisterInterface for host and device could have been expanded using similar steps. Both of the register interfaces objects are derived from the CRegisterArrayInterface class.

NOTE The **CUSBRegisterInterface** does have some dynamic attributes for endpoints in device mode and some dynamic attributes for ports in host mode. The dynamic portions of the interface are controlled by the core register values.

CUSBRegisterInterface has some blocks of commonly defined registers for both host and device mode. Be sure to correctly associate the block of common registers to both the host and the device when making additions. (See source code for more details and instances of common registers)

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 178-191

8.14.2 Creating A New Test for the USB Host Functionality

8.14.2.1.1 Step 1 - Define a new class to contain the test code and derive it from CTester. On create pass in all the objects needed to perform the testing.

For example:

```
class NewTest_123: public CTester
{
// Member functions
bool Create( CAsynchronousListTester* asynchronous_list_tester,
CPeriodicListTester* periodic_list_tester );
bool Start();
// Member functions default overrides
NewTest_123: ( ); // Default constructor
virtual ~ NewTest_123: ( ); // Destructor
protected:
// Member functions
char* GetName();
UnsignedLong GetTestNumber();
};
```

8.14.2.1.2 Step 2 - Define the Create() function.

For example:

```
bool NewTest_123::Create( CAsynchronousListTester* asynchronous_list_tester,
CPeriodicListTester* periodic_list_tester )
{
m_asynchronous_list_tester = asynchronous_list_tester;
m_periodic_list_tester = periodic_list_tester;
m_has_been_created = true;
return( true );
}
```

8.14.2.1.3 Step 3 - Define the Start() function.

For example:

```
bool NewTest_123::Start()
{
CUSBEndPointInformation request_end_point_information_out;
CUSBEndPointInformation request_end_point_information_in;

UnsignedLong usb_address = 1;
UnsignedLong usb_endpoint_number =3;
UnsignedLong hub_address = 0;
UnsignedLong port_number = 0;
UnsignedLong transfer_size = 123;
UnsignedLong root_hub_port_number = 0;

if( request_end_point_information_out.Create( usb_address, usb_endpoint_number,
USBDataDirection_OUT, USBDataType_BULK, usb_max_packet_length ,transfer_size,
endpoint_speed, hub_address, port_number, root_hub_port_number) == false )
{
PrintStringStdioErrorLF("0xEHCI484");
return( false );
}

if( request_end_point_information_in.Create( usb_address, usb_endpoint_number,
USBDataDirection_IN, USBDataType_BULK, usb_max_packet_length, transfer_size,
endpoint_speed, hub_address, port_number,
root_hub_port_number) == false )
{
PrintStringStdioErrorLF("0xEHCI385");
return( false );
}
```

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 179-191

```

if( m_asynchronous_list_tester->ProcessBulkLoopback( transfer_sizes ,
request_end_point_information_in, request_end_point_information_out, port_number ) ==
false )
{
    PrintStringStdioErrorLF("0xEHCI48");
    return( false );
}

return( true );
}

```

8.14.2.1.4 Step 4 - Insert the class as a member of CUSBHostControllerTester in the CUSBHostControllerTester.h file.

8.14.2.1.5 Step 5 - Call the create function in the CUSBHostControllerTester::Create function and pass in the needed objects for the new tester.

8.14.2.1.6 Step 6 - Insert the new tester into the CUSBHostControllerTester: code using the host tester class.

For example:

```

bool CUSBHostControllerTester::Start()
{
    CTestManager test_manager;

    if( test_manager.AppendTester( m_new_tester ) == false )
    {
        PrintStringStdioErrorLF("HTM0423");
        return( false );
    }

    //Tell the tester to start testing
    if( test_manager.Start() == false )
    {
        return( false );
    }

    if( ProcessTestsCompletedCommand() == false )
    {
        PrintStringStdioErrorLF("HTM0012");
        return( false );
    }
    return( true );
}

```

8.15 Reducing Test Initialization overhead in Non-BFM test environments.

Initialization of the USB HS controller requires actions on the System bus interface, and action on the USB port to put the controller into a connected and configured state. In the integration verification environment the normal sequence of port states are used to perform the USB speed negotiation protocol and enable a port after a connection event. Speed detection and connection are considered critical areas of verification for transceiver integration. However, some customers make use of verification environments that break tests into small units. In these environments the overhead of repeating the connection, reset and speed negation sequences at the beginning of each test can add significant overhead to the packet and data transfer testing that is being targeted by the test environment.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 180-191

The sections below describe the hardware EHCI extensions to the core and software actions required on the system bus to advance internal state of the port controller state machine to the connected and enabled state so that packet transfers may be initiated without the USB reset and speed negotiation overhead. The USB responder module used in the integration verification environment does not support skipping over the connection, reset and speed negotiation protocols specified in the USB specification.

8.15.1 Hardware additions for simulation acceleration

The port controller state machine dictates the operation of the ports in the VUSB_HS core. Test mode enhancements have been implemented in the port control state machine to force the port state into the connected and enabled state at Low, Full or High speed.

The controls for forcing the core speed and bypassing the reset and speed negotiation signaling have been added as extensions to the EHCI port test modes in the PTC field of the PORTSC register. The PTC field extensions are:

```
TEST_MODE_DISABLED == 0,
TEST_MODE_FORCE_HS == 5,
TEST_MODE_FORCE_FS == 6,
TEST_MODE_FORCE_LS == 7;
```

8.15.2 Steps to force the Peripheral controller into the connected state.

1. Initialize the peripheral controller normally for peripheral operation.
2. On a detection of the USB bus reset event from the testbench USB host model write the PORTSC.PTC = FORCE_ENABLE_{HS/FS}

Note: Low speed operations are not supported as a peripheral device.

3. Transition the testbench USB host model to the desired connect speed and terminate the USB bus reset.
4. Write the PORTSC.PTC = TEST_MODE_DISABLE

The peripheral controller port is now able to support packet traffic at the selected speed.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 181-191

9 Synthesis

This section is intended to assist customers when synthesizing the USB-HS core. It is assumed the user has an understanding of their synthesizers operation and this discussion is not tailored to any particular synthesizer.

The USB-HS core is shipped in either VHDL or Verilog source code format. The HDL code is hierarchical which allows the designer additional flexibility when synthesizing. A single file can be synthesized at a time, or any number can be synthesized together. Generally, the best results are achieved when the entire design is synthesized together.

The design flow is simple and straightforward. The designer simply configures the USB-HS core and then reads the files into the synthesizer. The designer can specify whether to optimize for speed or area, set a maximum delay, and other user constraints specific to the synthesizer. A target technology is also selected. The results from the synthesis should then be run through a static timing and analysis tool. This program will check the delays through all nets in the design to make sure that it can operate at your designated frequency. If any nets did not make timing, synthesis can be rerun with additional user constraints on the slow nets. This generally will continue until all timing constraints have been satisfied. This design flow is shown on the following page.

9.1 USB-HS HDL File Hierarchy

The following diagram illustrates the HDL hierarchy. Each file of the USB-HS core is briefly described below. See the header inside each file for more detail.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 183-191

9.2 Configuration

There are several parameters of the USB-HS core that can be customized to meet the requirements of a system. These options are available in the `vusb_hs_cfg.v(hdl)` file and is briefly described below.

System Configuration:

- Select the reset type and system clock source.
- Select the transceiver type & data width.

Device Configuration:

- Set the number of endpoints.

RX/TX Buffers:

- Select the depth, number of address bits and burst size.

Device Interface Version Number:

- Set the customer's device version number.

Please refer to [7 System Level Issues and Core Configuration](#), and the `vusb_hs_cfg.v(hdl)` file for more detailed information.

9.3 Synthesis Script

An example synthesis script [`tool_data/design_compiler/scripts/vusb_hs_core_dev_[bvci|amba].tcl`] is provided that shows how the USB-HS core may be synthesized. This file is targeted to the Synopsys version listed in the release notes or newer but it contains the information required to understand how to synthesize the USB-HS core using any synthesis tool.

Before using this script, several variables at the top of the synthesis scripts need to be set with values appropriate for the target synthesis library including:

- `search_path`,
- `link_library`,
- `target_library`,
- `symbol_library`.

The second group of variables in the synthesis script is used to specify the timing constraints for the design. These are several timing parameters that the user should take care in setting to ensure success. Of note are the:

- clock constraints.
- dpram setup & access.
- I/O constraints for the transceiver interface signals.

NOTE The default parameters included in the script are for a representative device and are likely not sufficient for the actual user system.

Also included in the synthesis scripts is the option to have scan test inserted into the core. The user can decline the insertion of scan altogether or they can have the scan flops inserted, with or without, stitching them together. If the option to add scan test is selected, then a report on the results will be generated as `vusb_hs_core.tst_rpt`. To insert a scan, carry out the following:

```
# insert scan on core?
#
# no : do not insert scan
# sfo : insert Scan Flops Only - do not add chains
# yes : insert scan & chains
#
```

		Title: CI13410-a High-Speed USB Device Controller Core				Revision: 1.0A
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 184-191

NOTE For those who wish to add scan test to the USB-HS core, the "sfo" option is most likely to be used. Stitching scan chains at the USB-HS core level is usually not desired. Scan chain stitching is usually done at a higher level - a "scan core" level where all scan testable circuitry has been encapsulated in a single piece of hierarchy".

After the `vusb_hs_core_dev_[bvci|amba].tcl` has been modified as described above, go to `tool_data/design_compiler` and type `dc_shell-t -f scripts/<script_file.tcl>`, pick one of the scripts: the amba or bvci versions.

Two reports are generated in the `tool_data/design_compiler/reports` folder after running the script: `vusb_hs_core_*.syn_rpt` and `vusb_hs_core_*.tim_rpt`. The file `vusb_hs_core_*.syn_rpt` is a complete report of the synthesis operation and `vusb_hs_core_*.tim_rpt` is a timing report that summarizes the worst-case path delays in the design and list any timing constraints that have not been met.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 185-191

10 References

- [1]. *USB 2.0 Transceiver Macrocell Interface (UTMI) Specification*, Version 1.05, March 2001, Jon Lueker, Steve McGowan (Editor) Ken Oliver, Dean Warren.
<http://www.intel.com>
- [2]. *VSI Alliance Virtual Component Interface Standard*, Version 2 (OCB 2 2.0), April 2001, On-Chip Bus Development Working Group.
<http://www.vsi.org>
- [3]. *Universal Serial Bus Specification*, Revision 2.0, April 2000, Compaq, Hewlett-Packard, Intel, Lucent, Microsoft, NEC, Philips. <http://www.usb.org>
- [4]. *On-The-Go Supplement to the USB 2.0 Specification*, Revision 1.0, Dec 2001, On-The-Go Working Group of the USB-IF.
<http://www.usb.org>
- [5]. *Enhanced Host Controller Interface Specification for Universal Serial Bus*, Revision 0.95, November 2000, Intel Corporation. <http://www.intel.com>
- [6]. *Universal Serial Bus Specification*, Revision 1.1, September 1998, Compaq, Intel, Microsoft, NEC.
<http://www.usb.org>
- [7]. *Software Design Document for USB 2.0 Device API*
- [8]. *AMBA Specification*, Revision 2.0, May 1999, ARM Limited. <http://www.arm.com>
- [9]. *UTMI+ Low Pin Interface (ULPI) Specification*, Revision 1.0 February 2004, ULPI Specification Organization.
<http://www.ulpi.org>

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 186-191

11 Glossary of Terms and Abbreviations

This chapter lists and defines terms and abbreviations used throughout this specification.

ACK	Handshake packet indicating a positive acknowledgment.
Active Device	A device that is powered and is not in the Suspend state.
Asynchronous Data	Data transferred at irregular intervals with relaxed latency requirements.
Asynchronous RA	The incoming data rate, F^s_i , and the outgoing data rate, F^s_o , of the RA process are independent (i.e., there is no shared master clock). See also rate adaptation.
Asynchronous SRC	The incoming sample rate, F^s_i , and outgoing sample rate, F^s_o , of the SRC process are independent (i.e., there is no shared master clock). See also sample rate conversion.
Audio Device	A device that sources or sinks sampled analog data.
AWG#	The measurement of a wire's cross-section, as defined by the American Wire Gauge standard.
b/s	Transmission rate expressed in bits per second.
B/s	Transmission rate expressed in bytes per second.
Babble	Unexpected bus activity that persists beyond a specified point in a (micro) frame.
Bandwidth	The amount of data transmitted per unit of time, typically bits per second (b/s) or bytes per second (B/s).
Big Endian	A method of storing data that places the most significant byte of multiple-byte values at a lower storage address. For example, a 16-bit integer stored in big endian format places the least significant byte at the higher address and the most significant byte at the lower address. See also little endian.
Bit	A unit of information used by digital computers. Represents the smallest piece of addressable memory within a computer. A bit expresses the choice between two possibilities and is typically represented by a logical one (1) or zero (0).
Bit Stuffing	Insertion of a "0" bit into a data stream to cause an electrical transition on the data wires, allowing a PLL to remain locked.
Buffer	Storage used to compensate for a difference in data rates or time of occurrence of events, when transmitting data from one device to another.
Bulk Transfer	One of the four USB transfer types. Bulk transfers are non-periodic, large burst communication typically used for a transfer that can use any available bandwidth and can be delayed until bandwidth is available. See also transfer type.
Bus Enumeration	Detecting and identifying USB devices.
Byte	A data element that is eight bits in size.
Capabilities	Those attributes of a USB device that are administrated by the host.
Characteristics	Those qualities of a USB device that are unchangeable; for example, the device class is a device characteristic.
Client	Software resident on the host that interacts with the USB System Software to arrange data transfer between a function and the host. The client is often the data provider and consumer for transferred data.
Configuring Software	Software resident on the host software that is responsible for configuring a USB device. This may be a system configurator or software specific to the device.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 187-191

Control Endpoint	A pair of device endpoints with the same endpoint number that are used by a control pipe. Control endpoints transfer data in both directions and, therefore, use both endpoint directions of a device address and endpoint number combination. Thus, each control endpoint consumes two endpoint addresses.
Control Pipe	Same as a message pipe.
Control Transfer	One of the four USB transfer types. Control transfers support configuration/command/status type communications between client and function. See also transfer type.
CRC	See Cyclic Redundancy Check.
CTI	Computer Telephony Integration.
Cyclic Redundancy Check (CRC)	A check performed on data to see if an error has occurred in transmitting, reading, or writing the data. The result of a CRC is typically stored or transmitted with the checked data. The stored or transmitted result is compared to a CRC calculated for the data to determine if an error has occurred.
Default Address	An address defined by the USB Specification and used by a USB device when it is first powered or reset. The default address is 00H.
Default Pipe	The message pipe created by the USB System Software to pass control and status information between the host and a USB device's endpoint zero.
Device	A logical or physical entity that performs a function. The actual entity described depends on the context of the reference. At the lowest level, device may refer to a single hardware component, as in a memory device. At a higher level, it may refer to a collection of hardware components that perform a particular function, such as a USB interface device. At an even higher level, device may refer to the function performed by an entity attached to the USB; for example, a data/FAX modem device. Devices may be physical, electrical, addressable, and logical. When used as a non-specific reference, a USB device is either a hub or a function.
Device Address	A seven-bit value representing the address of a device on the USB. The device address is the default address (00H) when the USB device is first powered or the device is reset. Devices are assigned a unique device address by the USB System Software.
Device Endpoint	A uniquely addressable portion of a USB device that is the source or sink of information in a communication flow between the host and device. See also endpoint address.
Device Resources	Resources provided by USB devices, such as buffer space and endpoints. See also Host Resources and Universal Serial Bus Resources.
Device Software	Software that is responsible for using a USB device. This software may or may not also be responsible for configuring the device for use.
Downstream	The direction of data flow from the host or away from the host. A downstream port is the port on a hub electrically farthest from the host that generates downstream data traffic from the hub. Downstream ports receive upstream data traffic.
Driver	When referring to hardware, an I/O pad that drives an external load. When referring to software, a program responsible for interfacing to a hardware device, that is, a device driver.
DWord	Double word. A data element that is two words (i.e., four bytes or 32 bits) in size.
Dynamic Insertion and Removal	The ability to attach and remove devices while the host is in operation.
E²PROM	See Electrically Erasable Programmable Read Only Memory.
EEPROM	See Electrically Erasable Programmable Read Only Memory.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 188-191

Electrically Erasable Programmable Read Only Memory (EEPROM)	Non-volatile rewriteable memory storage technology.
End User	The user of a host.
Endpoint	See device endpoint.
Endpoint Address	The combination of an endpoint number and an endpoint direction on a USB device. Each endpoint address supports data transfer in one direction.
Endpoint Direction	The direction of data transfer on the USB. The direction can be either IN or OUT. IN refers to transfers to the host; OUT refers to transfers from the host.
Endpoint Number	A four-bit value between 0H and FH, inclusive, associated with an endpoint on a USB device.
Envelope detector	An electronic circuit inside a USB device that monitors the USB data lines and detects certain voltage related signal characteristics.
EOF	End-of- (micro) Frame.
EOP	End-of-Packet.
External Port	See port.
Eye pattern	A representation of USB signaling that provides minimum and maximum voltage levels as well as signal jitter.
False EOP	A spurious, usually noise-induced event that is interpreted by a packet receiver as an EOP.
Flush (Endpoint)	A term used in this device controller implementation to describe the action of clearing an endpoint ready status.
Frame	A 1-millisecond time base established on full-/low-speed buses.
Frame Pattern	A sequence of frames that exhibit a repeating pattern in the number of samples transmitted per frame. For a 44.1 kHz audio transfer, the frame pattern could be nine frames containing 44 samples followed by one-frame containing 45 samples.
Fs	See sample rate.
Full-duplex	Computer data transmission occurring in both directions simultaneously.
Full-speed	USB operation at 12 Mb/s. See also low-speed and high-speed
Function	A USB device that provides a capability to the host, such as an ISDN connection, a digital microphone, or speakers.
Handshake Packet	A packet that acknowledges or rejects a specific condition. For examples, see ACK and NAK.
High-bandwidth endpoint	A high-speed device endpoint that transfers more than 1024 bytes and less than 3073 bytes per microframe.
High-speed	USB operation at 480 Mb/s. See also low-speed and full-speed
Host	The host computer system where the USB Host Controller is installed. This includes the host hardware platform (CPU, bus, etc.) and the operating system in use.
Host Controller	The host's USB interface.
Host Controller Driver (HCD)	The USB software layer that abstracts the Host Controller hardware. The Host Controller Driver provides an SPI for interaction with a Host Controller. The Host Controller Driver hides the specifics of the Host Controller hardware implementation.
Host Resources	Resources provided by the host, such as buffer space and interrupts. See also Device Resources and Universal Serial Bus Resources.
Hub	A USB device that provides additional connections to the USB.
Hub Tier	One plus the number of USB links in a communication path between the host and a function.
I/O Request Packet	An identifiable request by a software client to move data between itself (on the host) and an endpoint of a device in an appropriate direction.
Interrupt Request (IRQ)	A hardware signal that allows a device to request attention from a host. The host typically invokes an interrupt service routine to handle the condition that caused the request.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 189-191

Interrupt Transfer	One of the four USB transfer types. Interrupt transfer characteristics are small data, non-periodic, low frequency, and bounded-latency. Interrupt transfers are typically used to handle service needs. See also transfer type.
IRP	See I/O Request Packet.
IRQ	See Interrupt Request.
Isochronous Data	A stream of data whose timing is implied by its delivery rate
Isochronous Device	An entity with isochronous endpoints, as defined in the USB Specification, that sources or sinks sampled analog streams or synchronous data streams.
Isochronous Sink Endpoint	An endpoint that is capable of consuming an isochronous data stream that is sent by the host.
Isochronous Source Endpoint	An endpoint that is capable of producing an isochronous data stream and sending it to the host.
Isochronous Transfer	One of the four USB transfer types. Isochronous transfers are used when working with isochronous data. Isochronous transfers provide periodic, continuous communication between host and device. See also transfer type.
Jitter	A tendency toward lack of synchronization caused by mechanical or electrical changes. More specifically, the phase shift of digital pulses over a transmission medium.
kb/s	Transmission rate expressed in kilobits per second.
kB/s	Transmission rate expressed in kilobytes per second.
Little Endian	Method of storing data that places the least significant byte of multiple-byte values at lower storage addresses. For example, a 16-bit integer stored in little endian format places the least significant byte at the lower address and the most significant byte at the next address. See also big endian.
LOA	Loss of bus activity characterized by an SOP without a corresponding EOP.
Low-speed	USB operation at 1.5 Mb/s. See also full-speed and high-speed.
LSb	Least significant bit.
LSB	Least significant byte.
Mb/s	Transmission rate expressed in megabits per second.
MB/s	Transmission rate expressed in megabytes per second.
Message Pipe	A bi-directional pipe that transfers data using a request/data/status paradigm. The data has an imposed structure that allows requests to be reliably identified and communicated.
Microframe	A 125-microsecond time base established on high-speed buses.
MSb	Most significant bit.
MSB	Most significant byte.
NAK	Handshake packet indicating a negative acknowledgment.
Non Return to Zero Invert (NRZI)	A method of encoding serial data in which ones and zeros are represented by opposite and alternating high and low voltages where there is no return to zero (reference) voltage between encoded bits. Eliminates the need for clock pulses.
NRZI	See Non Return to Zero Invert.
Object	Host software or data structure representing a USB entity.
Packet	A bundle of data organized in a group for transmission. Packets typically contain three elements: control information (e.g., source, destination, and length), the data to be transferred, and error detection and correction bits.
Packet Buffer	The logical buffer used by a USB device for sending or receiving a single packet. This determines the maximum packet size the device can send or receive.
Packet ID (PID)	A field in a USB packet that indicates the type of packet and by inference, the format of the packet and the type of error detection applied to the packet.
Phase	A token, data, or handshake packet. A transaction has three phases
Phase Locked Loop (PLL)	A circuit that acts as a phase detector to keep an oscillator in phase with an incoming frequency.
Physical Device	A device that has a physical implementation; e.g., speakers, microphones, and CD players.
PID	See Packet ID.

		Title: CI13410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 190-191

Pipe	A logical abstraction representing the association between an endpoint on a device and software on the host. A pipe has several attributes; for example, a pipe may transfer data as streams (stream pipe) or messages (message pipe) See also stream pipe and message pipe.
PLL	See Phase Locked Loop.
Polling	Asking multiple devices, one at a time, if they have any data to transmit.
POR	See Power On Reset.
Port	Point of access to or from a system or circuit. For the USB, the point where a USB device is attached.
Power On Reset (POR)	Restoring a storage device, register, or memory to a predetermined state when power is applied.
Prime (Endpoint)	A term used in this device controller implementation to describe the action of readying an endpoint to transmit or receive data.
Programmable Data Rate	A fixed data rate (single-frequency endpoints), a limited number of data rates (32 kHz, 44.1 kHz, 48 kHz, ...), or a continuously programmable data rate. The exact programming capabilities of an endpoint must be reported in the appropriate class-specific endpoint descriptors.
Protocol	A specific set of rules, procedures, or conventions relating to format and timing of data transmission between two devices.
RA	See rate adaptation.
Rate Adaptation	The process by which an incoming data stream, sampled at F^s_i , is converted to an outgoing data stream, sampled at F^s_o , with a certain loss of quality, determined by the rate adaptation algorithm. Error control mechanisms are required for the process. F^s_i and F^s_o can be different and asynchronous. F^s_i is the input data rate of the RA; F^s_o is the output data rate of the RA.
Request	A request made to a USB device contained within the data portion of a SETUP packet.
Retire	The action of completing service for a transfer and notifying the appropriate software client of the completion.
Root Hub	A USB hub directly attached to the Host Controller. This hub (tier 1) is attached to the host.
Root Port	The downstream port on a Root Hub.
Sample	The smallest unit of data on which an endpoint operates; a property of an endpoint.
Sample Rate (Fs)	The number of samples per second, expressed in Hertz (Hz).
Sample Rate Conversion (SRC)	A dedicated implementation of the RA process for use on sampled analog data streams. The error control mechanism is replaced by interpolating techniques.
Service	A procedure provided by a System Programming Interface (SPI).
Service Interval	The period between consecutive requests to a USB endpoint to send or receive data.
Service Jitter	The deviation of service delivery from its scheduled delivery time.
Service Rate	The number of services to a given endpoint per unit time.
SOF	See Start-of-Frame.
SOP	Start-of-Packet.
SPI	See System Programming Interface.
Split transaction	A transaction type supported by host controllers and hubs. This transaction type allows full- and low-speed devices to be attached to hubs operating at high-speed.
SRC	See Sample Rate Conversion.
Stage	One part of the sequence composing a control transfer; stages include the Setup stage, the Data stage, and the Status stage.
Start-of-Frame (SOF)	The first transaction in each (micro) frame. An SOF allows endpoints to identify the start of the (micro) frame and synchronize internal endpoint clocks to the host.
Stream Pipe	A pipe that transfers data as a stream of samples with no defined USB structure.
Synchronization Type	A classification that characterizes an isochronous endpoint's capability to connect to other isochronous endpoints.

		Title: CII3410-a High-Speed USB Device Controller Core			Revision: 1.0A	
Template: TP02_DEP_GP	Department: IPCS	Prepared by: Joana Braga	Approved by: Luis Miguel Sampaio	Security Level: confidential	Date: 2006-05-11	Page: 191-191

Synchronous RA	The incoming data rate, F^s_i , and the outgoing data rate, F^s_o , of the RA process, are derived from the same master clock. There is a fixed relation between F^s_i and F^s_o .
Synchronous SRC	The incoming sample rate, F^s_i , and outgoing sample rate, F^s_o , of the SRC process are derived from the same master clock. There is a fixed relation between F^s_i and F^s_o .
System Programming Interface (SPI)	A defined interface to services provided by system software.
TDM	See Time Division Multiplexing.
TDR	See Time Domain Reflectometer.
Termination	Passive components attached at the end of cables to prevent signals from being reflected or echoed.
Time Division Multiplexing (TDM)	A method of transmitting multiple signals (data, voice, and/or video) simultaneously over one communications medium by interleaving a piece of each signal one after another.
Time Domain Reflectometer (TDR)	An instrument capable of measuring impedance characteristics of the USB signal lines.
Timeout	The detection of a lack of bus activity for some predetermined interval.
Token Packet	A type of packet that identifies what transaction is to be performed on the bus.
Transaction	The delivery of service to an endpoint; consists of a token packet, optional data packet, and optional handshake packet. Specific packets are allowed/required based on the transaction type.
Companion Controller	A functional component of a USB hub. The Companion Controller responds to special high-speed transactions and translates them to full/low-speed transactions with full/low-speed devices attached on downstream facing ports.
Transfer	One or more bus transactions to move information between a software client and its function.
Transfer Type	Determines the characteristics of the data flow between a software client and its function. Four standard transfer types are defined: control, interrupt, bulk, and isochronous.
Turn-around Time	The time a device needs to wait to begin transmitting a packet after a packet has been received to prevent collisions on the USB. This time is based on the length and propagation delay characteristics of the cable and the location of the transmitting device in relation to other devices on the USB.
Universal Serial Bus Driver (USB D)	The host resident software entity responsible for providing common services to clients that are manipulating one or more functions on one or more Host Controllers.
Universal Serial Bus Resources	Resources provided by the USB, such as bandwidth and power. See also Device Resources and Host Resources.
Upstream	The direction of data flow towards the host. An upstream port is the port on a device electrically closest to the host that generates upstream data traffic from the hub. Upstream ports receive downstream data traffic.
USB D	See Universal Serial Bus Driver.
USB-IF	USB Implementers Forum, Inc. is a nonprofit corporation formed to facilitate the development of USB compliant products and promote the technology.
Virtual Device	A device that is represented by a software interface layer. An example of a virtual device is a hard disk with its associated device driver and client software that makes it able to reproduce an audio .WAV file.
Word	A data element that is two bytes (16 bits) in size.